

**ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH**



**BÁO CÁO
ĐỒ ÁN TỐT NGHIỆP**

**NGHIÊN CỨU VÀ PHÁT TRIỂN ROBOT TRỢ LÝ ẢO
ỨNG DỤNG TRÍ TUỆ NHÂN TẠO HỖ TRỢ CHĂM SÓC
VÀ GIÁM SÁT NGƯỜI CAO TUỔI TẠI NHÀ**

Ngành: Kỹ thuật Máy tính

HỘI ĐỒNG: 10L

GVHD: TS. Lê Trọng Nhân

GVPB: TS. Phan Trần Minh Khuê

—o0o—

SVTH 1: Lê Đăng Khoa (2211601)

SVTH 2: Vũ Trọng Khánh (2211541)

SVTH 3: Lương Văn Khanh (2211486)

Tp. Hồ Chí Minh, tháng 12/2025

LỜI CAM ĐOAN

Chúng em xin cam đoan rằng đề án với đề tài “*Nghiên cứu và phát triển robot trợ lý ảo ứng dụng trí tuệ nhân tạo hỗ trợ chăm sóc và giám sát người cao tuổi tại nhà*” là công trình nghiên cứu do chính chúng em thực hiện dưới sự hướng dẫn khoa học của TS. Lê Trọng Nhân. Toàn bộ nội dung của đề án được xây dựng trên cơ sở quá trình học tập, nghiên cứu nghiêm túc, có sự tìm hiểu, phân tích và tổng hợp từ nhiều nguồn tài liệu khác nhau.

Chúng em xin cam đoan rằng các số liệu, kết quả thực nghiệm, mô hình đề xuất, hình ảnh minh họa và các nội dung trình bày trong đề án là trung thực, phản ánh đúng quá trình nghiên cứu và thực hiện đề tài.

Trong quá trình thực hiện đề án, chúng em có tham khảo và sử dụng các tài liệu, công trình nghiên cứu, sách, bài báo khoa học, cũng như các nguồn thông tin hợp pháp khác. Tất cả các tài liệu tham khảo đều đã được trích dẫn đầy đủ, chính xác và tuân thủ đúng các quy định về trích dẫn học thuật. Chúng em xin cam đoan không sao chép, không đạo văn, không sử dụng trái phép kết quả nghiên cứu của các tác giả khác dưới bất kỳ hình thức nào.

Chúng em xin hoàn toàn chịu trách nhiệm trước Khoa, Nhà trường và Hội đồng chấm đề án về tính trung thực, chính xác và tính pháp lý của toàn bộ nội dung được trình bày trong đề án này. Nếu có bất kỳ sai sót hoặc vi phạm nào liên quan đến bản quyền, đạo đức học thuật hoặc tính xác thực của dữ liệu, chúng em xin hoàn toàn chịu trách nhiệm theo các quy định hiện hành.

Nhóm tác giả

Lê Đăng Khoa

Vũ Trọng Khánh

Lương Văn Khanh

LỜI CẢM ƠN

Trước hết, chúng em xin bày tỏ lòng biết ơn sâu sắc đến TS. Lê Trọng Nhân, người đã trực tiếp hướng dẫn, định hướng và đồng hành cùng chúng em trong suốt quá trình thực hiện đề tài “*Nghiên cứu và phát triển robot trợ lý ảo ứng dụng trí tuệ nhân tạo hỗ trợ chăm sóc và giám sát người cao tuổi tại nhà*”. Trong quá trình thực hiện đồ án, thầy đã tận tình chỉ dẫn về phương pháp nghiên cứu, định hướng triển khai nội dung, cũng như thường xuyên góp ý, chỉnh sửa và giải đáp những khó khăn mà chúng em gặp phải. Sự hướng dẫn tận tâm và những ý kiến chuyên môn quý báu của thầy đã giúp chúng em từng bước hoàn thiện đề tài theo hướng khoa học, thực tiễn và có giá trị ứng dụng.

Chúng em xin chân thành cảm ơn quý thầy cô trong Khoa/Viện đã giảng dạy và truyền đạt cho chúng em những kiến thức nền tảng và chuyên môn trong suốt quá trình học tập. Những kiến thức và kinh nghiệm mà quý thầy cô đã chia sẻ không chỉ giúp chúng em hoàn thành đồ án này mà còn là hành trang quan trọng cho quá trình học tập và công tác sau này.

Bên cạnh đó, chúng em xin gửi lời cảm ơn đến Khoa và Nhà trường đã tạo điều kiện thuận lợi về cơ sở vật chất, môi trường học tập và nghiên cứu, giúp chúng em có thể triển khai và hoàn thành đề tài đúng tiến độ.

Chúng em cũng xin chân thành cảm ơn gia đình và bạn bè đã luôn quan tâm, động viên, chia sẻ và hỗ trợ chúng em cả về tinh thần lẫn thời gian trong suốt quá trình học tập và thực hiện đồ án. Sự động viên đó là nguồn khích lệ to lớn giúp chúng em vượt qua những khó khăn và áp lực trong quá trình nghiên cứu.

Mặc dù đã có nhiều cố gắng và nỗ lực trong quá trình thực hiện, song do hạn chế về thời gian, kiến thức và kinh nghiệm nghiên cứu, đồ án không tránh khỏi những thiếu sót. Chúng em rất mong nhận được sự góp ý và chỉ dẫn từ quý thầy cô để đề tài được hoàn thiện hơn.

Chúng em xin chân thành cảm ơn.

Nhóm tác giả

Lê Đăng Khoa

Vũ Trọng Khánh

Lương Văn Khanh

TÓM TẮT

Trong bối cảnh già hóa dân số và nhu cầu chăm sóc người cao tuổi tại nhà ngày càng tăng. Việc giám sát an toàn, duy trì liên lạc và hỗ trợ sinh hoạt hằng ngày cho người cao tuổi đặt ra nhiều khó khăn với các gia đình có người chăm sóc thường xuyên vắng mặt. Đồ án nghiên cứu và xây dựng hệ thống robot trợ lý ảo ứng dụng trí tuệ nhân tạo (Artificial Intelligence – AI), sử dụng thiết bị Android gắn trên robot làm bộ não xử lý. Hệ thống hướng đến việc tăng khả năng hỗ trợ kịp thời khi xảy ra sự cố, tạo phương thức tương tác thuận tiện bằng giọng nói cho người cao tuổi và giúp người chăm sóc theo dõi từ xa hiệu quả hơn.

Giải pháp gồm hai ứng dụng Android. Ứng dụng Robot khai thác camera để xử lý thị giác máy tính tại biên. Chuỗi xử lý kết hợp mô hình ước lượng tư thế You Only Look Once (YOLO), thuật toán Simple Online and Realtime Tracking (SORT) và đặc trưng tái định danh để theo dõi, điều khiển robot tự động di chuyển theo người cần chăm sóc; mô hình Long Short-Term Memory (LSTM) cùng bước hậu kiểm được sử dụng để nhận biết tình huống nghi té ngã. Khi phát hiện dấu hiệu bất thường, robot chủ động hỏi thăm bằng giọng nói để xác nhận tình trạng của người dùng; trong trường hợp người dùng cần trợ giúp hoặc không có phản hồi, hệ thống gửi cảnh báo và thiết lập cuộc gọi đến người chăm sóc. Bên cạnh đó, robot có khả năng trò chuyện, giúp nhắc nhở và điều khiển qua giọng nói, qua đó hỗ trợ một số nhu cầu giao tiếp và sinh hoạt thường ngày. Ứng dụng Caregiver cho phép người chăm sóc quản lý hồ sơ, thiết lập các nhắc nhở, xem nhật ký các nhắc nhở, cảnh báo té ngã và cuộc gọi; đồng thời có thể bật hoặc tắt các chức năng giám sát, gọi video hai chiều và điều khiển robot bằng cần điều khiển ảo.

Kết quả của đồ án là một nguyên mẫu tích hợp hai ứng dụng di động, khối xử lý trí tuệ nhân tạo, hạ tầng kết nối và phần cứng robot. Là sự tận dụng năng lực tính toán và cảm biến của thiết bị Android để giảm yêu cầu về phần cứng chuyên dụng. Là tiền đề cho việc tiếp tục nghiên cứu và phát triển các ứng dụng, sản phẩm công nghệ hỗ trợ chăm sóc người cao tuổi tại nhà. Hệ thống có thể được hoàn thiện thêm về độ tin cậy, bảo mật, khả năng ghép cặp nhiều thiết bị và các chức năng theo dõi sức khỏe, qua đó góp phần mở rộng khả năng ứng dụng trí tuệ nhân tạo và robot trong lĩnh vực chăm sóc người cao tuổi.

Từ khóa: *assistive robot, older adult care, fall detection, human pose estimation, person tracking, edge AI, voice interaction.*

MỤC LỤC

Lời cam đoan	i
Lời cảm ơn	ii
Tóm tắt	iii
Mục lục	iv
Danh mục hình ảnh	xi
Danh mục bảng biểu	xii
Danh mục từ viết tắt	xiv
Danh mục thuật ngữ	xviii
1 Giới thiệu	1
1.1 Bối cảnh và tính cấp thiết của đề tài	1
1.2 Mục tiêu đề tài	2
1.2.1 Mục tiêu tổng quát	2
1.2.2 Mục tiêu cụ thể	2
1.3 Đối tượng và phạm vi nghiên cứu	3
1.3.1 Đối tượng nghiên cứu	3
1.3.2 Phạm vi chức năng	4
1.3.3 Giới hạn nghiên cứu	5
1.4 Phương pháp nghiên cứu	6
1.4.1 Khảo cứu và phân tích tài liệu	6
1.4.2 Phân tích yêu cầu và thiết kế hệ thống	6
1.4.3 Xây dựng và tích hợp nguyên mẫu	7
1.4.4 Thực nghiệm và đánh giá	7
1.5 Các công trình và giải pháp liên quan	7
1.5.1 Các hướng tiếp cận cho bài toán phát hiện té ngã	8
1.5.2 Các giải pháp theo dõi người và bám mục tiêu	15
1.5.3 Chatbot AI và hội thoại hỗ trợ người cao tuổi	17
1.5.4 Các nền tảng robot tích hợp và bài học áp dụng	21

1.5.5	Khoảng trống nghiên cứu và định vị của đề tài	21
1.6	Đóng góp của đề tài	22
1.7	Cấu trúc báo cáo	22
1.8	Kết luận chương	23
2	Cơ sở lý thuyết	25
2.1	Nền tảng Intel OpenBot (OpenBot)	25
2.1.1	Giới thiệu và ý tưởng cốt lõi	25
2.1.2	Kiến trúc và các thành phần liên quan	25
2.1.3	Giải pháp kết nối và điều khiển	26
2.1.4	AI và cơ chế theo dõi đối tượng	27
2.1.5	Khả năng áp dụng cho đồ án	28
2.2	Nền tảng Thị giác máy tính (Computer Vision)	28
2.2.1	Bài toán Object Detection	29
2.2.2	Lịch sử phát triển Object Detection	29
2.2.3	Khái niệm Bounding Box	30
2.2.4	Intersection over Union (IoU)	30
2.2.5	True Positive, False Positive, False Negative	31
2.2.6	Đánh giá hiệu suất Mean Average Precision (mAP)	32
2.2.7	Các kiến trúc phát hiện đối tượng phù hợp thiết bị di động	33
2.2.8	Theo dõi đối tượng và tạo chuỗi thời gian	34
2.3	Họ mô hình YOLO và YOLO26-Pose	35
2.3.1	Quá trình phát triển của họ YOLO	35
2.3.2	Cấu trúc backbone–neck–head	36
2.3.3	Các thay đổi chính của YOLO26	37
2.3.4	Quy trình suy luận và hậu xử lý YOLO-Pose	37
2.3.5	Input và Output của YOLO-Pose	38
2.3.6	Các mức kích thước của YOLO26-Pose	39
2.4	Mô hình chuỗi thời gian (Sequence Modeling)	40
2.4.1	Mạng nơ-ron hồi quy (RNN)	40
2.4.2	Mạng Bộ nhớ Ngắn-Dài hạn (LSTM)	40
2.5	Giới thiệu Dataset Fall Detection – LE2I	42
2.5.1	Mục tiêu và đặc điểm của dataset LE2I	42
2.5.2	Dữ liệu video và Annotation	43

2.5.3	Ưu điểm và hạn chế của dataset LE2I	43
2.5.4	So sánh với các Dataset quy mô lớn (DiverseFALL10500)	44
2.6	Suy luận AI trên thiết bị và TensorFlow Lite	44
2.6.1	Suy luận AI trên thiết bị	44
2.6.2	Kiến trúc và quy trình suy luận TensorFlow Lite	45
2.6.3	Tối ưu hóa và đánh giá mô hình sau chuyển đổi	45
2.7	Firebase Authentication và Realtime Database	46
2.7.1	Firebase Authentication	46
2.7.2	Firebase Realtime Database	46
2.8	WebRTC, signaling và ICE	47
2.8.1	Tổng quan và kiến trúc truyền thông WebRTC	47
2.8.2	Signaling và mô hình Offer–Answer	47
2.8.3	ICE và vòng đời kết nối WebRTC	48
2.9	Bluetooth Low Energy và GATT	51
2.9.1	Tổng quan và quá trình thiết lập kết nối BLE	51
2.9.2	ATT, GATT và phương thức trao đổi dữ liệu	52
2.9.3	Hiệu năng, bảo mật và khả năng áp dụng	53
2.10	Phần cứng robot và thư viện điều khiển động cơ	54
2.11	Kết luận chương	57
3	Phân tích và thiết kế hệ thống	58
3.1	Phân tích yêu cầu hệ thống	58
3.1.1	Bài toán, mục tiêu và phạm vi	58
3.1.2	Tác nhân, điều kiện vận hành và ranh giới hệ thống	59
3.1.3	Yêu cầu chức năng	60
3.1.4	Yêu cầu phi chức năng	63
3.2	Kiến trúc tổng quan hệ thống	65
3.2.1	Cơ sở xây dựng kiến trúc tổng quan	65
3.2.2	Các khối chức năng và luồng phối hợp	66
3.3	Các luồng xử lý chính	69
3.3.1	Các use case chính	69
3.3.2	Luồng giám sát và xử lý té ngã	71
3.3.3	Luồng cuộc gọi khẩn cấp và điều khiển từ xa	72
3.3.4	Luồng hội thoại và nhắc nhở	74

3.4	Thiết kế và lựa chọn công nghệ cho các module	75
3.4.1	Module ứng dụng người chăm sóc và điều phối tại robot	77
3.4.2	Module thu nhận hình ảnh, ước lượng tư thế và theo dõi mục tiêu	78
3.4.3	Module tự bám theo và điều khiển chuyển động	81
3.4.4	Module phát hiện, hậu kiểm và xác nhận té ngã	83
3.4.5	Module hội thoại và lời nhắc	84
3.4.6	Module dữ liệu dùng chung và liên lạc thời gian thực	85
3.4.7	Module cầu nối BLE và chấp hành thân robot	86
3.4.8	Module chuẩn bị và triển khai mô hình AI	86
3.5	Thiết kế dữ liệu, giao thức truyền thông và phương thức kết nối	86
3.5.1	Cây dữ liệu Firebase	86
3.5.2	Cấu trúc dữ liệu điều khiển trên RTDB	88
3.5.3	Giao tiếp BLE và điều khiển động cơ trên ESP32	90
3.5.4	Cơ chế dừng an toàn nhiều tầng	90
3.6	Kết luận chương	91
4	Hiện thực hệ thống	92
4.1	Kiến trúc hiện thực và ánh xạ công nghệ	92
4.2	Môi trường phát triển và tổ chức các thành phần	94
4.3	Hiện thực RobotApp	95
4.3.1	Khởi động, điều hướng và vòng đời dịch vụ	95
4.3.2	TrackingFragment và quản lý camera	97
4.3.3	Nạp và suy luận mô hình YOLO26n-Pose	97
4.3.4	SORT, Kalman, Hungarian và Re-ID	101
4.3.5	Duy trì mục tiêu ID0 và tạo lệnh bám theo	105
4.3.6	LSTM phát hiện té ngã	110
4.3.7	Hậu kiểm té ngã	110
4.3.8	Luồng xác nhận và cảnh báo	112
4.3.9	Chatbot giọng nói	112
4.3.10	Nhắc nhở theo lịch	113
4.3.11	WebRTC trong RobotApp	114
4.3.12	RobotControlManager và BLE	114
4.4	Hiện thực CaregiverApp	117
4.4.1	Khởi động, xác thực và hồ sơ	117

4.4.2	Bật/tắt chức năng theo dõi	117
4.4.3	Lịch và nhật ký	119
4.4.4	Tiếp nhận cuộc gọi khi ứng dụng chạy nền	122
4.4.5	Gọi video và điều khiển robot từ xa	123
4.5	Hiện thực dữ liệu dùng chung	123
4.6	Huấn luyện và chuyển đổi mô hình AI	125
4.6.1	Xử lý dữ liệu	125
4.6.2	Trích xuất đặc trưng	126
4.6.3	Huấn luyện mô hình	127
4.6.4	Chuyển đổi và tối ưu mô hình	127
4.6.5	Quy trình kiểm tra tương đương sau chuyển đổi	128
4.7	Hiện thực firmware phần thân robot	129
4.7.1	Khởi tạo phần cứng	129
4.7.2	Tiếp nhận lệnh qua BLE GATT	129
4.7.3	Xử lý vùng không đáp ứng và dừng khi mất lệnh	130
4.8	Kết luận chương	130
5	Kết quả và thảo luận	131
5.1	Mục tiêu đánh giá	131
5.2	Cấu hình và tiêu chí đánh giá	131
5.3	Kết quả hiện thực và artifact	132
5.3.1	Mức độ hoàn thành thành phần	132
5.3.2	Kết quả đóng gói Android	133
5.3.3	Kết quả artifact mô hình	134
5.4	Kiểm thử tự động ở mức logic	134
5.4.1	Ánh xạ lệnh và chính sách bám người	134
5.4.2	Hậu kiểm té ngã và hình học tư thế	135
5.4.3	Phân vùng đường dẫn Firebase	136
5.5	Đánh giá tính nhất quán khi tích hợp bằng kiểm tra tĩnh	136
5.5.1	Nhất quán Firebase	136
5.5.2	Nhất quán BLE	136
5.5.3	Chuỗi timeout	137
5.5.4	Quyền sở hữu tài nguyên	137
5.6	Kịch bản kiểm thử chức năng đầu cuối	137

5.7	Thiết kế phép đo hiệu năng	139
5.7.1	Độ trễ pipeline AI	139
5.7.2	Độ trễ điều khiển	139
5.7.3	WebRTC	140
5.7.4	Độ chính xác té ngã	140
5.8	Kết quả Benchmark Mô hình AI	140
5.8.1	Dữ liệu, quy ước gán nhãn và giao thức đánh giá	140
5.8.2	Quy trình lựa chọn cấu hình mô hình	142
5.8.3	Độ chính xác của mô hình	147
5.8.4	Kết quả đóng gói mô hình	149
5.8.5	Benchmark Tốc độ suy luận (Inference Speed / FPS)	150
5.9	Thảo luận kết quả	151
5.9.1	Điểm mạnh	151
5.9.2	Hạn chế	151
5.9.3	Mối đe dọa tới tính hợp lệ	153
5.10	Mức đáp ứng mục tiêu	153
5.11	Khuyến nghị trước nghiệm thu và trình diễn	154
5.12	Kết luận chương	154
6	Kết luận	156
6.1	Tổng kết	156
6.2	Kết quả đạt được	156
6.3	Hạn chế	157
6.4	Hướng phát triển	158
6.4.1	Hoàn thiện dữ liệu và phương pháp đánh giá	158
6.4.2	Nâng cao độ ổn định của theo dõi và phát hiện té ngã	158
6.4.3	Hoàn thiện khả năng di chuyển và an toàn vận hành	158
6.4.4	Mở rộng quản lý người dùng và khả năng duy trì liên lạc	159
6.4.5	Cải thiện trải nghiệm và tối ưu tài nguyên	159
6.5	Kết luận cuối cùng	159
	Tài liệu tham khảo	161
	Phụ lục	169
	Phụ lục A – Đặc tả use case	169

Phụ lục B – Cấu trúc dữ liệu và giao thức truyền thông	173
Phụ lục C – Danh mục module mã nguồn	174
Phụ lục D – Manifest artifact AI	176
Phụ lục E – Quy trình build và triển khai	177
Phụ lục F – Phiếu ghi kết quả kiểm thử	178
Phụ lục G – Checklist an toàn trình diễn	180

DANH SÁCH HÌNH VẼ

2.1	Minh họa hộp giới hạn của đối tượng trong ảnh.	31
2.2	Cấu trúc khái quát của một detector YOLO gồm backbone, neck và head.	36
2.3	Cấu trúc chi tiết của một tế bào LSTM (LSTM cell) với hệ thống các cổng kiểm soát thông tin.	41
2.4	Một số khung hình minh họa trong môi trường của dataset LE2I.	43
3.1	Kiến trúc tổng quan của hệ thống robot hỗ trợ chăm sóc người cao tuổi	66
3.2	Biểu đồ hoạt động của luồng giám sát và xử lý tình huống nghi té ngã	71
3.3	Biểu đồ tuần tự của quá trình xác nhận và hỗ trợ khi có tình huống nghi té ngã	73
3.4	Biểu đồ hoạt động của luồng hội thoại và nhắc nhở	74
3.5	Minh họa các đại lượng trong hàm liên kết và hàm lựa chọn người mục tiêu	80
3.6	Luồng lựa chọn và thực thi lệnh điều khiển chuyển động	83
3.7	Sơ đồ trạng thái của quá trình hậu kiểm và xác nhận té ngã	84
4.1	Kiến trúc hiện thực của hệ thống	93
4.2	Ảnh xạ từ module thiết kế đến các thành phần hiện thực	94
4.3	Giao diện khởi động và điều hướng của RobotApp	96
4.4	Lớp phủ theo dõi và duy trì mục tiêu ID0 trên RobotApp	109
4.5	Hiện thị kết quả suy luận LSTM trên lớp phủ theo dõi	111
4.6	Màn hình chatbot của RobotApp ở trạng thái chờ đánh thức	113
4.7	Chức năng gọi video trên RobotApp	115
4.8	Màn hình quản lý hồ sơ trên CaregiverApp	118
4.9	Màn hình bật, tắt bám theo tự động và phát hiện té ngã	119
4.10	Quản lý lịch nhắc nhở trên CaregiverApp	120
4.11	Ba nhóm nhật ký hoạt động trên CaregiverApp	121
4.12	Thông báo cuộc gọi đến từ robot khi ứng dụng chạy nền	122
4.13	Chức năng gọi video trên CaregiverApp	124
4.14	Cần điều khiển ảo dùng để điều khiển robot từ xa trong cuộc gọi	124
4.15	Ví dụ khung hình té ngã trong dữ liệu dùng để chuẩn bị mô hình	126

DANH SÁCH BẢNG

1.1	So sánh bằng chứng và giới hạn của các hướng phát hiện té ngã tiêu biểu	11
1.2	Một số nghiên cứu thị giác gần đây về phát hiện té ngã	14
1.3	So sánh các phương pháp theo dõi người liên quan đến đề tài	17
1.4	So sánh các phương án tạo phản hồi hội thoại	20
2.1	Các ý tưởng OpenBot được tham khảo trong đồ án	28
2.2	So sánh định tính các họ detector liên quan đến đồ án	34
2.3	Thứ tự keypoint người theo cấu hình COCO-Pose của YOLO26	39
2.4	So sánh các mức YOLO26-Pose ở chế độ end-to-end trên COCO	40
2.5	Các loại ICE candidate trong WebRTC	49
2.6	Các phương thức trao đổi dữ liệu cơ bản trong GATT	53
2.7	Các thành phần chính của phần cứng thân robot	55
3.1	Cụ thể hóa yêu cầu luận văn theo phạm vi đề tài	58
3.2	Tác nhân và các thành phần hỗ trợ của hệ thống	60
3.3	Danh sách yêu cầu chức năng	60
3.4	Yêu cầu phi chức năng	63
3.5	Phân chia trách nhiệm giữa các khu vực trong kiến trúc hệ thống	66
3.6	Vai trò của các khối trong kiến trúc tổng quan	67
3.7	Tóm tắt các use case chính	70
3.8	So sánh các lựa chọn công nghệ xuyên suốt hệ thống	75
3.9	Cấu trúc dữ liệu của Firebase Realtime Database	87
3.10	Các điểm chuẩn ánh xạ lệnh điều khiển	90
4.1	Các thành phần hiện thực của hệ thống	95
4.2	Hai mô hình YOLO26n-Pose được đóng gói trong RobotApp	98
4.3	Các hệ tọa độ trong quy trình xử lý YOLO26n-Pose của RobotApp	100
4.4	Các mô hình chính sau khi huấn luyện và chuyển đổi	128
5.1	Cấu hình phần mềm được đánh giá	131
5.2	Kết quả hiện thực theo thành phần	132
5.3	Artifact đóng gói hiện có	133
5.4	Đánh giá định tính các biến thể mô hình	134
5.5	Các trường hợp kiểm thử ánh xạ điều khiển	135
5.6	Các trường hợp kiểm thử hậu kiểm té ngã	135
5.7	Danh mục kiểm thử đầu cuối	137

5.8	So sánh kiến trúc mạng hồi quy, cửa sổ 15 khung hình (chỉ số lớp Fall ở mức cửa sổ)	143
5.9	Ảnh hưởng của độ dài cửa sổ trượt, mô hình LSTM32 (chỉ số lớp Fall ở mức cửa sổ)	143
5.10	So sánh cách biểu diễn đặc trưng đầu vào, LSTM32 và cửa sổ 15 khung hình (chỉ số lớp Fall ở mức cửa sổ)	144
5.11	Ảnh hưởng của khối lượng dữ liệu huấn luyện (chỉ số lớp Fall ở mức cửa sổ)	145
5.12	Ảnh hưởng của trọng số lớp dương trong hàm mất mát (chỉ số lớp Fall ở mức cửa sổ)	146
5.13	Ảnh hưởng của số vòng huấn luyện trên cấu hình đã chọn (F1 lớp Fall ở mức cửa sổ)	146
5.14	Độ chính xác của cấu hình đã chọn ở mức cửa sổ, chia tập theo video, trung bình ba lần chạy	148
5.15	Ảnh hưởng của ngưỡng hậu kiểm theo số cửa sổ dương liên tiếp	149
5.16	Kết quả đo trên tập TFLite được đóng gói	150
5.17	Thời gian xử lý trung bình của cấu hình FP16 thử nghiệm trên Android	151
5.18	Đối chiếu mục tiêu và kết quả	154
6.11	Thông số BLE triển khai	175
6.12	Module chính của RobotApp	175
6.13	Module chính của CaregiverApp	176
6.15	Mẫu tổng hợp benchmark AI	179
6.16	Mẫu tổng hợp phát hiện té ngã theo sự cố	179
6.17	Mẫu tổng hợp độ trễ và an toàn điều khiển	179

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Thuật ngữ tiếng Anh	Ý nghĩa tiếng Việt
ABI	Application Binary Interface	Giao diện nhị phân của ứng dụng
AI	Artificial Intelligence	Trí tuệ nhân tạo
AP	Average Precision	Độ chính xác trung bình
API	Application Programming Interface	Giao diện lập trình ứng dụng
APK	Android Package Kit	Gói cài đặt ứng dụng Android
ASR	Automatic Speech Recognition	Nhận dạng tiếng nói tự động
ATT	Attribute Protocol	Giao thức thuộc tính trong BLE
BLE	Bluetooth Low Energy	Bluetooth năng lượng thấp
BLDC	Brushless Direct Current	Động cơ điện một chiều không chổi than
CCCD	Client Characteristic Configuration Descriptor	Bộ mô tả cấu hình đặc tính phía máy khách
CNN	Convolutional Neural Network	Mạng nơ-ron tích chập
COCO	Common Objects in Context	Bộ dữ liệu các đối tượng phổ biến trong ngữ cảnh
CPU	Central Processing Unit	Bộ xử lý trung tâm
DTLS	Datagram Transport Layer Security	Bảo mật tầng truyền tải cho dữ liệu dạng datagram
ECG	Electrocardiogram	Điện tâm đồ
EMA	Exponential Moving Average	Trung bình trượt lũy thừa
FN	False Negative	Âm tính giả
FOC	Field-Oriented Control	Điều khiển định hướng từ thông
FP	False Positive	Dương tính giả
FP16	16-bit Floating Point	Số dấu phẩy động 16 bit
FPS	Frames Per Second	Số khung hình trên giây
GAP	Generic Access Profile	Hồ sơ truy cập tổng quát của BLE

Từ viết tắt	Thuật ngữ tiếng Anh	Ý nghĩa tiếng Việt
GATT	Generic Attribute Profile	Hồ sơ thuộc tính tổng quát của BLE
GFLOPs	Giga Floating-Point Operations	Số tỷ phép toán dấu phẩy động
GPU	Graphics Processing Unit	Bộ xử lý đồ họa
GRU	Gated Recurrent Unit	Đơn vị hồi quy có cổng
HSV	Hue, Saturation, Value	Không gian màu sắc độ, độ bão hòa và độ sáng
HTTPS	Hypertext Transfer Protocol Secure	Giao thức truyền siêu văn bản có bảo mật
ICE	Interactive Connectivity Establishment	Cơ chế thiết lập đường kết nối qua NAT
IMU	Inertial Measurement Unit	Khối đo lường quán tính
INT8	8-bit Integer	Số nguyên 8 bit
IoT	Internet of Things	Internet vạn vật
IoU	Intersection over Union	Tỷ lệ phần giao trên phần hợp
JSON	JavaScript Object Notation	Định dạng biểu diễn và trao đổi dữ liệu
LLM	Large Language Model	Mô hình ngôn ngữ lớn
LSTM	Long Short-Term Memory	Mạng bộ nhớ dài-ngắn hạn
mAP	mean Average Precision	Độ chính xác trung bình trên các lớp hoặc ngưỡng đánh giá
MCU	Microcontroller Unit	Bộ vi điều khiển
MFCC	Mel-Frequency Cepstral Coefficients	Các hệ số phổ cepstrum theo thang Mel
MQTT	MQ Telemetry Transport	Giao thức truyền thông nhẹ theo mô hình xuất bản–đăng ký
MTU	Maximum Transmission Unit	Kích thước đơn vị truyền tối đa
NAT	Network Address Translation	Chuyển đổi địa chỉ mạng
NMS	Non-Maximum Suppression	Ước chế không cực đại
NNAPI	Android Neural Networks API	Giao diện tăng tốc mạng nơ-ron trên Android
NPU	Neural Processing Unit	Bộ xử lý chuyên dụng cho mạng nơ-ron

Từ viết tắt	Thuật ngữ tiếng Anh	Ý nghĩa tiếng Việt
OKS	Object Keypoint Similarity	Độ tương đồng giữa các điểm khớp của đối tượng
ONNX	Open Neural Network Exchange	Định dạng trao đổi mô hình mạng nơ-ron mở
P2P	Peer-to-Peer	Kết nối ngang hàng
PID	Proportional–Integral–Derivative	Bộ điều khiển tỉ lệ–tích phân–vi phân
PR	Precision–Recall	Độ chính xác–độ thu hồi
PWM	Pulse-Width Modulation	Điều chế độ rộng xung
RAM	Random Access Memory	Bộ nhớ truy cập ngẫu nhiên
R-CNN	Region-based Convolutional Neural Network	Mạng nơ-ron tích chập dựa trên vùng
Re-ID	Person Re-identification	Tái định danh người
REST	Representational State Transfer	Kiểu kiến trúc truyền thông dựa trên tài nguyên
RGB	Red, Green, Blue	Hệ màu đỏ, xanh lá và xanh dương
RNN	Recurrent Neural Network	Mạng nơ-ron hồi quy
RTDB	Realtime Database	Cơ sở dữ liệu thời gian thực Firebase
RTP	Real-time Transport Protocol	Giao thức vận chuyển dữ liệu thời gian thực
RTSP	Real-Time Streaming Protocol	Giao thức điều khiển luồng thời gian thực
SCTP	Stream Control Transmission Protocol	Giao thức truyền điều khiển luồng
SDK	Software Development Kit	Bộ công cụ phát triển phần mềm
SDP	Session Description Protocol	Giao thức mô tả phiên truyền thông
SMP	Security Manager Protocol	Giao thức quản lý bảo mật của BLE
SORT	Simple Online and Realtime Tracking	Thuật toán theo dõi trực tuyến thời gian thực
SRTP	Secure Real-time Transport Protocol	Giao thức vận chuyển thời gian thực có bảo mật
SSD	Single Shot MultiBox Detector	Mô hình phát hiện đối tượng một giai đoạn
SSE	Server-Sent Events	Cơ chế máy chủ truyền sự kiện một chiều đến máy khách

Từ viết tắt	Thuật ngữ tiếng Anh	Ý nghĩa tiếng Việt
STT	Speech-to-Text	Chuyển giọng nói thành văn bản
STUN	Session Traversal Utilities for NAT	Giao thức hỗ trợ xác định địa chỉ công khai qua NAT
SVM	Support Vector Machine	Máy véc-tơ hỗ trợ
TFLite	TensorFlow Lite	Nền tảng suy luận TensorFlow trên thiết bị biên
TP	True Positive	Dương tính đúng
TTS	Text-to-Speech	Chuyển văn bản thành giọng nói
TURN	Traversal Using Relays around NAT	Giao thức chuyển tiếp dữ liệu qua máy chủ trung gian
UART	Universal Asynchronous Receiver–Transmitter	Bộ thu phát nối tiếp không đồng bộ
UDP	User Datagram Protocol	Giao thức dữ liệu người dùng
UI	User Interface	Giao diện người dùng
UID	User Identifier	Định danh người dùng
USB	Universal Serial Bus	Chuẩn kết nối nối tiếp đa dụng
UUID	Universally Unique Identifier	Định danh duy nhất toàn cục
WebRTC	Web Real-Time Communication	Giao tiếp thời gian thực trên nền web
WHO	World Health Organization	Tổ chức Y tế Thế giới
XML	Extensible Markup Language	Ngôn ngữ đánh dấu mở rộng
YOLO	You Only Look Once	Họ mô hình thị giác máy tính thời gian thực

DANH MỤC THUẬT NGỮ

Thuật ngữ	Giải thích trong phạm vi báo cáo
Activity	Thành phần Android đại diện cho một màn hình hoặc điểm tương tác chính với người dùng và có vòng đời do hệ điều hành quản lý.
Actor	Tác nhân tương tác với hệ thống trong mô hình use case, có thể là người sử dụng hoặc một hệ thống bên ngoài.
Annotation	Thông tin chú thích gắn với dữ liệu, chẳng hạn nhãn lớp, bounding box, keypoint hoặc thời điểm xảy ra hành động trong video.
Artifact	Sản phẩm có thể lưu trữ và kiểm tra được tạo ra trong quá trình phát triển, chẳng hạn tệp mô hình, ứng dụng cài đặt, nhật ký hoặc kết quả kiểm thử.
Augmentation	Kỹ thuật tạo thêm biến thể từ dữ liệu huấn luyện bằng các phép biến đổi như lật ảnh, thay đổi độ sáng, co giãn hoặc xoay ảnh.
Backbone	Phần mạng nơ-ron đảm nhiệm trích xuất đặc trưng nền tảng từ dữ liệu đầu vào trước khi các đầu dự đoán thực hiện nhiệm vụ cụ thể.
Batch	Nhóm mẫu dữ liệu được đưa qua mô hình trong cùng một bước huấn luyện hoặc suy luận.
Bonding	Cơ chế lưu thông tin khóa sau khi hai thiết bị BLE ghép đôi để có thể tái lập liên kết bảo mật ở những lần kết nối sau.
Bounding box	Hình chữ nhật bao quanh đối tượng trong ảnh, dùng để biểu diễn vị trí và kích thước của đối tượng được phát hiện.
Callback	Hàm được đăng ký để hệ thống gọi lại khi một tác vụ hoàn tất hoặc khi một sự kiện tương ứng xảy ra.
Calibration	Quá trình hiệu chỉnh tham số, ngưỡng hoặc kết quả đo để hệ thống phù hợp hơn với thiết bị và điều kiện vận hành cụ thể.
Candidate	Địa chỉ IP và cổng có khả năng tạo đường truyền giữa hai đầu cuối trong quá trình thiết lập kết nối WebRTC bằng ICE.
Characteristic	Thành phần dữ liệu nằm trong một GATT service, có giá trị, định danh và các thuộc tính quy định thao tác đọc, ghi hoặc thông báo.
Closed-loop control	Điều khiển vòng kín; đầu ra thực tế được đo và phản hồi liên tục để bộ điều khiển điều chỉnh tín hiệu tác động theo giá trị đặt.
Codec	Phương thức mã hóa và giải mã âm thanh hoặc hình ảnh để phục vụ lưu trữ và truyền dữ liệu media.
Confidence score	Giá trị thể hiện mức độ tin cậy của mô hình đối với một kết quả dự đoán, thường được dùng để lọc các phát hiện không chắc chắn.
Data channel	Kênh truyền dữ liệu tùy ý giữa hai đầu cuối WebRTC, hoạt động bên cạnh các luồng âm thanh và hình ảnh.

Thuật ngữ	Giải thích trong phạm vi báo cáo
Data split	Cách phân chia dữ liệu thành các tập huấn luyện, xác thực và kiểm tra nhằm huấn luyện mô hình và đánh giá khả năng khái quát.
Dataset	Tập dữ liệu được tổ chức để huấn luyện, xác thực hoặc đánh giá thuật toán và mô hình học máy.
Delegate	Thành phần chuyển một phần hoặc toàn bộ đồ thị suy luận sang bộ tăng tốc như GPU hoặc NPU thay vì chỉ thực thi trên CPU.
Descriptor	Thành phần mô tả bổ sung cho characteristic trong GATT, cung cấp thông tin cấu hình hoặc cách diễn giải giá trị.
Differential drive	Cơ cấu dẫn động vi sai sử dụng hai bánh chủ động độc lập; hướng và vận tốc robot được xác định từ chênh lệch vận tốc giữa hai bánh.
Drop frame	Việc chủ động bỏ qua một khung hình khi hệ thống chưa xử lý xong dữ liệu trước đó, nhằm tránh hàng đợi và độ trễ tiếp tục tăng.
Edge AI	Cách triển khai trong đó mô hình AI được thực thi gần nguồn tạo dữ liệu, chẳng hạn trực tiếp trên điện thoại gắn trên robot.
Embedding	Vector đặc trưng số biểu diễn nội dung hoặc đặc điểm của dữ liệu, có thể được dùng để so sánh mức độ tương đồng giữa các đối tượng.
End-to-end	Cách tổ chức mô hình hoặc quy trình xử lý trực tiếp từ đầu vào đến kết quả cuối với ít khâu trung gian được xây dựng thủ công.
Epoch	Một lượt mô hình sử dụng toàn bộ tập dữ liệu huấn luyện để cập nhật các tham số.
Feature extraction	Quá trình trích xuất các đặc trưng có ý nghĩa từ dữ liệu đầu vào để phục vụ nhận dạng, phân loại hoặc theo dõi.
Fine-tuning	Quá trình tiếp tục huấn luyện một mô hình đã được huấn luyện trước trên dữ liệu của bài toán cụ thể.
Firmware	Chương trình chạy trên vi điều khiển, đảm nhiệm giao tiếp với phần cứng và thực hiện các chức năng điều khiển ở mức thiết bị.
Foreground service	Dịch vụ Android thực hiện tác vụ người dùng có thể nhận biết trong thời gian dài và phải duy trì thông báo thường trực.
Frame	Một khung hình đơn trong chuỗi video; nhiều frame liên tiếp tạo thành dữ liệu chuyển động theo thời gian.
Fragment	Thành phần giao diện Android có vòng đời riêng và thường được đặt bên trong Activity để tổ chức từng khu vực chức năng của màn hình.
GATT service	Nhóm các characteristic phục vụ cùng một chức năng trong mô hình dữ liệu GATT và được nhận diện bằng UUID.
Ground truth	Nhãn tham chiếu được xem là đúng, dùng làm cơ sở huấn luyện hoặc đối chiếu khi đánh giá kết quả dự đoán.

Thuật ngữ	Giải thích trong phạm vi báo cáo
Heuristic	Quy tắc kinh nghiệm giúp đưa ra quyết định nhanh dựa trên đặc điểm của bài toán nhưng không bảo đảm tối ưu trong mọi trường hợp.
Hungarian algorithm	Thuật toán tìm phương án gán có tổng chi phí nhỏ nhất, được dùng để liên kết kết quả phát hiện với các đối tượng đang được theo dõi.
Hysteresis	Cơ chế sử dụng ngưỡng chuyển vào và ngưỡng chuyển ra khác nhau để hạn chế trạng thái thay đổi liên tục khi giá trị dao động gần biên.
Inference	Quá trình sử dụng mô hình đã huấn luyện để tạo kết quả dự đoán từ dữ liệu đầu vào mới.
Interpreter	Thành phần nạp mô hình, quản lý tensor và thực thi các toán tử của mô hình trong môi trường suy luận.
Kalman filter	Bộ lọc ước lượng trạng thái bằng cách kết hợp dự báo từ mô hình chuyển động với thông tin quan sát mới.
Keypoint	Điểm mốc biểu diễn vị trí một bộ phận trên cơ thể người, chẳng hạn vai, hông, gối hoặc mắt cá chân.
Letterbox	Phép đổi kích thước ảnh nhưng vẫn giữ tỷ lệ khung hình bằng cách bổ sung vùng đệm vào phần còn thiếu.
Listener	Thành phần đăng ký để nhận thông báo khi dữ liệu hoặc trạng thái được theo dõi có thay đổi.
Media stream	Luồng dữ liệu âm thanh hoặc hình ảnh được thu nhận và truyền liên tục trong một phiên liên lạc thời gian thực.
Metadata	Dữ liệu mô tả bổ sung về cấu trúc, định dạng hoặc ý nghĩa của một tệp, mô hình hay tập dữ liệu.
Model quantization	Kỹ thuật giảm độ chính xác biểu diễn của trọng số hoặc tensor nhằm giảm dung lượng mô hình và chi phí tính toán.
Object detection	Bài toán xác định lớp và vị trí của một hoặc nhiều đối tượng trong ảnh hoặc video.
Object tracking	Bài toán duy trì định danh và ước lượng vị trí của đối tượng qua nhiều khung hình liên tiếp.
Offer–Answer	Cơ chế thương lượng phiên trong WebRTC, trong đó một đầu cuối gửi đề nghị cấu hình và đầu cuối còn lại phản hồi bằng cấu hình tương thích.
On-device AI	Hình thức thực thi mô hình AI trực tiếp trên thiết bị đầu cuối thay vì gửi dữ liệu đến máy chủ để suy luận.
Payload	Phần dữ liệu nội dung được mang trong một bản tin truyền thông, không bao gồm các thông tin điều khiển của giao thức.
Pipeline	Chuỗi các bước xử lý có thứ tự, trong đó đầu ra của một bước trở thành đầu vào cho bước tiếp theo.
Pose estimation	Bài toán ước lượng tư thế cơ thể thông qua vị trí và độ tin cậy của các keypoint trên ảnh.

Thuật ngữ	Giải thích trong phạm vi báo cáo
Prompt	Nội dung hướng dẫn hoặc ngữ cảnh được gửi đến mô hình ngôn ngữ để định hướng cách tạo phản hồi.
Preprocessing/Post-processing	Tiền xử lý chuẩn bị dữ liệu trước khi đưa vào mô hình; hậu xử lý diễn giải và lọc kết quả sau suy luận.
Re-identification	Quá trình nhận dạng lại một đối tượng đã xuất hiện trước đó sau khi bị che khuất, mất dấu hoặc rời khỏi vùng quan sát.
Setpoint	Giá trị đặt mà hệ thống điều khiển hướng tới, chẳng hạn vận tốc mong muốn của từng bánh xe.
Signaling	Quá trình trao đổi thông tin điều khiển để thiết lập và duy trì phiên WebRTC; dữ liệu media không truyền qua kênh này sau khi phiên được hình thành.
Softmax	Hàm chuyển các giá trị đầu ra của mô hình thành phân bố xác suất trên các lớp dự đoán.
Streaming	Phương thức truyền và xử lý dữ liệu liên tục theo từng phần thay vì chờ toàn bộ nội dung hoàn tất.
Tensor	Cấu trúc dữ liệu nhiều chiều dùng để biểu diễn đầu vào, đầu ra và các giá trị trung gian của mô hình học sâu.
Threshold	Giá trị ngưỡng dùng để quyết định giữ, loại hoặc chuyển trạng thái dựa trên một đại lượng đo hay kết quả dự đoán.
Timestamp	Giá trị biểu diễn thời điểm một dữ liệu hoặc sự kiện được tạo ra, dùng để sắp xếp và kiểm tra tính còn hiệu lực.
Track/tracklet	Chuỗi các quan sát được liên kết và gán cùng một định danh; tracklet thường chỉ một đoạn theo dõi ngắn hoặc chưa hoàn chỉnh.
Tracking-by-detection	Hướng theo dõi sử dụng kết quả phát hiện ở từng khung hình, sau đó dự báo và liên kết các kết quả để duy trì định danh theo thời gian.
Transfer learning	Kỹ thuật tận dụng tri thức từ mô hình đã huấn luyện trên một bài toán hoặc tập dữ liệu khác để huấn luyện bài toán mới.
Use case	Mô tả một mục tiêu tương tác giữa tác nhân và hệ thống, bao gồm các điều kiện và luồng hành vi cần thiết để đạt mục tiêu đó.
Watchdog	Cơ chế giám sát thời gian cập nhật; khi lệnh hoặc tín hiệu không được làm mới trong khoảng quy định, hệ thống chuyển về trạng thái an toàn.
Write without response	Phương thức ghi GATT không yêu cầu phản hồi xác nhận ở tầng ATT, phù hợp với dữ liệu được cập nhật thường xuyên và cần độ trễ thấp.

Chương 1 – Giới thiệu

1.1 Bối cảnh và tính cấp thiết của đề tài

Già hóa dân số đã trở thành bài toán hiện hữu đối với hệ thống y tế, an sinh xã hội và từng gia đình. Tổ chức Y tế Thế giới (WHO) dự báo số người từ 60 tuổi trở lên trên toàn cầu tăng từ khoảng 1 tỷ người năm 2020 lên 1,4 tỷ người vào năm 2030 và 2,1 tỷ người vào năm 2050; khoảng hai phần ba số người cao tuổi sẽ sống tại các quốc gia thu nhập thấp và trung bình [1]. Quá trình chuyển đổi này diễn ra nhanh hơn trước, trong khi nguồn nhân lực chăm sóc dài hạn, cơ sở hạ tầng và khả năng chi trả không tăng với cùng tốc độ.

Tại Việt Nam, áp lực còn rõ rệt hơn vì thời gian chuyển từ cơ cấu dân số “già hóa” sang “dân số già” tương đối ngắn. Dự báo dựa trên Tổng điều tra dân số và nhà ở năm 2019 cho thấy Việt Nam có thể trở thành quốc gia có dân số già vào khoảng năm 2036, khi nhóm từ 65 tuổi trở lên đạt xấp xỉ 14% dân số; đến năm 2069, người cao tuổi có thể chiếm 27,1% dân số [2]. Nghiên cứu về kinh tế chăm sóc người cao tuổi tại Việt Nam còn dự báo số người cần hỗ trợ thực hiện các hoạt động sống hằng ngày tăng từ khoảng 4,7 triệu năm 2025 lên 6,5 triệu năm 2035 [3]. Mô hình chỉ dựa vào chăm sóc trực tiếp bởi thành viên gia đình vì vậy sẽ ngày càng khó duy trì, đặc biệt tại đô thị khi người thân thường làm việc xa nhà.

Té ngã là một rủi ro tiêu biểu cần được phát hiện sớm. WHO định nghĩa té ngã là sự kiện khiến một người ngoài ý muốn nằm lại trên mặt đất, sàn hoặc một mức thấp hơn. Mỗi năm thế giới ghi nhận khoảng 684.000 ca tử vong do té ngã và 37,3 triệu trường hợp đủ nghiêm trọng để cần chăm sóc y tế; nhóm trên 60 tuổi chịu số ca tử vong cao nhất [4]. Trong nhà, sự cố có thể xảy ra tại thời điểm không có người chăm sóc. Khoảng thời gian từ khi xảy ra sự cố đến khi người thân biết và can thiệp vì vậy có ý nghĩa trực tiếp đối với mức độ hậu quả.

Bên cạnh rủi ro té ngã, người cao tuổi còn phải ghi nhớ và sắp xếp nhiều công việc trong sinh hoạt hằng ngày, đặc biệt là sử dụng nhiều loại thuốc đúng thời điểm. Khi trí nhớ và khả năng nhận thức suy giảm, họ có thể bỏ quên lịch uống thuốc hoặc những công việc đã dự định. Một nghiên cứu tổng quan có hệ thống cho thấy suy giảm nhận thức, mắc đồng thời nhiều bệnh và phác đồ điều trị phức tạp có liên quan đến mức độ tuân thủ điều trị bằng thuốc thấp hơn ở nhóm tuổi này [5]. Cô đơn và cô lập xã hội cũng là các yếu tố nguy cơ quan trọng đối với sức khỏe tinh thần ở tuổi già [6]. Do đó, một

giải pháp hỗ trợ tại nhà không nên chỉ là camera giám sát thụ động; hệ thống cần kết hợp hỗ trợ nhắc nhở, tương tác tự nhiên và liên lạc.

Sự phổ biến của điện thoại thông minh tạo điều kiện xây dựng robot chi phí thấp. Điện thoại cung cấp camera, microphone, loa, kết nối mạng và năng lực tính toán đủ cho nhiều tác vụ thị giác máy tính. Công trình OpenBot chứng minh một điện thoại Android có thể đóng vai trò bộ não và hệ cảm biến cho thân xe robot giá thấp, đồng thời hỗ trợ các tác vụ như bám theo người và điều hướng [7]. Việc tận dụng những thiết bị sẵn có giúp giảm chi phí ban đầu và rào cản kỹ thuật, qua đó tạo điều kiện thuận lợi hơn cho sinh viên và người nghiên cứu tiếp cận, thử nghiệm cũng như phát triển các ứng dụng robot kết hợp trí tuệ nhân tạo.

Từ các nhu cầu trên, đề tài **“Nghiên cứu và phát triển robot trợ lý ảo ứng dụng trí tuệ nhân tạo hỗ trợ chăm sóc và giám sát người cao tuổi tại nhà”** xây dựng một nguyên mẫu tích hợp thay vì các chức năng rời rạc. Hệ thống gồm ứng dụng RobotApp trên thiết bị Android gắn với phần thân robot hỗ trợ di chuyển, ứng dụng CaregiverApp cho người chăm sóc, firmware ESP32 điều khiển hai động cơ BLDC ở thân robot và chuỗi mô hình AI được chuyển đổi để suy luận trên thiết bị. Giá trị cốt lõi nằm ở việc kết nối các giải pháp công nghệ thành một hệ thống tương đối hoàn chỉnh hỗ trợ chăm sóc người cao tuổi tại nhà.

1.2 Mục tiêu đề tài

1.2.1 Mục tiêu tổng quát

Nghiên cứu, thiết kế và xây dựng nguyên mẫu robot trợ lý ảo hỗ trợ chăm sóc người cao tuổi tại nhà. Hệ thống hướng đến khả năng tương tác bằng giọng nói, nhắc nhở theo lịch, giám sát an toàn, phát hiện tình huống nghi té ngã, liên lạc trực tuyến và hỗ trợ người chăm sóc can thiệp từ xa thông qua một nền tảng robot di động sử dụng điện thoại thông minh làm bộ xử lý chính.

1.2.2 Mục tiêu cụ thể

1. **Nhận biết và giám sát an toàn:** xây dựng khả năng thu nhận hình ảnh từ môi trường trong nhà, phát hiện người và duy trì đúng đối tượng cần theo dõi khi người hoặc robot di chuyển. Trên cơ sở thông tin tư thế theo thời gian, hệ thống cần nhận biết tình huống nghi té ngã, phân biệt với một số hoạt động thông thường và thực hiện bước xác nhận trước khi phát cảnh báo. Mô hình AI phải được chuẩn bị, chuyển đổi

và đánh giá để có thể suy luận trực tiếp trên điện thoại gắn với robot.

2. **Tương tác và hỗ trợ chăm sóc:** xây dựng khả năng trò chuyện bằng giọng nói tiếng Việt theo hướng gần gũi, dễ sử dụng đối với người cao tuổi. Hệ thống cần hỗ trợ quản lý và thực hiện lịch nhắc nhở, ghi nhận phản hồi của người dùng, đồng thời kết hợp hội thoại với quy trình xác nhận khi phát hiện dấu hiệu bất thường. Nội dung nhắc nhở và thông tin cá nhân phải được quản lý rõ ràng để người chăm sóc có thể cập nhật khi cần.
3. **Liên lạc và điều khiển từ xa:** xây dựng hai ứng dụng dành riêng cho robot và người chăm sóc, cho phép đồng bộ hồ sơ, lịch nhắc nhở, trạng thái giám sát, cảnh báo và lịch sử hoạt động. Khi cần can thiệp, hệ thống phải hỗ trợ cuộc gọi âm thanh, hình ảnh hai chiều và cho phép người chăm sóc điều khiển chuyển động của robot từ xa. Phần thân robot cần tiếp nhận lệnh từ điện thoại và điều khiển hai bánh xe theo yêu cầu di chuyển.
4. **An toàn và đánh giá hệ thống:** thiết kế cơ chế dừng robot khi lệnh không hợp lệ, quá thời gian cho phép hoặc đường truyền bị gián đoạn; đồng thời tránh xung đột giữa chế độ tự bám theo và điều khiển thủ công. Dữ liệu cá nhân, lịch sử chăm sóc và thông tin liên lạc cần được giới hạn theo đúng người sử dụng. Cuối cùng, các chức năng chính, tốc độ xử lý, độ ổn định của kết nối, chất lượng phát hiện té ngã và khả năng dừng an toàn phải được kiểm thử trên nguyên mẫu để xác định mức độ đáp ứng và những giới hạn còn tồn tại.

1.3 Đối tượng và phạm vi nghiên cứu

1.3.1 Đối tượng nghiên cứu

Đối tượng kỹ thuật của đề tài gồm nền tảng robot sử dụng điện thoại thông minh làm bộ xử lý chính; hai ứng dụng Android dành cho robot và người chăm sóc; thị giác máy tính và học sâu trên thiết bị; bài toán theo dõi đối tượng và phân tích chuỗi tư thế; công nghệ chuyển văn bản thành giọng nói (Text-to-Speech – TTS) và chuyển giọng nói thành văn bản (Speech-to-Text – STT); mô hình ngôn ngữ lớn; các phương thức đồng bộ dữ liệu và truyền thông thời gian thực; kết nối giữa điện thoại với robot; thiết kế firmware cho hệ thống điều khiển phần thân robot; cùng quy trình chuyển đổi mô hình AI để triển khai trên thiết bị di động.

Đối tượng sử dụng gồm người cao tuổi sinh hoạt tại nhà và người thân/người chăm sóc từ xa. Trong nguyên mẫu, RobotApp và CaregiverApp của một cặp thiết bị đăng nhập cùng tài khoản Firebase; toàn bộ kênh điều khiển, trạng thái AI, signaling và nhật ký thời gian thực được cách ly dưới `pair_sessions/{uid}`. Cách tổ chức này tách dữ liệu giữa các tài khoản, nhưng mỗi tài khoản hiện mới đại diện cho một cặp robot–caregiver và chưa mô hình hóa nhiều robot hay nhiều vai trò trong cùng gia đình.

1.3.2 Phạm vi chức năng

- **Hỗ trợ tương tác với người cao tuổi:** robot tiếp nhận lời nói, tạo phản hồi và phát lại bằng giọng nói để phục vụ các cuộc trò chuyện hằng ngày. Nội dung tương tác có thể được cá nhân hóa ở mức cơ bản dựa trên hồ sơ và cách xưng hô do người chăm sóc thiết lập.
- **Hỗ trợ nhắc nhở:** người chăm sóc có thể tạo, chỉnh sửa, bật hoặc tắt các lịch nhắc nhở cho những công việc cần thực hiện. Đến thời điểm đã thiết lập, robot phát lời nhắc, tiếp nhận phản hồi của người cao tuổi và lưu lại kết quả để người chăm sóc theo dõi.
- **Giám sát và bám theo người:** robot sử dụng hình ảnh từ camera để phát hiện, lựa chọn và duy trì một người làm đối tượng theo dõi chính. Từ vị trí của người trong khung hình, robot thực hiện các chuyển động tiến, lùi, quay hoặc dừng nhằm giữ khoảng cách quan sát phù hợp trong môi trường trong nhà.
- **Phát hiện và xử lý tình huống nghi té ngã:** hệ thống phân tích sự thay đổi tư thế của người được theo dõi theo thời gian để nhận biết dấu hiệu té ngã. Khi mức nghi ngờ đủ lớn, robot dừng chuyển động và hỏi xác nhận bằng giọng nói; trường hợp người dùng cho biết đang gặp nguy hiểm, trả lời không rõ ràng hoặc không phản hồi sẽ được ghi nhận và thông báo cho người chăm sóc.
- **Liên lạc và can thiệp từ xa:** người chăm sóc có thể nhận thông báo, thực hiện cuộc gọi âm thanh và hình ảnh với người cao tuổi, đồng thời điều khiển chuyển động cơ bản của robot khi cần quan sát hoặc hỗ trợ từ xa. Hệ thống bảo đảm chế độ điều khiển thủ công không hoạt động đồng thời với chế độ tự động bám theo.
- **Quản lý thông tin chăm sóc:** ứng dụng của người chăm sóc hỗ trợ quản lý hồ sơ, lịch nhắc nhở và trạng thái của các chức năng giám sát; đồng thời cung cấp lịch sử thực hiện lời nhắc, cảnh báo té ngã và cuộc gọi để phục vụ việc theo dõi sau sự kiện.

1.3.3 Giới hạn nghiên cứu

Đề tài được thực hiện ở mức nguyên mẫu kỹ thuật nhằm kiểm chứng sự phối hợp giữa các chức năng hỗ trợ chăm sóc, không hướng đến chẩn đoán bệnh hoặc thay thế quyết định của nhân viên y tế và người chăm sóc. Phạm vi nghiên cứu được giới hạn như sau:

- **Đối tượng và dữ liệu đánh giá:** các tình huống té ngã được sử dụng trong quá trình phát triển và kiểm thử chủ yếu là dữ liệu công khai hoặc kịch bản dựng lại có kiểm soát. Đề tài chưa tiến hành thử nghiệm dài hạn trên người cao tuổi trong điều kiện sinh hoạt thực tế và chưa thực hiện đánh giá lâm sàng.
- **Không gian hoạt động:** robot được định hướng sử dụng trong nhà, trên bề mặt tương đối bằng phẳng và trong điều kiện camera có thể quan sát người dùng. Các môi trường ngoài trời, cầu thang, ánh sáng quá yếu, vật cản phức tạp hoặc rung động mạnh của camera chưa được khảo sát đầy đủ.
- **Khả năng giám sát:** hệ thống ưu tiên theo dõi một người mục tiêu tại một thời điểm và phát hiện tình huống nghi té ngã từ hình ảnh. Kết quả có thể bị ảnh hưởng khi cơ thể bị che khuất, người đi ra ngoài vùng quan sát hoặc xuất hiện nhiều người có đặc điểm gần giống nhau. Hệ thống không đo dấu hiệu sinh tồn, không dự báo nguy cơ té ngã và không bảo đảm phát hiện được mọi dạng sự cố.
- **Khả năng di chuyển:** robot chỉ thực hiện tự bám theo và các lệnh di chuyển cơ bản. Các bài toán lập bản đồ, định vị trong toàn bộ ngôi nhà, lập kế hoạch đường đi và tránh vật cản chủ động chưa thuộc phạm vi nghiên cứu.
- **Kết nối và quy mô sử dụng:** các chức năng đồng bộ, hội thoại trực tuyến, thông báo và gọi từ xa cần có kết nối mạng. Nguyên mẫu được xây dựng cho một robot và một tài khoản chăm sóc; việc quản lý đồng thời nhiều robot, nhiều người cao tuổi hoặc nhiều thành viên chăm sóc chưa được triển khai.
- **Mức độ hoàn thiện sản phẩm:** đề tài chưa bao gồm đánh giá độ bền trong thời gian dài, khả năng chống nước, an toàn điện và cơ khí theo tiêu chuẩn sản phẩm, bảo trì từ xa hoặc chứng nhận dành cho thiết bị y tế. Những nội dung này cần được tiếp tục nghiên cứu trước khi hệ thống có thể được sử dụng như một sản phẩm thương mại.

1.4 Phương pháp nghiên cứu

Đề tài được thực hiện qua bốn giai đoạn có quan hệ kế tiếp và bổ sung lẫn nhau: khảo cứu tài liệu, phân tích và thiết kế hệ thống, xây dựng nguyên mẫu, sau đó tiến hành thực nghiệm và đánh giá. Kết quả của mỗi giai đoạn được sử dụng để điều chỉnh nội dung ở giai đoạn tiếp theo, nhờ đó hệ thống được hoàn thiện dần theo các mục tiêu đã đặt ra.

1.4.1 Khảo cứu và phân tích tài liệu

Phương pháp khảo cứu tài liệu được sử dụng để xác định bối cảnh của bài toán chăm sóc người cao tuổi, các yêu cầu đối với robot trợ lý và những hướng tiếp cận đã được công bố. Nguồn tham khảo gồm báo cáo của các tổ chức có uy tín, bài báo khoa học, tiêu chuẩn kỹ thuật và tài liệu chuyên ngành liên quan đến robot hỗ trợ, thị giác máy tính, theo dõi người, phát hiện té ngã, tương tác bằng giọng nói và hệ thống nhúng. Kết quả khảo cứu là cơ sở để hình thành yêu cầu, lựa chọn hướng giải quyết và xác định phạm vi phù hợp với đề án.

Đối với bài toán phát hiện té ngã, các nghiên cứu được xem xét theo loại dữ liệu đầu vào và phương pháp nhận biết sự kiện, từ thiết bị đeo, cảm biến môi trường đến phương pháp dựa trên thị giác [8]. Các phương án được phân tích về khả năng ứng dụng trong nhà, mức độ thuận tiện đối với người cao tuổi, yêu cầu phần cứng và khả năng tích hợp với robot di động. Trên cơ sở đó, đề tài tập trung vào hướng phân tích tư thế và chuyển động từ camera gắn trên robot.

1.4.2 Phân tích yêu cầu và thiết kế hệ thống

Từ các tình huống chăm sóc tại nhà, nhóm xác định những người tham gia, nhu cầu sử dụng, chức năng cần có, điều kiện vận hành và các trường hợp bất thường có thể xảy ra. Các yêu cầu sau đó được phân nhóm và chuyển thành kiến trúc tổng quan, các khối chức năng cùng những luồng xử lý chính của hệ thống. Quá trình thiết kế chú trọng phân chia rõ trách nhiệm giữa ứng dụng trên robot, ứng dụng của người chăm sóc, khối xử lý AI và phần thân robot; đồng thời xem xét khả năng phối hợp, sử dụng tài nguyên, bảo vệ dữ liệu và dừng an toàn ngay từ đầu.

1.4.3 Xây dựng và tích hợp nguyên mẫu

Trên cơ sở thiết kế đã xây dựng, nguyên mẫu được phát triển theo từng thành phần gồm ứng dụng trên robot, ứng dụng của người chăm sóc, khối xử lý AI và chương trình điều khiển phần thân robot. Mỗi thành phần được hoàn thiện và kiểm tra riêng trước khi tích hợp vào hệ thống chung. Quá trình tích hợp được thực hiện lần lượt theo các nhóm chức năng: quản lý hồ sơ và lịch nhắc nhở, hội thoại bằng giọng nói, giám sát và phát hiện té ngã, liên lạc từ xa, điều khiển chuyển động và xử lý các tình huống mất kết nối. Những vấn đề phát sinh trong quá trình chạy thử được ghi nhận để điều chỉnh thiết kế và hoàn thiện nguyên mẫu.

1.4.4 Thực nghiệm và đánh giá

Nguyên mẫu được kiểm tra từ từng chức năng riêng lẻ đến các luồng hoạt động hoàn chỉnh trong môi trường trong nhà. Phần xử lý AI được đánh giá về chất lượng nhận biết, khả năng duy trì người mục tiêu và tốc độ xử lý trên điện thoại. Các chức năng tương tác, nhắc nhở, cảnh báo, cuộc gọi và điều khiển từ xa được kiểm tra theo những tình huống sử dụng dự kiến. Phần thân robot được thử nghiệm về hướng chuyển động, khả năng đáp ứng lệnh và trạng thái dừng khi xảy ra bất thường. Ngoài điều kiện vận hành bình thường, các tình huống như mất dấu người, người dùng không phản hồi, kết nối bị gián đoạn hoặc một thành phần ngừng hoạt động cũng được xem xét. Kết quả thực nghiệm được dùng để đối chiếu với mục tiêu đề tài, phân tích hạn chế và đề xuất hướng phát triển tiếp theo.

1.5 Các công trình và giải pháp liên quan

Robot hỗ trợ người cao tuổi là một bài toán liên ngành, trong đó khả năng di chuyển, quan sát, tương tác và kết nối với người chăm sóc phải được phối hợp trong cùng một hệ thống. Vì vậy, phần tổng quan tập trung vào bốn nhóm công trình có liên hệ trực tiếp với đề tài: phát hiện té ngã, theo dõi và bám người mục tiêu, hội thoại bằng giọng nói và các nền tảng robot hỗ trợ tại nhà. Mỗi nhóm được xem xét theo ý tưởng chính, cách tổ chức luồng xử lý, ưu điểm, hạn chế và khả năng kế thừa vào nguyên mẫu.

Đối với phát hiện té ngã, các bài tổng quan cung cấp cái nhìn hệ thống về cảm biến và phương pháp đã được sử dụng [9][8], trong khi những nghiên cứu gần đây cho thấy xu hướng chuyển sang mô hình thị giác nhẹ, khai thác tư thế theo thời gian và xử lý trên thiết bị biên [10][11][12][13]. Việc kết hợp hai nhóm tài liệu này giúp tránh đánh

giá một giải pháp chỉ qua độ chính xác được báo cáo, đồng thời làm rõ hơn các yêu cầu về dữ liệu, tốc độ, che khuất và khả năng vận hành trong điều kiện thực tế.

1.5.1 Các hướng tiếp cận cho bài toán phát hiện té ngã

Bức tranh chung về các phương pháp phát hiện té ngã. Các hệ thống phát hiện té ngã được phát triển từ nhiều nguồn dữ liệu khác nhau, bao gồm gia tốc và vận tốc góc, tín hiệu sinh học như điện tâm đồ (ECG) thu từ thiết bị đeo, âm thanh, ảnh chiều sâu, radar và video RGB. Hai tổng quan gần đây có cách phân nhóm khác nhau nhưng cùng cho thấy hiệu quả của hệ thống phụ thuộc đồng thời vào cảm biến, thuật toán, bộ dữ liệu và phương pháp đánh giá [9][8]. Vì vậy, tỷ lệ xuất hiện của một hướng nghiên cứu không đồng nghĩa với ưu thế tuyệt đối về độ chính xác hoặc khả năng triển khai.

Xét theo cách thu nhận bằng chứng, các phương pháp có thể quy về ba nhóm chính: cảm biến gắn trên cơ thể, cảm biến bố trí trong môi trường và camera quan sát. Điện thoại thông minh, kết nối IoT hoặc điện toán đám mây chủ yếu giữ vai trò xử lý, lưu trữ và chuyển tiếp cảnh báo nên có thể xuất hiện đồng thời trong cả ba nhóm. Cách phân chia theo nguồn dữ liệu phù hợp hơn khi đánh giá một robot hoạt động trong nhà, bởi nó làm rõ người dùng có phải mang thiết bị hay không, phạm vi quan sát, mức độ riêng tư và phần cứng cần bổ sung.

Phương pháp sử dụng thiết bị đeo và tín hiệu cơ thể. Các hệ thống thuộc nhóm này thường sử dụng gia tốc kế, con quay hồi chuyển hoặc IMU gắn tại thắt lưng, ngực, cổ tay hay tích hợp sẵn trong điện thoại. Tín hiệu được lấy mẫu, lọc nhiễu và chia thành các cửa sổ thời gian trước khi đưa qua luật ngưỡng hoặc mô hình phân loại để xác nhận sự kiện. Một đại lượng thường được sử dụng là độ lớn vector gia tốc

$$a = \sqrt{a_x^2 + a_y^2 + a_z^2}, \quad (1.1)$$

sau đó đổi chiều diễn biến giảm tải, va chạm, thay đổi góc thân và trạng thái bất động. Bourke, O'Brien và Lyons đã đánh giá một thuật toán ngưỡng sử dụng gia tốc kế ba trục gắn trên cơ thể, qua đó cho thấy khả năng hiện thực bộ phát hiện có chi phí tính toán thấp [14]. Trên nền tảng smartphone, Abbate và cộng sự bổ sung bước nhận biết những hoạt động thường ngày dễ bị nhầm với té ngã trước khi tự động gửi yêu cầu trợ giúp [15]. Việc xem xét cả trạng thái trước và sau va chạm giúp giảm sự phụ thuộc vào một đỉnh gia tốc đơn lẻ.

Các phương pháp dựa trên chuyển động có chi phí tính toán thấp, không ghi lại hình ảnh trong không gian sống, ít chịu ảnh hưởng của điều kiện chiếu sáng và vẫn hoạt động khi người dùng ở ngoài tầm quan sát của camera. Khi luật ngưỡng được thay bằng SVM, random forest, CNN một chiều hoặc LSTM/GRU, mô hình có khả năng học quan hệ giữa nhiều pha chuyển động. Bộ dữ liệu SisFall, được thu từ hai gia tốc kế và một con quay đặt tại thắt lưng, bao gồm nhiều hoạt động thường ngày và kiểu ngã khác nhau [16]. Kết quả đánh giá thay đổi đáng kể giữa các nhóm tuổi cho thấy mô hình huấn luyện chủ yếu bằng những cú ngã dàn dựng của người trẻ có thể suy giảm khả năng khái quát khi áp dụng cho người cao tuổi.

Bên cạnh tín hiệu chuyển động, một số nghiên cứu khai thác ECG để ghi nhận hoạt động điện của tim và những biến đổi sinh lý có thể xuất hiện trước, trong hoặc sau sự cố. Butt và cộng sự sử dụng dữ liệu ECG của các trường hợp ngã và không ngã, sau đó áp dụng học chuyển giao với AlexNet và GoogLeNet để phân loại; mô hình tốt nhất đạt độ chính xác 98,44% trong điều kiện thực nghiệm của nghiên cứu [17]. Kết quả này cho thấy tín hiệu sinh học có thể cung cấp thêm bằng chứng mà gia tốc hoặc hình ảnh không phản ánh trực tiếp, đồng thời mở ra khả năng kết hợp phát hiện sự cố với theo dõi trạng thái tim mạch.

Tuy nhiên, biến đổi ECG không phải lúc nào cũng bắt nguồn từ té ngã và bản thân tín hiệu này không cho biết vị trí hay tư thế của người trong không gian. Chất lượng đo còn phụ thuộc vị trí điện cực, mức tiếp xúc với da và nhiễu do chuyển động. Các phương pháp dùng thiết bị đeo nói chung cũng phụ thuộc vào việc người dùng đeo đúng vị trí, duy trì đủ pin và không bỏ quên thiết bị. Với cảm biến quán tính, những hoạt động như chạy, nhảy hoặc ngồi mạnh có thể tạo xung gia tốc gần giống va chạm, trong khi một cú trượt chậm lại có khả năng không vượt ngưỡng phát hiện. Vì vậy, ECG và IMU phù hợp hơn khi được sử dụng như nguồn thông tin bổ trợ. Trong phạm vi đề tài, camera vẫn là lựa chọn chính vì vừa cung cấp tư thế, vừa xác định vị trí người để phục vụ theo dõi và điều khiển robot; tín hiệu sinh học là một hướng mở rộng cho chức năng giám sát sức khỏe sau này.

Phương pháp dựa trên cảm biến môi trường. Thay vì gắn cảm biến lên cơ thể, nhóm phương pháp này bố trí thiết bị thu nhận trong không gian cần giám sát. Acoustic-FADE sử dụng mảng microphone và SRP-PHAT để định vị nguồn âm, beamforming để tăng cường tín hiệu, MFCC để biểu diễn đặc trưng phổ, đồng thời khai thác độ cao nguồn âm nhằm hỗ trợ phân biệt tiếng ngã với các âm thanh khác [18]. Trên tập thử nghiệm gồm

120 âm thanh ngã và 120 âm thanh không ngã được mô phỏng, hệ thống đạt độ nhạy 100% tại độ đặc hiệu 97%. Mặc dù cho kết quả cao trong điều kiện thí nghiệm, tín hiệu âm thanh dễ thay đổi theo tiếng nền, độ vang, vật liệu sàn và âm thanh do đồ vật rơi; dữ liệu dàn dựng cũng chưa phản ánh đầy đủ sự đa dạng của tai nạn tự nhiên.

Camera chiều sâu và hệ stereo bổ sung quan hệ hình học ba chiều giữa cơ thể với mặt sàn. Stone và Skubic xây dựng một quy trình hai giai đoạn trên Kinect: trước hết xác định các đoạn chuỗi trong đó người ở gần sàn, sau đó dùng tập hợp cây quyết định để ước lượng khả năng đã xảy ra cú ngã [19]. Công trình được đánh giá trong thời gian dài tại 13 căn hộ, ghi nhận 454 lần ngã nhưng chỉ có chín trường hợp diễn ra tự nhiên. Sự chênh lệch này phản ánh khó khăn trong việc thu thập dữ liệu té ngã thực tế có nhãn đáng tin cậy. Solbach và Tsotsos kết hợp CNN ước lượng tư thế với camera stereo để khôi phục pose 3D và mặt phẳng sàn [20]. Thông tin hình học giúp giảm ảnh hưởng của màu sắc và nền cảnh, nhưng yêu cầu phần cứng, phạm vi đo và quy trình hiệu chuẩn riêng.

Radar mmWave khai thác khoảng cách, vận tốc Doppler hoặc point cloud mà không tạo ra hình ảnh có thể nhận dạng khuôn mặt, đồng thời vẫn hoạt động trong điều kiện thiếu sáng. Hệ thống mmFall sử dụng point cloud 4D và hybrid variational RNN autoencoder để học phân bố của các hoạt động bình thường, sau đó kết hợp điểm bất thường với sự hạ thấp của tâm cơ thể [21]. Trong 50 cú ngã thử nghiệm, phương pháp phát hiện được 98% và tạo hai báo động giả. Tuy nhiên, độ thưa của point cloud, phản xạ đa đường và cách bố trí phòng có thể làm thay đổi đáng kể tín hiệu. Để khắc phục giới hạn của từng cảm biến riêng lẻ, Kwolek và Kepski sử dụng IMU để kích hoạt một sự kiện tiềm năng, rồi xác minh bằng ảnh chiều sâu, đặc trưng hình dạng, khoảng cách tới sàn và SVM [22]. Hợp nhất cảm biến giúp các nguồn dữ liệu bù trừ điểm mù cho nhau, nhưng kéo theo chi phí phần cứng, đồng bộ thời gian, hiệu chuẩn và bảo trì cao hơn.

Bảng 1.1: So sánh bằng chứng và giới hạn của các hướng phát hiện té ngã tiêu biểu

Hướng	Bằng chứng chính	Ưu điểm	Giới hạn khi triển khai
IMU/thiết bị đeo	Gia tốc, vận tốc góc, góc thân, bất động	Nhẹ, riêng tư, không phụ thuộc camera	Phải đeo/sạc đúng cách; nhạy vị trí và người dùng
Tín hiệu sinh học	ECG, nhịp tim và biến đổi sinh lý	Bổ sung thông tin về trạng thái cơ thể và nguy cơ liên quan	Cần tiếp xúc cơ thể; nhạy nhiều chuyển động và không cung cấp vị trí người
Âm thanh	Va chạm, MFCC, vị trí và độ cao nguồn âm	Không đeo, không lưu ảnh	Nhạy tiếng nền, độ vang, đồ vật rơi và vật liệu sàn
Depth/stereo	Silhouette/pose 3D, tâm người, khoảng cách tới sàn	Hình học ba chiều rõ, giảm phụ thuộc màu sắc	Cần phần cứng và hiệu chuẩn; vùng quan sát cố định
Radar mmWave	Doppler, point cloud, quỹ đạo tâm cơ thể	Hoạt động thiếu sáng, bảo vệ riêng tư tốt hơn RGB	Dữ liệu thừa, đa đường, chi phí cảm biến
RGB một khung hình	Bounding box và đặc trưng hình ảnh của người	Nhanh, định vị trực tiếp, tận dụng camera sẵn có	Dễ nhầm lẫn/cúi; nhạy ánh sáng, góc nhìn và che khuất
RGB pose–thời gian	Keypoint, quỹ đạo khớp, thay đổi tư thế theo chuỗi	Mô tả được diễn biến cú ngã, phù hợp xử lý biên	Lỗi pose/tracking lan sang LSTM; cần dữ liệu chuỗi đa dạng
Đa cảm biến	Quyết định từ hai hoặc nhiều nguồn	Bù điểm mù và tăng khả năng xác minh	Tăng chi phí, đồng bộ, hiệu chuẩn và bảo trì

Phương pháp sử dụng camera và thị giác máy tính. Camera cung cấp nhiều thông tin hơn một tín hiệu chuyển động đơn lẻ: từ video, hệ thống có thể xác định vị trí người, kích thước và tỷ lệ của bounding box, các điểm khớp, tốc độ hạ thấp cơ thể và trạng thái sau va chạm. Khung hình gắn với sự kiện còn giúp người chăm sóc quan sát lại tình huống khi cần thiết. Sự phổ biến của camera tích hợp và bộ xử lý di động làm cho hướng tiếp cận này ngày càng phù hợp với xử lý tại biên. Tuy nhiên, chất lượng nhận biết vẫn chịu ảnh hưởng của ánh sáng, góc nhìn, che khuất và chuyển động nền; dữ liệu hình ảnh cũng đòi hỏi nguyên tắc bảo vệ quyền riêng tư rõ ràng.

Các phương pháp thị giác ban đầu thường dùng silhouette, tỷ lệ chiều cao–chiều rộng, hướng trục chính hoặc lịch sử chuyển động. Hướng optical flow biểu diễn chuyển động giữa các khung hình; Núñez-Marcos, Azkune và Arganda-Carreras xếp chồng optical flow rồi dùng CNN dựa trên VGG-16 để phân lớp đoạn video [23]. Cách này nắm được động học tốt hơn ảnh tĩnh, nhưng bước dòng quang học tốn tài nguyên và đặc biệt nhạy với chuyển động camera của robot.

Với detector một giai đoạn, ảnh RGB đi qua mạng YOLO, ngưỡng độ tin cậy và NMS để trả về bounding box của người ngã. Công trình “Visionary Vigilance” xây dựng DiverseFALL10500 với khoảng 10.500 ảnh đa dạng và tối ưu YOLOv8s bằng Focus cùng CBAM [10]. Detector ảnh có tốc độ tốt và định vị trực tiếp sự cố, nhưng một tư thế nằm trong ảnh đơn không chứng minh đã có chuyển tiếp té ngã: người dùng có thể nằm nghỉ, cúi nhặt đồ, tập thể dục hoặc bị quan sát ở góc bất lợi. Vì vậy, accuracy theo khung hình cao vẫn có thể dẫn đến nhiều cảnh báo giả khi camera hoạt động hàng giờ.

Ước lượng pose chuyển người thành tập keypoint (x, y, c) , trong đó c là độ tin cậy. Sau khi chuẩn hóa theo bounding box của người hoặc kích thước cơ thể, hệ thống có thể tạo các đặc trưng như góc thân, tỷ lệ chiều cao–chiều rộng, độ cao tương đối và vận tốc của hông, vai, gối. So với pixel RGB, skeleton giảm phụ thuộc vào màu quần áo và nền, đồng thời làm rõ diễn biến động học. Tuy nhiên, che khuất và góc nhìn xấu có thể làm mất keypoint; bởi vậy pose phải được gắn với đúng track trước khi ghép thành chuỗi. Juraev và cộng sự cũng chỉ ra các khó khăn về thiếu dữ liệu ngã thực, thay đổi góc camera/ánh sáng và che khuất; dữ liệu tổng hợp có thể hỗ trợ, nhưng lựa chọn mô hình thời gian vẫn phải được kiểm chứng trên miền triển khai [24].

Kết hợp tư thế với thông tin thời gian. Hạn chế của quyết định dựa trên từng khung hình tạo động lực kết hợp bộ ước lượng tư thế với mô hình chuỗi. Trong kiến trúc này, YOLO-pose xác định vị trí người và các khớp ở mỗi thời điểm, còn LSTM khai thác sự phụ thuộc giữa các khung liên tiếp để phân biệt một quá trình té ngã với tư thế nằm hoặc cúi người thông thường. Luồng xử lý tổng quát gồm

*Video RGB $\rightarrow$ ước lượng tư thế và gán ID $\rightarrow$ chuỗi keypoint đã chuẩn hóa
 $\rightarrow$ mô hình thời gian $\rightarrow$ xác nhận sự kiện $\rightarrow$ phát cảnh báo.*

Nhờ duy trì trạng thái theo thời gian, LSTM có thể học sự nối tiếp giữa mất thăng bằng, hạ thấp nhanh, va chạm và trạng thái nằm bất thường thay vì chỉ nhận dạng một dáng người đơn lẻ [25]. YOLO-pose và LSTM vì thế đảm nhiệm hai vai trò bổ sung: mô hình

đầu trích xuất biểu diễn không gian, còn mô hình sau diễn giải sự biến đổi của biểu diễn đó theo thời gian.

FallNet là một hiện thực tiêu biểu của kiến trúc này. Mô hình được phát triển từ YOLOv8n-pose, trong đó khối C2F được thay bằng DyC2F nhằm cải thiện khả năng trích xuất đặc trưng động và giảm chi phí tính toán; quỹ đạo keypoint sau đó được đưa vào LSTM để nhận biết sớm xu hướng ngã [13]. Nghiên cứu báo cáo độ chính xác 97%, precision 92%, recall 97% và F1-score 94% với 3,4 triệu tham số cùng 6,4 GFLOPs. Khi triển khai trên Raspberry Pi 3B+, nguyên mẫu đạt 13 FPS với độ trễ xử lý tại thiết bị khoảng 100 ms. Việc báo cáo đồng thời chất lượng phân loại, quy mô mô hình và tốc độ trên phần cứng thực làm cho công trình có giá trị tham khảo trực tiếp đối với các hệ thống AI biên.

Vai trò của bước theo dõi được thể hiện rõ hơn trong YOSAP-LSTM. Hệ thống dùng YOLOv8 để phát hiện người, SORT để duy trì định danh, AlphaPose để trích xuất skeleton và mạng 1D CNN-LSTM để phân loại chuỗi [12]. Trên bộ dữ liệu công trường của nghiên cứu, mô hình đạt độ chính xác 98,66%, độ nhạy 97,32%, độ đặc hiệu 99,10% và tốc độ 6,44 FPS trên NVIDIA Jetson Xavier NX. Khả năng xử lý nhiều người và các mức che khuất khác nhau cho thấy tracking là mất xích cần thiết trước khi hình thành chuỗi pose. Tuy nhiên, miền công trường, cấu hình Jetson và dữ liệu chuyên biệt khác đáng kể so với môi trường nhà ở trên smartphone; do đó kết quả 98,66% không thể được xem là hiệu năng kỳ vọng của đồ án.

Ở một biến thể khác, Liu và cộng sự đề xuất HFDMIA-Pose, cải tiến YOLOv8s bằng SPD-Conv, bổ sung đầu phát hiện vật thể nhỏ và sử dụng BCIOU; kết quả của detector được chuyển sang AlphaPose và kết hợp với các đặc trưng tức thời cũng như trạng thái để xác định cú ngã [11]. Trên Le2i và bộ MPFDD nhiều người, phương pháp cải thiện tương đối so với AlphaPose 4,30% về độ chính xác, 4,57% về F1 và 37,50% về FPS. Mặc dù không sử dụng LSTM, nghiên cứu cho thấy chất lượng phát hiện người và keypoint quyết định trực tiếp chất lượng của tầng nhận dạng hành động phía sau; đồng thời, bằng chứng tức thời chỉ trở nên đáng tin cậy khi được đặt trong một trạng thái kéo dài theo thời gian.

Bảng 1.2 tóm tắt một số công trình thị giác tiêu biểu được công bố gần đây. Các nghiên cứu này không hoàn toàn cùng bài toán, nhưng thể hiện rõ ba hướng cải tiến đang được quan tâm: tăng tính đa dạng của dữ liệu, khai thác tư thế theo thời gian và giảm chi phí tính toán để đưa mô hình tới thiết bị biên.

Bảng 1.2: Một số nghiên cứu thị giác gần đây về phát hiện té ngã

Công trình	Hướng tiếp cận	Kết quả và điểm cần lưu ý
Visionary Vigilance (2024) [10]	Cải tiến YOLOv8s và xây dựng DiverseFALL10500	Bộ dữ liệu có quy mô và bối cảnh đa dạng; phát hiện trực tiếp trên ảnh nhanh nhưng chưa mô tả đầy đủ diễn biến của sự kiện.
HFDmia-Pose (2025) [11]	YOLOv8s cải tiến, AlphaPose và đặc trưng tức thời–trạng thái	Hướng đến nhiều người và thiết bị biên; hiệu quả vẫn phụ thuộc chất lượng pose và mức che khuất.
YOSAP-LSTM (2025) [12]	YOLOv8, SORT, AlphaPose và 1D CNN–LSTM	Đạt 98,66% accuracy và 6,44 FPS trên Jetson Xavier NX; miễn dữ liệu công trường khác với môi trường nhà ở.
FallNet (2026) [13]	YOLOv8n-pose cải tiến kết hợp LSTM	Đạt 97% accuracy với 3,4 triệu tham số; cho thấy tiềm năng của pose–LSTM trên thiết bị có tài nguyên hạn chế.

Các con số trên không nên đặt cạnh nhau như một bảng xếp hạng. FallNet, YOSAP-LSTM và HFDmia-Pose dùng miền dữ liệu, định nghĩa nhãn, cách chia tập, phần cứng và đơn vị đo khác nhau. Accuracy còn có thể che giấu mất cân bằng lớp; trong chăm sóc dài hạn, recall của cú ngã, precision, F1, số báo giả mỗi giờ, độ trễ phát hiện và khả năng tổng quát sang người/căn phòng chưa thấy đều quan trọng. Đánh giá đáng tin cậy cần tách train/test theo người hoặc video, không chia ngẫu nhiên các khung gần nhau của cùng một cảnh, và phải thử bổ sung trên camera thực của robot. CAUCAFall là một nguồn dữ liệu hữu ích do có thay đổi khoảng cách, hướng ngã và ánh sáng trong môi trường ít kiểm soát, nhưng các cú ngã vẫn chủ yếu phải dàn dựng vì lý do đạo đức và an toàn [26].

Hướng tiếp cận được lựa chọn cho đề tài. Từ các kết quả đã phân tích, hướng kết hợp pose và mô hình thời gian phù hợp với nguyên mẫu robot chăm sóc trên cả phương diện chức năng lẫn khả năng triển khai. Camera trên điện thoại cho phép quan sát không tiếp xúc, không yêu cầu người cao tuổi luôn mang thêm cảm biến. Bounding box của người và keypoint còn có thể được sử dụng chung cho chức năng theo dõi mục tiêu, trong khi chuỗi tư thế cung cấp thêm bằng chứng để phân biệt cú ngã với hành động nằm hoặc cúi người có chủ đích.

Nhược điểm của kiến trúc này là sai số có thể truyền qua nhiều giai đoạn: bỏ sót người làm gián đoạn track, track không ổn định làm trộn chuỗi tư thế và keypoint sai tiếp

tục ảnh hưởng đến kết quả phân loại. Do đó, dự đoán của mô hình chỉ nên được xem là một nguồn bằng chứng và cần được kiểm tra theo nhiều khung hình trước khi tạo cảnh báo. Đề tài lựa chọn hướng YOLO-pose kết hợp LSTM vì đạt được sự cân bằng giữa thông tin tư thế, chi phí tính toán và khả năng tích hợp với robot di động; các tham số và cơ chế hậu kiểm cụ thể được trình bày tại phần thiết kế và hiện thực hệ thống.

1.5.2 Các giải pháp theo dõi người và bám mục tiêu

Phát hiện người xác định vị trí của đối tượng trong từng khung hình, còn theo dõi tạo sự liên tục giữa các kết quả phát hiện theo thời gian. Trong đề tài, thông tin này được sử dụng với hai mục đích chính: lựa chọn người cần giám sát và xác định sai lệch của người đó so với tâm ảnh để robot điều chỉnh chuyển động. Bài toán được đặt trong môi trường gia đình, nơi phần lớn thời gian chỉ có một người cao tuổi trong vùng quan sát và robot di chuyển ở tốc độ thấp. Vì vậy, yêu cầu cốt lõi không phải là duy trì đồng thời nhiều định danh trong thời gian dài, mà là duy trì ổn định người đã được chọn làm mục tiêu và dừng an toàn khi kết quả quan sát không còn đủ tin cậy.

SORT. SORT là phương pháp *tracking-by-detection*, tức sử dụng kết quả của bộ phát hiện làm quan sát đầu vào cho bộ theo dõi. Bộ lọc Kalman dự đoán vị trí hộp ở khung hình kế tiếp; độ chồng lấp IoU biểu diễn mức tương đồng về vị trí giữa hộp dự đoán và hộp vừa phát hiện; thuật toán Hungarian sau đó tìm cách ghép các cặp phù hợp [27][28][30]. Phương pháp có cấu trúc gọn, không cần huấn luyện thêm mạng nhận dạng và đáp ứng tốt yêu cầu xử lý thời gian thực trên điện thoại. Hạn chế chính là khả năng giữ ID suy giảm khi mục tiêu bị che khuất lâu, camera chuyển động mạnh hoặc nhiều người đi gần nhau.

Các phương pháp như Deep SORT và ByteTrack cải thiện một số hạn chế trên theo những hướng khác nhau. Deep SORT bổ sung đặc trưng ngoại hình để nhận biết lại đối tượng sau khi bị che khuất, nhờ đó giảm hiện tượng đổi ID trong cảnh có nhiều người [31]. ByteTrack tận dụng cả những hộp có độ tin cậy thấp trong bước liên kết, thay vì loại bỏ chúng ngay từ đầu, nên có thể duy trì quỹ đạo tốt hơn khi đối tượng chỉ bị che một phần [32]. Những cải tiến này có ý nghĩa rõ rệt đối với bài toán theo dõi đa đối tượng, nhưng đồng thời làm tăng chi phí tính toán hoặc độ phức tạp của cơ chế quản lý track.

Đối với tình huống một người chuyển nhanh từ tư thế đứng sang nằm, tracker phức tạp cũng không mặc nhiên cho kết quả tốt hơn. Sự thay đổi lớn về chiều cao, tỉ lệ và vị trí của bounding box có thể làm giảm độ tương đồng hình học; trong khi đặc trưng

ngoại hình cũng kém ổn định khi góc nhìn và phần cơ thể quan sát được thay đổi đột ngột. Quan trọng hơn, các tracker chỉ giải quyết bài toán liên kết đối tượng, không xác định ý nghĩa của biến đổi tư thế. Do đó, việc nhận biết một cú ngã cần dựa vào keypoint và diễn biến theo thời gian; tracker chỉ hỗ trợ bảo đảm chuỗi tư thế được thu thập từ cùng một người.

Từ theo dõi đến bám người. Trong hệ thống của đề tài, người xuất hiện trong vùng quan sát được đánh giá dựa trên kích thước bounding box, độ tin cậy của phép phát hiện, số keypoint nhìn thấy và vị trí trong khung hình. Người có chất lượng quan sát tốt nhất được chọn làm mục tiêu chính và được duy trì trong các khung hình tiếp theo. Vị trí tâm của bounding box cho biết độ lệch theo phương ngang để robot điều chỉnh hướng quay, còn kích thước bounding box kết hợp với khoảng cách giữa các keypoint trên cơ thể được dùng để ước lượng mức độ gần–xa. Từ hai đại lượng này, hệ thống xác định thành phần chuyển động tiến–lùi và thành phần điều hướng, sau đó làm mượt lệnh nhằm hạn chế rung khi mục tiêu dao động quanh vùng cân bằng.

Luồng xử lý có thể khái quát thành *thu nhận hình ảnh* → *phát hiện người và keypoint* → *liên kết với mục tiêu chính* → *ước lượng sai lệch hướng và khoảng cách* → *tạo lệnh chuyển động*. Lệnh được ghi nhận trên kênh điều khiển của hệ thống và chuyển xuống thân robot để thực hiện. Cách phân chia này kế thừa tư tưởng của OpenBot, trong đó điện thoại đảm nhiệm cảm nhận và tính toán, còn phần thân robot thực thi lệnh chuyển động [7].

Phương pháp theo dõi được xây dựng trên nền tảng SORT nhưng có bổ sung thông tin về khoảng cách tâm, vị trí tương đối của các keypoint và sự thay đổi kích thước bounding box. Histogram màu chỉ được sử dụng khi cần phân biệt giữa nhiều ứng viên hoặc nhận dạng lại mục tiêu sau khi mất dấu, do đó không phải chạy thêm một mạng Re-ID sâu ở mọi khung hình. Cơ chế này giúp giảm nguy cơ đổi ID khi người chuyển từ đứng sang nằm, bởi sự thay đổi mạnh về hình dạng bounding box không còn được đánh giá chỉ bằng IoU. Nếu ID vẫn bị thay đổi, hệ thống tìm lại mục tiêu trong vùng lân cận của vị trí cuối, yêu cầu kết quả ổn định qua nhiều khung hình rồi mới khôi phục ID chính và tiếp tục chuỗi dữ liệu dùng cho phát hiện té ngã.

Trong chế độ bám thông thường, hệ thống dành một khoảng thời gian ngắn để tái liên kết khi mục tiêu tạm thời biến mất; nếu không tìm lại được trong khoảng hai giây, lệnh dừng được gửi tới robot. Khi đã ghi nhận người đang nằm hoặc đang xử lý một tình

huống nghi ngờ té ngã, yêu cầu an toàn được đặt cao hơn: robot dừng ngay khi mất mục tiêu và không tiếp tục tiến về phía trước. Nếu người vẫn còn trong ảnh nhưng vùng quan sát chưa phù hợp, robot chỉ lùi chậm hoặc quay tại chỗ để đưa cơ thể vào gần tâm khung hình, sau đó dừng lại. Như vậy, tracker hỗ trợ duy trì đúng người và cung cấp dữ liệu cho điều khiển, còn kết luận té ngã vẫn do quá trình phân tích tư thế và chuỗi thời gian đảm nhiệm. Bảng 1.3 tóm tắt các lựa chọn được cân nhắc.

Bảng 1.3: So sánh các phương pháp theo dõi người liên quan đến đề tài

Phương pháp	Cơ sở liên kết	Ưu điểm và giới hạn	Khả năng áp dụng
SORT	Kalman, IoU, Hungarian	Gọn, nhanh, không cần mạng Re-ID; dễ đổi ID khi che khuất hoặc giao nhau	Phù hợp làm lõi tracker trên smartphone
Deep SORT	Chuyển động và embedding ngoại hình	Giảm đổi ID; cần suy luận đặc trưng và tài nguyên lớn hơn	Hữu ích nếu thiết bị đủ mạnh và nhiều người xuất hiện
ByteTrack	Ghép hai lượt với hộp điểm cao và thấp	Khôi phục quan sát bị che khuất; nhạy với ngưỡng và chất lượng detector	Gợi ý giữ bằng chứng yếu có kiểm soát
Hướng của đề tài	SORT mở rộng với hình học, keypoint và histogram màu	Duy trì mục tiêu qua thay đổi tư thế mà không cần mạng Re-ID sâu; khả năng phục hồi vẫn phụ thuộc chất lượng ảnh	Phù hợp với bối cảnh một mục tiêu và xử lý thời gian thực trên điện thoại

1.5.3 Chatbot AI và hội thoại hỗ trợ người cao tuổi

Trong đồ án, chatbot được định hướng như một người bạn trò chuyện bằng giọng nói trên robot. Người dùng có thể gọi robot, đặt câu hỏi, kể chuyện hoặc trao đổi về những chủ đề hằng ngày mà không phải thao tác với bàn phím và màn hình nhỏ. Mục tiêu chính là tạo một kênh giao tiếp tự nhiên, phản hồi ngắn gọn và dễ sử dụng; chatbot không được xem là công cụ chẩn đoán, trị liệu tâm lý hay tư vấn y khoa.

Kiến trúc của một chatbot giọng nói. Một lượt hội thoại hoàn chỉnh thường đi qua chuỗi kích hoạt → thu âm → nhận dạng tiếng nói (STT) → mô hình ngôn ngữ lớn (LLM) → tổng hợp tiếng nói (TTS) → phát âm thanh. Bên cạnh ba khối xử lý chính, ứng dụng còn cần phát hiện khoảng lặng, quản lý lịch sử hội thoại, ngắt câu trả lời đang phát, xử lý

mất mạng và biểu diễn trạng thái để người dùng biết robot đang nghe, suy nghĩ hay nói. Chất lượng trải nghiệm vì thế phụ thuộc vào độ trễ đầu cuối và cách phối hợp các thành phần, không chỉ vào chất lượng văn bản do LLM sinh ra.

Xiaozhi-ESP32 là một hiện thực tham khảo tiêu biểu cho cách tổ chức này trên thiết bị có tài nguyên hạn chế. Nền tảng thực hiện đánh thức bằng từ khóa ngay trên thiết bị, sau đó truyền âm thanh nén Opus qua WebSocket hoặc MQTT/UDP để sử dụng chuỗi ASR–LLM–TTS; giao thức còn quy định các trạng thái bắt đầu nghe, bắt đầu nói, kết thúc nói và hủy lượt hội thoại [33]. Điểm phù hợp với đồ án không nằm ở phần cứng ESP32 mà ở nguyên tắc kiến trúc: thiết bị đầu cuối chỉ duy trì vòng đời phiên, thu/phát âm thanh và giao diện trạng thái, còn tác vụ ngôn ngữ nặng có thể được chuyển tới dịch vụ bên ngoài. Cách phân vai này cho phép một thiết bị nhỏ tạo cảm giác hội thoại liên tục mà không phải chứa toàn bộ mô hình AI.

Các lựa chọn cho khối STT và TTS. Các giải pháp tiếng nói có thể được chia thành hai nhóm theo vị trí xử lý. Nhóm thứ nhất thực hiện STT và TTS ngay trên thiết bị. Android cung cấp khả năng nhận dạng cục bộ trên những thiết bị có dịch vụ tương thích, đồng thời có thể tổng hợp giọng nói từ các gói giọng đã được cài đặt [34][35]. Ưu điểm của lựa chọn này là ứng dụng không phải đóng gói thêm mô hình, độ trễ thấp và có thể hoạt động khi mất mạng. Tuy nhiên, mức hỗ trợ tiếng Việt và chất lượng nhận dạng phụ thuộc vào phiên bản hệ điều hành, nhà sản xuất và dữ liệu ngôn ngữ có sẵn; dịch vụ mặc định trên một số thiết bị vẫn có thể cần kết nối mạng. Vì vậy, STT/TTS của Android nên được xem là giải pháp có khả năng xử lý cục bộ, thay vì một cơ chế ngoại tuyến đồng nhất trên mọi điện thoại.

Khi cần chủ động hơn về khả năng ngoại tuyến, mô hình tiếng nói có thể được tích hợp cùng ứng dụng. Vosk hỗ trợ nhận dạng luồng và cung cấp các mô hình nhỏ dành cho thiết bị di động, trong đó có tiếng Việt [36]. Sherpa-onnx có phạm vi rộng hơn, bao gồm nhận dạng tiếng nói, phát hiện hoạt động giọng nói, nhận biết từ khóa và tổng hợp tiếng nói trên Android [37]. Hai nền tảng này giữ dữ liệu âm thanh trên thiết bị và không phát sinh chi phí theo lượt sử dụng, nhưng làm tăng dung lượng cài đặt, mức sử dụng CPU/RAM và công việc lựa chọn mô hình. Chất lượng thực tế cũng cần được đánh giá trong điều kiện có tiếng động cơ và tạp âm sinh hoạt trong nhà.

Nhóm thứ hai sử dụng dịch vụ tiếng nói trực tuyến. Âm thanh được gửi theo từng đoạn ngắn lên máy chủ; dịch vụ STT trả dần kết quả tạm thời và kết quả hoàn chỉnh,

trong khi TTS có thể truyền các khối âm thanh về để phát trước khi toàn bộ câu được tổng hợp xong. Azure Speech là một ví dụ cung cấp SDK cho luồng âm thanh thời gian thực và hỗ trợ cả nhận dạng lẫn giọng đọc tiếng Việt [38][39]. Cách tiếp cận này có ưu thế về chất lượng mô hình, khả năng cập nhật và độ nhất quán giữa các thiết bị. Tuy nhiên, quá trình xử lý phụ thuộc vào đường truyền và thường phát sinh chi phí theo thời lượng âm thanh hoặc lượng văn bản sau khi vượt phạm vi dùng thử. Trong điều kiện thời gian và kinh phí của đề án, hướng trực tuyến cho STT/TTS được xem như một phương án tham khảo, còn giải pháp sẵn có trên thiết bị thuận lợi hơn cho việc xây dựng nguyên mẫu.

Lựa chọn giữa LLM qua API và LLM chạy trên thiết bị. LLM qua API cho phép điện thoại gửi nội dung đã nhận dạng cùng một phần ngữ cảnh hội thoại và nhận lại phản hồi theo từng đoạn. Khả năng streaming giúp ứng dụng bắt đầu xử lý câu trả lời sớm, nhờ đó giảm thời gian chờ cảm nhận của người dùng. Phương án này giữ ứng dụng gọn, tận dụng được mô hình có năng lực tốt hơn phần cứng di động và cho phép thay đổi mô hình tương đối linh hoạt. Hạn chế chủ yếu là sự phụ thuộc vào mạng và chi phí có thể phát sinh nếu nhu cầu sử dụng trong quá trình thực hiện đề án vượt quá giới hạn của gói miễn phí.

Groq cung cấp API hội thoại và phản hồi streaming, phù hợp để khảo sát chatbot có độ trễ thấp [40]. Gói miễn phí giới hạn số yêu cầu và token nhưng vẫn đáp ứng nhu cầu xây dựng, điều chỉnh và đánh giá nguyên mẫu [41]. Gemini Developer API cũng cung cấp một số mô hình có thể sử dụng miễn phí trong phạm vi hạn mức nhất định [42].

Ở hướng ngược lại, LLM cục bộ giữ nội dung hội thoại trên điện thoại và không phụ thuộc mạng sau khi mô hình đã được tải xuống. Google AI Edge Gallery và MLC LLM cho thấy các mô hình đã lượng tử hóa có thể được triển khai trên Android [43][44]. Dù vậy, mô hình vẫn chiếm đáng kể dung lượng lưu trữ và bộ nhớ, đồng thời cạnh tranh tài nguyên với pipeline thị giác của robot. Việc giảm kích thước mô hình còn có thể ảnh hưởng đến khả năng hiểu tiếng Việt và chất lượng hội thoại. Vì vậy, LLM cục bộ phù hợp hơn với thiết bị có cấu hình đủ mạnh hoặc yêu cầu ngoại tuyến và riêng tư cao; LLM qua API phù hợp hơn với nguyên mẫu cần hội thoại tự nhiên nhưng có tài nguyên tính toán giới hạn.

Bảng 1.4: So sánh các phương án tạo phản hồi hội thoại

Phương án	Ưu điểm	Hạn chế	Mức phù hợp
Luật/kho câu trả lời	Rất nhẹ, ngoại tuyến, kết quả xác định	Hội thoại cứng và khó bao phủ cách diễn đạt tự nhiên	Tương tác ngắn và nội dung chuẩn bị trước
LLM qua API	Ứng dụng nhẹ, phản hồi tốt, hỗ trợ streaming, dễ đổi mô hình	Cần mạng; chi phí tăng khi vượt giới hạn miễn phí	Phù hợp với nguyên mẫu hội thoại hằng ngày
LLM cục bộ	Ngoại tuyến, dữ liệu không rời thiết bị, không tính phí theo lượt	Mô hình lớn, dùng nhiều RAM và năng lượng; phụ thuộc phần cứng	Hướng phát triển khi thiết bị đáp ứng tài nguyên

Tổ chức hội thoại phù hợp với người cao tuổi. Một chatbot trò chuyện hằng ngày không cần câu trả lời dài hoặc lập luận phức tạp. Giá trị sử dụng đến từ việc kích hoạt dễ nhớ, nghe được nhiều cách diễn đạt, trả lời đúng ngôn ngữ, duy trì một lượng ngữ cảnh vừa đủ và phản hồi sớm. Giao diện nên thể hiện rõ các trạng thái đang nghe, đang xử lý và đang nói; cho phép lặp lại khi STT nhận sai; tránh đọc ký hiệu, đường dẫn hoặc đoạn văn quá dài. Hồ sơ như tên gọi, cách xưng hô và sở thích có thể giúp cuộc trò chuyện tự nhiên hơn, nhưng chỉ nên đưa vào prompt những trường cần thiết cho lượt hội thoại hiện tại.

Ba phương án trong bảng không loại trừ lẫn nhau. LLM thích hợp với hội thoại mở vì có thể tạo phản hồi linh hoạt, trong khi các câu thông báo và tương tác ngắn có thể sử dụng nội dung được chuẩn bị trước. Việc phối hợp hai cách tiếp cận giúp giới hạn vai trò của mô hình ngôn ngữ ở những phần cần khả năng trò chuyện, thay vì dùng nó cho mọi phản hồi của ứng dụng. Cách phân chia này phù hợp với nhận định từ các nghiên cứu về conversational agent rằng khả năng tiếp nhận ngôn ngữ tự nhiên không đồng nghĩa với việc toàn bộ hoạt động của hệ thống phải do mô hình sinh điều khiển [45].

Hướng lựa chọn cho đề tài. Với mục tiêu tạo một người bạn trò chuyện bằng tiếng Việt trên điện thoại Android, đề tài lựa chọn các dịch vụ nhận dạng và tổng hợp tiếng nói sẵn có trên thiết bị kết hợp với LLM qua API. Phương án này không yêu cầu đóng gói thêm mô hình STT/TTS hoặc LLM dung lượng lớn, phù hợp với giới hạn tài nguyên

của điện thoại đồng thời đáp ứng nhu cầu thử nghiệm hội thoại hằng ngày.

Do LLM được cung cấp qua API, chức năng trò chuyện vẫn phụ thuộc Internet ngay cả khi STT và TTS có thể chạy cục bộ. Việc tách riêng ba khối STT, LLM và TTS tạo điều kiện khảo sát, đánh giá và thay thế từng thành phần bằng giải pháp trực tuyến hoặc cục bộ khi yêu cầu và khả năng phần cứng thay đổi.

1.5.4 Các nền tảng robot tích hợp và bài học áp dụng

SocialRobot là một ví dụ về robot di động hỗ trợ chăm sóc theo hướng lấy người dùng làm trung tâm. Nền tảng kết hợp khả năng di chuyển và cảm nhận tại chỗ với hệ thống quản lý hồ sơ, dịch vụ cá nhân hóa và nhóm chăm sóc từ xa [46]. Điểm đáng tham khảo là người thân và người chăm sóc được xem như những tác nhân tham gia vào quá trình hỗ trợ, thay vì chỉ nhận thông báo khi sự cố đã xảy ra.

OpenBot tiếp cận bài toán từ hướng đơn giản hóa nền tảng robot: điện thoại thông minh đảm nhiệm cảm nhận, tính toán, giao diện và kết nối, còn phần thân xe tập trung thực hiện lệnh chuyển động [7][47]. Cách tổ chức này làm giảm số phần cứng chuyên dụng và tạo điều kiện thuận lợi cho học tập, thử nghiệm. Đề tài kế thừa tư tưởng sử dụng điện thoại làm trung tâm xử lý, đồng thời bổ sung ứng dụng dành cho người chăm sóc và các chức năng phù hợp với bối cảnh hỗ trợ người cao tuổi tại nhà.

1.5.5 Khoảng trống nghiên cứu và định vị của đề tài

Các công trình được khảo sát thường tập trung giải quyết tốt một bài toán riêng. Nghiên cứu phát hiện té ngã chủ yếu đánh giá mô hình trên bộ dữ liệu; nghiên cứu theo dõi chú trọng duy trì danh tính; chatbot tập trung vào chất lượng tương tác; còn các nền tảng robot giá thấp quan tâm đến khả năng điều khiển và mở rộng. Những nghiên cứu thị giác gần đây đã cải thiện đáng kể khả năng xử lý nhiều người, che khuất và triển khai trên thiết bị biên [11][12][13], nhưng việc đưa kết quả nhận biết vào một quy trình chăm sóc hoàn chỉnh vẫn cần được xem xét thêm.

Khoảng trống mà đề tài hướng tới là sự phối hợp từ quan sát, nhận biết tình huống đến tương tác và thông báo cho người chăm sóc. Khi các chức năng cùng hoạt động trên robot di động, kết quả của bước trước ảnh hưởng trực tiếp đến bước sau; vì vậy hệ thống không chỉ cần mô hình nhận biết tốt mà còn phải duy trì đúng người mục tiêu, xử lý trường hợp thiếu dữ liệu và chuyển về trạng thái an toàn khi không đủ cơ sở ra quyết định.

1.6 Đóng góp của đề tài

Đề tài hướng đến đóng góp ở khả năng tổ chức nhiều chức năng hỗ trợ thành một hệ thống thống nhất, phù hợp với bối cảnh chăm sóc người cao tuổi tại nhà. Những kết quả chính đạt được gồm:

- Xây dựng nguyên mẫu robot trợ lý có khả năng hỗ trợ giao tiếp, nhắc nhở, giám sát và kết nối người cao tuổi với người chăm sóc. Sự phối hợp giữa các chức năng này góp phần hình thành một hệ sinh thái chăm sóc liên tục, trong đó nhu cầu sinh hoạt hằng ngày và các tình huống cần trợ giúp được tiếp nhận trên cùng một hệ thống.
- Đề xuất cách phân chia chức năng và luồng phối hợp giữa người cao tuổi, robot và người chăm sóc. Kết quả này cho thấy khả năng kết hợp trợ lý ảo, giám sát an toàn và robot di động trong một giải pháp hỗ trợ tại nhà, đồng thời tạo cơ sở để bổ sung các dịch vụ chăm sóc khác trong tương lai.
- Xây dựng một nền tảng thử nghiệm phục vụ việc nghiên cứu các bài toán liên quan đến tương tác người–robot, hỗ trợ sinh hoạt, nhận biết tình huống bất thường và phối hợp chăm sóc từ xa. Nguyên mẫu có thể được tiếp tục sử dụng để khảo sát phương pháp mới, so sánh giải pháp hoặc đánh giá trải nghiệm của người dùng trong những nghiên cứu tiếp theo.
- Góp phần làm rõ tiềm năng ứng dụng của trí tuệ nhân tạo và robot trong hỗ trợ người cao tuổi, qua đó cung cấp một tài liệu tham khảo cho sinh viên và các nhóm nghiên cứu khi phát triển những ứng dụng có mục tiêu tương tự. Kết quả của đề tài cũng đặt nền tảng cho việc hoàn thiện giải pháp theo hướng dễ tiếp cận, tin cậy và phù hợp hơn với điều kiện chăm sóc thực tế.

1.7 Cấu trúc báo cáo

Báo cáo được tổ chức thành sáu chương, sắp xếp theo trình tự từ xác định bài toán, xây dựng nền tảng lý thuyết đến thiết kế, hiện thực và đánh giá hệ thống. Nội dung cụ thể của từng chương như sau:

- **Chương 1 – Giới thiệu:** trình bày bối cảnh già hóa dân số và nhu cầu hỗ trợ người cao tuổi tại nhà; từ đó xác định mục tiêu, đối tượng, phạm vi, phương pháp nghiên cứu, các công trình liên quan và đóng góp của đề tài.

- **Chương 2 – Cơ sở lý thuyết:** hệ thống hóa những kiến thức làm nền tảng cho việc xây dựng robot trợ lý, bao gồm nền tảng robot sử dụng điện thoại thông minh, thị giác máy tính, mô hình nhận dạng tư thế và chuỗi thời gian, xử lý hội thoại, suy luận trên thiết bị, truyền thông và điều khiển phần thân robot.
- **Chương 3 – Phân tích và thiết kế hệ thống:** phân tích yêu cầu chức năng và phi chức năng, xây dựng kiến trúc tổng quan, mô tả các luồng hoạt động chính, thiết kế các khối chức năng, tổ chức dữ liệu và những yêu cầu về an toàn, bảo mật.
- **Chương 4 – Hiện thực hệ thống:** trình bày quá trình xây dựng các thành phần của nguyên mẫu, gồm ứng dụng trên robot, ứng dụng dành cho người chăm sóc, các chức năng trí tuệ nhân tạo, quá trình chuẩn bị mô hình và chương trình điều khiển phần thân robot.
- **Chương 5 – Kết quả và thảo luận:** trình bày phương pháp và tiêu chí đánh giá, kết quả kiểm thử chức năng, kết quả đánh giá mô hình trí tuệ nhân tạo và hiệu năng hệ thống; đồng thời phân tích những kết quả đạt được và các hạn chế còn tồn tại.
- **Chương 6 – Kết luận:** tổng kết quá trình thực hiện, đối chiếu kết quả với mục tiêu đề tài và đề xuất các hướng nghiên cứu, hoàn thiện hệ thống trong tương lai.

Phần phụ lục bổ sung các đặc tả trường hợp sử dụng, cấu trúc dữ liệu, danh mục thành phần, thông tin mô hình và biểu mẫu kiểm thử nhằm hỗ trợ việc tra cứu và kiểm chứng nội dung được trình bày trong báo cáo.

1.8 Kết luận chương

Chương 1 đã làm rõ bối cảnh và tính cấp thiết của việc nghiên cứu giải pháp hỗ trợ chăm sóc người cao tuổi tại nhà. Quá trình già hóa dân số, nguy cơ té ngã, sự suy giảm trí nhớ, nhu cầu duy trì lịch sinh hoạt và khoảng cách giữa người cao tuổi với người chăm sóc đặt ra yêu cầu về một phương thức hỗ trợ chủ động, thuận tiện và có khả năng phản hồi kịp thời. Trong bối cảnh đó, robot trợ lý được xem là một phương tiện hỗ trợ cho hoạt động chăm sóc, giúp duy trì tương tác hằng ngày, hỗ trợ giám sát và tạo thêm kênh kết nối với người thân, thay vì thay thế vai trò của con người hoặc các thiết bị y tế chuyên dụng.

Từ bài toán đặt ra, chương đã xác định mục tiêu xây dựng một nguyên mẫu tích hợp các chức năng giao tiếp, nhắc nhở, theo dõi, nhận biết tình huống nghi té ngã và hỗ

trợ liên lạc từ xa. Đối tượng, phạm vi chức năng và giới hạn nghiên cứu cũng được xác định nhằm bảo đảm nội dung thực hiện phù hợp với điều kiện của đồ án. Bên cạnh đó, phương pháp nghiên cứu được tổ chức từ khảo cứu tài liệu, phân tích và thiết kế đến xây dựng nguyên mẫu, thực nghiệm và đánh giá. Việc khảo sát các công trình liên quan cho thấy mỗi hướng tiếp cận đều có những ưu điểm và giới hạn riêng; đồng thời cung cấp cơ sở để lựa chọn giải pháp phù hợp với môi trường trong nhà và khả năng triển khai trên robot di động.

Trên cơ sở đó, đóng góp của đề tài được định hướng vào việc kết nối các chức năng hỗ trợ thành một hệ thống thống nhất dành cho người cao tuổi và người chăm sóc, đồng thời hình thành một nền tảng có thể tiếp tục mở rộng trong các nghiên cứu sau. Những vấn đề đã xác lập trong chương này là cơ sở để Chương 2 trình bày các kiến thức lý thuyết cần thiết, trước khi tiến hành phân tích và thiết kế hệ thống ở Chương 3.

Chương 2 – Cơ sở lý thuyết

Chương này trình bày các nền tảng lý thuyết phục vụ quá trình phân tích, thiết kế và hiện thực hệ thống. Nội dung bắt đầu từ kiến trúc robot sử dụng điện thoại thông minh của OpenBot, sau đó trình bày cơ sở của thị giác máy tính, phát hiện đối tượng, các chỉ số đánh giá, theo dõi mục tiêu, họ mô hình YOLO và YOLO26-Pose. Phần tiếp theo đề cập mô hình chuỗi RNN–LSTM, tập dữ liệu LE2I và quá trình triển khai mô hình trên thiết bị bằng TensorFlow Lite. Cuối cùng, chương giới thiệu Firebase, WebRTC, Bluetooth Low Energy, phần cứng thân robot và thư viện điều khiển động cơ, tạo cơ sở cho các lựa chọn thiết kế ở Chương 3.

2.1 Nền tảng Intel OpenBot (OpenBot)

2.1.1 Giới thiệu và ý tưởng cốt lõi

OpenBot là nền tảng robot mã nguồn mở khởi nguồn từ công trình của Matthias Müller và Vladlen Koltun, được công bố tại IEEE ICRA 2021. Ý tưởng chính của OpenBot là sử dụng điện thoại thông minh làm “bộ não” cho một thân xe robot chi phí thấp. Thay vì trang bị riêng camera, máy tính nhúng mạnh, màn hình và mô-đun mạng, nền tảng tận dụng camera, cảm biến, CPU/GPU, pin và khả năng kết nối đã có trên smartphone [7].

Trong kiến trúc này, điện thoại được gắn trực tiếp lên robot và tham gia vào vòng xử lý: thu hình ảnh, chạy mô hình AI, quyết định hướng di chuyển và gửi lệnh xuống vi điều khiển. Thân xe đảm nhiệm cấp nguồn, điều khiển động cơ và đọc các cảm biến thấp tầng. Công trình gốc chứng minh cách tiếp cận này có thể thực hiện các tác vụ như bám theo người và điều hướng tự động thời gian thực trên điện thoại Android phổ thông [7]. Website và kho mã chính thức tiếp tục phát triển OpenBot theo hướng một nền tảng mở phục vụ nghiên cứu, giáo dục robot và AI [48][47].

Ý tưởng này có liên hệ trực tiếp với đồ án. RobotApp cũng sử dụng smartphone để thu hình ảnh, chạy mô hình AI, giao tiếp mạng và điều phối hoạt động; ESP32 tập trung vào giao tiếp BLE và điều khiển động cơ. Nhờ đó, các thuật toán thị giác có thể thay đổi mà không phải thiết kế lại toàn bộ phần thân robot.

2.1.2 Kiến trúc và các thành phần liên quan

OpenBot có thể được khái quát thành ba lớp chính:

- **Lớp smartphone:** Robot App truy cập camera và cảm biến, chạy mô hình TensorFlow Lite, quản lý chế độ điều khiển và truyền lệnh cho thân xe.
- **Lớp vi điều khiển và thân robot:** Arduino Nano hoặc ESP32 nhận lệnh, tạo PWM cho driver động cơ, đọc encoder, sonar, bumper và điện áp pin rồi gửi trạng thái trở lại ứng dụng [49].
- **Lớp điều khiển bên ngoài:** gamepad, điện thoại thứ hai, trình duyệt hoặc chương trình Python có thể gửi lệnh và nhận hình ảnh từ robot [50].

Kho mã OpenBot cũng phản ánh cách phân chia trên: thư mục `android` chứa Robot App và Controller App; `firmware` chứa chương trình cho MCU; `body` chứa thiết kế thân xe; `controller` chứa các phương thức điều khiển; còn `policy` hỗ trợ thu thập dữ liệu và huấn luyện chính sách lái [47]. Cấu trúc mô-đun này là một tham khảo hữu ích cho việc tách RobotApp, CaregiverApp, firmware và phần AI trong đồ án.

Điểm quan trọng của kiến trúc OpenBot là phân công theo năng lực của thiết bị. Smartphone phù hợp với camera, AI, giao diện và kết nối Internet; MCU phù hợp với vòng điều khiển động cơ có chu kỳ ngắn. Việc không đặt toàn bộ xử lý trên MCU giúp triển khai được các mô hình thị giác phức tạp, trong khi vẫn giữ lớp chấp hành độc lập và có khả năng dừng khi mất lệnh.

2.1.3 Giải pháp kết nối và điều khiển

OpenBot hỗ trợ USB serial và BLE giữa smartphone với vi điều khiển. Với USB, Robot App thường sử dụng baud rate 115200 để gửi lệnh và nhận telemetry. Với các cấu hình ESP32, BLE cho phép bố trí điện thoại không dây và thuận tiện hơn, nhưng ứng dụng phải xử lý quét thiết bị, kết nối lại và mất liên lạc [51][49].

Firmware OpenBot sử dụng giao thức văn bản ngắn gọn. Lệnh điều khiển chứa giá trị cho phía trái và phải; các lệnh khác cấu hình đèn, chu kỳ đo sonar, tốc độ bánh hoặc điện áp. Firmware còn hỗ trợ heartbeat: nếu không nhận được gói mới trong thời gian quy định, robot phải dừng. Đây là giải pháp quan trọng vì smartphone và kết nối không dây không bảo đảm hoạt động liên tục [49].

Ở mức điều khiển từ xa, OpenBot cung cấp nhiều lựa chọn: gamepad Bluetooth, Controller App trên điện thoại thứ hai, Node.js controller qua Wi-Fi, Python controller với RTSP và web controller qua Internet. Robot App có thể truyền hình ảnh bằng WebRTC hoặc RTSP tùy client [50].

Trong đồ án, ý tưởng kết nối được điều chỉnh theo mục tiêu chăm sóc người cao tuổi:

- RobotApp kết nối ESP32 bằng BLE GATT thay cho giao thức serial nguyên bản của OpenBot;
- CaregiverApp gửi lệnh từ xa qua Firebase RTDB, do người chăm sóc có thể không ở cùng mạng với robot;
- WebRTC được dùng cho cuộc gọi audio/video giữa caregiver và robot;
- lệnh có timestamp, RobotApp có watchdog 500 ms và firmware có watchdog 800 ms để robot dừng khi mất luồng điều khiển.

Như vậy, đồ án kế thừa nguyên tắc phân tầng và dừng an toàn của OpenBot, nhưng xây dựng lại kênh truyền để phù hợp mô hình điều khiển từ xa qua Internet.

2.1.4 AI và cơ chế theo dõi đối tượng

Robot App của OpenBot hỗ trợ suy luận TensorFlow Lite trực tiếp trên điện thoại. Người dùng có thể lựa chọn model, ngưỡng confidence, số luồng và backend CPU, GPU hoặc NNAPI. Tài liệu OpenBot cung cấp các detector như MobileNet SSD, YOLO và EfficientDet, đồng thời benchmark chúng trên nhiều mẫu smartphone. Cách thiết kế này cho thấy model triển khai trên robot phải cân bằng độ chính xác, tốc độ, nhiệt độ và khả năng tương thích của thiết bị [51].

Trong chức năng Object Tracking, detector nhận dạng các đối tượng thuộc tập lớp COCO và trả về vùng bao. Sai lệch giữa tâm vùng bao với tâm ảnh có thể dùng để điều chỉnh hướng quay; kích thước vùng bao được dùng như một dấu hiệu gần đúng về khoảng cách để giảm tốc khi robot tiến gần mục tiêu. Luồng xử lý cơ bản gồm:

Camera → phát hiện đối tượng → chọn mục tiêu → tính sai lệch vị trí → lệnh bánh xe.

Đối với bám theo người, OpenBot cung cấp ý tưởng nền tảng là đặt nhận thức và điều khiển trong cùng một vòng kín trên smartphone. Tuy nhiên, chỉ chọn detection có nhãn “person” ở từng frame có thể làm đổi mục tiêu khi nhiều người xuất hiện hoặc khi người đang bám bị che khuất. Vì vậy, đồ án phát triển thêm cơ chế tracking theo thời gian.

Pipeline của đồ án sử dụng mô hình pose để phát hiện người và keypoint, SORT/Kalman/Hungarian để liên kết detection giữa các frame, cùng Re-ID nhẹ để hỗ trợ duy trì người mục tiêu. Người được chọn được bảo vệ bằng ID0; vị trí và kích thước vùng bao được chuyển thành lệnh bấm. Chuỗi pose cũng được đưa vào LSTM và bộ hậu kiểm để phát hiện té ngã. Đây là phần mở rộng từ ý tưởng object/person tracking của OpenBot sang yêu cầu nhận diện đúng người và giám sát an toàn.

2.1.5 Khả năng áp dụng cho đồ án

Bảng 2.1 tổng hợp những nội dung OpenBot có liên quan trực tiếp đến hệ thống hiện tại.

Bảng 2.1: Các ý tưởng OpenBot được tham khảo trong đồ án

Nội dung	OpenBot	Áp dụng trong đồ án
Smartphone làm bộ não	Camera, AI, giao diện và logic điều phối đặt trên điện thoại	RobotApp chạy pose, tracking, LSTM, hội thoại và cuộc gọi.
Tách xử lý và chấp hành	Smartphone xử lý bậc cao; MCU điều khiển PWM	RobotApp sinh lệnh; ESP32 điều khiển hai BLDC bằng SimpleFOC.
Kết nối thân robot	USB serial hoặc BLE	BLE GATT với giao thức hai setpoint và watchdog.
Điều khiển từ thiết bị khác	Gamepad, Controller App, web/Python controller	CaregiverApp gửi lệnh qua Firebase RTDB và nhận trạng thái từ xa.
AI tracking	Detector TFLite, vùng bao và luật bấm mục tiêu	Pose, SORT, Kalman, Hungarian, Re-ID và bảo vệ ID0.
Truyền hình ảnh	WebRTC hoặc RTSP tùy bộ điều khiển	WebRTC cho cuộc gọi audio/video caregiver-robot.

OpenBot vì vậy là nền tảng tham khảo quan trọng về ý tưởng và các giải pháp kỹ thuật nền, đặc biệt là smartphone-as-a-robot-brain, xử lý AI tại biên, kết nối smartphone-MCU và vòng kín theo dõi mục tiêu. Đồ án kế thừa các nguyên tắc này nhưng bổ sung kiến trúc hai ứng dụng, nghiệp vụ chăm sóc, phát hiện té ngã, tương tác giọng nói, Firebase và phần thân BLDC/FOC riêng.

2.2 Nền tảng Thị giác máy tính (Computer Vision)

Thị giác máy tính cung cấp dữ liệu cảm nhận cho hai chức năng chính của robot: giám sát tư thế nhằm phát hiện té ngã và xác định vị trí người để tự động bấm theo. Một pipeline điển hình gồm thu nhận khung hình, trích xuất đặc trưng bằng mạng nơ-ron tích chập (Convolutional Neural Network – CNN), phát hiện người và các điểm khớp, liên

kết quả qua thời gian, sau đó chuyển chuỗi đặc trưng sang bộ phân loại hành vi. CNN học trực tiếp các bộ lọc không gian từ dữ liệu, nhờ đó thay thế phần lớn quá trình thiết kế đặc trưng thủ công của các phương pháp thị giác cổ điển [52]. Các khái niệm từ phát hiện đối tượng đến đánh giá mô hình dưới đây là nền tảng chung để phân tích MobileNet SSD, YOLO và EfficientDet, không phụ thuộc vào một họ kiến trúc cụ thể.

2.2.1 Bài toán Object Detection

Phát hiện đối tượng (Object Detection) trả lời đồng thời hai câu hỏi: trong ảnh có những đối tượng thuộc lớp nào và chúng nằm ở đâu. Vì vậy, khác với phân loại ảnh chỉ sinh một hoặc một số nhãn cho toàn ảnh, đầu ra của bộ phát hiện là một tập

$$\mathcal{D} = \{(b_i, c_i, s_i)\}_{i=1}^M, \quad (2.1)$$

trong đó b_i là hộp giới hạn, c_i là nhãn lớp và $s_i \in [0, 1]$ là độ tin cậy của dự đoán thứ i . Số lượng đối tượng M thay đổi theo từng khung hình. Tổng quan của Zou và cộng sự phân Object Detection thành hai nhóm lớn: detector hai giai đoạn, trước hết sinh vùng ứng viên rồi phân loại/tinh chỉnh vùng; và detector một giai đoạn, dự đoán trực tiếp lớp cùng vị trí trên bản đồ đặc trưng [53].

Trong đồ án, yêu cầu đầu ra không dừng ở việc định vị người bằng bounding box. Bộ ước lượng tư thế còn trả về các điểm khớp của cơ thể; bounding box hỗ trợ theo dõi và điều khiển bám mục tiêu, trong khi tọa độ khớp được dùng để tạo chuỗi đặc trưng cho mô hình LSTM. Do đó, phát hiện một người trong từng khung hình mới là tầng cảm nhận ban đầu, chưa đồng nghĩa với kết luận người đó đã té ngã.

2.2.2 Lịch sử phát triển Object Detection

Quá trình phát triển của Object Detection có thể khái quát qua ba giai đoạn [53]:

- **Đặc trưng thủ công và cửa sổ trượt:** các bộ phát hiện như Viola–Jones, HOG và DPM sử dụng đặc trưng được thiết kế trước kết hợp bộ phân loại. Chúng đặt nền móng cho phát hiện khuôn mặt và người đi bộ, nhưng việc quét nhiều vị trí, tỉ lệ và thiết kế đặc trưng riêng cho từng bài toán làm hạn chế khả năng mở rộng.
- **Detector hai giai đoạn dựa trên học sâu:** R-CNN áp dụng CNN lên các vùng ứng viên [54]; Faster R-CNN đưa mạng sinh vùng đề xuất vào cùng kiến trúc, cải thiện đáng kể tính thống nhất và tốc độ [55]. Nhóm này thường có lợi thế về độ chính xác

định vị nhưng chi phí suy luận và bộ nhớ có thể lớn đối với thiết bị di động.

- **Detector một giai đoạn và tối ưu cho thiết bị biên:** SSD dự đoán đối tượng trên nhiều bản đồ đặc trưng chỉ trong một lần suy luận [56]; YOLO hợp nhất bài toán thành ánh xạ trực tiếp từ ảnh đến hộp và xác suất lớp [57]. Sau đó, các thiết kế backbone nhẹ, hợp nhất đặc trưng đa tỉ lệ và tìm kiếm kiến trúc tạo ra MobileNet SSD, EfficientDet và nhiều biến thể YOLO hướng đến cân bằng độ chính xác–độ trễ.

Xu hướng hiện nay không chỉ tối ưu độ chính xác trên tập kiểm thử mà còn quan tâm độ trễ, bộ nhớ, mức tiêu thụ năng lượng và khả năng chuyển đổi mô hình. Đây là các tiêu chí đặc biệt quan trọng khi camera và suy luận AI cùng hoạt động liên tục trên smartphone gắn trên robot.

2.2.3 Khái niệm Bounding Box

Bounding box là hình chữ nhật song song với các trục ảnh dùng để mô tả vị trí và kích thước đối tượng. Hai cách biểu diễn thường gặp là $(x_{min}, y_{min}, x_{max}, y_{max})$, với hai góc đối diện của hộp, và (x_c, y_c, w, h) , với tâm, chiều rộng và chiều cao. Tọa độ có thể ở đơn vị pixel hoặc được chuẩn hóa về $[0, 1]$ theo kích thước ảnh.

Với ảnh có kích thước $W \times H$, phép chuyển từ tọa độ hai góc sang dạng tâm đã chuẩn hóa được xác định bởi

$$x_c = \frac{x_{min} + x_{max}}{2W}, \quad y_c = \frac{y_{min} + y_{max}}{2H}, \quad (2.2)$$

$$w = \frac{x_{max} - x_{min}}{W}, \quad h = \frac{y_{max} - y_{min}}{H}. \quad (2.3)$$

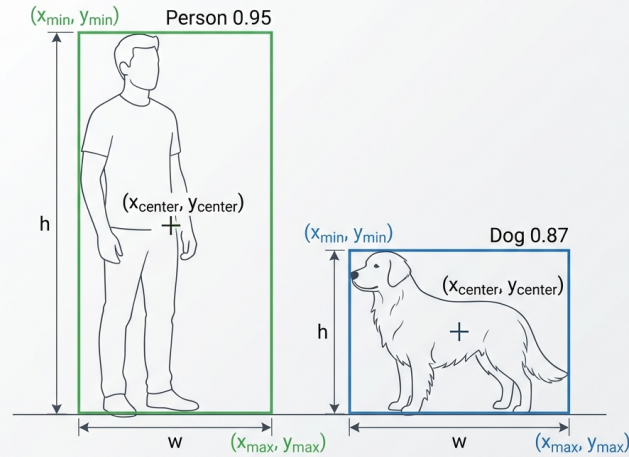
Khi triển khai, cần phân biệt hệ tọa độ của ảnh gốc, tensor đầu vào sau khi co giãn hoặc thêm biên và lớp hiển thị trên màn hình. Hộp dự đoán phải được ánh xạ ngược đúng tỉ lệ; nếu không, sai số tiền xử lý có thể làm lệch cả kết quả theo dõi lẫn lệnh quay của robot dù mô hình vẫn dự đoán chính xác trong không gian tensor.

2.2.4 Intersection over Union (IoU)

Intersection over Union đo mức chồng lấp giữa hộp dự đoán B_p và hộp nhãn chuẩn B_g :

$$\text{IoU}(B_p, B_g) = \frac{|B_p \cap B_g|}{|B_p \cup B_g|}. \quad (2.4)$$

Bounding Box Concepts in Object Detection



University Thesis Illustration: Computer Vision Fundamentals

Hình 2.1: Minh họa hộp giới hạn của đối tượng trong ảnh.

IoU bằng 0 khi hai hộp không giao nhau và bằng 1 khi trùng khít. Trong đánh giá, một dự đoán chỉ có thể ghép với nhãn chuẩn khi đúng lớp và IoU đạt ngưỡng quy định. Trong hậu xử lý, IoU còn được dùng bởi Non-Maximum Suppression (NMS) để loại các dự đoán trùng lặp; trong tracking-by-detection, nó có thể đóng vai trò độ tương đồng giữa hộp dự báo của track và detection mới.

Không có một ngưỡng IoU phù hợp cho mọi mục đích. PASCAL VOC thường báo cáo AP tại IoU 0,5 [58], trong khi giao thức COCO đánh giá trên dãy ngưỡng từ 0,50 đến 0,95, bước 0,05, nhằm phản ánh chặt chẽ hơn chất lượng định vị [59]. Vì vậy, mọi kết quả mAP phải đi kèm giao thức và ngưỡng IoU; không nên so sánh trực tiếp các con số được tính theo chuẩn khác nhau.

2.2.5 True Positive, False Positive, False Negative

Tại một lớp và một ngưỡng IoU xác định, các dự đoán được sắp theo độ tin cậy giảm dần rồi ghép một-một với nhãn chuẩn:

- **True Positive (TP)** là dự đoán đúng lớp, đạt ngưỡng IoU và được ghép với một nhãn chuẩn chưa có dự đoán nào nhận trước đó.
- **False Positive (FP)** là dự đoán sai lớp, không đạt ngưỡng IoU, không có nhãn tương

ứng hoặc là dự đoán trùng lặp cho một nhãn đã được ghép.

- **False Negative (FN)** là đối tượng có trong nhãn chuẩn nhưng không được bất kỳ dự đoán hợp lệ nào phát hiện.

Quy tắc ghép một–một là cần thiết: nếu nhiều hộp cùng bao quanh một người, chỉ hộp hợp lệ có độ tin cậy cao nhất được tính là TP, các hộp dư được tính là FP. Ba đại lượng này là cơ sở để xây dựng Precision, Recall, đường cong Precision–Recall, AP và cuối cùng là mAP.

2.2.6 Đánh giá hiệu suất Mean Average Precision (mAP)

Precision và Recall. Precision biểu thị mức đáng tin của các cảnh báo dương, còn Recall biểu thị khả năng tìm đủ các đối tượng thực sự tồn tại:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}. \quad (2.5)$$

Tăng ngưỡng confidence thường loại bớt dự đoán yếu, có thể làm Precision tăng nhưng Recall giảm. Với hệ thống chăm sóc người cao tuổi, Recall thấp có nghĩa là nguy cơ bỏ sót người hoặc sự kiện cần giám sát; ngược lại, Precision thấp làm tăng nhiễu cho tracker và các bước xử lý sau. Vì thế không thể đánh giá detector chỉ bằng một trong hai chỉ số.

Đường cong Precision–Recall. Với từng lớp, các dự đoán được sắp theo confidence giảm dần. Khi lần lượt chấp nhận thêm dự đoán, cặp $(\text{Recall}, \text{Precision})$ thay đổi và tạo thành đường cong Precision–Recall (PR). Đường cong mô tả hiệu năng trên toàn bộ miền ngưỡng confidence, thay vì tại một ngưỡng vận hành duy nhất.

Average Precision. AP tóm tắt đường cong PR của một lớp tại một ngưỡng khớp xác định. Với bao chính xác nội suy $p_{\text{interp}}(r) = \max_{r' \geq r} p(r')$, AP có thể được diễn đạt như diện tích dưới đường cong:

$$AP = \int_0^1 p_{\text{interp}}(r) dr. \quad (2.6)$$

Trong tính toán rời rạc, giao thức đánh giá quy định cụ thể các mức recall lấy mẫu và cách nội suy. Do đó AP không chỉ phụ thuộc mô hình mà còn phụ thuộc tập dữ liệu và quy ước đánh giá.

Mean Average Precision. Với tập C lớp và một ngưỡng IoU τ , mAP là trung

bình AP trên các lớp:

$$mAP_{\tau} = \frac{1}{|C|} \sum_{c \in C} AP_{c,\tau}. \quad (2.7)$$

Theo COCO, chỉ số chính tiếp tục lấy trung bình trên mười ngưỡng $T = \{0,50, 0,55, \dots, 0,95\}$ [59]:

$$mAP_{[.50:.95]} = \frac{1}{|C||T|} \sum_{c \in C} \sum_{\tau \in T} AP_{c,\tau}. \quad (2.8)$$

AP_{50} dễ đạt hơn AP_{75} vì chấp nhận hộp ít khít hơn; $mAP_{[.50:.95]}$ phản ánh đồng thời khả năng nhận biết và chất lượng định vị. Ngoài ra, COCO còn tách kết quả theo đối tượng nhỏ, vừa và lớn. Đối với mô hình pose, chuỗi Precision–Recall–AP–mAP vẫn được giữ nguyên nhưng điều kiện ghép keypoint thường dựa trên Object Keypoint Similarity (OKS), vì vậy cần phân biệt rõ box mAP và pose mAP khi trình bày benchmark.

2.2.7 Các kiến trúc phát hiện đối tượng phù hợp thiết bị di động

Sau khi thống nhất đầu ra, quy tắc ghép và cách đánh giá, có thể so sánh các kiến trúc ứng viên trên cùng hệ tiêu chí. Ba hướng phổ biến đối với ứng dụng Android là MobileNet SSD, YOLO và EfficientDet.

MobileNet SSD. SSD thực hiện dự đoán hộp và lớp trên nhiều mức bản đồ đặc trưng để xử lý các kích thước đối tượng khác nhau [56]. Khi ghép với MobileNet—backbone sử dụng tích chập tách theo chiều sâu nhằm giảm lượng phép tính—mô hình có kích thước nhỏ, dễ lượng tử hóa và phù hợp với TensorFlow Lite [60]. Hạn chế là cấu hình chuẩn chủ yếu trả về bounding box; nếu cần 17 điểm khớp cơ thể, hệ thống phải bổ sung một mô hình pose hoặc thay đổi đầu dự đoán.

YOLO. YOLO mô hình hóa detection như một mạng hợp nhất, dự đoán trực tiếp vị trí và xác suất lớp trong một lần forward [57]. Các biến thể nhỏ phù hợp với xử lý thời gian thực, đồng thời họ mô hình có phiên bản pose trả về cả bounding box của người và keypoint. Lợi thế này giúp tránh một tầng ước lượng tư thế riêng; đổi lại, việc chuyển đổi sang TensorFlow Lite và giải mã đầu ra phải khớp đúng phiên bản xuất mô hình, còn tốc độ thực tế phụ thuộc thiết bị, kích thước đầu vào và delegate được sử dụng.

EfficientDet. EfficientDet kết hợp backbone EfficientNet, mạng hợp nhất đặc trưng hai chiều có trọng số (BiFPN) và compound scaling để đồng thời thay đổi độ sâu, độ rộng và độ phân giải [61]. Họ mô hình tạo nhiều mức quy mô để lựa chọn theo ngân

sách tính toán. Tuy nhiên, tương tự SSD chuẩn, EfficientDet là detector hộp; pipeline cần thêm thành phần pose nếu muốn tạo trực tiếp chuỗi khớp cho LSTM.

Bảng 2.2: So sánh định tính các họ detector liên quan đến đề án

Họ mô hình	Đặc điểm chính	Ưu điểm khi triển khai Android	Hạn chế đối với bài toán của đề án
MobileNet SSD	SSD một giai đoạn với backbone nhẹ	Mô hình gọn; hệ sinh thái TensorFlow Lite phổ biến; dễ tạo baseline phát hiện người	Đầu ra chuẩn chỉ có hộp và lớp; cần thêm tầng pose để cung cấp keypoint cho LSTM
YOLO	Detector hộp nhất; có các biến thể detection và pose	Cân bằng tốc độ–độ chính xác; biến thể pose tạo đồng thời bounding box và keypoint của người	Hậu xử lý và tensor đầu ra phụ thuộc phiên bản export; cần kiểm chứng trên đúng thiết bị đích
EfficientDet	EfficientNet, BiFPN và compound scaling	Có nhiều mức mô hình; khai thác đặc trưng đa tỉ lệ hiệu quả	Phiên bản chuẩn không sinh keypoint; cấu hình lớn có thể vượt ngân sách độ trễ của smartphone

Các giá trị FPS hoặc mAP công bố trong những bài báo khác nhau không được đặt cạnh nhau như một phép so sánh trực tiếp nếu khác tập dữ liệu, độ phân giải, phần cứng và giao thức đo. Với yêu cầu của đề án, tiêu chí quyết định là khả năng tạo đồng thời bounding box và 17 keypoint của người trong một mô hình đủ nhẹ để chạy trên Android. Vì vậy hướng YOLO-Pose được chọn để huấn luyện và chuyển đổi sang TensorFlow Lite; phiên bản mô hình, delegate và số đo thực nghiệm được trình bày tại Chương 4 và Chương 5.

2.2.8 Theo dõi đối tượng và tạo chuỗi thời gian

Detector xử lý từng khung hình độc lập nên chưa bảo đảm cùng một người giữ nguyên định danh theo thời gian. Tracking-by-detection bổ sung bước dự báo trạng thái và liên kết detection để tạo tracklet. SORT sử dụng bộ lọc Kalman cho dự báo chuyển động, IoU cho độ tương đồng và thuật toán Hungarian cho bài toán gán tối ưu [27]. Deep SORT bổ sung embedding ngoại hình nhằm giảm đổi ID khi che khuất [31]; ByteTrack khai thác cả detection có confidence thấp trong bước liên kết thứ hai để phục hồi đối tượng khó quan sát [32].

Với trạng thái x_t , ma trận chuyển trạng thái F , điều khiển u_t và quan sát z_t , hai pha dự báo–hiệu chỉnh của bộ lọc Kalman rời rạc được biểu diễn như sau [28][29]:

$$\hat{x}_{t|t-1} = F\hat{x}_{t-1|t-1} + Bu_t, \quad (2.9)$$

$$P_{t|t-1} = FP_{t-1|t-1}F^T + Q, \quad (2.10)$$

$$K_t = P_{t|t-1}H^T(H P_{t|t-1}H^T + R)^{-1}, \quad (2.11)$$

$$\hat{x}_{t|t} = \hat{x}_{t|t-1} + K_t(z_t - H\hat{x}_{t|t-1}), \quad (2.12)$$

$$P_{t|t} = (I - K_tH)P_{t|t-1}. \quad (2.13)$$

Ma trận chi phí giữa các track và detection được giải bằng thuật toán Hungarian [30]. Tuy nhiên, IoU đơn lẻ có thể suy giảm mạnh khi người chuyển nhanh từ tư thế đứng sang nằm. Theo mã nguồn hiện tại, đồ án dùng SORT được mở rộng bằng khoảng cách tâm, tỉ lệ kích thước, khoảng cách pose và histogram vùng thân; mục tiêu chính được gán ID0, có tuổi track dài hơn và cơ chế phục hồi có điều kiện. Cách lựa chọn này giữ pipeline nhẹ trên Android nhưng vẫn cung cấp chuỗi keypoint nhất quán cho LSTM và tín hiệu vị trí ổn định cho bộ điều khiển bám người.

2.3 Họ mô hình YOLO và YOLO26-Pose

2.3.1 Quá trình phát triển của họ YOLO

YOLOv1 đặt nền móng cho detector một giai đoạn bằng cách dự đoán đồng thời hộp và xác suất lớp từ toàn ảnh [57]. Từ ý tưởng này, nhiều nhánh nghiên cứu và hiện thực cùng sử dụng tên YOLO nhưng không phải tất cả đều do một nhóm tác giả phát triển liên tục. Vì vậy, số phiên bản chỉ nên được xem là một phần của định danh; khi so sánh còn phải xác định đơn vị phát triển, nhiệm vụ, kích thước mô hình, tập dữ liệu và chế độ xuất.

Những mốc có ảnh hưởng trực tiếp đến lựa chọn mô hình trong đồ án được tóm tắt như sau:

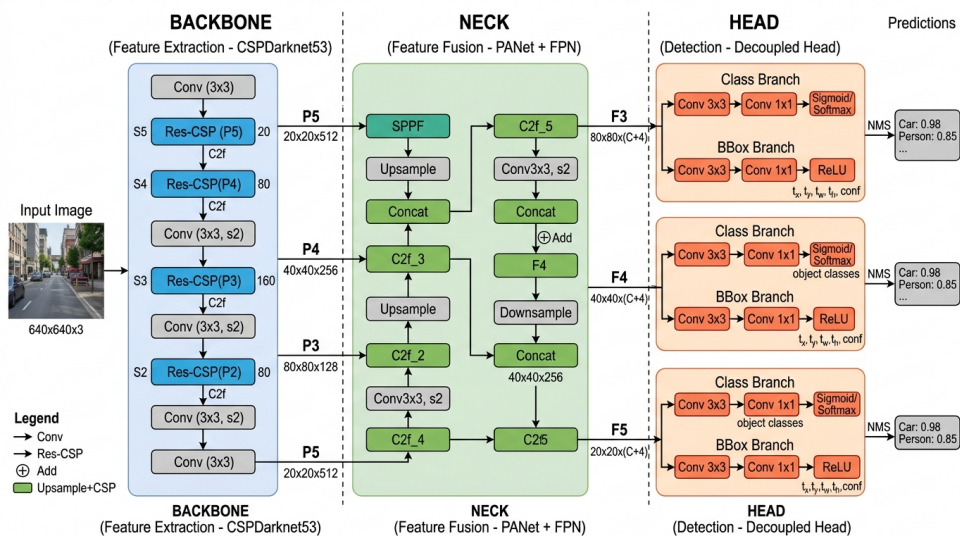
- **YOLOv1** hợp nhất định vị và phân loại trong một mạng, ưu tiên tốc độ thời gian thực nhưng còn hạn chế với vật thể nhỏ và các đối tượng nằm gần nhau [57].
- **YOLOv3** tăng cường backbone và dự đoán ở nhiều tỉ lệ, giúp xử lý tốt hơn sự thay đổi kích thước đối tượng [62].
- **YOLOv5** nhấn mạnh tính thực dụng của pipeline huấn luyện–suy luận–xuất mô

hình, tạo tiền đề cho việc triển khai YOLO trên nhiều runtime [63].

- **YOLOv8** chuyển sang đầu dự đoán anchor-free và tách nhánh hồi quy hợp với phân loại, giảm phụ thuộc vào thiết kế anchor thủ công [64].
- **YOLOv10** sử dụng hai chiến lược gán nhãn one-to-many và one-to-one để huấn luyện detector end-to-end không cần NMS ở đường suy luận [65].
- **YOLO11** tiếp tục tinh chỉnh các khối trích xuất và hợp nhất đặc trưng, đồng thời duy trì nhiều tác vụ như detection, segmentation và pose [66].
- **YOLO26** kế thừa nền tảng YOLO11 nhưng tập trung vào suy luận end-to-end, đầu hồi quy nhẹ và các đầu dự đoán chuyên biệt cho từng tác vụ [67].

2.3.2 Cấu trúc backbone–neck–head

Dù chi tiết từng phiên bản khác nhau, các mô hình YOLO hiện đại có thể được đọc theo ba khối. Backbone biến ảnh thành các bản đồ đặc trưng phân cấp; các lớp nông giữ nhiều chi tiết vị trí, còn lớp sâu biểu diễn ngữ nghĩa mạnh hơn. Neck hợp nhất đặc trưng theo cả hướng từ sâu đến nông và từ nông đến sâu để tạo các mức P3, P4, P5. Với stride lần lượt là 8, 16 và 32, mỗi phần tử trên các mức này quan sát một vùng ảnh có kích thước và ngữ cảnh khác nhau. Head chuyển đặc trưng đa tỉ lệ thành dự đoán hộp, độ tin cậy, lớp và—đối với biến thể pose—tọa độ keypoint. Cách phân tách này hữu ích khi đọc tệp YAML của mô hình, kiểm tra lớp không được runtime hỗ trợ hoặc xác định phần đầu ra cần giải mã [64].



Hình 2.2: Cấu trúc khái quát của một detector YOLO gồm backbone, neck và head.

2.3.3 Các thay đổi chính của YOLO26

YOLO26 sử dụng kiến trúc hai đầu trong quá trình huấn luyện. Đầu one-to-many tạo nhiều ứng viên dương cho mỗi đối tượng, cung cấp tín hiệu học dày; đầu one-to-one học ánh xạ một dự đoán chính cho mỗi nhãn chuẩn. Ở chế độ end-to-end mặc định, chỉ đầu one-to-one được giữ lại khi suy luận, trả về tối đa 300 detection và không cần NMS. Khi ưu tiên độ chính xác hoặc runtime không hỗ trợ toán tử top- K của đường end-to-end, mô hình có thể dùng đầu one-to-many và NMS như detector dense thông thường [67].

YOLO26 đồng thời loại Distribution Focal Loss (DFL) khỏi đầu hồi quy hộp, thể hiện trong cấu hình bởi `reg_max=1`. Thay vì dự đoán phân phối trên nhiều bin cho mỗi cạnh hộp, đầu mới hồi quy trực tiếp, nhờ đó giảm số kênh và chi phí giải mã. Trong huấn luyện, MuSGD được dùng để cải thiện hội tụ; Progressive Loss dịch dần trọng số giám sát từ đầu one-to-many sang one-to-one; STAL bảo đảm các đối tượng rất nhỏ vẫn nhận được ứng viên dương. Ba kỹ thuật sau tác động đến quá trình tối ưu trọng số, không làm tăng giao diện đầu vào của ứng dụng Android [67].

2.3.4 Quy trình suy luận và hậu xử lý YOLO-Pose

Quy trình suy luận YOLO-Pose bắt đầu bằng việc sửa hướng ảnh, chuyển về thứ tự kênh màu mà mô hình yêu cầu, resize hoặc letterbox và chuẩn hóa miền giá trị. Tensor ảnh sau đó đi qua interpreter để tạo tensor dự đoán. Với các mức đặc trưng có stride $s \in \mathcal{S}$, số vị trí ứng viên của đầu dense có dạng tổng quát

$$N = \sum_{s \in \mathcal{S}} \left(\frac{W_i}{s} \right) \left(\frac{H_i}{s} \right). \quad (2.14)$$

Mỗi ứng viên chứa thông tin hộp, score/lớp và $3K$ giá trị keypoint. Tensor có thể sắp theo $[1, N, D]$ hoặc $[1, D, N]$, vì vậy kích thước và thứ tự trục phải được đọc trước khi duyệt các detection.

Ở đầu raw, hậu xử lý đổi bounding box sang dạng $xyxy$, lọc score và áp dụng NMS để loại các hộp trùng. Ở đầu end-to-end, mô hình đã chọn một tập dự đoán cuối nên thường không cần NMS bên ngoài. Sau đó, bộ giải mã tách các bộ ba keypoint, lọc theo confidence và ánh xạ đồng nhất cả hộp lẫn keypoint về không gian ảnh nguồn. Kết quả có cấu trúc mới được chuyển cho tracker, mô hình chuỗi thời gian hoặc tầng hiển thị. Trình tự này giúp cô lập sự khác nhau giữa các exporter ở bộ giải mã, thay vì để định dạng tensor lan sang toàn bộ ứng dụng.

2.3.5 Input và Output của YOLO-Pose

Input. Về mặt logic, đầu vào của YOLO-Pose là tensor ảnh $\mathbf{X}$ có batch B , chiều cao H_i , chiều rộng W_i và ba kênh màu. Khi triển khai, cần xác định chính xác thứ tự chiều mà runtime yêu cầu, chẳng hạn channels-last $[B, H_i, W_i, 3]$ hoặc channels-first $[B, 3, H_i, W_i]$. Ảnh phải được sửa đúng hướng, chuyển sang RGB, resize hoặc letterbox và biến đổi về kiểu dữ liệu cũng như miền giá trị khớp với lúc export. Mô hình float thường nhận float32 đã chuẩn hóa về $[0, 1]$; mô hình lượng tử có thể nhận uint8 hoặc int8 cùng scale và zero-point riêng. Sai thứ tự RGB/BGR hoặc chuẩn hóa lặp hai lần không làm tensor sai shape nhưng có thể làm giảm mạnh chất lượng dự đoán.

Kích thước $W_i \times H_i$ quyết định mức chi tiết mà backbone quan sát và số vị trí dự đoán ở các đầu đa tỉ lệ. Độ phân giải lớn có lợi cho người ở xa hoặc keypoint nhỏ nhưng làm tăng số phép tính, bộ nhớ trung gian và độ trễ gần theo diện tích ảnh. Với các mức đặc trưng có stride 8, 16 và 32, kích thước đầu vào thường được chọn chia hết cho 32 để các bản đồ đặc trưng có kích thước nguyên. Resize trực tiếp đơn giản nhưng có thể làm biến dạng tỉ lệ cơ thể; letterbox giữ tỉ lệ ảnh nguồn nhưng bổ sung vùng đệm cần được loại khỏi tọa độ khi hậu xử lý.

Output. YOLO26-Pose dự đoán đồng thời người và tư thế trong một lần suy luận, thay vì phát hiện người rồi chạy một mạng pose riêng trên từng vùng. Với cấu hình COCO, mỗi detection có $K = 17$ keypoint và mỗi điểm được biểu diễn bởi

$$k_j = (x_j, y_j, p_j), \quad j = 1, \dots, 17, \quad (2.15)$$

trong đó (x_j, y_j) là tọa độ và $p_j \in [0, 1]$ là confidence. Tập cấu hình chính thức mô tả giao diện này bằng `kpt_shape=[17, 3]` [68]. Trong huấn luyện, YOLO26 kết hợp loss dựa trên OKS với Residual Log-Likelihood Estimation (RLE): OKS chuẩn hóa sai số theo kích thước người và độ khó của từng loại khớp, còn RLE mô hình hóa phần dư và độ bất định của tọa độ [67]. Các thành phần loss thuộc giai đoạn huấn luyện; đầu ra suy luận vẫn là hộp, score/lớp và các bộ ba keypoint.

Bounding box có thể ở dạng tâm–kích thước $B_{cxcywh} = (c_x, c_y, w_b, h_b)$ hoặc dạng hai góc $B_{xxyy} = (x_{\min}, y_{\min}, x_{\max}, y_{\max})$. Phép chuyển đổi là

$$\begin{aligned} x_{\min} &= c_x - \frac{w_b}{2}, & y_{\min} &= c_y - \frac{h_b}{2}, \\ x_{\max} &= c_x + \frac{w_b}{2}, & y_{\max} &= c_y + \frac{h_b}{2}. \end{aligned} \quad (2.16)$$

Dạng $xyxy$ phù hợp để tính IoU, cắt vùng ảnh và kiểm tra biên; dạng $cxcywh$ thuận tiện cho đầu hồi quy. Mỗi bounding box liên kết với đúng bộ keypoint của detection đó. Keypoint không phải tọa độ cục bộ tính từ góc hộp mà nằm trong cùng hệ tọa độ ảnh; hộp mô tả phạm vi tổng thể, còn keypoint mô tả cấu trúc hình học bên trong.

Tọa độ output có thể ở đơn vị pixel hoặc được chuẩn hóa về $[0, 1]$ theo $x_n = x/W$, $y_n = y/H$. Nếu ảnh nguồn (W_s, H_s) được resize trực tiếp thành ảnh mô hình (W_i, H_i) , phép ánh xạ ngược là

$$x_s = x_i \frac{W_s}{W_i}, \quad y_s = y_i \frac{H_s}{H_i}. \quad (2.17)$$

Với letterbox, cần trừ phần đệm (d_x, d_y) rồi chia hệ số $r = \min(W_i/W_s, H_i/H_s)$. Với ảnh phản chiếu ngang, điểm đổi thành $x' = W - x$ và bounding box phải hoán đổi hai cạnh theo $x'_{\min} = W - x_{\max}$, $x'_{\max} = W - x_{\min}$. Mọi phép biến đổi phải được áp dụng đồng nhất cho hộp và toàn bộ keypoint. Confidence của từng điểm cũng cần được giữ lại để các tầng sau loại hoặc giảm ảnh hưởng của khớp bị che khuất.

Thứ tự keypoint phải được giữ nhất quán từ dữ liệu huấn luyện, mô hình đến bộ giải mã:

Bảng 2.3: Thứ tự keypoint người theo cấu hình COCO-Pose của YOLO26

Chỉ số	Điểm cơ thể	Chỉ số	Điểm cơ thể
0	Mũi	9	Cổ tay trái
1	Mắt trái	10	Cổ tay phải
2	Mắt phải	11	Hông trái
3	Tai trái	12	Hông phải
4	Tai phải	13	Gối trái
5	Vai trái	14	Gối phải
6	Vai phải	15	Cổ chân trái
7	Khuỷu tay trái	16	Cổ chân phải
8	Khuỷu tay phải	—	—

2.3.6 Các mức kích thước của YOLO26-Pose

Họ mô hình cung cấp năm mức n, s, m, l và x. Tăng mức mô hình thường làm tăng khả năng biểu diễn và pose mAP, đồng thời tăng số tham số, phép tính, bộ nhớ và độ trễ. Bảng 2.4 sử dụng kết quả E2E chính thức tại đầu vào 640×640 trên COCO Keypoints val2017 [67]; số liệu CPU ONNX và T4 TensorRT chỉ cho thấy xu hướng giữa các mức,

không phải tốc độ dự kiến trên Android/TensorFlow Lite.

Bảng 2.4: So sánh các mức YOLO26-Pose ở chế độ end-to-end trên COCO

Mô hình	Tham số (M)	GFLOPs	Pose mAP	CPU ONNX (ms)	T4 TRT (ms)
YOLO26n-Pose	2,9	7,5	57,2	40,3	1,8
YOLO26s-Pose	10,4	23,9	63,0	85,3	2,7
YOLO26m-Pose	21,5	73,1	68,8	218,0	5,0
YOLO26l-Pose	25,9	91,3	70,4	275,4	6,5
YOLO26x-Pose	57,6	201,7	71,6	565,4	12,2

Mức nano có chi phí thấp nhất trong khi vẫn giữ đầy đủ đầu pose; các mức lớn hơn đòi hỏi thêm tham số và độ trễ để đạt mAP cao hơn. Vì vậy, lựa chọn cho một hệ thống cụ thể không chỉ dựa vào mAP lớn nhất mà phải xét đồng thời độ phân giải, tần số xử lý, bộ nhớ, nhiệt độ thiết bị và chất lượng chuỗi keypoint cung cấp cho tác vụ phía sau.

2.4 Mô hình chuỗi thời gian (Sequence Modeling)

2.4.1 Mạng nơ-ron hồi quy (RNN)

Mạng nơ-ron hồi quy (Recurrent Neural Network - RNN) là một lớp mạng nơ-ron nhân tạo chuyên biệt để xử lý dữ liệu dạng chuỗi (sequence data) như văn bản, âm thanh, hoặc chuỗi thời gian (time-series). Trong ngữ cảnh phát hiện té ngã, luồng video chứa các khung hình liên tiếp tạo thành một chuỗi dữ liệu thời gian ghi nhận chuyển động của con người [69].

Khác với mạng truyền thẳng (Feedforward Neural Network) nơi dữ liệu chỉ đi một chiều từ đầu vào đến đầu ra, RNN sở hữu các "vòng lặp" (loops) cho phép thông tin có thể lưu lại và truyền qua các bước thời gian. Nhờ cơ chế trạng thái ẩn (hidden state), mạng RNN có thể nhớ được thông tin từ quá khứ để ảnh hưởng đến quyết định ở hiện tại. Tuy nhiên, RNN truyền thống gặp phải vấn đề **triệt tiêu đạo hàm (vanishing gradient)** và **bùng nổ đạo hàm (exploding gradient)** khi phải học các phụ thuộc xa (long-term dependencies) trong những chuỗi dữ liệu dài.

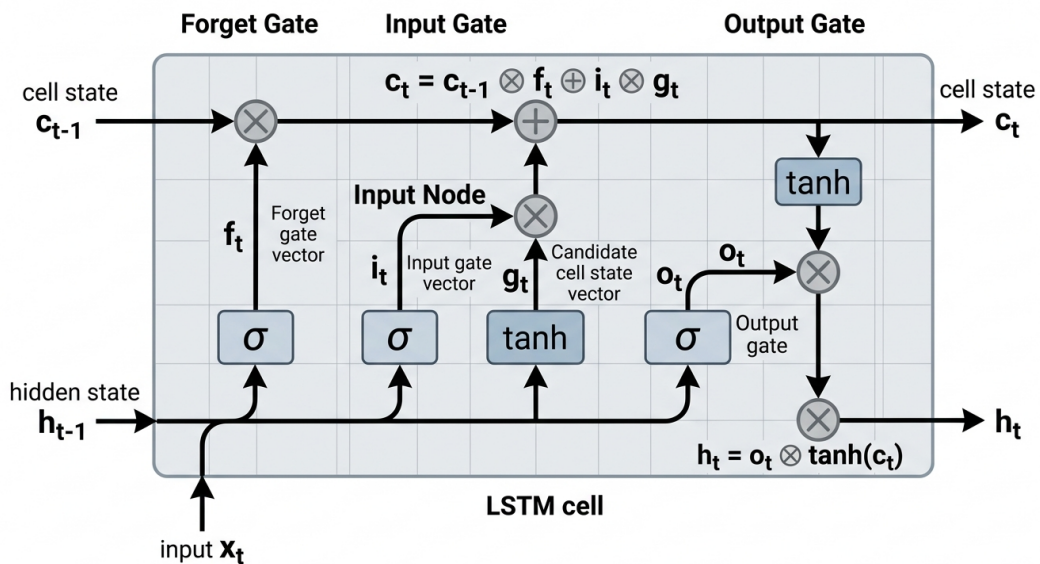
2.4.2 Mạng Bộ nhớ Ngắn-Dài hạn (LSTM)

Để khắc phục nhược điểm của RNN, mạng **Long Short-Term Memory (LSTM)** được đề xuất bởi Hochreiter và Schmidhuber (1997) [25]. LSTM có khả năng học các phụ thuộc dài hạn cực kỳ xuất sắc nhờ cấu trúc "Cell State" (trạng thái tế bào) hoạt động như một băng chuyền thông tin xuyên suốt các bước thời gian, giúp đạo hàm không bị

suy giảm quá nhanh.

Cốt lõi của kiến trúc LSTM nằm ở ba cổng (gates) đóng vai trò kiểm soát luồng thông tin:

- **Cổng quên (Forget Gate):** Quyết định lượng thông tin từ quá khứ (trạng thái tế bào trước đó) cần bị loại bỏ hoặc lãng quên.
- **Cổng đầu vào (Input Gate):** Quyết định lượng thông tin mới (từ dữ liệu hiện tại và trạng thái ẩn trước đó) nào sẽ được cập nhật vào trạng thái tế bào.
- **Cổng đầu ra (Output Gate):** Quyết định thông tin nào từ trạng thái tế bào sẽ được xuất ra dưới dạng trạng thái ẩn mới cho bước thời gian tiếp theo.



Hình 2.3: Cấu trúc chi tiết của một tế bào LSTM (LSTM cell) với hệ thống các cổng kiểm soát thông tin.

Trong kiến trúc phát hiện té ngã, YOLO-Pose trích xuất các điểm khớp ở từng khung hình và bộ theo dõi SORT mở rộng liên kết chúng theo đúng người. LSTM tiếp nhận chuỗi tọa độ đã chuẩn hóa để học sự biến đổi tư thế theo thời gian. Nhờ vậy, quyết định không chỉ dựa trên một ảnh tức thời mà còn xét quá trình chuyển động, hỗ trợ phân biệt té ngã với các hoạt động gần giống như ngồi xuống, cúi người hoặc chủ động nằm.

Một detector trên từng ảnh chỉ quan sát tư thế tại một thời điểm. Trong bài toán té ngã, quá trình chuyển từ đứng/ngồi sang hạ thấp nhanh rồi nằm mới là tín hiệu quan trọng. Recurrent Neural Network (RNN) đưa trạng thái ẩn từ bước $t - 1$ sang bước t ,

nhưng có thể gặp mất hoặc bùng nổ gradient khi chuỗi dài. Long Short-Term Memory (LSTM) bổ sung cell state và ba cổng vào, quên, ra để điều tiết thông tin [25].

Với input x_t , hidden state h_{t-1} và cell state c_{t-1} :

$$f_t = \sigma(W_f[x_t, h_{t-1}] + b_f), \quad (2.18)$$

$$i_t = \sigma(W_i[x_t, h_{t-1}] + b_i), \quad (2.19)$$

$$\tilde{c}_t = \tanh(W_c[x_t, h_{t-1}] + b_c), \quad (2.20)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \quad (2.21)$$

$$o_t = \sigma(W_o[x_t, h_{t-1}] + b_o), \quad (2.22)$$

$$h_t = o_t \odot \tanh(c_t). \quad (2.23)$$

Trong hệ thống, một mẫu là tensor $X \in \mathbb{R}^{15 \times 69}$. Số 15 là chiều thời gian; 69 là vector pose/hình học mỗi frame. Đầu ra softmax hai lớp được diễn giải là normal/fall. Threshold xác suất chỉ là tầng đầu; hysteresis theo số frame liên tiếp và hậu kiểm hình học giúp ngăn trạng thái dao động. Cần lưu ý nếu FPS thay đổi, 15 frame tương ứng thời gian thực khác nhau; một triển khai tốt nên log timestamp hoặc resample chuỗi theo thời gian.

2.5 Giới thiệu Dataset Fall Detection – LE2I

2.5.1 Mục tiêu và đặc điểm của dataset LE2I

LE2I Fall Detection Dataset là một trong những bộ dữ liệu chuẩn mực và phổ biến nhất dành cho bài toán phát hiện té ngã, được phát triển bởi phòng thí nghiệm Le2i (Đại học Burgundy, Pháp). Bộ dữ liệu cung cấp các video mô phỏng các hoạt động thường ngày (Activities of Daily Living - ADL) và các tình huống té ngã đa dạng [70].

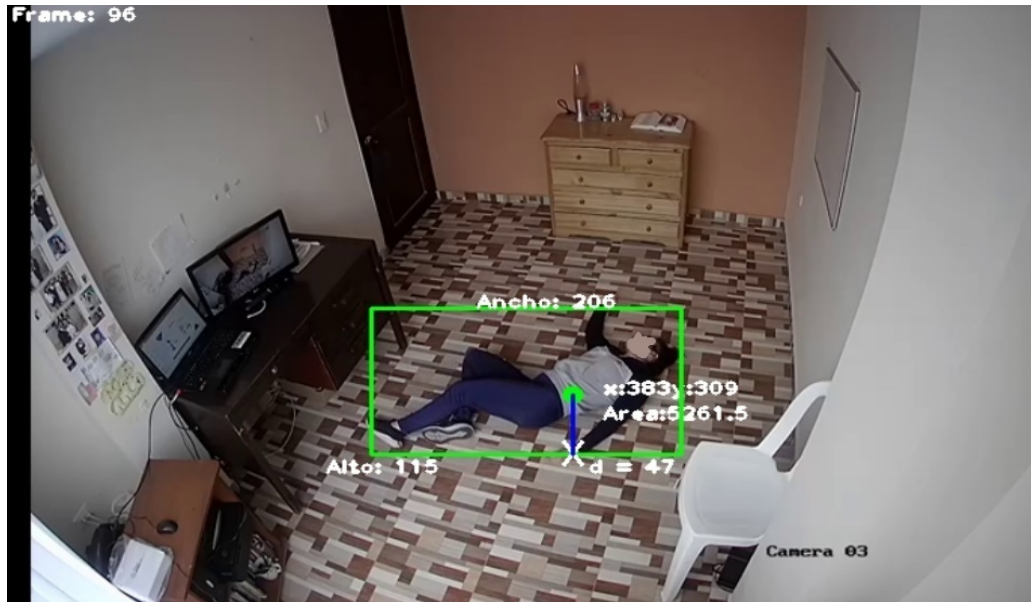
Đặc điểm nổi bật của dataset LE2I:

- **Môi trường thực tế:** Được quay ở 4 bối cảnh khác nhau bao gồm: Nhà ở (Home), Phòng cafe (Coffee room), Văn phòng (Office) và Phòng học (Lecture room).
- **Đa dạng kịch bản ngã:** Bao gồm ngã về phía trước, phía sau, ngã do mất thăng bằng, hoặc ngã khi đang cố gắng đứng lên.
- **Nhiều tự nhiên:** Video chứa các thách thức thực tế như đổ bóng (shadows), thay

đổi ánh sáng (illumination variations), và che khuất (occlusions) do đồ vật.

2.5.2 Dữ liệu video và Annotation

Dataset LE2I bao gồm khoảng 191 video với các đặc điểm:



Hình 2.4: Một số khung hình minh họa trong môi trường của dataset LE2I.

- Độ phân giải video là 320x240 pixels với tốc độ khung hình 25 FPS.
- Annotation cung cấp khung hình bắt đầu và kết thúc quá trình ngã (ground truth frame numbers), kết hợp với thông tin bounding box của người.
- Các nhãn cơ bản gồm trạng thái ngã (fall) và hoạt động bình thường (no_fall).

2.5.3 Ưu điểm và hạn chế của dataset LE2I

Ưu điểm:

- Bối cảnh phong phú, từ không gian chật hẹp (văn phòng) đến rộng rãi (phòng học).
- Các tình huống ngã được diễn viên thực hiện tự nhiên, bao gồm các hoạt động khó phân biệt như cúi nhặt đồ hoặc ngồi xuống nhanh.
- Là chuẩn đối sánh (benchmark) uy tín được sử dụng rộng rãi trong các nghiên cứu Fall Detection.

Hạn chế:

- Độ phân giải video tương đối thấp (320x240) so với các chuẩn camera HD hiện đại.
- Số lượng chủ thể (diễn viên) còn hạn chế so với các dataset quy mô lớn gần đây.

2.5.4 So sánh với các Dataset quy mô lớn (DiverseFALL10500)

Gần đây, một số nghiên cứu đề xuất các bộ dữ liệu không lồ như DiverseFALL10500 (10,500 ảnh) để khắc phục nhược điểm của các dataset cũ [10].

So sánh LE2I và DiverseFALL10500:

Tiêu chí	LE2I Dataset	DiverseFALL10500
Số lượng video/ảnh	191 videos	10,500 images
Môi trường	4 môi trường trong nhà	Indoor + Outdoor
Độ phân giải	320x240	Đa dạng (HD)
Yếu tố che khuất	Tích hợp sẵn (bàn, ghế)	Đa dạng

Đồ án này lựa chọn sử dụng (hoặc kết hợp) LE2I do đặc thù bộ dữ liệu này phản ánh rất sát các kịch bản trong môi trường nhà ở và có chứa chuỗi chuyển động thời gian (video), vô cùng phù hợp để trích xuất Tracklet cho mô hình LSTM.

2.6 Suy luận AI trên thiết bị và TensorFlow Lite

2.6.1 Suy luận AI trên thiết bị

Suy luận AI trên thiết bị, còn gọi là *on-device AI* hoặc *Edge AI*, là quá trình thực thi mô hình ngay tại nơi dữ liệu được thu nhận thay vì chuyển toàn bộ dữ liệu đến máy chủ đám mây. Cách tiếp cận này giúp rút ngắn đường truyền dữ liệu, giảm độ trễ phản hồi và cho phép một số chức năng tiếp tục hoạt động khi kết nối mạng không ổn định. Dữ liệu đầu vào được xử lý cục bộ cũng làm giảm phạm vi truyền tải thông tin nhạy cảm ra ngoài thiết bị.

Đổi lại, thiết bị biên bị giới hạn về năng lực tính toán, bộ nhớ, dung lượng pin và khả năng tản nhiệt. Mô hình triển khai vì vậy phải đạt sự cân bằng giữa độ chính xác, kích thước, thời gian suy luận và mức tiêu thụ tài nguyên. Hiệu năng không chỉ phụ thuộc số lượng tham số hoặc phép toán mà còn chịu ảnh hưởng bởi kích thước tensor trung gian, khả năng tối ưu toán tử và đặc điểm phần cứng của thiết bị.

2.6.2 Kiến trúc và quy trình suy luận TensorFlow Lite

TensorFlow Lite, hiện được Google tiếp tục phát triển trong hệ sinh thái LiteRT, là nền tảng dành cho việc triển khai mô hình học máy trên thiết bị di động và thiết bị biên. Mô hình được lưu dưới định dạng `.tflite` dựa trên FlatBuffers, giúp biểu diễn cấu trúc mạng, trọng số và metadata trong một tệp có thể nạp trực tiếp bởi runtime [71].

Một quy trình triển khai thông thường bắt đầu từ mô hình đã huấn luyện, sau đó sử dụng công cụ chuyển đổi để ánh xạ các toán tử và tham số sang định dạng TensorFlow Lite. Ở phía suy luận, *interpreter* nạp mô hình, cấp phát bộ nhớ cho tensor, nhận dữ liệu đầu vào, thực thi các toán tử theo đồ thị và trả về tensor đầu ra. Dữ liệu trước khi đưa vào mô hình phải được biến đổi đúng kích thước, thứ tự chiều, kiểu số và miền giá trị mà mô hình yêu cầu. Tương tự, tensor đầu ra cần được diễn giải dựa trên cấu trúc đầu ra và metadata của mô hình.

TensorFlow Lite thực thi mô hình trên CPU thông qua các kernel được tối ưu hoặc chuyển một phần đồ thị sang bộ tăng tốc bằng *delegate*. Delegate đóng vai trò ánh xạ những toán tử được hỗ trợ sang GPU, NPU hoặc bộ xử lý chuyên dụng; các toán tử còn lại có thể tiếp tục chạy trên CPU. Do phạm vi hỗ trợ khác nhau giữa các delegate, một mô hình hợp lệ về mặt định dạng chưa chắc đạt cùng hiệu năng trên mọi thiết bị.

2.6.3 Tối ưu hóa và đánh giá mô hình sau chuyển đổi

Tối ưu hóa sau huấn luyện được sử dụng để giảm kích thước mô hình và chi phí tính toán. Lượng tử hóa FP16 chuyển trọng số từ số thực 32 bit sang 16 bit, qua đó giảm dung lượng và băng thông bộ nhớ. Lượng tử hóa dải động lưu trọng số ở dạng số nguyên nhưng lượng tử hóa activation trong khi suy luận. Lượng tử hóa số nguyên toàn phần chuyển cả trọng số và activation sang INT8; phương pháp này thường cần một tập dữ liệu đại diện để hiệu chuẩn miền giá trị của các tensor.

Lượng tử hóa có thể cải thiện tốc độ và mức sử dụng bộ nhớ nhưng cũng tạo sai số số học. Mức ảnh hưởng phụ thuộc kiến trúc mạng, phân bố dữ liệu và độ nhạy của từng đầu ra. Vì vậy, mô hình sau chuyển đổi cần được đánh giá lại bằng cùng tập kiểm thử và chỉ số của mô hình ban đầu, thay vì chỉ kiểm tra khả năng nạp và thực thi.

Ba yếu tố cần được kiểm chứng trong quá trình chuyển đổi là tính tương thích của toán tử, tính nhất quán của tensor đầu vào–đầu ra và sai lệch kết quả suy luận. Bên cạnh độ chính xác, việc đánh giá trên thiết bị đích cần xem xét thời gian khởi động, độ trễ trung bình, độ biến thiên độ trễ, bộ nhớ sử dụng và ảnh hưởng của cơ chế tăng tốc.

Những chỉ số này phản ánh đầy đủ hơn khả năng triển khai của mô hình trong điều kiện tài nguyên giới hạn.

2.7 Firebase Authentication và Realtime Database

2.7.1 Firebase Authentication

Firebase Authentication là dịch vụ quản lý danh tính và xác thực người dùng. Dịch vụ hỗ trợ nhiều phương thức đăng nhập như tài khoản email–mật khẩu, số điện thoại, nhà cung cấp danh tính liên kết hoặc hệ thống xác thực tùy chỉnh. Sau khi đăng nhập thành công, mỗi tài khoản được gắn với một định danh duy nhất (UID), cho phép các dịch vụ Firebase phân biệt người dùng và liên kết dữ liệu theo danh tính [72].

Kết quả xác thực được thể hiện bằng token chứa thông tin định danh và các thuộc tính liên quan. Token được gửi kèm khi client truy cập tài nguyên Firebase và được phía dịch vụ kiểm tra trước khi áp dụng các quy tắc phân quyền. Cần phân biệt xác thực với phân quyền: xác thực trả lời người dùng là ai, trong khi phân quyền xác định người dùng đó được phép thực hiện thao tác nào trên dữ liệu.

2.7.2 Firebase Realtime Database

Firebase Realtime Database (RTDB) là cơ sở dữ liệu NoSQL được lưu trữ trên đám mây. Dữ liệu được biểu diễn dưới dạng một cây JSON thay vì các bảng và quan hệ. Mỗi vị trí trong cây được xác định bởi một đường dẫn; client có thể đọc, ghi hoặc đăng ký lắng nghe thay đổi tại một đường dẫn cụ thể. Khi dữ liệu thay đổi, máy chủ đồng bộ giá trị mới đến các client đang theo dõi vị trí liên quan [73].

Cơ chế lắng nghe theo sự kiện phù hợp với dữ liệu cần cập nhật nhanh giữa nhiều thiết bị. SDK có thể duy trì bộ nhớ đệm cục bộ và hàng đợi thao tác ghi khi mất kết nối; khi kết nối được khôi phục, dữ liệu tiếp tục được đồng bộ với máy chủ. Tuy nhiên, cập nhật theo thời gian thực không đồng nghĩa mọi client luôn quan sát cùng một trạng thái tại mọi thời điểm, do thông tin vẫn chịu ảnh hưởng của độ trễ mạng, trạng thái kết nối và thứ tự xử lý sự kiện.

Do RTDB sử dụng cây JSON, cấu trúc dữ liệu cần được thiết kế dựa trên cách truy cập. Cây lồng quá sâu làm tăng sự phụ thuộc giữa các nhánh, còn việc đọc tại một nút cha có thể kéo theo toàn bộ dữ liệu bên dưới. Một số trường hợp cần phi chuẩn hóa dữ liệu hoặc cập nhật nhiều vị trí để giảm lượng dữ liệu tải xuống và đơn giản hóa truy vấn. Các thao tác cần bảo đảm tính nhất quán khi nhiều client cùng sửa một giá trị có thể

sử dụng transaction.

Ưu điểm của RTDB là khả năng đồng bộ sự kiện nhanh, hỗ trợ hoạt động gián đoạn mạng và cho phép client truy cập dữ liệu mà không cần tự xây dựng toàn bộ máy chủ đồng bộ. Hạn chế chính là mô hình dữ liệu phi quan hệ, khả năng truy vấn chuyên biệt hơn cơ sở dữ liệu quan hệ và chi phí phụ thuộc lượng dữ liệu lưu trữ, truyền tải cũng như số kết nối. Firebase Authentication và Security Rules bổ sung lớp định danh và kiểm soát truy cập, tạo thành nền tảng chung cho các ứng dụng cần chia sẻ trạng thái giữa nhiều thiết bị.

2.8 WebRTC, signaling và ICE

2.8.1 Tổng quan và kiến trúc truyền thông WebRTC

Web Real-Time Communication (WebRTC) là tập hợp các giao diện lập trình và giao thức hỗ trợ truyền âm thanh, video và dữ liệu với độ trễ thấp giữa hai đầu cuối. Thành phần trung tâm của một phiên liên lạc là `RTCPeerConnection`, có nhiệm vụ thương lượng khả năng truyền thông, tìm đường kết nối, quản lý các luồng media và theo dõi trạng thái của phiên. Âm thanh và hình ảnh được biểu diễn dưới dạng các *MediaStreamTrack*; ngoài media, hai đầu cuối còn có thể trao đổi dữ liệu tùy ý thông qua `RTCDataChannel` [74].

WebRTC ưu tiên thiết lập đường truyền trực tiếp giữa các peer để giảm độ trễ và hạn chế việc media phải đi qua máy chủ ứng dụng. Tuy nhiên, hai thiết bị thường nằm sau Network Address Translator (NAT), tường lửa hoặc sử dụng các loại mạng khác nhau. Vì vậy, một phiên WebRTC không chỉ bao gồm quá trình truyền media mà còn cần cơ chế signaling để trao đổi thông tin phiên và ICE để tìm đường kết nối khả dụng.

2.8.2 Signaling và mô hình Offer–Answer

Signaling là quá trình hai đầu cuối trao đổi các thông tin cần thiết trước và trong khi thiết lập phiên. WebRTC không quy định giao thức signaling cụ thể; ứng dụng có thể sử dụng WebSocket, HTTP, cơ sở dữ liệu thời gian thực hoặc một dịch vụ truyền tin khác. Kênh signaling chỉ chuyển tiếp thông tin điều khiển, không phải là đường truyền âm thanh và video sau khi kết nối WebRTC đã được thiết lập.

Thông tin quan trọng nhất được trao đổi qua signaling là Session Description Protocol (SDP). Một mô tả SDP có thể chứa loại media, codec được hỗ trợ, hướng truyền nhận, tham số mã hóa, thông tin ICE và dấu vân tay chứng thư dùng khi thiết lập bảo

mật. WebRTC sử dụng mô hình *Offer–Answer*, trong đó một peer đưa ra cấu hình phiên và peer còn lại trả lời bằng cấu hình tương thích. Trình tự cơ bản gồm:

1. Peer khởi tạo tạo một *offer*, lưu offer làm mô tả cục bộ và gửi nó qua kênh signaling.
2. Peer nhận đặt offer làm mô tả từ xa, lựa chọn các khả năng tương thích, sau đó tạo và lưu *answer* làm mô tả cục bộ.
3. Answer được gửi ngược lại qua kênh signaling và được peer khởi tạo đặt làm mô tả từ xa. Khi đó, hai bên có cùng thông tin về cấu hình media và các tham số cần thiết để tiếp tục thiết lập kết nối.

Nếu cấu hình phiên thay đổi, chẳng hạn bổ sung hoặc loại bỏ một luồng media, quá trình thương lượng có thể được thực hiện lại. Do các thao tác tạo và thiết lập SDP diễn ra bất đồng bộ, ứng dụng cần tuân theo trạng thái signaling để tránh xử lý đồng thời nhiều lần thương lượng không tương thích.

2.8.3 ICE và vòng đời kết nối WebRTC

Interactive Connectivity Establishment (ICE) là cơ chế tìm kiếm và lựa chọn đường truyền giữa hai peer trong điều kiện tồn tại NAT và tường lửa. Mỗi đầu cuối thu thập một tập *candidate*, tức các địa chỉ IP và cổng có khả năng được sử dụng để liên lạc. Những candidate này được chuyển cho đầu cuối còn lại qua signaling. ICE sau đó ghép candidate cục bộ với candidate từ xa thành các cặp và thực hiện kiểm tra kết nối để xác định cặp có thể truyền dữ liệu hai chiều [75].

Các loại ICE candidate chính được trình bày trong Bảng 2.5.

Bảng 2.5: Các loại ICE candidate trong WebRTC

Loại candidate	Nguồn hình thành	Đặc điểm
Host	Giao diện mạng cục bộ	Là địa chỉ IP và cổng trực tiếp của thiết bị; thường hữu ích khi hai peer ở cùng mạng hoặc có thể định tuyến trực tiếp.
Server-reflexive	Thu được thông qua STUN	Là địa chỉ công khai mà máy chủ STUN quan sát được sau phép ánh xạ NAT; cho phép thử thiết lập đường truyền trực tiếp qua NAT.
Peer-reflexive	Phát hiện khi kiểm tra ICE	Là địa chỉ ánh xạ mới được peer phát hiện trong quá trình kiểm tra kết nối nhưng chưa có trong tập candidate ban đầu.
Relay	Được cấp bởi TURN	Đại diện cho địa chỉ trên máy chủ chuyển tiếp; dữ liệu đi qua TURN khi đường truyền trực tiếp không khả dụng.

Mỗi candidate còn mang các thông tin như giao thức truyền, địa chỉ, cổng, mức ưu tiên, *foundation* và loại candidate. ICE sử dụng độ ưu tiên để sắp xếp các cặp cần kiểm tra nhưng không chỉ chọn theo địa chỉ có vẻ ngắn nhất. Một cặp candidate chỉ được sử dụng sau khi các bản tin kiểm tra xác nhận rằng đường truyền đó hoạt động. Khi tìm được cặp phù hợp, ICE đề cử cặp này làm đường truyền của phiên.

Candidate có thể được gửi ngay khi vừa được phát hiện thay vì chờ thu thập toàn bộ; kỹ thuật này được gọi là *Trickle ICE*. Nhờ tiến hành trao đổi candidate và kiểm tra kết nối song song, thời gian thiết lập cuộc gọi có thể được rút ngắn. Khi môi trường mạng thay đổi hoặc đường truyền cũ không còn sử dụng được, quá trình *ICE restart* có thể được thực hiện để thu thập thông tin mới và tìm lại đường kết nối.

Session Traversal Utilities for NAT (STUN) cho phép một thiết bị gửi yêu cầu đến máy chủ trên Internet để xác định địa chỉ IP và cổng công khai mà NAT đã ánh xạ cho nó. Kết quả được dùng để tạo server-reflexive candidate. STUN không chuyển tiếp media và không bảo đảm hai peer sẽ kết nối trực tiếp được, vì khả năng thành công còn phụ thuộc kiểu NAT, quy tắc tường lửa và chính sách của mạng.

Traversal Using Relays around NAT (TURN) được sử dụng khi các candidate trực tiếp và server-reflexive không tạo được đường truyền khả dụng. Peer yêu cầu TURN cấp một địa chỉ relay; peer còn lại gửi dữ liệu đến địa chỉ này và máy chủ chuyển tiếp dữ liệu tới đích. TURN làm tăng khả năng thiết lập phiên trong các mạng hạn chế nhưng toàn bộ lưu lượng đi qua máy chủ, do đó làm tăng nhu cầu băng thông, độ trễ và chi phí vận

hành. Trong cấu hình thực tế, ICE thường ưu tiên đường truyền trực tiếp và chỉ sử dụng relay khi cần thiết.

Sau khi ICE lựa chọn được đường truyền, âm thanh và video được vận chuyển bằng RTP và được mã hóa dưới dạng Secure Real-time Transport Protocol (SRTP). Khóa mã hóa cho SRTP được thiết lập thông qua Datagram Transport Layer Security (DTLS). Với `RTCDatChannel`, dữ liệu ứng dụng được truyền bằng SCTP trên DTLS. Như vậy, signaling có nhiệm vụ giúp hai peer trao đổi thông tin thiết lập, còn media và `DataChannel` sử dụng đường truyền do ICE lựa chọn.

Các codec và tham số media được thương lượng qua SDP. Trong thời gian truyền, WebRTC theo dõi điều kiện mạng để điều chỉnh tốc độ bit và chất lượng nhằm hạn chế tắc nghẽn. Sự thay đổi băng thông, độ trễ, mất gói và dao động thời gian truyền có thể làm chất lượng âm thanh hoặc hình ảnh thay đổi dù kết nối chưa bị ngắt.

WebRTC yêu cầu bảo vệ media trong quá trình truyền, nhưng tính an toàn của toàn bộ cuộc gọi còn phụ thuộc vào lớp ứng dụng. Kênh signaling cần được bảo vệ để ngăn sửa đổi SDP hoặc ICE candidate; ứng dụng cũng cần kiểm soát danh tính peer và quyền truy cập camera, microphone. Ngoài ra, SDP và candidate có thể chứa thông tin liên quan đến cấu trúc mạng của thiết bị, vì vậy dữ liệu signaling và nhật ký phiên cần được quản lý phù hợp [76].

Một `RTCPeerConnection` duy trì nhiều nhóm trạng thái thay vì chỉ có hai trạng thái kết nối và ngắt kết nối. `signalingState` phản ánh tiến trình thương lượng SDP; `iceGatheringState` cho biết quá trình thu thập candidate; `iceConnectionState` mô tả kết quả kiểm tra ICE; còn `connectionState` tổng hợp trạng thái các transport bên dưới. Việc phân biệt các trạng thái này giúp xác định phiên đang chờ signaling, đang tìm đường truyền, đã kết nối, tạm thời gián đoạn hay thất bại.

Khi kết thúc phiên, các luồng media cần được dừng, peer connection được đóng và dữ liệu signaling liên quan được loại bỏ. Nếu chỉ xóa trạng thái hiển thị mà không giải phóng camera, microphone hoặc đối tượng kết nối, tài nguyên có thể tiếp tục bị chiếm dụng. Ngược lại, việc chỉ đóng peer connection nhưng giữ lại offer, answer hoặc candidate cũ trên kênh signaling có thể làm lần thiết lập tiếp theo nhận nhầm dữ liệu của phiên trước.

WebRTC phù hợp với giao tiếp âm thanh và hình ảnh thời gian thực nhờ khả năng thương lượng codec, thích nghi với điều kiện mạng, mã hóa media và hỗ trợ xuyên NAT.

Hạn chế của công nghệ này nằm ở quy trình thiết lập nhiều bước, sự phụ thuộc vào signaling do ứng dụng xây dựng và nhu cầu triển khai TURN để bảo đảm khả năng kết nối trong các mạng hạn chế. Vì vậy, một hệ thống sử dụng WebRTC cần quản lý đồng thời trạng thái signaling, trạng thái peer connection và vòng đời của tài nguyên media.

2.9 Bluetooth Low Energy và GATT

2.9.1 Tổng quan và quá trình thiết lập kết nối BLE

Bluetooth Low Energy (BLE) là một nhánh của công nghệ Bluetooth được giới thiệu từ Bluetooth Core 4.0, hướng đến các thiết bị cần trao đổi lượng dữ liệu vừa và nhỏ với mức tiêu thụ năng lượng thấp. BLE hoạt động trong băng tần ISM 2,4 GHz và sử dụng 40 kênh vô tuyến có độ rộng 2 MHz. Trong đó, các kênh quảng bá chính hỗ trợ việc phát hiện và thiết lập kết nối, còn các kênh dữ liệu được sử dụng để trao đổi thông tin sau khi kết nối được hình thành. Cơ chế truyền theo từng sự kiện và khả năng chuyển về trạng thái nghỉ giữa các lần trao đổi giúp thiết bị giảm thời gian hoạt động của bộ thu phát vô tuyến [77].

Ngăn xếp BLE được chia thành hai khối logic là *Controller* và *Host*. Controller bao gồm lớp vật lý và lớp liên kết dữ liệu, đảm nhiệm việc phát, thu, lựa chọn kênh và duy trì liên kết vô tuyến. Host chứa các giao thức bậc cao như L2CAP, ATT, GATT, GAP và SMP. Hai khối giao tiếp thông qua Host Controller Interface (HCI). Cách phân lớp này cho phép phần cứng vô tuyến và phần mềm ứng dụng phát triển tương đối độc lập, đồng thời giúp BLE có thể được tích hợp trên nhiều loại vi điều khiển và hệ điều hành khác nhau.

Generic Access Profile (GAP) quy định cách thiết bị xuất hiện, được phát hiện và thiết lập kết nối trong mạng BLE. Khi chưa kết nối, thiết bị có thể phát các gói *advertising* chứa địa chỉ, tên, cờ khả năng hoặc UUID của dịch vụ. Thiết bị quét tiếp nhận các gói này và có thể gửi yêu cầu tạo kết nối đến thiết bị quảng bá. Sau khi kết nối thành công, hai bên chuyển sang trao đổi dữ liệu theo các sự kiện kết nối định kỳ.

Trong một liên kết, thiết bị khởi tạo kết nối giữ vai trò *central*, còn thiết bị chấp nhận kết nối giữ vai trò *peripheral*. Các vai trò này thuộc GAP và không đồng nhất với vai trò client–server của GATT. Trên thực tế, điện thoại thường là central và GATT client, trong khi thiết bị ngoại vi thường là peripheral và GATT server; tuy nhiên, hai cặp vai trò này độc lập về mặt giao thức. Sau khi liên kết được hình thành, GATT client có thể thực hiện quá trình khám phá để xác định các dịch vụ và đặc tính mà server cung cấp.

2.9.2 ATT, GATT và phương thức trao đổi dữ liệu

Attribute Protocol (ATT) tổ chức dữ liệu của server dưới dạng một bảng thuộc tính. Mỗi thuộc tính gồm một *handle* dùng để định địa chỉ trong kết nối, một kiểu dữ liệu được nhận diện bằng UUID, một giá trị và các quyền truy cập. ATT quy định các bản tin mức thấp để client đọc, ghi hoặc nhận dữ liệu từ bảng này. Generic Attribute Profile (GATT) được xây dựng trên ATT và bổ sung cách tổ chức thuộc tính thành các đối tượng có ý nghĩa đối với ứng dụng [78].

Cấu trúc dữ liệu GATT gồm ba thành phần chính:

- **Service:** nhóm các dữ liệu và hành vi phục vụ cùng một chức năng. Một server có thể công bố nhiều service; mỗi service được nhận diện bởi một UUID.
- **Characteristic:** đại diện cho một giá trị dữ liệu bên trong service. Characteristic có UUID, giá trị và tập thuộc tính cho biết những thao tác được hỗ trợ, chẳng hạn *read*, *write*, *write without response*, *notify* hoặc *indicate*.
- **Descriptor:** cung cấp thông tin bổ sung hoặc cấu hình cho characteristic. Client Characteristic Configuration Descriptor (CCCD) là descriptor thường gặp, cho phép client đăng ký hoặc hủy đăng ký nhận notification và indication.

Bluetooth SIG quy định các UUID 16 bit cho những dịch vụ và đặc tính tiêu chuẩn. Với chức năng do nhà phát triển tự định nghĩa, UUID 128 bit thường được sử dụng để nhận diện service và characteristic riêng. Cần phân biệt *property* với quyền truy cập: *property* mô tả thao tác mà characteristic hỗ trợ, còn quyền truy cập xác định điều kiện để thao tác đó được chấp nhận, chẳng hạn chỉ cho phép khi liên kết đã được mã hóa.

GATT hỗ trợ nhiều phương thức trao đổi có đặc điểm khác nhau về hướng truyền, phản hồi và chi phí giao thức. Bảng 2.6 tóm tắt các phương thức thường dùng.

Bảng 2.6: Các phương thức trao đổi dữ liệu cơ bản trong GATT

Phương thức	Hướng truyền	Phản hồi ATT	Đặc điểm
Read	Client đến server	Có response	Client yêu cầu và nhận giá trị hiện tại của characteristic.
Write Request	Client đến server	Có response	Server xác nhận kết quả xử lý; phù hợp với dữ liệu cần biết thao tác ghi đã được chấp nhận hay từ chối.
Write Command	Client đến server	Không	Còn được gọi là <i>write without response</i> ; giảm số bản tin ATT và phù hợp với dữ liệu cập nhật thường xuyên.
Notification	Server đến client	Không	Server chủ động gửi giá trị cho client đã đăng ký mà không chờ xác nhận ở tầng ATT.
Indication	Server đến client	Có confirmation	Server chỉ gửi indication tiếp theo sau khi nhận xác nhận; phù hợp với thông tin cần độ tin cậy cao hơn.

Không có phản hồi ở tầng ATT không đồng nghĩa với việc gói tin hoàn toàn không được bảo vệ. Trong chế độ kết nối, lớp liên kết BLE vẫn thực hiện kiểm tra lỗi và truyền lại gói tin khi cần thiết. Tuy nhiên, *Write Command* và *Notification* không cung cấp xác nhận cho ứng dụng rằng dữ liệu đã được xử lý. Vì vậy, lựa chọn phương thức truyền cần dựa trên ý nghĩa của dữ liệu: luồng cập nhật liên tục có thể ưu tiên độ trễ thấp, trong khi cấu hình hoặc sự kiện quan trọng có thể cần cơ chế phản hồi.

2.9.3 Hiệu năng, bảo mật và khả năng áp dụng

Khối lượng dữ liệu có thể mang trong mỗi thao tác ATT phụ thuộc vào Maximum Transmission Unit (MTU). Giá trị ATT MTU mặc định là 23 byte; với các bản tin phổ biến phải dành byte cho mã thao tác và handle, phần giá trị characteristic thường còn tối đa 20 byte. Thiết bị có thể thương lượng MTU lớn hơn sau khi kết nối. Nếu dữ liệu vượt quá kích thước cho phép, giao thức phải chia dữ liệu thành nhiều phần hoặc sử dụng thủ tục đọc, ghi dài. Vì vậy, định dạng dữ liệu càng ngắn gọn thì số lần truyền và thời gian xử lý càng giảm.

Độ trễ và mức tiêu thụ năng lượng còn chịu ảnh hưởng bởi *connection interval*,

tức khoảng thời gian giữa hai sự kiện kết nối. Khoảng thời gian ngắn cho phép dữ liệu được trao đổi thường xuyên hơn nhưng làm bộ thu phát hoạt động nhiều hơn; khoảng thời gian dài tiết kiệm năng lượng nhưng tăng độ trễ. Thông lượng thực tế còn phụ thuộc vào MTU, kích thước gói ở lớp liên kết, số gói trong mỗi sự kiện kết nối, chất lượng tín hiệu và cách hệ điều hành lập lịch giao tiếp Bluetooth. Do đó, tốc độ danh nghĩa của lớp vật lý không phải là tốc độ dữ liệu mà ứng dụng luôn đạt được.

Security Manager Protocol (SMP) hỗ trợ ghép đôi, xác thực và thiết lập khóa mã hóa cho liên kết BLE. Quá trình *bonding* cho phép hai thiết bị lưu thông tin khóa để tái lập kết nối an toàn trong những lần sử dụng sau. Bên cạnh bảo vệ liên kết, server có thể đặt quyền đọc hoặc ghi riêng cho từng attribute. Mức bảo vệ cần được lựa chọn dựa trên tính nhạy cảm của dữ liệu và khả năng tương tác của thiết bị.

BLE có ưu điểm là được hỗ trợ rộng rãi trên điện thoại và vi điều khiển, không đòi hỏi hạ tầng mạng trung gian và phù hợp với liên kết cự ly gần. GATT cho phép xây dựng mô hình dữ liệu riêng bằng các service và characteristic, nhờ đó có thể tách lệnh điều khiển, dữ liệu trạng thái và thông tin cấu hình hình thành các kênh logic khác nhau trên cùng một kết nối. Hạn chế của BLE là thông lượng thấp hơn Wi-Fi, kích thước bản tin tương đối nhỏ và chất lượng liên kết có thể thay đổi do khoảng cách hoặc nhiễu trong băng tần 2,4 GHz. Đối với các hệ thống điều khiển, các đặc điểm này cần được xem xét khi lựa chọn phương thức ghi, chu kỳ cập nhật và cơ chế xử lý khi kết nối bị gián đoạn.

2.10 Phần cứng robot và thư viện điều khiển động cơ

Phần thân robot sử dụng cơ cấu dẫn động vi sai với hai động cơ BLDC bố trí ở hai bên. Mỗi động cơ được nối với một mạch driver 3-PWM và một cảm biến góc từ AS5600, tạo thành hai kênh điều khiển độc lập cho bánh trái và bánh phải. ESP32 là bộ điều khiển trung tâm, đảm nhiệm tiếp nhận lệnh, cập nhật tốc độ đặt và thực hiện vòng điều khiển động cơ. Cấu trúc hai bánh cho phép robot tiến, lùi hoặc đổi hướng bằng cách thay đổi chiều và độ lớn vận tốc giữa hai động cơ.

Bảng 2.7: Các thành phần chính của phần cứng thân robot

Thành phần	Cấu hình trong firmware	Vai trò
ESP32	Hai bus I ² C, BLE và UART Serial	Điều phối giao tiếp, đọc cảm biến và thực hiện hai vòng điều khiển động cơ
Hai động cơ BLDC	7 cặp cực; giới hạn vận tốc 40 rad/s	Tạo chuyển động cho bánh trái và bánh phải
Hai driver 3-PWM	Nguồn khai báo 17,5 V; giới hạn điện áp điều khiển 10 V	Phát ba tín hiệu PWM để cấp điện cho từng động cơ
Hai cảm biến AS5600	Mỗi cảm biến dùng một bus I ² C riêng ở 400 kHz	Cung cấp góc rotor để ước lượng vận tốc và khép kín vòng điều khiển

Giám sát và bảo vệ vận hành. UART Serial ở tốc độ 115200 baud được dùng để theo dõi trạng thái khởi tạo, kết nối BLE, lệnh nhận được và cảnh báo timeout. Chương trình kiểm tra động cơ riêng còn cho phép nhập các lệnh tiến, lùi, quay trái, quay phải và dừng qua Serial, thuận tiện khi đánh giá từng phần trước khi ghép với ứng dụng. Trong firmware tích hợp, nếu BLE bị ngắt kết nối hoặc không có lệnh mới trong 800 ms, tốc độ đặt của cả hai động cơ được đưa về 0. Vùng dừng và vùng bù nhỏ quanh giá trị 0 cũng được áp dụng để hạn chế rung hoặc trường hợp động cơ không thắng được ma sát ban đầu.

Giao diện lập trình của thư viện SimpleFOC. SimpleFOC là thư viện mã nguồn mở hỗ trợ điều khiển động cơ BLDC và động cơ bước trên nhiều nền tảng vi điều khiển, trong đó có ESP32. Thư viện tổ chức hệ thống điều khiển thành các đối tượng tương ứng với cảm biến, mạch driver và động cơ; các đối tượng này được liên kết với nhau thông qua một giao diện lập trình thống nhất. Nhờ đó, người phát triển có thể xây dựng vòng điều khiển FOC bằng cách cấu hình và kết hợp các thành phần có sẵn thay vì phải tự hiện thực toàn bộ thuật toán ở mức thấp. Một số đối tượng, thuộc tính và hàm chính của thư viện gồm:

- **Đối tượng cảm biến `MagneticSensorI2C`:** biểu diễn cảm biến vị trí từ giao tiếp qua I²C. Đối tượng này cung cấp góc cơ học của rotor và dữ liệu cần thiết để xác định vận tốc. Hàm `init()` được dùng để khởi tạo cảm biến và giao tiếp với vi điều khiển.

- **Đối tượng driver `BLDCDriver3PWM`:** mô tả mạch công suất điều khiển động cơ BLDC bằng ba tín hiệu PWM. Các chân PWM và chân cho phép hoạt động được khai báo khi tạo đối tượng; thuộc tính `voltage_power_supply` cho biết điện áp nguồn của driver. Hàm `init()` thiết lập các ngõ ra PWM và chuẩn bị driver cho quá trình điều khiển.
- **Đối tượng động cơ `BLDCMotor`:** là thành phần trung tâm của vòng điều khiển và được khởi tạo với số cặp cực của động cơ. Các hàm `linkSensor()` và `linkDriver()` lần lượt gắn cảm biến phản hồi và driver công suất với đối tượng động cơ, qua đó hình thành chuỗi đo lường, tính toán và tác động.
- **Thuộc tính cấu hình điều khiển:** `controller` xác định đại lượng chuyển động cần điều khiển, chẳng hạn mô-men, vận tốc hoặc vị trí. Thuộc tính `foc_modulation` lựa chọn phương pháp điều chế điện áp ba pha, trong đó `FOCModulationType::SpaceVectorPWM` tương ứng với phương pháp điều chế vector không gian.
- **Bộ điều khiển và giới hạn:** các tham số `PID_velocity.P`, `PID_velocity.I` và `PID_velocity.D` dùng để hiệu chỉnh vòng điều khiển vận tốc. `LPF_velocity.Tf` thiết lập hằng số thời gian của bộ lọc thông thấp cho tín hiệu vận tốc. Bên cạnh đó, `voltage_limit` và `velocity_limit` giới hạn đầu ra điện áp và vận tốc nhằm giữ hệ thống trong phạm vi vận hành đã xác định.
- **Các hàm khởi tạo:** `motor.init()` khởi tạo đối tượng động cơ và kiểm tra các thành phần đã liên kết. Sau đó, `motor.initFOC()` thực hiện quá trình căn chỉnh cảm biến, xác định chiều quay và góc điện ban đầu cần thiết cho điều khiển FOC.
- **Các hàm điều khiển trong thời gian chạy:** `loopFOC()` được gọi lặp lại để đọc phản hồi từ cảm biến và cập nhật điện áp điều khiển các pha. Hàm `move(target)` tiếp nhận giá trị đặt; ý nghĩa của `target` phụ thuộc vào chế độ điều khiển đã lựa chọn, có thể là mô-men, vận tốc hoặc vị trí.

Cách tổ chức này phân tách rõ ba thành phần cảm biến, driver và động cơ, đồng thời phân biệt giai đoạn khởi tạo với quá trình cập nhật điều khiển theo thời gian thực. Đây là cơ sở để tái sử dụng cùng một cấu trúc chương trình cho nhiều cấu hình phần cứng và chế độ điều khiển khác nhau.

2.11 Kết luận chương

Chương này đã hệ thống hóa những cơ sở lý thuyết cần thiết cho quá trình xây dựng robot hỗ trợ chăm sóc người cao tuổi. Trước hết, mô hình robot sử dụng điện thoại thông minh được trình bày nhằm làm rõ khả năng khai thác camera, microphone, năng lực xử lý và kết nối sẵn có của thiết bị di động. Trên nền tảng đó, các kiến thức về thị giác máy tính, phát hiện và theo dõi đối tượng, ước lượng tư thế cùng mô hình chuỗi thời gian cung cấp cơ sở cho bài toán nhận biết trạng thái và phân tích diễn biến chuyển động của con người.

Bên cạnh khối nhận thức, chương cũng trình bày các nền tảng phục vụ triển khai mô hình trên thiết bị, quản lý và đồng bộ dữ liệu, truyền thông thời gian thực cũng như kết nối giữa điện thoại với phần cứng robot. Phần cuối chương giới thiệu cấu trúc phần cứng chuyển động và các đối tượng điều khiển động cơ do thư viện SimpleFOC cung cấp. Các nội dung này hình thành nền tảng để xác định mối quan hệ giữa khối cảm nhận, xử lý, giao tiếp và chấp hành. Từ cơ sở đó, Chương 3 sẽ phân tích yêu cầu và xây dựng kiến trúc tổng thể, các module chức năng cùng những luồng xử lý chính của hệ thống.

Chương 3 – Phân tích và thiết kế hệ thống

Chương này trình bày quá trình phân tích yêu cầu và thiết kế hệ thống robot trợ lý hỗ trợ chăm sóc người cao tuổi tại nhà. Từ mục tiêu giao tiếp, nhắc nhở, giám sát, cảnh báo té ngã và hỗ trợ can thiệp từ xa, các yêu cầu chức năng và yêu cầu chất lượng được xác định làm cơ sở xây dựng kiến trúc cùng các luồng xử lý. Kiến trúc vận hành gồm ứng dụng trên robot, ứng dụng của người chăm sóc, hệ thống dịch vụ & kết nối, cùng phần thân robot. Quy trình chuẩn bị mô hình AI được xem xét riêng để cung cấp mô hình cho ứng dụng trên robot. Chương cũng phân tích các phương án công nghệ, giao thức truyền thông và cơ chế an toàn làm cơ sở cho quá trình hiện thực ở Chương 4.

3.1 Phân tích yêu cầu hệ thống

3.1.1 Bài toán, mục tiêu và phạm vi

Đề tài hướng đến xây dựng một robot di động hoạt động trong môi trường trong nhà, sử dụng điện thoại hoặc máy tính bảng gắn trên robot làm bộ xử lý chính. Hệ thống hỗ trợ người cao tuổi thông qua hội thoại bằng giọng nói, nhắc nhở theo lịch, giám sát bằng thị giác, phát hiện tình huống nghi té ngã và liên lạc với người chăm sóc. Một ứng dụng riêng dành cho người chăm sóc cung cấp khả năng quản lý thông tin, theo dõi sự kiện và can thiệp từ xa. Phần thân robot tiếp nhận lệnh chuyển động từ thiết bị gắn trên robot và điều khiển hai bánh xe theo yêu cầu.

Các định hướng của luận văn được cụ thể hóa trong Bảng 3.1. Trong phạm vi đề tài, khả năng hiểu ngôn ngữ tự nhiên tập trung vào hội thoại hằng ngày, nhận biết yêu cầu liên lạc với người chăm sóc và diễn giải câu trả lời khi xác nhận tình trạng sau một sự kiện nghi té ngã. Hệ thống chỉ đóng vai trò hỗ trợ chăm sóc, giám sát và liên lạc; không phải thiết bị chẩn đoán y khoa, không thay thế người chăm sóc và không tự đưa ra quyết định điều trị.

Bảng 3.1: Cụ thể hóa yêu cầu luận văn theo phạm vi đề tài

Định hướng ban đầu	Mục tiêu cụ thể	Kết quả cần đạt
Tìm hiểu Intel OpenBot và giao tiếp điện thoại-robot	Khảo sát mô hình sử dụng điện thoại thông minh làm bộ xử lý và giao tiếp với phần thân robot	Xây dựng được phương thức kết nối để truyền lệnh chuyển động từ điện thoại đến thân robot

Định hướng ban đầu	Mục tiêu cụ thể	Kết quả cần đạt
Nhận dạng giọng nói và hiểu ngôn ngữ tự nhiên	Xây dựng khả năng tiếp nhận tiếng nói, diễn giải nội dung và phản hồi bằng giọng nói	Hỗ trợ hội thoại hằng ngày, yêu cầu liên lạc và xác nhận tình trạng của người dùng
AI và IoT cho xử lý, điều khiển và tích hợp	Kết hợp xử lý thông minh trên robot với khả năng trao đổi dữ liệu và điều khiển giữa các thành phần	Đồng bộ được thông tin chăm sóc, trạng thái giám sát, sự kiện và lệnh điều khiển
Thị giác máy tính cho giám sát và theo dõi	Nhận biết người, theo dõi người mục tiêu và phân tích sự thay đổi tư thế theo thời gian	Hỗ trợ robot bám theo người và phát hiện tình huống nghi tế ngã
Đánh giá thuật toán và hệ thống kết nối	Đánh giá chất lượng nhận biết, tốc độ xử lý và khả năng trao đổi dữ liệu giữa các thành phần	Xác định được mức đáp ứng của nguyên mẫu, các giới hạn và hướng cải tiến

Phạm vi của đề tài gồm quản lý hồ sơ và lịch nhắc nhở; hội thoại bằng giọng nói; ước lượng tư thế, theo dõi một người mục tiêu, tự động bám theo và phát hiện té ngã; gọi video hai chiều; điều khiển robot bằng cần điều khiển ảo; ghi nhật ký nhắc nhở, cảnh báo và cuộc gọi. Các chức năng định vị và lập bản đồ toàn bộ ngôi nhà, tránh vật cản bằng cảm biến chuyên dụng, đo chỉ số sinh tồn, đánh giá bảo mật chuyên sâu và triển khai hệ thống ở quy mô lớn trong môi trường thực tế chưa được xem xét trong phạm vi này.

3.1.2 Tác nhân, điều kiện vận hành và ranh giới hệ thống

Hai tác nhân chính của hệ thống là người cao tuổi và người chăm sóc. Người cao tuổi tương tác trực tiếp với robot để nhận hỗ trợ, trong khi người chăm sóc quản lý thông tin và can thiệp từ xa. Bên trong ranh giới hệ thống gồm ứng dụng trên robot, ứng dụng của người chăm sóc, khối xử lý AI và phần thân robot. Các dịch vụ trực tuyến nằm ngoài ranh giới này nhưng cung cấp khả năng xác thực, đồng bộ dữ liệu, xử lý ngôn ngữ và hỗ trợ thiết lập liên lạc.

Bảng 3.2: Tác nhân và các thành phần hỗ trợ của hệ thống

Đối tượng	Vai trò	Tương tác với hệ thống
Người cao tuổi	Người sử dụng được hỗ trợ	Trò chuyện với robot, nhận lời nhắc, được giám sát trong phạm vi cho phép và tham gia cuộc gọi
Người chăm sóc	Người quản lý và hỗ trợ từ xa	Quản lý hồ sơ và lịch nhắc nhở, cấu hình chức năng, xem nhật ký, thực hiện cuộc gọi và điều khiển robot khi cần
Dịch vụ trực tuyến	Thành phần hỗ trợ bên ngoài	Cung cấp xác thực, đồng bộ dữ liệu, xử lý hội thoại và hỗ trợ thiết lập liên lạc từ xa
Phần thân robot	Thành phần chấp hành	Tiếp nhận lệnh chuyển động, điều khiển hai bánh xe và dừng khi mất lệnh

Hệ thống giả định thiết bị gắn trên robot có camera, microphone, loa và khả năng kết nối không dây với phần thân robot. Thiết bị trên robot và thiết bị của người chăm sóc cần có kết nối Internet.

3.1.3 Yêu cầu chức năng

Trên cơ sở mục tiêu, phạm vi và các tác nhân đã xác định, hệ thống cần cung cấp một nhóm chức năng xuyên suốt từ hỗ trợ người cao tuổi đến giám sát và điều khiển robot. Các yêu cầu này được tổng hợp trong Bảng 3.3; mỗi yêu cầu được gán một mã riêng để thuận tiện liên kết với các khối thiết kế và đối chiếu với kết quả kiểm thử ở những phần tiếp theo.

Bảng 3.3: Danh sách yêu cầu chức năng

Mã	Yêu cầu	Mô tả chức năng
FR01	Xác thực và liên kết người dùng	Hệ thống cho phép đăng ký, đăng nhập và xác định đúng không gian dữ liệu dùng chung giữa robot với người chăm sóc.

Mã	Yêu cầu	Mô tả chức năng
FR02	Quản lý hồ sơ chăm sóc	Người chăm sóc có thể xem và cập nhật thông tin nhận diện, cách xưng hô, thông tin liên hệ, tình trạng, ghi chú và sở thích của người được chăm sóc.
FR03	Quản lý lịch nhắc nhở	Người chăm sóc có thể tạo, chỉnh sửa, bật, tắt và xóa lịch nhắc nhở theo các chế độ một lần, hằng ngày, hằng tuần hoặc hằng tháng.
FR04	Thực hiện và ghi nhận lời nhắc	Khi đến thời điểm đã thiết lập, robot phát lời nhắc, tiếp nhận phản hồi của người cao tuổi và lưu lại kết quả thực hiện.
FR05	Hội thoại bằng giọng nói	Hệ thống tiếp nhận lời nói, tạo câu trả lời phù hợp và phản hồi bằng giọng nói; đồng thời nhận biết yêu cầu liên lạc với người chăm sóc.
FR06	Cấu hình chức năng giám sát	Người chăm sóc có thể bật hoặc tắt chức năng bám theo người và phát hiện té ngã; trạng thái mới phải được chuyển đến robot.
FR07	Phát hiện người và tư thế	Hệ thống xác định vị trí người trong khung hình và các điểm đặc trưng tư thế cần thiết cho chức năng theo dõi, bám người và phát hiện té ngã.
FR08	Duy trì người mục tiêu	Hệ thống lựa chọn và duy trì một người làm mục tiêu theo dõi, đồng thời có khả năng nhận lại mục tiêu sau khi bị che khuất hoặc mất dấu trong thời gian ngắn.
FR09	Tự động bám theo người	Robot điều chỉnh hướng và tốc độ di chuyển để duy trì người mục tiêu ở vị trí và khoảng cách phù hợp; robot phải dừng khi không còn đủ thông tin để tiếp tục bám.
FR10	Phân tích diễn biến té ngã	Hệ thống phân tích chuỗi tư thế của người mục tiêu theo thời gian để phân biệt hoạt động bình thường với tình huống nghi té ngã.

Mã	Yêu cầu	Mô tả chức năng
FR11	Kiểm tra sự kiện nghi té ngã	Kết quả nhận biết ban đầu được kiểm tra thêm dựa trên tính liên tục theo thời gian và trạng thái cơ thể trước khi tạo sự kiện cần xác nhận.
FR12	Xác nhận tình trạng bằng giọng nói	Khi phát hiện tình huống nghi té ngã, robot dừng di chuyển, hỏi thăm tình trạng và chờ phản hồi của người cao tuổi trong một khoảng thời gian xác định.
FR13	Cảnh báo và liên lạc khẩn cấp	Khi người cao tuổi xác nhận cần hỗ trợ hoặc không thể đưa ra phản hồi rõ ràng, hệ thống lưu cảnh báo và thiết lập liên lạc với người chăm sóc.
FR14	Gọi âm thanh và hình ảnh hai chiều	Hệ thống cho phép robot và người chăm sóc thiết lập, duy trì và kết thúc cuộc gọi hai chiều, đồng thời lưu thông tin cơ bản của cuộc gọi.
FR15	Điều khiển robot từ xa	Người chăm sóc có thể điều chỉnh hướng và tốc độ chuyển động của robot; robot phải dừng khi thao tác điều khiển kết thúc và không để lệnh từ xa xung đột với chế độ bám tự động.
FR16	Chuyển tiếp lệnh chuyển động	Hệ thống tiếp nhận lệnh từ chức năng bám người hoặc điều khiển từ xa, kiểm tra tính hợp lệ và chuyển đổi thành lệnh vận tốc cho hai bánh xe.
FR17	Điều khiển động cơ	Phần thân robot tiếp nhận lệnh vận tốc, điều khiển độc lập hai động cơ và giới hạn chuyển động trong phạm vi vận hành cho phép.
FR18	Nhật ký chăm sóc	Người chăm sóc có thể xem lịch sử thực hiện lời nhắc, cảnh báo té ngã và cuộc gọi theo trình tự thời gian.

Mã	Yêu cầu	Mô tả chức năng
FR19	Chuyển đổi và triển khai mô hình	Hệ thống hỗ trợ chuyển mô hình đã huấn luyện sang định dạng phù hợp với thiết bị di động và kiểm tra tính nhất quán của đầu vào, đầu ra sau chuyển đổi.
FR20	Dừng an toàn khi xảy ra lỗi	Robot phải đưa vận tốc về 0 khi lệnh điều khiển không hợp lệ, đã quá hạn hoặc kết nối điều khiển bị gián đoạn.

3.1.4 Yêu cầu phi chức năng

Các yêu cầu phi chức năng mô tả những thuộc tính chất lượng mà hệ thống cần đáp ứng trong quá trình vận hành. Các thuộc tính được lựa chọn dựa trên mô hình ISO/IEC 25010 [79] và đặc điểm của một robot hoạt động gần người cao tuổi. Những yêu cầu này là cơ sở xây dựng tiêu chí kiểm thử; các giá trị định lượng cụ thể sẽ được xác định trong thiết kế và đánh giá bằng thực nghiệm ở Chương 5.

Bảng 3.4: Yêu cầu phi chức năng

Mã	Thuộc tính	Yêu cầu chất lượng
NFR01	An toàn chuyển động	Robot phải dừng khi chức năng di chuyển bị tắt, thao tác điều khiển kết thúc, mục tiêu không còn được xác định, lệnh không hợp lệ hoặc kết nối điều khiển bị gián đoạn.
NFR02	Tính kịp thời	Dữ liệu giám sát và lệnh điều khiển phải được xử lý trong khoảng thời gian phù hợp với chức năng; lệnh đã quá thời hạn cho phép không được tiếp tục thực thi.
NFR03	Độ tin cậy	Hệ thống phải hạn chế phát lặp cùng một sự kiện, giải phóng tài nguyên khi chức năng kết thúc và có khả năng trở về trạng thái xác định sau gián đoạn tạm thời.

Mã	Thuộc tính	Yêu cầu chất lượng
NFR04	Khả năng sử dụng	Giao diện dành cho người cao tuổi phải ưu tiên tương tác đơn giản và phản hồi rõ ràng; giao diện dành cho người chăm sóc phải tổ chức chức năng theo nhóm để nhận biết và thao tác.
NFR05	Bảo mật	Dữ liệu cá nhân và chức năng điều khiển chỉ được truy cập bởi người dùng đã xác thực và có quyền phù hợp; dữ liệu truyền qua mạng phải được bảo vệ khỏi truy cập trái phép.
NFR06	Quyền riêng tư	Hình ảnh giám sát phải được ưu tiên xử lý cục bộ; âm thanh và hình ảnh chỉ được truyền khi chức năng liên lạc được thiết lập và không được lưu trữ nếu không cần thiết.
NFR07	Hiệu quả tài nguyên	Các chức năng xử lý liên tục phải sử dụng hợp lý năng lực tính toán, bộ nhớ và năng lượng của thiết bị, không làm gián đoạn các tương tác quan trọng của người dùng.
NFR08	Khả năng bảo trì	Các nhóm chức năng chính phải được phân tách rõ trách nhiệm, sử dụng cấu trúc dữ liệu và giao diện trao đổi nhất quán để thuận tiện sửa đổi hoặc thay thế từng thành phần.
NFR09	Khả năng giải thích	Cảnh báo cần kèm theo thời điểm và thông tin hỗ trợ giải thích nguyên nhân tạo sự kiện; kết quả nhận biết không được trình bày như một kết luận chẩn đoán y khoa.
NFR10	Khả năng mở rộng	Cấu trúc hệ thống cần cho phép mở rộng quan hệ giữa người chăm sóc, người cao tuổi và robot mà không phải thay đổi toàn bộ các chức năng hiện có.
NFR11	Khả năng kiểm thử	Các chức năng chính, trạng thái lỗi, chất lượng mô hình, tốc độ xử lý và độ trễ điều khiển phải có khả năng quan sát, đo lường hoặc tái hiện trong quá trình thử nghiệm.

Các yêu cầu trên cho thấy hệ thống phải phối hợp nhiều nhóm chức năng, từ quản lý chăm sóc, tương tác và giám sát tại nhà đến liên lạc từ xa và điều khiển chuyển động. Mỗi nhóm cần được giao cho một thành phần có trách nhiệm rõ ràng, đồng thời vẫn bảo đảm khả năng trao đổi dữ liệu trong các tình huống cần phối hợp. Trên cơ sở đó, mục tiếp theo phân rã hệ thống theo chức năng và xây dựng kiến trúc tổng quan thể hiện mối liên hệ giữa các thành phần.

3.2 Kiến trúc tổng quan hệ thống

3.2.1 Cơ sở xây dựng kiến trúc tổng quan

Các yêu cầu ở Mục 3.1 cho thấy hệ thống phải hỗ trợ đồng thời hoạt động chăm sóc tại nhà và khả năng theo dõi từ xa. Những chức năng tương tác trực tiếp với người cao tuổi, xử lý hình ảnh và điều khiển chuyển động cần được bố trí trên robot. Thông tin hồ sơ, lịch nhắc nhở, cảnh báo và trạng thái hoạt động phải được chia sẻ giữa hai ứng dụng. Trong khi đó, người chăm sóc cần một giao diện riêng để quản lý thông tin, tiếp nhận sự kiện, liên lạc và điều khiển robot khi cần. Từ đó, hệ thống được phân chia thành ba khu vực chính: hệ thống robot tại nhà, hệ thống dịch vụ & kết nối, và hệ thống của người chăm sóc từ xa.

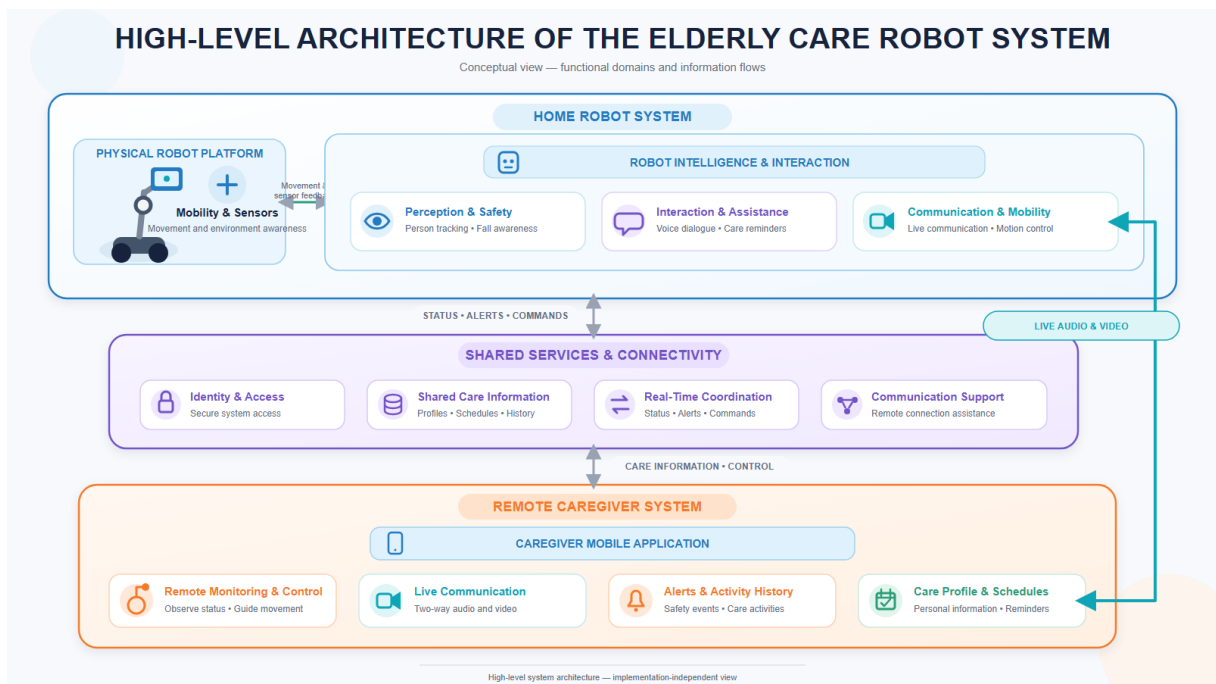
Bảng 3.5 trình bày trách nhiệm của từng khu vực và mối liên hệ với các yêu cầu đã xác định. Một số yêu cầu xuất hiện ở nhiều khu vực vì cần sự phối hợp giữa các thành phần. Chẳng hạn, điều khiển từ xa bắt đầu từ thao tác của người chăm sóc, được chuyển qua hệ thống dịch vụ & kết nối, sau đó được RobotApp kiểm tra trước khi gửi đến phần thân robot.

Yêu cầu FR19 về chuẩn bị mô hình AI thuộc giai đoạn phát triển, không tham gia như một khu vực vận hành độc lập. Mô hình sau khi chuyển đổi được tích hợp vào khối nhận biết và an toàn của RobotApp. Cách phân chia này giúp Hình 3.1 tập trung mô tả các thành phần hoạt động và trao đổi dữ liệu khi hệ thống được sử dụng.

Hình 3.1 thể hiện kiến trúc tổng quan của hệ thống. Phần phía trên là hệ thống robot tại nhà, gồm thiết bị Android đảm nhiệm xử lý và phần thân robot thực hiện chuyển động. Phần giữa là hệ thống dịch vụ & kết nối hỗ trợ hai ứng dụng trao đổi dữ liệu. Phần phía dưới là ứng dụng dành cho người chăm sóc. Các mũi tên trong hình biểu diễn bốn nhóm trao đổi chính: lệnh chuyển động và phản hồi từ phần thân robot; trạng thái, cảnh báo và lệnh giữa robot với hệ thống dịch vụ & kết nối; thông tin chăm sóc và thao tác điều khiển từ phía người chăm sóc; âm thanh và hình ảnh của cuộc gọi giữa hai thiết bị.

Bảng 3.5: Phân chia trách nhiệm giữa các khu vực trong kiến trúc hệ thống

Khu vực	Yêu cầu liên quan	Trách nhiệm chính
Hệ thống robot tại nhà	FR04–FR17, FR20	Tương tác với người cao tuổi; thu nhận và xử lý hình ảnh; theo dõi người, phát hiện tình huống nghi tế ngã; thực hiện lời nhắc và cuộc gọi; tạo hoặc tiếp nhận lệnh chuyển động; điều khiển và dừng phần thân robot
Hệ thống dịch vụ & kết nối	FR01–FR04, FR06, FR13–FR16, FR18	Xác thực người dùng; lưu trữ và đồng bộ hồ sơ, lịch, nhật ký; chuyển trạng thái chức năng, cảnh báo và lệnh điều khiển; hỗ trợ thiết lập liên lạc giữa hai ứng dụng
Hệ thống người chăm sóc từ xa	FR01–FR03, FR06, FR13–FR15, FR18	Cung cấp giao diện quản lý hồ sơ và lịch; bật hoặc tắt chức năng giám sát; tiếp nhận cảnh báo, xem lịch sử; gọi video và điều khiển robot từ xa



Hình 3.1: Kiến trúc tổng quan của hệ thống robot hỗ trợ chăm sóc người cao tuổi

3.2.2 Các khối chức năng và luồng phối hợp

Trong mỗi khu vực, Hình 3.1 tiếp tục chia hệ thống thành các khối chức năng. Bảng 3.6 sử dụng đúng cách phân nhóm trong hình để làm rõ nhiệm vụ và thông tin trao đổi của từng khối.

Bảng 3.6: *Vai trò của các khối trong kiến trúc tổng quan*

Khu vực	Khối chức năng	Nhiệm vụ	Thông tin trao đổi chính
Hệ thống robot tại nhà	Nền tảng robot vật lý	Thực hiện chuyển động bằng hai bánh xe, cung cấp phản hồi vị trí rotor và dừng khi mất lệnh	Vận tốc đặt của hai bánh; phản hồi từ cảm biến góc
Hệ thống robot tại nhà	Nhận biết và an toàn	Thu nhận hình ảnh, ước lượng tư thế, duy trì người mục tiêu và phân tích tình huống nghi tế ngã	Khung hình; bounding box của người; điểm khớp; trạng thái mục tiêu; sự kiện nghi tế ngã
Hệ thống robot tại nhà	Tương tác và hỗ trợ	Tiếp nhận tiếng nói, tạo câu trả lời, phát lời nhắc và hỏi xác nhận khi cần	Lời nói; nội dung hội thoại; lịch nhắc nhở; phản hồi của người dùng
Hệ thống robot tại nhà	Liên lạc và di chuyển	Thiết lập cuộc gọi, phối hợp nguồn lệnh bám theo hoặc điều khiển từ xa và chuyển lệnh đến phần thân robot	Trạng thái cuộc gọi; âm thanh, hình ảnh; vector điều khiển
Hệ thống dịch vụ & kết nối	Định danh và truy cập	Xác thực tài khoản và xác định vùng dữ liệu mà cặp thiết bị được phép sử dụng	Thông tin xác thực; định danh người dùng
Hệ thống dịch vụ & kết nối	Thông tin chăm sóc	Lưu trữ và đồng bộ hồ sơ, lịch nhắc nhở cùng các bản ghi hoạt động	Hồ sơ; lịch; nhật ký lời nhắc, cảnh báo và cuộc gọi

Khu vực	Khối chức năng	Nhiệm vụ	Thông tin trao đổi chính
Hệ thống dịch vụ & kết nối	Điều phối thời gian thực	Chuyển trạng thái chức năng, cảnh báo, trạng thái cuộc gọi và lệnh điều khiển giữa hai ứng dụng	Trạng thái bật, tắt; cảnh báo; lệnh; thông tin thiết lập cuộc gọi
Hệ thống dịch vụ & kết nối	Hỗ trợ liên lạc	Hỗ trợ hai thiết bị tìm đường kết nối và thiết lập phiên truyền thông từ xa	Thông tin thiết lập kết nối
Hệ thống người chăm sóc	Giám sát và điều khiển từ xa	Hiển thị trạng thái robot, bật hoặc tắt chức năng giám sát và tạo lệnh từ cần điều khiển ảo	Trạng thái chức năng; thao tác điều khiển; lệnh dừng
Hệ thống người chăm sóc	Liên lạc trực tiếp	Thực hiện cuộc gọi âm thanh và hình ảnh với thiết bị đặt trên robot	Yêu cầu gọi; âm thanh; hình ảnh; trạng thái cuộc gọi
Hệ thống người chăm sóc	Cảnh báo và lịch sử hoạt động	Thông báo sự kiện cần hỗ trợ và cho phép xem lại các hoạt động đã ghi nhận	Cảnh báo té ngã; nhật ký lời nhắc và cuộc gọi
Hệ thống người chăm sóc	Hồ sơ và lịch nhắc nhở	Quản lý thông tin người cao tuổi và thiết lập nội dung, thời điểm nhắc nhở	Hồ sơ chăm sóc; lịch nhắc nhở

Các luồng trong hình thể hiện cách các khối phối hợp mà không phụ thuộc vào công nghệ hiện thực. Thứ nhất, thiết bị Android trên robot gửi vận tốc đặt đến phần thân và nhận dữ liệu phản hồi phục vụ điều khiển động cơ. Thứ hai, RobotApp trao đổi trạng thái chức năng, cảnh báo, lệnh và thông tin cuộc gọi với hệ thống dịch vụ & kết nối. Thứ ba, CaregiverApp đọc hoặc cập nhật hồ sơ, lịch, nhật ký và lệnh điều khiển thông qua hệ thống này. Thứ tư, khi cuộc gọi được thiết lập, âm thanh và hình ảnh được truyền giữa

hai thiết bị theo kênh riêng, không lưu cùng hồ sơ và lịch chăm sóc.

3.3 Các luồng xử lý chính

3.3.1 Các use case chính

Các use case được chia thành bốn nhóm: quản lý hồ sơ, lịch nhắc nhở và nhật ký; hội thoại và thực hiện lời nhắc; giám sát, bám theo và phát hiện té ngã; liên lạc và điều khiển robot. Người chăm sóc chủ yếu thao tác thông qua ứng dụng quản lý, trong khi người cao tuổi tương tác trực tiếp với robot bằng giọng nói. Bảng 3.7 tóm tắt tác nhân và kết quả của các use case chính.

Bảng 3.7: Tóm tắt các use case chính

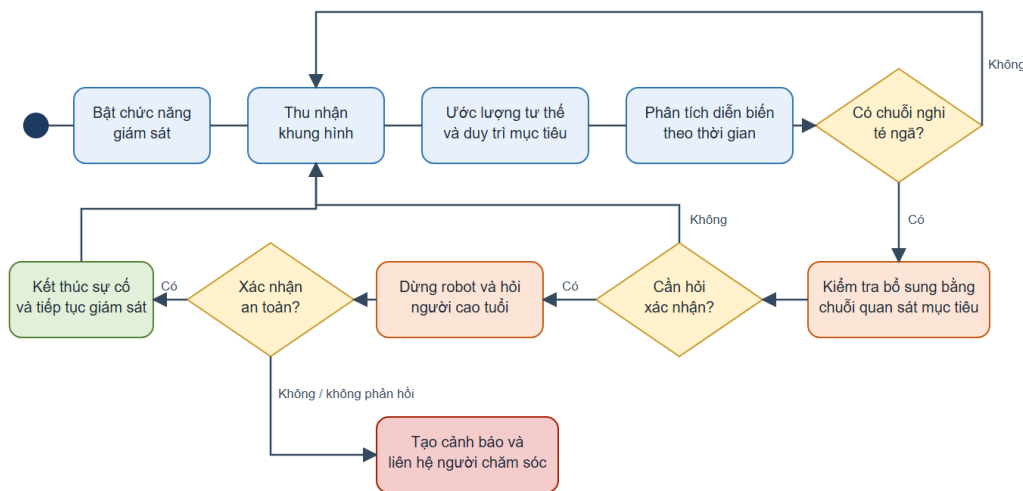
Mã	Use case	Tác nhân chính	Kết quả
UC01	Đăng ký/đăng nhập	Người chăm sóc, thiết bị robot	Tài khoản được tạo hoặc người dùng được xác thực để truy cập đúng dữ liệu chăm sóc
UC02	Quản lý hồ sơ	Người chăm sóc	Thông tin của người cao tuổi được lưu hoặc cập nhật để phục vụ quá trình chăm sóc
UC03	Quản lý lịch nhắc nhở	Người chăm sóc	Lịch nhắc nhở được tạo, cập nhật, bật, tắt hoặc xóa theo thao tác của người chăm sóc
UC04	Nhận và xác nhận lời nhắc	Người cao tuổi	Lời nhắc được phát đúng thời điểm và phản hồi của người cao tuổi được ghi nhận
UC05	Hội thoại trợ lý	Người cao tuổi	Người cao tuổi nhận được câu trả lời bằng giọng nói phù hợp với nội dung trao đổi
UC06	Bật/tắt giám sát	Người chăm sóc	Chức năng bám theo và phát hiện té ngã được bật hoặc tắt theo lựa chọn của người chăm sóc
UC07	Phát hiện và xác nhận té ngã	Người cao tuổi, RobotApp	Tình huống được xác nhận là an toàn hoặc cảnh báo cần hỗ trợ được tạo
UC08	Gọi video	Hai người dùng	Người cao tuổi và người chăm sóc có thể trao đổi âm thanh, hình ảnh và kết thúc cuộc gọi khi cần
UC09	Điều khiển robot	Người chăm sóc	Robot di chuyển theo thao tác của người chăm sóc và dừng khi việc điều khiển kết thúc
UC10	Xem nhật ký	Người chăm sóc	Lịch sử lời nhắc, cảnh báo té ngã và cuộc gọi được hiển thị để người chăm sóc theo dõi

3.3.2 Luồng giám sát và xử lý té ngã

Hình 3.2 mô tả luồng xử lý ở mức chức năng sau khi người chăm sóc bật chế độ giám sát té ngã. Khung hình được chuyển qua các bước ước lượng tư thế, duy trì người mục tiêu và phân tích diễn biến theo thời gian. Kết quả nghi ngờ chưa tạo cảnh báo ngay mà được đưa qua bước hậu kiểm bằng những quan sát tiếp theo. Cách tổ chức này tách bằng chứng do mô hình cung cấp khỏi quyết định xử lý sự cố, nhờ đó hệ thống có thể xem xét thêm trạng thái của người trước khi liên lạc với người chăm sóc.

Khi bằng chứng hậu kiểm cho thấy cần xác nhận, luồng điều khiển chuyển robot về trạng thái dừng và kích hoạt hội thoại với người cao tuổi. Câu trả lời xác nhận an toàn kết thúc nghi ngờ có sự cố té ngã cố và đưa hệ thống trở lại giám sát sau một khoảng phục hồi ổn định. Ngược lại, phản hồi cần trợ giúp, không rõ nghĩa hoặc không có phản hồi sẽ tạo cảnh báo và kích hoạt luồng liên lạc khẩn cấp. Nguyên tắc chuyển trạng thái được làm rõ ở Mục 3.4; các ngưỡng và số lượng quan sát liên tiếp được trình bày cùng cấu hình hiện thực ở Chương 4.

Luồng giám sát và xử lý tình huống nghi té ngã

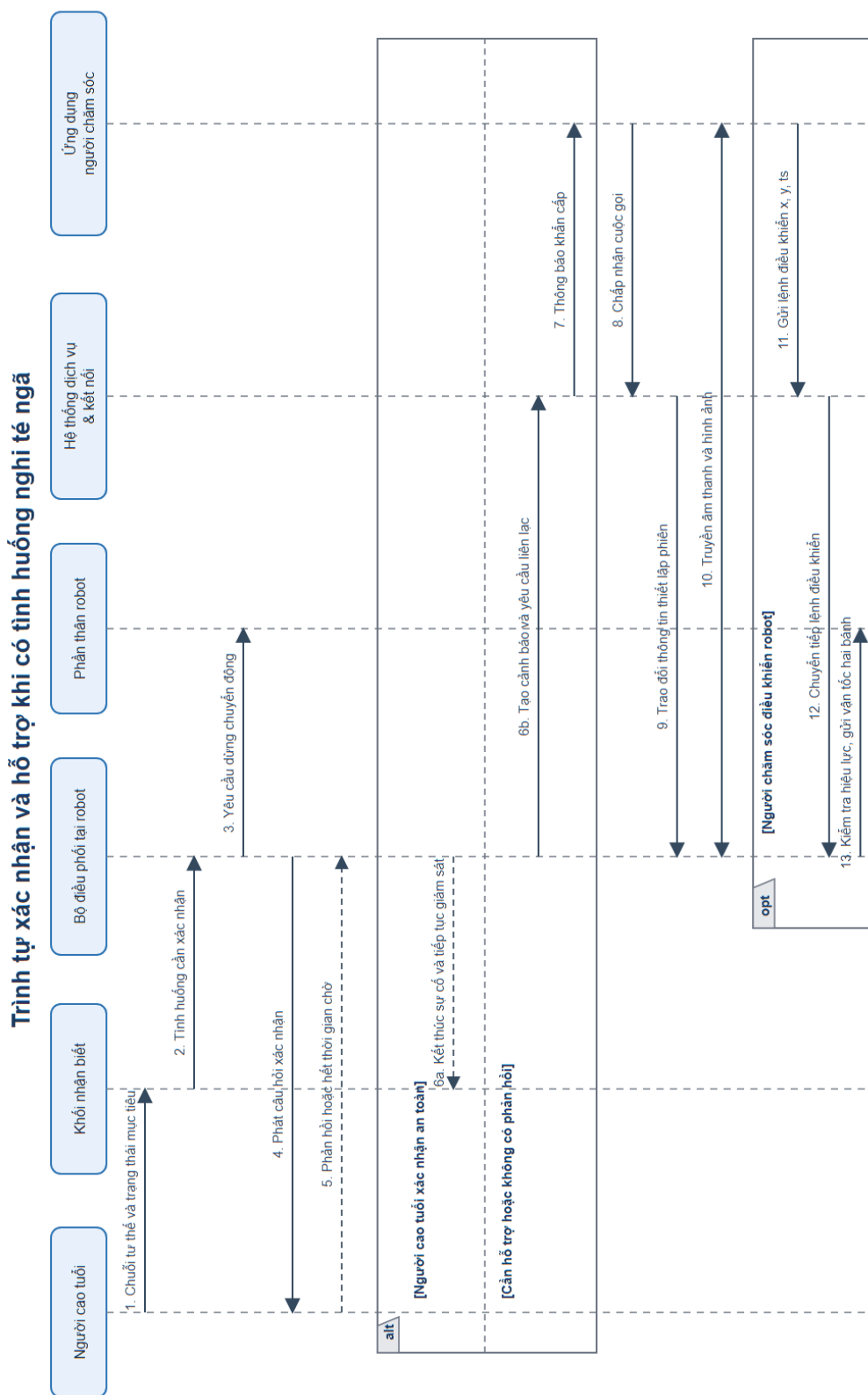


Hình 3.2: Biểu đồ hoạt động của luồng giám sát và xử lý tình huống nghi té ngã

3.3.3 Luồng cuộc gọi khẩn cấp và điều khiển từ xa

Hình 3.3 trình bày sự phối hợp giữa người cao tuổi, hệ thống robot tại nhà, hệ thống dịch vụ & kết nối và người chăm sóc. Khi khối nhận biết phát hiện tình huống cần xác nhận, bộ điều phối tại robot dừng chuyển động và đặt câu hỏi cho người cao tuổi. Nếu người cao tuổi yêu cầu hỗ trợ hoặc hệ thống không nhận được phản hồi xác nhận an toàn, cảnh báo được chuyển đến ứng dụng của người chăm sóc và phiên liên lạc được thiết lập.

Trong cuộc gọi, luồng âm thanh và hình ảnh được tách khỏi luồng dữ liệu chăm sóc. Lệnh điều khiển từ xa đi qua hệ thống dịch vụ & kết nối, được bộ điều phối tại robot kiểm tra trước khi chuyển đến phần thân robot. Việc kiểm tra hiệu lực lệnh và giới hạn chỉ một nguồn điều khiển tại một thời điểm giúp tránh trường hợp lệnh bám theo tự động xung đột với thao tác của người chăm sóc. Công nghệ dùng cho từng kênh truyền được lựa chọn ở Mục 3.4.



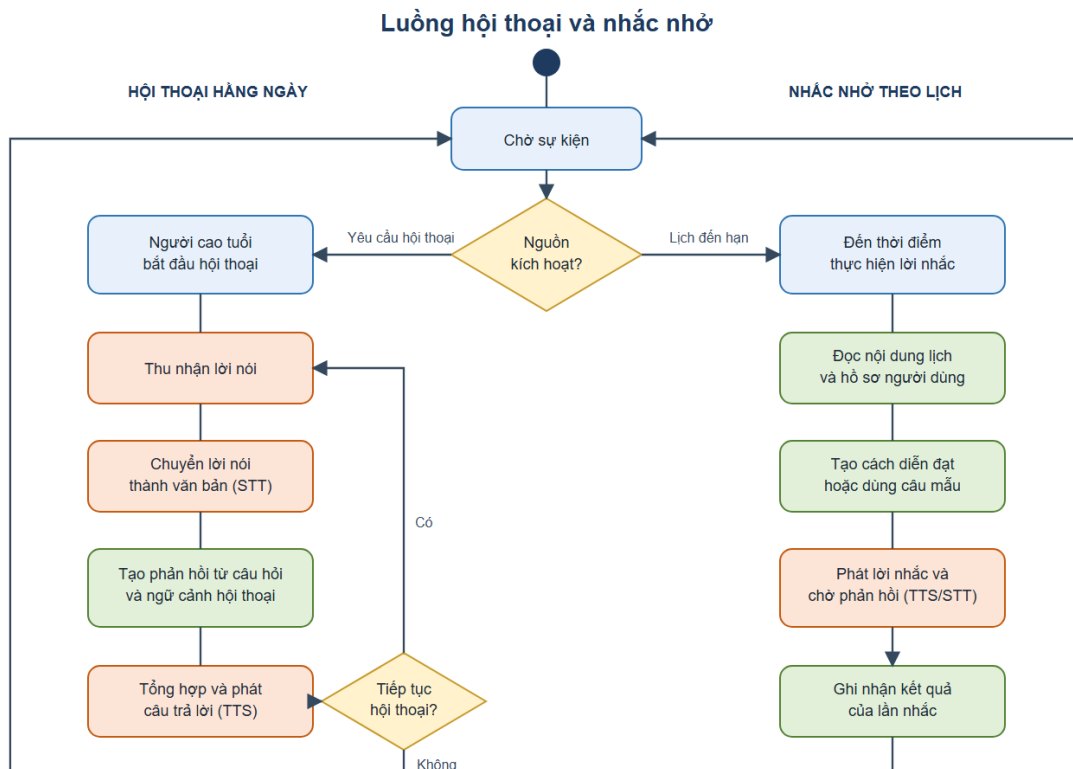
Hình 3.3: Biểu đồ tuần tự của quá trình xác nhận và hỗ trợ khi có tình huống nghi té ngã

3.3.4 Luồng hội thoại và nhắc nhở

Trong hội thoại thông thường, lời nói của người cao tuổi được chuyển thành văn bản và đưa đến module xử lý hội thoại cùng với ngữ cảnh cần thiết. Nội dung phản hồi sau đó được tổng hợp thành giọng nói để robot phát lại. Luồng được tổ chức theo trạng thái để tránh việc hệ thống đồng thời thu âm và phát tiếng, đồng thời cho phép ngắt hoặc trở về trạng thái chờ khi xảy ra lỗi.

Đối với chức năng nhắc nhở, người chăm sóc thiết lập nội dung và thời điểm trên ứng dụng quản lý. Lịch được đồng bộ đến robot; khi điều kiện thời gian thỏa mãn, module nhắc nhở phát nội dung và chờ phản hồi của người cao tuổi. Mỗi lần thực thi được lưu thành một bản ghi lịch sử riêng, tách biệt với cấu hình lịch, giúp người chăm sóc xem lại quá trình thực hiện mà không ảnh hưởng đến lịch đã thiết lập.

Hình 3.4 biểu diễn hai nhánh hoạt động bắt đầu từ trạng thái chờ. Hội thoại được kích hoạt bởi người cao tuổi và có thể tiếp tục qua nhiều lượt, trong khi nhắc nhở được kích hoạt khi lịch đến hạn và kết thúc bằng việc ghi nhận phản hồi. Hai nhánh cùng sử dụng chức năng tổng hợp tiếng nói nhưng duy trì trạng thái riêng, nhờ đó một lời nhắc đang xử lý không bị trộn với nội dung hội thoại thông thường.



Hình 3.4: Biểu đồ hoạt động của luồng hội thoại và nhắc nhở

3.4 Thiết kế và lựa chọn công nghệ cho các module

Sau khi xác định các module và luồng phối hợp, những công nghệ phù hợp được lựa chọn dựa trên độ trễ, tài nguyên của điện thoại, khả năng tích hợp và phạm vi hệ thống. Mỗi loại dữ liệu có đặc điểm riêng: giải pháp phù hợp để lưu trữ hồ sơ chưa chắc đáp ứng việc truyền video thời gian thực, còn một mô hình đạt độ chính xác cao trên máy trạm có thể không duy trì được tốc độ suy luận trên điện thoại. Bảng 3.8 tóm tắt các quyết định lựa chọn công nghệ sử dụng; các mục tiếp theo trình bày thiết kế chi tiết của từng khối.

Bảng 3.8: So sánh các lựa chọn công nghệ xuyên suốt hệ thống

Bài toán	Phương án xem xét	Ưu điểm và hạn chế chính	Lựa chọn cho đề tài
Ứng dụng di động	Phát triển trực tiếp trên Android; nền tảng phát triển đa hệ điều hành; ứng dụng web	Phát triển trực tiếp trên Android thuận tiện khi tích hợp camera, BLE và dịch vụ nền; nền tảng đa hệ điều hành hỗ trợ dùng chung mã giao diện nhưng có thể cần lớp tích hợp riêng	Kotlin, XML và ViewBinding cho hai ứng dụng Android
Suy luận AI	Suy luận trên máy chủ; suy luận tại thiết bị	Máy chủ thuận tiện nâng cấp mô hình nhưng cần truyền ảnh qua mạng; xử lý tại thiết bị giảm phụ thuộc mạng và hạn chế truyền ảnh ra ngoài, song chịu giới hạn tài nguyên	TFLite trên RobotApp với cơ chế dự phòng giữa các bộ xử lý [71]

Bài toán	Phương án xem xét	Ưu điểm và hạn chế chính	Lựa chọn cho đề tài
Đồng bộ dữ liệu	REST tự xây dựng; MQTT; cơ sở dữ liệu thời gian thực	REST cho phép kiểm soát cao nhưng cần xây dựng máy chủ; MQTT phù hợp dữ liệu đo và điều khiển nhưng cần broker cùng cơ chế lưu trữ bổ sung; RTDB cung cấp listener và cây JSON nhưng đòi hỏi quản lý chặt cấu trúc dữ liệu và quy tắc truy cập	Firebase Authentication và Realtime Database cho mô hình một cặp thiết bị [73]
Truyền âm thanh và hình ảnh	Truyền qua cơ sở dữ liệu; máy chủ truyền phát tập trung; WebRTC	Cơ sở dữ liệu không phù hợp truyền hình liên tục; máy chủ tập trung cần thêm hạ tầng; WebRTC hỗ trợ liên lạc thời gian thực nhưng cần quản lý bảo hiệu và phiên kết nối	WebRTC truyền âm thanh, hình ảnh; RTDB trao đổi thông tin thiết lập cuộc gọi [74]

Bài toán	Phương án xem xét	Ưu điểm và hạn chế chính	Lựa chọn cho đề tài
Liên kết điện thoại–thân robot	USB; BLE	USB có độ ổn định cao nhưng ràng buộc thiết bị bằng cáp; BLE phù hợp với các gói lệnh điều khiển có kích thước nhỏ, tiêu thụ ít năng lượng và hỗ trợ kết nối không dây trong phạm vi gần, song có băng thông hạn chế	BLE GATT vì dữ liệu điều khiển có kích thước nhỏ và không bao gồm hình ảnh [77]
Hội thoại	Mô hình cục bộ; LLM qua API; kịch bản cố định	Mô hình cục bộ cần nhiều tài nguyên; LLM qua API hỗ trợ hội thoại linh hoạt nhưng phụ thuộc mạng và chi phí; kịch bản cố định dễ kiểm soát nhưng hạn chế nội dung	Android STT/TTS kết hợp LLM qua API, bổ sung từ khóa nhận biết phản hồi và câu mẫu dự phòng

3.4.1 Module ứng dụng người chăm sóc và điều phối tại robot

Ứng dụng của người chăm sóc được thiết kế như điểm tương tác từ xa, gồm các nhóm chức năng quản lý hồ sơ và lịch nhắc nhở, cấu hình giám sát, tiếp nhận cảnh báo, liên lạc và điều khiển robot. Thành phần này không thực hiện suy luận thị giác hoặc điều khiển động cơ; dữ liệu và lệnh do người chăm sóc tạo ra được chuyển đến các module tương ứng ở phía robot.

Ứng dụng đặt trên robot giữ vai trò điều phối cục bộ. Các nguồn dữ liệu gồm hình ảnh, tiếng nói, lịch nhắc nhở, trạng thái chức năng và lệnh từ xa được phân phối đến đúng

module xử lý. Bộ điều phối đồng thời quản lý quyền sử dụng các tài nguyên dùng chung như camera và kết nối với phần thân robot, đồng thời ưu tiên trạng thái an toàn khi nhiều chức năng cùng yêu cầu điều khiển chuyển động.

Hai ứng dụng được lựa chọn phát triển trực tiếp trên Android vì hệ thống cần sử dụng camera, microphone, Bluetooth, nhận dạng và tổng hợp tiếng nói cùng các tác vụ nền. Thiết kế phần mềm được phân chia theo chức năng, trong đó giao diện tiếp nhận thao tác và hiển thị trạng thái, còn các tác vụ kéo dài hoặc liên quan đến thiết bị được giao cho thành phần có vòng đời phù hợp. Tên lớp và cách tổ chức cụ thể được trình bày ở Chương 4.

3.4.2 Module thu nhận hình ảnh, ước lượng tư thế và theo dõi mục tiêu

Để giám sát và phát hiện té ngã trên điện thoại gắn trên robot, module thị giác cần cung cấp đồng thời bounding box của người và các điểm khớp cơ thể với tốc độ phù hợp cho việc phân tích chuyển động theo thời gian. Việc kết hợp một bộ phát hiện người với một mô hình ước lượng tư thế riêng cho phép thay thế từng thành phần độc lập, nhưng cần hai lượt suy luận và bước chuyển vùng ảnh giữa hai mô hình. Mô hình tích hợp phát hiện người và ước lượng tư thế tạo cả hai loại kết quả trong cùng một lượt suy luận, giúp đơn giản hóa chuỗi xử lý.

Các mô hình MobileNet SSD và EfficientDet tiêu chuẩn phục vụ phát hiện đối tượng nhưng không trực tiếp cung cấp 17 điểm khớp, nên cần thêm mô hình ước lượng tư thế. YOLO-Pose cung cấp đồng thời bounding box và điểm khớp, đồng thời hỗ trợ xuất sang TFLite/LiteRT [80]. Từ quy trình ở Mục 2.3.4, cấu trúc đầu vào, đầu ra ở Mục 2.3.5 và so sánh quy mô ở Mục 2.3.6, đề tài lựa chọn YOLO26n-Pose. Biến thể nano có số tham số và chi phí tính toán thấp nhất trong các mức n, s, m, l và x, phù hợp với điện thoại còn phải thực hiện theo dõi, hội thoại, đồng bộ dữ liệu và điều khiển. Tuy nhiên, chỉ số mAP ước lượng tư thế thấp hơn các biến thể lớn, nên lựa chọn này cần được đánh giá thêm về độ trễ và chất lượng điểm khớp trên thiết bị sử dụng.

Module tiếp nhận một nguồn khung hình hợp lệ tại mỗi thời điểm và hướng đến nhịp thu nhận 25 FPS để chuỗi tư thế có khoảng lấy mẫu tương đồng với dữ liệu dùng cho mô hình phát hiện té ngã. Khung hình được thu phóng nhưng vẫn giữ nguyên tỷ lệ, sau đó bổ sung vùng đệm để phù hợp với đầu vào vuông của mô hình. Sau suy luận, phần đệm và hệ số thu phóng được loại khỏi tọa độ dự đoán trước khi chuyển kết quả về ảnh nguồn. Bounding box và 17 điểm khớp của mỗi người được đóng gói trong cùng một cấu trúc dữ liệu để các module theo dõi, phân tích té ngã và bám người có thể sử dụng

mà không phụ thuộc vào định dạng tensor của mô hình.

Việc gửi khung hình lên dịch vụ đám mây cho phép sử dụng phần cứng mạnh hơn nhưng làm tăng độ trễ, độ ổn định và lưu lượng mạng, đồng thời yêu cầu dữ liệu camera phải được truyền ra ngoài thiết bị làm giảm khả năng riêng tư và bảo mật. Vì vậy, đề tài lựa chọn suy luận trực tiếp trên điện thoại bằng TFLite [71]. Cách xử lý tại biên này giúp dữ liệu hình ảnh dùng cho suy luận được giữ tại robot, giảm tải tính toán cho hạ tầng máy chủ và phân tán khối lượng xử lý trên từng thiết bị, qua đó thuận lợi hơn khi mở rộng hệ thống cho nhiều robot. Thiết kế giới hạn số lượt suy luận đang thực hiện và ưu tiên khung hình mới để không hình thành hàng đợi ảnh cũ; kích thước đầu vào, ngưỡng lọc và thứ tự lựa chọn bộ tăng tốc được xem là cấu hình hiện thực ở Chương 4.

Đối với theo dõi người, SORT có cấu trúc gọn và tốc độ xử lý cao nhưng phụ thuộc vào chất lượng phát hiện. ByteTrack tận dụng thêm các kết quả có độ tin cậy thấp để duy trì đối tượng khi bị che khuất, đồng thời cần thêm bước liên kết kết quả [27][32]. Mạng tái định danh dựa trên học sâu có khả năng phân biệt ngoại hình tốt hơn nhưng làm tăng dung lượng mô hình và thời gian suy luận. Trong bối cảnh của đề tài, robot chủ yếu bám theo một người và cơ chế điều khiển có xu hướng giữ mục tiêu gần vùng trung tâm khung hình. Vì vậy, việc liên kết đối tượng giữa các khung hình liên tiếp không đòi hỏi một mô hình tái định danh phức tạp. Khó khăn đáng chú ý hơn xuất hiện khi người chuyển nhanh giữa các tư thế đứng, ngồi và nằm, làm hình dạng bounding box thay đổi đáng kể hoặc kết quả phát hiện bị gián đoạn trong thời gian ngắn. Trên cơ sở đó, đề tài lựa chọn cấu trúc SORT, đồng thời bổ sung thông tin điểm khớp và đặc trưng màu để tăng khả năng duy trì đúng người mục tiêu.

Trong mỗi khung hình, bộ lọc Kalman của từng track dự báo vị trí và kích thước bounding box dựa trên trạng thái chuyển động trước đó. Các hộp dự báo được so sánh với những detection mới từ YOLO để lập ma trận chi phí. Thuật toán Hungarian tìm phép ghép một–một có tổng chi phí nhỏ nhất trên toàn bộ ma trận, qua đó tránh trường hợp nhiều track cùng chọn một detection hoặc ngược lại. Hàm chi phí liên kết được xác định như sau:

$$C = 0,30(1 - \text{IoU}) + 0,30\hat{d}_c + 0,30\hat{d}_p + 0,10p_a, \quad (3.1)$$

Trong đó, $\hat{d}_c$ biểu diễn khoảng cách giữa tâm hộp dự báo và tâm detection, $\hat{d}_p$ biểu diễn độ dịch chuyển của vùng vai–hông, còn p_a phản ánh mức thay đổi diện tích bounding box. Các đại lượng được chuẩn hóa để hạn chế ảnh hưởng của kích thước người trong

ảnh. Vì vậy, chi phí C nhỏ khi hai hộp chồng lấp tốt, tâm và tư thế ít dịch chuyển, đồng thời kích thước hộp không thay đổi đột ngột. Cách tính và các ngưỡng chuẩn hóa cụ thể được trình bày ở phần hiện thực bộ theo dõi trong Chương 4.

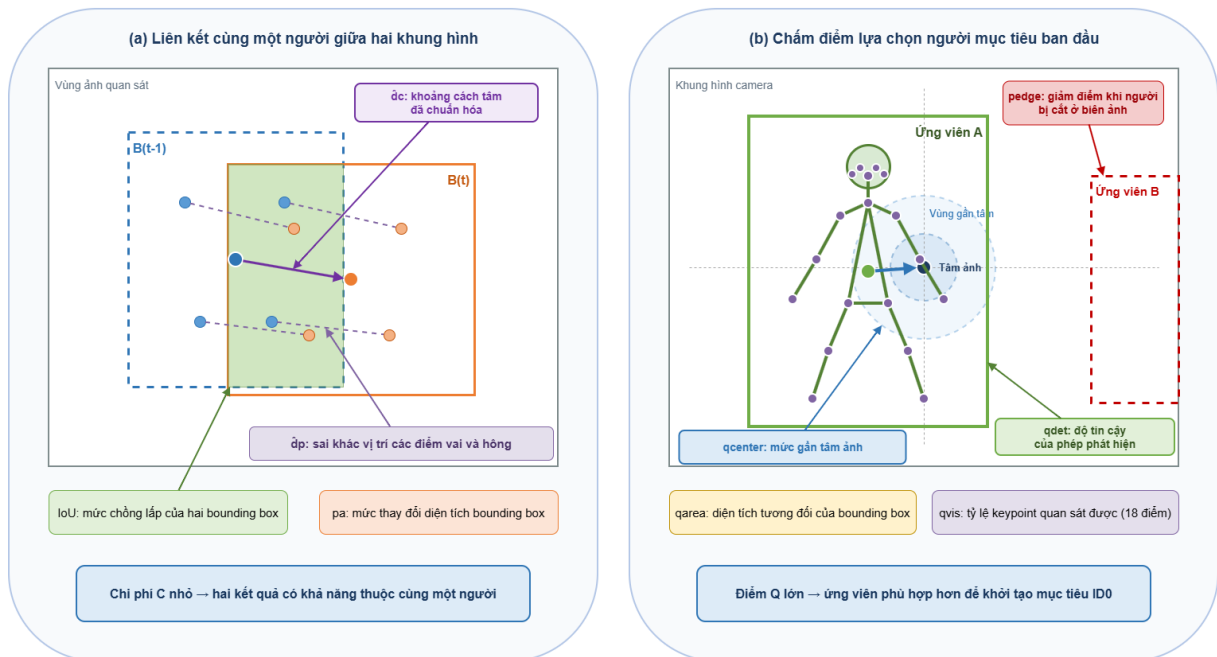
Sau bước gán ghép, detection phù hợp được dùng để hiệu chỉnh trạng thái Kalman và duy trì định danh của track. Track chưa tìm được detection tương ứng vẫn được giữ trong một khoảng thời gian ngắn bằng kết quả dự báo, còn detection chưa được ghép sẽ được xem xét để nhận lại mục tiêu cũ hoặc khởi tạo một track mới. Đặc trưng màu H-S chỉ đóng vai trò hỗ trợ trong các trường hợp có nhiều ứng viên phù hợp hoặc khi mục tiêu xuất hiện lại sau khi mất dấu; quyết định theo dõi thông thường vẫn dựa chủ yếu trên vị trí, độ chồng lấp và chuyển động của cơ thể.

Khi lựa chọn người mục tiêu ban đầu, mỗi đối tượng được đánh giá theo điểm số:

$$Q = 0,60q_{area} + 0,15q_{det} + 0,15q_{vis} + 0,10q_{center} - p_{edge}. \quad (3.2)$$

Trong biểu thức trên, q_{area} phản ánh kích thước tương đối của bounding box, q_{det} là độ tin cậy phát hiện, q_{vis} là tỷ lệ keypoint quan sát được và q_{center} thể hiện mức độ gần tâm ảnh. Thành phần p_{edge} làm giảm điểm của người bị cắt ở biên. Do đó, ứng viên có chất lượng quan sát tốt, nằm gần tâm và xuất hiện tương đối đầy đủ trong khung hình sẽ được ưu tiên. Hình 3.5 minh họa trực quan các đại lượng này.

Minh họa các đại lượng trong hàm liên kết và lựa chọn mục tiêu



Hình 3.5: Minh họa các đại lượng trong hàm liên kết và hàm lựa chọn người mục tiêu

Đối tượng có điểm số cao nhất được gán định danh ID0 để dùng chung cho bám theo và phát hiện té ngã. Mục tiêu chính được duy trì lâu hơn các đối tượng khác và chỉ được khôi phục khi ứng viên mới thỏa mãn đồng thời điều kiện về vị trí, hình học và tính ổn định theo thời gian. Khi đang xử lý một sự kiện nghi té ngã, hệ thống khóa mục tiêu hiện tại và không tự chuyển sang người khác. Các khoảng thời gian và ngưỡng dùng trong nguyên mẫu được trình bày cùng phần hiện thực bộ theo dõi ở Chương 4.

3.4.3 Module tự bám theo và điều khiển chuyển động

Bộ điều khiển chỉ dựa vào tâm bounding box có chi phí tính toán thấp, nhưng có thể đánh giá sai khoảng cách khi phần chân bị cắt khỏi ảnh. Cảm biến chiều sâu cung cấp khoảng cách trực tiếp hơn nhưng làm tăng độ phức tạp phần cứng và nằm ngoài phạm vi đề tài. Do đó, hệ thống kết hợp bounding box với các keypoint và áp dụng chính sách chuyển động thận trọng: khi bounding box có kích thước quá lớn hoặc phần chân không còn trong khung hình, robot ưu tiên lùi để quan sát đầy đủ hơn. Sau khi xuất hiện kết quả FALL, robot không tiến về phía người mà chỉ dừng, lùi chậm hoặc quay tại chỗ; nếu mất ID0 trong trạng thái này, robot dừng ngay.

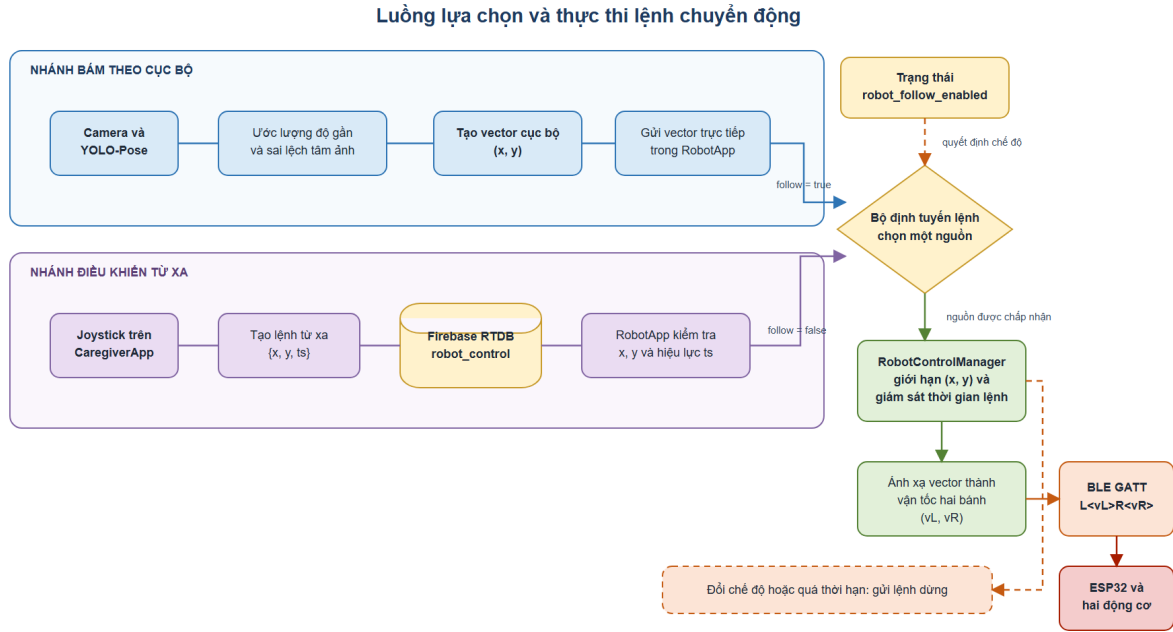
Module bám theo tự động nhận bounding box và các điểm khớp của mục tiêu ID0, sau đó tạo vector điều khiển tịnh tiến và quay đã chuẩn hóa. Độ gần được ước lượng từ kích thước bounding box kết hợp với các mốc trên cơ thể và được làm mượt bằng trung bình trượt lũy thừa (Exponential Moving Average – EMA). Giá trị này được chia thành ba vùng: quá xa, phù hợp và quá gần; các ngưỡng chuyển trạng thái có độ trễ nhằm hạn chế robot đổi hướng liên tục khi kết quả dao động gần biên. Vị trí tương đối của tâm mục tiêu so với tâm ảnh được dùng để xác định hướng quay. Các dao động nhỏ quanh tâm được bỏ qua, còn độ lớn lệnh quay được giới hạn để robot chuyển hướng ổn định và tránh phản ứng quá mạnh.

Gọi c_x là tọa độ ngang của tâm bounding box, W là chiều rộng ảnh và d là độ gần đã làm mượt. Sai lệch ngang của mục tiêu so với tâm ảnh được xác định bởi $e_x = c_x/W - 1/2$. Từ hai đại lượng d và e_x , vector bám theo được hình thành theo nguyên tắc:

$$\begin{aligned}
 x &= \begin{cases} -s_f(d), & d \text{ thuộc vùng quá xa,} \\ 0, & d \text{ thuộc vùng phù hợp,} \\ s_b(d), & d \text{ thuộc vùng quá gần,} \end{cases} \\
 y &= \begin{cases} 0, & |e_x| \leq \varepsilon, \\ \text{clip}_{[-y_{\max}; y_{\max}]}(k_e e_x), & |e_x| > \varepsilon. \end{cases}
 \end{aligned} \tag{3.3}$$

Trong đó, $x < 0$ biểu diễn yêu cầu tiến, $x > 0$ biểu diễn yêu cầu lùi; $s_f(d)$ và $s_b(d)$ là các hàm tăng dần độ lớn chuyển động theo mức sai lệch khoảng cách. Thành phần y điều khiển hướng quay, ε tạo vùng chết quanh tâm ảnh, còn ký hiệu $\text{clip}_{[a; b]}(z)$ biểu diễn việc giới hạn z trong khoảng $[a; b]$. Trước khi được chuyển thành vận tốc hai bánh, hai thành phần tiếp tục được làm mượt để giảm thay đổi đột ngột. Các hàm, hệ số và ngưỡng cụ thể được trình bày trong Chương 4.

Lệnh bám theo tự động và lệnh từ cần điều khiển ảo cùng sử dụng vector chuẩn hóa (x, y) , nhưng đi theo hai đường truyền khác nhau. Vector bám theo được tạo và chuyển trực tiếp đến bộ điều khiển ngay trên RobotApp. Đối với điều khiển từ xa, CaregiverApp ghi thêm thời điểm tạo lệnh ts lên RTDB để RobotApp có thể loại bỏ dữ liệu cũ trước khi thực thi. Do đồng hồ của hai điện thoại có thể sai lệch, thời điểm tạo và thời điểm kiểm tra được quy đổi về một mốc thời gian chung do dịch vụ đồng bộ cung cấp, thay vì so sánh trực tiếp thời gian cục bộ của hai thiết bị. Trạng thái bật/tắt bám theo quyết định nguồn lệnh được chấp nhận: khi bám theo hoạt động, bộ điều khiển chỉ nhận vector cục bộ; khi chuyển sang điều khiển từ xa, luồng bám theo được dừng và bộ lắng nghe lệnh joystick mới được kích hoạt. Hai đường truyền hội tụ tại bước giới hạn giá trị, ánh xạ sang vận tốc hai bánh và gửi lệnh qua BLE, nhờ đó tránh trường hợp hai nguồn đồng thời điều khiển thân robot. Hình 3.6 minh họa quá trình lựa chọn và thực thi lệnh của hai nhánh điều khiển.



Hình 3.6: Luồng lựa chọn và thực thi lệnh điều khiển chuyển động

3.4.4 Module phát hiện, hậu kiểm và xác nhận té ngã

Bài toán phát hiện té ngã có thể được tiếp cận bằng ngưỡng hình học trên từng khung hình, mô hình phân loại trực tiếp từ ảnh hoặc đoạn video, hoặc mô hình phân tích chuỗi điểm khớp. Ngưỡng hình học có chi phí tính toán thấp và dễ giải thích nhưng nhạy với góc nhìn. Mô hình học trực tiếp từ video có thể khai thác cả hình ảnh và chuyển động, song cần nhiều tài nguyên và dữ liệu huấn luyện. Chuỗi điểm khớp có kích thước đầu vào nhỏ hơn và thể hiện được diễn biến tư thế, nhưng phụ thuộc vào chất lượng ước lượng các điểm này. Đề tài lựa chọn YOLO-Pose kết hợp LSTM, sau đó kiểm tra bổ sung theo quy tắc và hỏi xác nhận bằng giọng nói để hạn chế cảnh báo nhầm.

Mỗi khung hình của mục tiêu ID0 được chuyển thành vector 69 đặc trưng:

$$\mathbf{f}_t = [x_i, y_i, \Delta x_i, \Delta y_i]_{i=1}^{17} \oplus r_t, \quad (3.4)$$

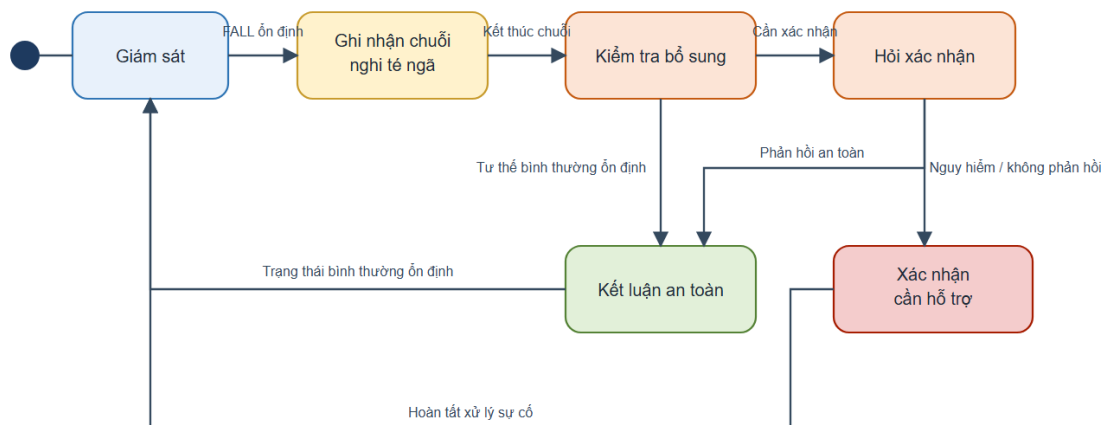
trong đó tọa độ điểm khớp được chuẩn hóa theo tâm hông và kích thước vùng tư thế, $\Delta x_i, \Delta y_i$ là độ dịch chuyển so với khung hình trước, còn r_t là tỷ lệ rộng/cao. Tại khung hình đầu tiên của một mục tiêu, chưa tồn tại quan sát trước đó nên hai thành phần dịch chuyển được đặt bằng 0. Với nhịp lấy mẫu mục tiêu 25 FPS, cửa sổ 15 khung hình biểu diễn khoảng 0,6 s chuyển động và tạo tensor $[1, 15, 69]$. LSTM trả về $[p_{normal}, p_{fall}]$; kết quả được gán nhãn FALL khi $p_{fall} > p_{normal}$. LSTM khai thác quan hệ theo thời gian

của chuỗi tư thế để bổ sung thông tin chuyển động cho việc phân loại [25].

Bộ hậu kiểm được thiết kế như một máy trạng thái gồm các giai đoạn giám sát, ghi nhận chuỗi nghi té ngã, kiểm tra bổ sung, hỏi xác nhận và kết thúc sự cố. Một kết quả FALL đơn lẻ không đủ làm hệ thống chuyển trạng thái; bộ xử lý yêu cầu bằng chứng ổn định qua nhiều khung hình. Khi chuỗi nghi ngờ kết thúc, vị trí tương đối của phần thân trên và khả năng tiếp tục quan sát mục tiêu được dùng để phân biệt trường hợp người đã trở lại tư thế bình thường với trường hợp cần hỏi xác nhận. Sau khi sự cố được kết luận, hệ thống chỉ trở về giám sát khi trạng thái bình thường đã ổn định, qua đó hạn chế tạo nhiều yêu cầu cho cùng một sự kiện. Giá trị cụ thể của các bộ đếm và ngưỡng hình học được trình bày ở phần hiện thực để phản ánh đúng cấu hình của nguyên mẫu.

Hình 3.7 biểu diễn các trạng thái chính và điều kiện chuyển tiếp của quá trình hậu kiểm. Sơ đồ tách rõ kết quả dự đoán của mô hình với quyết định phát cảnh báo: mô hình cung cấp bằng chứng ban đầu, còn việc hỏi xác nhận và kết thúc sự cố được quyết định theo chuỗi quan sát cùng phản hồi của người dùng.

Trạng thái hậu kiểm và xác nhận té ngã



Hình 3.7: Sơ đồ trạng thái của quá trình hậu kiểm và xác nhận té ngã

3.4.5 Module hội thoại và lời nhắc

Mô hình hội thoại chạy hoàn toàn tại thiết bị hạn chế phụ thuộc mạng và việc gửi dữ liệu ra ngoài, nhưng cần chia sẻ tài nguyên với chức năng thị giác. Phương án dùng kịch bản cố định có chi phí xử lý thấp và dễ kiểm soát nội dung, song khó đáp ứng hội thoại hằng ngày với nhiều chủ đề. Vì vậy, đề tài kết hợp Android SpeechRecognizer để

chuyển tiếng nói thành văn bản, LLM qua API để tạo câu trả lời và Android TextToSpeech để tổng hợp tiếng nói [34][40][35]. Những phản hồi thể hiện rõ tình trạng an toàn hoặc nguy hiểm được nhận biết trước bằng từ khóa tại thiết bị; chức năng nhắc nhở sử dụng câu mẫu dự phòng khi không nhận được phản hồi từ API.

Hoạt động hội thoại được tổ chức theo các trạng thái nghỉ, lắng nghe, xử lý câu hỏi, phát câu trả lời, xác nhận lời nhắc, xác nhận té ngã và đang gọi. Hồ sơ người dùng và một phần lịch sử gần được sử dụng làm ngữ cảnh cho hội thoại. Phản hồi dạng truyền luồng được ghép thành từng câu trước khi đưa vào TTS để robot có thể bắt đầu phát lời sớm hơn. Những tác vụ gắn liền quan đến an toàn sử dụng luồng xử lý tách biệt và không làm thay đổi lịch sử trò chuyện. Với chức năng nhắc nhở, nội dung lịch do người chăm sóc thiết lập vẫn là dữ liệu chính; câu sinh bởi LLM chỉ hỗ trợ cách diễn đạt và luôn có phương án câu mẫu khi dịch vụ không khả dụng.

3.4.6 Module dữ liệu dùng chung và liên lạc thời gian thực

Firestore Realtime Database được lựa chọn để đồng bộ các dữ liệu có kích thước nhỏ như trạng thái chức năng, lệnh điều khiển và sự kiện. Các bộ lắng nghe thông báo thay đổi đến hai ứng dụng gần thời gian thực, kết hợp với Firebase Authentication để xác định tài khoản truy cập. Giải pháp này giảm khối lượng xây dựng máy chủ riêng so với REST hoặc việc bổ sung cơ chế lưu trữ cho MQTT. Tuy nhiên, hai ứng dụng cần thống nhất cấu trúc cây JSON và đường dẫn; việc thay đổi cấu trúc dữ liệu phải được cập nhật đồng bộ. Các đường dẫn vì vậy được quản lý tập trung và phân vùng theo UID. Lệnh điều khiển được gắn thời điểm tạo theo mốc thời gian dùng chung để kiểm tra hiệu lực mà không phụ thuộc vào độ lệch đồng hồ giữa hai thiết bị. Hai thiết bị dùng chung tài khoản nên hệ thống hiện chưa phân quyền riêng cho robot và người chăm sóc [73][81].

Âm thanh và hình ảnh được truyền bằng WebRTC, còn RTDB chuyển trạng thái và thông tin thiết lập cuộc gọi, gồm bản mô tả phiên đề nghị, bản trả lời và các địa chỉ ứng viên ICE. WebRTC hỗ trợ kết nối trực tiếp khi điều kiện mạng cho phép và chuyển tiếp qua TURN khi cần [74][75]. Thiết kế quy định camera chỉ có một thành phần sở hữu tại mỗi thời điểm: ngoài cuộc gọi, nguồn hình ảnh phục vụ xem trước và suy luận; trong cuộc gọi, module truyền thông tiếp quản camera nhưng vẫn cung cấp bản sao khung hình cục bộ cho khối nhận biết. Quy tắc này tránh xung đột tài nguyên mà không làm gián đoạn chức năng giám sát.

3.4.7 Module cầu nối BLE và chấp hành thân robot

Dữ liệu gửi đến thân robot chỉ gồm vận tốc đặt của hai bánh, có kích thước nhỏ và không chứa hình ảnh. BLE GATT đáp ứng nhu cầu trao đổi này trong phạm vi gần, đồng thời cho phép kết nối không dây giữa điện thoại và ESP32. Khi sử dụng thao tác ghi không phản hồi, phía gửi không nhận xác nhận ở mức GATT cho từng lệnh. Vì vậy, hệ thống kết hợp kiểm tra thời hạn lệnh và cơ chế tự dừng khi không nhận cập nhật, bên cạnh việc chủ động gửi lệnh dừng [77].

Module cầu nối chịu trách nhiệm kiểm tra thời điểm tạo lệnh, giới hạn miền giá trị và chuyển vector điều khiển thành vận tốc đặt của hai bánh trước khi truyền qua BLE. Ở phía thân robot, vi điều khiển tiếp nhận hai giá trị đặt, áp dụng giới hạn an toàn và cập nhật điều khiển vận tốc cho từng động cơ. Hai kênh động cơ có cùng cấu trúc cảm biến–driver–động cơ nhưng hoạt động với phản hồi và giá trị đặt độc lập, phù hợp với cơ cấu dẫn động vi sai. Cấu hình chân kết nối, thông số động cơ, giới hạn vận hành và tham số bộ điều khiển được trình bày cùng phần hiện thực firmware ở Chương 4.

3.4.8 Module chuẩn bị và triển khai mô hình AI

Quy trình chuẩn bị mô hình được thiết kế thành các bước liên tục gồm tổ chức dữ liệu, huấn luyện, chuyển đổi định dạng, kiểm tra tương đương và đóng gói vào ứng dụng. Mô hình triển khai phải công bố rõ kích thước đầu vào, kiểu dữ liệu, cấu trúc tensor đầu ra và quy tắc tiền xử lý; bộ giải mã ở ứng dụng chỉ phụ thuộc vào giao diện này thay vì phụ thuộc trực tiếp vào môi trường huấn luyện.

Quá trình chuyển đổi có thể làm thay đổi toán tử, độ chính xác số hoặc quy ước đầu ra. Vì vậy, cùng một tập mẫu cần được chạy qua mô hình gốc và mô hình triển khai để đối chiếu kết quả trước khi tích hợp. Cấu hình dữ liệu, tham số huấn luyện, phiên bản công cụ và mã băm của tệp mô hình cũng cần được lưu cùng nhau để quy trình có thể được lặp lại. Định dạng trung gian, công cụ chuyển đổi và các tệp tạo ra được trình bày ở Chương 4.

3.5 Thiết kế dữ liệu, giao thức truyền thông và phương thức kết nối

3.5.1 Cây dữ liệu Firebase

Firebase RTDB lưu dữ liệu theo cấu trúc cây JSON và chuyển thay đổi đến các bộ lắng nghe của hai ứng dụng [73]. Dữ liệu vận hành của từng cặp thiết bị được gom

dưới `pair_sessions/{uid}`, trong khi hồ sơ, lịch và nhật ký lời nhắc được tổ chức thành các nhánh theo UID ở cấp gốc.

Bảng 3.9: Cấu trúc dữ liệu của *Firebase Realtime Database*

Đường dẫn	Trường chính	Mục đích
<code>Users/{uid}</code>	<code>fullName</code> , <code>robotName</code> , <code>pronoun</code> , <code>caregiver</code> , <code>medicalConditions</code> , <code>notes</code> , <code>hobbies</code>	Hồ sơ và ngữ cảnh hội thoại
<code>Schedules/{uid}/{id}</code>	<code>title</code> , <code>note</code> , <code>dateMillis</code> , <code>hour</code> , <code>minute</code> , <code>repeatMode</code> , <code>active</code>	Lịch một lần hoặc lặp
<code>reminder_logs/{uid}/{id}</code>	<code>scheduleId</code> , <code>triggeredAt</code> , <code>status</code> , <code>elderlyResponse</code> , <code>retryCount</code> , <code>confirmedAt</code>	Nhật ký thực thi lời nhắc
<code>pair_sessions/{uid}/robot_follow_enabled</code>	Boolean	Bật/tắt auto-follow
<code>pair_sessions/{uid}/ai_fall_detection_enabled</code>	Boolean	Bật/tắt LSTM/hậu kiểm
<code>pair_sessions/{uid}/robot_control</code>	<code>x</code> , <code>y</code> , <code>ts</code>	Lệnh joystick từ CaregiverApp
<code>pair_sessions/{uid}/webrtc_signal</code>	<code>status</code> , <code>callerRole</code> , <code>emergencyReason</code> , <code>offer</code> , <code>answer</code> , <code>robotCandidates</code> , <code>caregiverCandidates</code>	Signaling và trạng thái cuộc gọi

Đường dẫn	Trường chính	Mục đích
pair_sessions/ {uid}/fall_ alerts/{pushId}	timestamp, trackId, probFall, verificationReason, confirmationStatus, severity, evidence	Cảnh báo nguy hiểm/không phản hồi
pair_sessions/ {uid}/call_ history/{pushId}	timestamp, initiator, durationSec, status	Nhật ký cuộc gọi

Quy tắc bảo mật mặc định đặt `.read=false` và `.write=false`; mỗi nhánh chỉ cho phép truy cập khi `auth != null` và `auth.uid === $uid`. Hai nhánh lịch sử được lập chỉ mục theo thời điểm để hỗ trợ truy vấn. Các quy tắc này giới hạn người dùng vào vùng dữ liệu thuộc UID tương ứng [81]. Tuy nhiên, robot và người chăm sóc chưa có quyền truy cập riêng vì hai thiết bị đang sử dụng cùng tài khoản.

3.5.2 Cấu trúc dữ liệu điều khiển trên RTDB

```

1 pair_sessions/{uid}/robot_control = {
2   "x": Number,    // translation, [-1, 1]
3   "y": Number,    // steering,      [-1, 1]
4   "ts": Long      // Unix epoch milliseconds
5 }
```

Listing 1: Cấu trúc lệnh điều khiển trong phiên ghép cặp

Ba trường trong lệnh điều khiển từ xa có ý nghĩa như sau:

- $x \in [-1, 1]$ là thành phần tịnh tiến dọc theo thân robot. Theo quy ước đã hiệu chỉnh, $x < 0$ yêu cầu robot tiến, $x > 0$ yêu cầu robot lùi và $x = 0$ không tạo chuyển động tịnh tiến. Độ lớn $|x|$ biểu diễn mức yêu cầu tương đối, không có đơn vị m/s.
- $y \in [-1, 1]$ là thành phần quay. Hai dấu của y tương ứng với hai chiều quay đã hiệu chỉnh trên robot; $y = 0$ yêu cầu đi thẳng hoặc không quay. Với cần điều khiển trên màn hình, kéo sang trái tạo $y > 0$ và kéo sang phải tạo $y < 0$.
- ts là thời điểm tạo lệnh theo Unix epoch, tính bằng mili giây và được quy đổi về mốc thời gian của dịch vụ Firebase. Trường này không biểu diễn tốc độ hoặc thời

gian duy trì chuyển động mà được RobotApp dùng để loại lệnh cũ hay lệnh có thời điểm bất thường. Sau bước kiểm tra, ts không được chuyển tiếp đến ESP32.

Cần điều khiển ảo gồm một vùng tròn cố định và một núm điều khiển di chuyển bên trong. Gọi $(\Delta x, \Delta y)$ là độ lệch của núm so với tâm vùng điều khiển theo hệ tọa độ màn hình và r là bán kính di chuyển cực đại. Khi thao tác vượt ra ngoài đường tròn, vector độ lệch được co về biên nhưng vẫn giữ nguyên hướng. Sau đó CaregiverApp chuẩn hóa độ lệch thành lệnh:

$$x = \frac{\Delta y}{r}, \quad y = -\frac{\Delta x}{r}. \quad (3.5)$$

Do tọa độ dọc trên màn hình tăng từ trên xuống, thao tác kéo lên tạo $x < 0$, kéo xuống tạo $x > 0$; dấu âm trước Δx chuyển thao tác ngang sang quy ước quay đã hiệu chỉnh. Khi núm nằm ở tâm hoặc được thả ra, ứng dụng gửi $x = 0, y = 0$. Mỗi lệnh joystick được ghi kèm thời điểm tạo đã quy đổi về mốc thời gian chung để RobotApp đánh giá hiệu lực của dữ liệu nhận qua mạng. Vector bấm theo được tạo cục bộ theo Công thức (3.3), vì vậy không được ghi vào nhánh `robot_control` và không sử dụng timestamp của RTDB.

Đối với lệnh từ RTDB, RobotApp chỉ tiếp nhận khi x, y là các giá trị hữu hạn, có thời điểm tạo lệnh và đã xác định được mốc thời gian dùng chung. Tuổi lệnh được tính trên cùng mốc này; lệnh quá 2.000 ms hoặc có thời điểm vượt quá thời gian hiện tại hơn 5.000 ms đều bị loại bỏ. Bản ghi đang lưu tại thời điểm bộ lắng nghe vừa được kích hoạt cũng không được xem là thao tác điều khiển mới. Sau khi nguồn điều khiển được lựa chọn, vector từ xa hợp lệ hoặc vector bấm theo cục bộ đều được giới hạn trong $[-1, 1]$ và ánh xạ thành vận tốc đặt của hai động cơ như sau:

$$t = \text{clip}_{[-1; 1]}(x), \quad (3.6)$$

$$s = 0,15 \text{ clip}_{[-1; 1]}(y), \quad (3.7)$$

$$v_L = (s + t)20, \quad (3.8)$$

$$v_R = (s - t)20. \quad (3.9)$$

Nếu $|x| > 0,3$ và $|y| < 0,3$, thành phần quay được đưa về 0 để giảm dao động khi đi thẳng. Do hai động cơ được lắp đối xứng, dấu của vận tốc đặt tuân theo chiều trục

động cơ đã hiệu chỉnh. Vì vậy, hướng chuyển động của robot phải được xác định theo quy ước lắp đặt, không chỉ dựa vào việc hai giá trị vận tốc cùng dấu hay trái dấu.

Bảng 3.10: Các điểm chuẩn ánh xạ lệnh điều khiển

x	y	(v_L, v_R)	Ý nghĩa hiệu chỉnh
-1	0	$(-20, 20)$	Tiến
1	0	$(20, -20)$	Lùi
0	1	$(3, 3)$	Quay theo chiều dương
0	-1	$(-3, -3)$	Quay theo chiều âm
0	0	$(0, 0)$	Dừng

3.5.3 Giao tiếp BLE và điều khiển động cơ trên ESP32

RobotApp đóng vai trò máy khách GATT, còn ESP32 là máy chủ GATT có tên OhmniRobot. Mã định danh dịch vụ là 4FAFC201-1FB5-459E-8FCC-C5C9C331914B; đặc tính nhận lệnh có UUID BEB5483E-36E1-4688-B7F5-EA07361B26A8. Gói lệnh là chuỗi ASCII dạng $L\langle float \rangle R\langle float \rangle \backslash n$, định dạng số theo Locale.US và được gửi bằng thao tác ghi không phản hồi WRITE_NO_RESPONSE. Cách tổ chức dịch vụ và đặc tính tuân theo mô hình GATT [77].

Ở phía nhận, hai giá trị vận tốc phải được kiểm tra định dạng và tính hữu hạn trước khi cập nhật giá trị đặt. Firmware áp dụng vùng không đáp ứng, giới hạn vận tốc và cơ chế dừng khi mất kết nối hoặc quá thời gian chờ. Trong mỗi chu kỳ, phản hồi góc của từng động cơ được cập nhật trước khi giá trị đặt mới được đưa vào bộ điều khiển. Các hàm và hằng số thực tế dùng để hiện thực những yêu cầu này được trình bày ở Mục 4.7.

3.5.4 Cơ chế dừng an toàn nhiều tầng

Hệ thống kết hợp bốn cơ chế dừng tại các vị trí khác nhau trong chuỗi điều khiển:

1. Ứng dụng gửi lệnh dừng khi người dùng thả cần điều khiển, ấn phần điều khiển, tắt bảm theo tự động hoặc kết thúc cuộc gọi.
2. Ứng dụng tại robot chỉ tiếp nhận lệnh khi đã đồng bộ được mốc thời gian, đồng thời loại bỏ bản cập nhật đã hết hiệu lực, sai kiểu hoặc chứa giá trị không hữu hạn.
3. Module cầu nối chủ động gửi lệnh dừng khi không nhận được bản cập nhật điều khiển trong thời gian quy định.

4. Firmware độc lập đưa vận tốc đặt của hai động cơ về 0 khi quá thời gian chờ hoặc khi kết nối BLE bị ngắt.

Cơ chế giám sát thời gian nhận lệnh được đặt ở cả ứng dụng và ESP32. Thời gian chờ tại ứng dụng ngắn hơn phía firmware để ứng dụng có cơ hội chủ động gửi lệnh dừng trước. ESP32 vẫn kiểm tra độc lập và đưa vận tốc về 0 nếu ứng dụng không tiếp tục cập nhật hoặc kết nối bị ngắt trước khi lệnh dừng được truyền đến. Các giá trị thời gian chờ cụ thể được nêu ở Chương 4.

3.6 Kết luận chương

Chương đã xác định phạm vi, yêu cầu chức năng và yêu cầu chất lượng làm cơ sở phân chia nhiệm vụ cho RobotApp, CaregiverApp, hệ thống dịch vụ & kết nối, cùng phần thân robot. Trên kiến trúc tổng quan đó, các luồng giám sát, hội thoại, nhắc nhở, liên lạc và điều khiển được phân tích để lựa chọn công nghệ phù hợp. Thiết kế cũng xác định cách chuẩn bị mô hình AI, tổ chức dữ liệu, truyền nhận lệnh và phối hợp các cơ chế dừng an toàn. Những nội dung này là cơ sở để hiện thực hệ thống ở Chương 4 và đánh giá kết quả ở Chương 5.

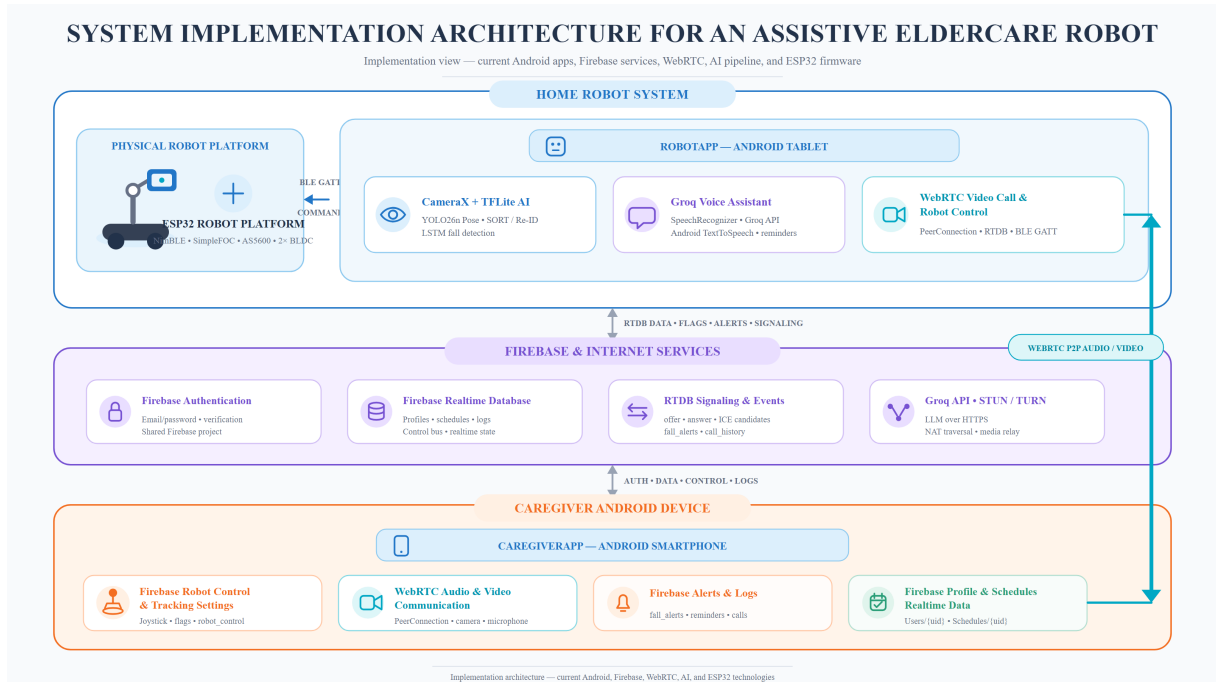
Chương 4 – Hiện thực hệ thống

Trên cơ sở kiến trúc và các đặc tả ở Chương 3, chương này trình bày quá trình chuyển các module thiết kế thành những thành phần phần mềm và phần cứng cụ thể. Nội dung tập trung vào cấu trúc lớp, dịch vụ, mô hình, cấu hình vận hành và firmware, đồng thời làm rõ cách các thành phần phối hợp trong hệ thống. Kết quả kiểm thử và mức độ đáp ứng yêu cầu được trình bày riêng ở Chương 5.

4.1 Kiến trúc hiện thực và ánh xạ công nghệ

Hình 4.1 thể hiện các thành phần và công nghệ được sử dụng theo thiết kế ở Chương 3. CaregiverApp cung cấp giao diện cho người chăm sóc. Firebase Authentication, Firebase Realtime Database, dịch vụ LLM và hạ tầng STUN/TURN hỗ trợ xác thực, đồng bộ và liên lạc. RobotApp đảm nhiệm xử lý hình ảnh, hội thoại và điều phối tại robot. Phần thân gồm ESP32, cảm biến góc, mạch công suất và hai động cơ BLDC; firmware sử dụng BLE GATT để nhận lệnh và SimpleFOC để điều khiển động cơ.

Firebase RTDB lưu dữ liệu dưới dạng cây JSON và đồng bộ trạng thái chức năng, lệnh điều khiển cùng thông tin thiết lập cuộc gọi gần thời gian thực. Âm thanh và hình ảnh được truyền bằng WebRTC, trực tiếp giữa hai thiết bị hoặc qua máy chủ TURN. Dịch vụ Groq xử lý hội thoại và các yêu cầu phân loại ngôn ngữ. RobotApp tiếp nhận kết quả AI, phối hợp các trạng thái xử lý và chuyển lệnh chuyển động từ RTDB hoặc chức năng bấm theo đến ESP32 qua BLE. Firmware thực hiện lệnh, điều khiển động cơ và dừng khi không còn nhận được cập nhật.



Hình 4.1: Kiến trúc hiện thực của hệ thống

Để làm rõ mối liên hệ giữa hai chương, Hình 4.2 đối chiếu các module được xác định ở giai đoạn thiết kế với những thành phần hiện thực chính. Mỗi module có thể được triển khai bởi nhiều lớp hoặc dịch vụ, nhưng vẫn giữ nguyên trách nhiệm, dữ liệu vào–ra và ranh giới giao tiếp đã mô tả ở Chương 3. Các mục tiếp theo lần lượt trình bày cấu trúc và cách phối hợp của từng nhóm thành phần này.



Hình 4.2: Ảnh xạ từ module thiết kế đến các thành phần hiện thực

4.2 Môi trường phát triển và tổ chức các thành phần

Hệ thống gồm bốn thành phần được phát triển độc lập nhưng sử dụng cấu trúc dữ liệu và quy ước trao đổi thống nhất. RobotApp và CaregiverApp được xây dựng bằng Kotlin, giao diện XML và ViewBinding. RobotApp sử dụng package `com.example.datn_v1`, còn CaregiverApp sử dụng package `com.example.datn_caregiver`; cả hai cùng đặt `minSdk=23`, `targetSdk=36` và Java/Kotlin JVM target 11. Firmware được xây dựng dưới dạng chương trình Arduino cho ESP32. Thành phần AI gồm mô hình sau huấn luyện, mô hình ONNX trung gian, mô hình TFLite và các chương trình chuyển đổi, kiểm tra suy luận.

RobotApp sử dụng CameraX, các gói TensorFlow Lite (CPU, GPU, Support và Select TF Ops), OpenCV Android, Firebase Authentication, Firebase Database, WebRTC native, OkHttp và Kotlin Coroutines. CaregiverApp sử dụng Firebase Authentication, Firebase Database, Android Credentials, RecyclerView và WebRTC. Tập mô hình TFLite được cấu hình `noCompress` để giữ nguyên dạng khi đóng gói, cho phép truy cập bằng cơ chế ánh xạ tệp vào bộ nhớ.

Bảng 4.1: Các thành phần hiện thực của hệ thống

Khối	Công nghệ chính	Vai trò
RobotApp	Android/Kotlin, CameraX, TFLite, OpenCV, Firebase, WebRTC, BLE, OkHttp	Xử lý AI, hội thoại, cuộc gọi và kết nối với thân robot
CaregiverApp	Android/Kotlin, Firebase Auth/RTDB, WebRTC	Ứng dụng cho người thân quản lý, giám sát và điều khiển
Mô hình AI	Python, Keras/TensorFlow, Ultralytics, ONNX, onnx2tf, TFLite	Chuẩn bị dữ liệu, huấn luyện và chuyển đổi mô hình cho Android
Firmware	ESP32, NimBLE, SimpleFOC, AS5600, 3PWM	Tiếp nhận, phân tích lệnh qua BLE và điều khiển hai động cơ BLDC

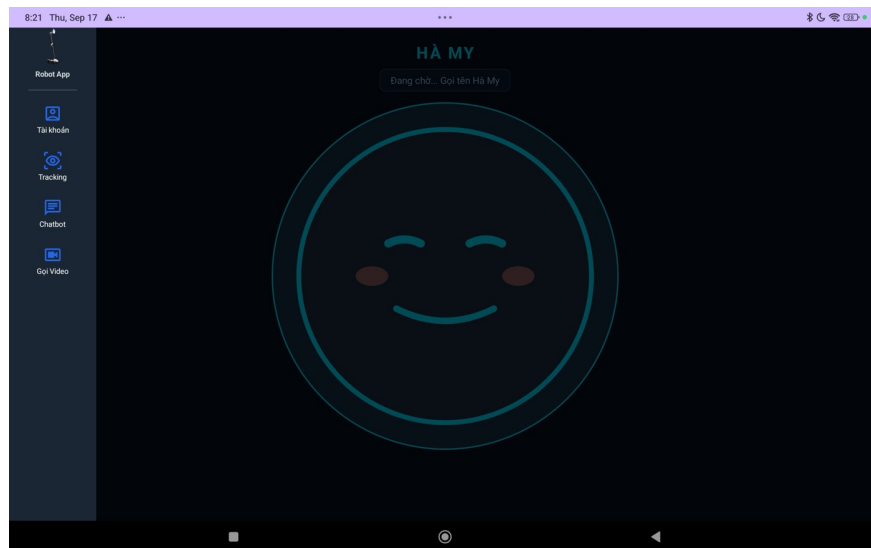
4.3 Hiện thực RobotApp

4.3.1 Khởi động, điều hướng và vòng đời dịch vụ

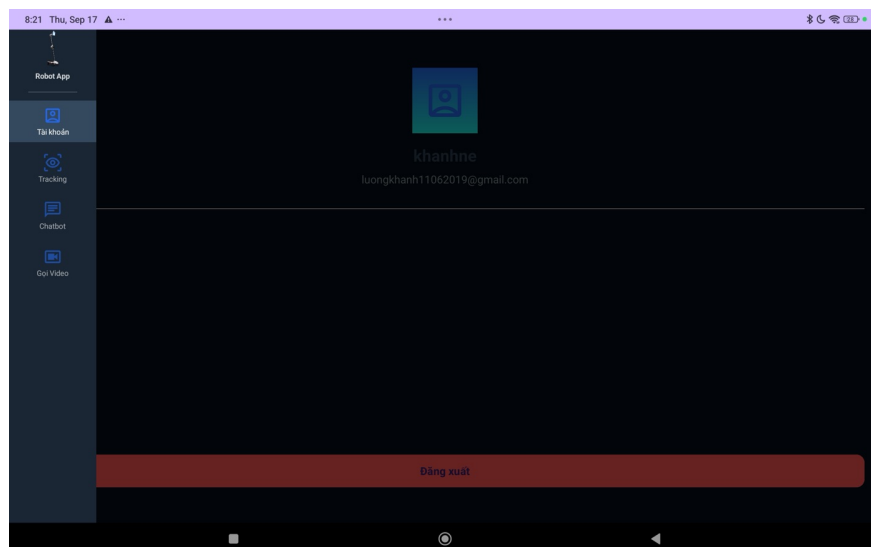
LoginActivity là màn hình khởi động của RobotApp. Sau khi xác thực, MainActivity quản lý các fragment hồ sơ, theo dõi, chatbot và cuộc gọi, đồng thời lắng nghe nhánh webrtc_signal để hiển thị yêu cầu gọi đến. ReminderService theo dõi lịch nhắc nhở; TrackingService duy trì thông báo của dịch vụ chạy nền và chuyển sự kiện té ngã từ khối theo dõi đến chatbot. Việc thu nhận hình ảnh và suy luận AI được thực hiện trong tracking_fragment. Tập AndroidManifest khai báo các quyền sử dụng camera, microphone, Internet, thông báo, dịch vụ nền và Bluetooth theo phiên bản hệ điều hành.

Quyền Bluetooth được khai báo theo từng nhóm phiên bản hệ điều hành. Android 11 trở xuống sử dụng BLUETOOTH, BLUETOOTH_ADMIN và quyền vị trí để quét thiết bị; từ Android 12, ứng dụng sử dụng BLUETOOTH_SCAN và BLUETOOTH_CONNECT theo hướng dẫn của Android [84]. Camera được khai báo required=false để không ngăn cài đặt chỉ vì thiết bị thiếu cờ tính năng tương ứng, nhưng các chức năng thị giác chỉ hoạt động khi có camera.

Hình 4.3 thể hiện bố cục màn hình của RobotApp trên máy tính bảng. Thanh điều hướng bên trái cho phép chuyển giữa bốn fragment hồ sơ, theo dõi, chatbot và cuộc gọi; vùng nội dung bên phải hiển thị fragment đang được chọn. Hồ sơ tại Users/{uid} được hiển thị ở chế độ chỉ đọc vì việc chỉnh sửa thông tin được thực hiện từ CaregiverApp.



(a) Màn hình chính với thanh điều hướng bốn chức năng



(b) Fragment hồ sơ hiển thị thông tin tài khoản dùng chung

Hình 4.3: Giao diện khởi động và điều hướng của RobotApp

4.3.2 TrackingFragment và quản lý camera

`tracking_fragment.kt` điều phối CameraX, mô hình ước lượng tư thế, bộ theo dõi, LSTM, bộ hậu kiểm và chức năng bấm người. CameraX cung cấp `Preview` để hiển thị hình ảnh xem trước và `ImageAnalysis` để phân tích khung hình. Dải tốc độ khung hình của hai use case được yêu cầu ở mức [25, 25] FPS thông qua Camera2Interop nhằm duy trì nhịp lấy mẫu thống nhất cho chuỗi tư thế. Bộ phân tích chuyển dữ liệu từ `ImageProxy` sang ảnh bitmap và tensor, sau đó giải phóng khung hình đã xử lý. Hai chức năng camera được liên kết với vòng đời của fragment theo cơ chế CameraX [85]. Khả năng duy trì đúng tốc độ yêu cầu phụ thuộc vào dải FPS mà camera của thiết bị hỗ trợ.

Khi không có cuộc gọi, CameraX được gắn với vòng đời của fragment. Trước khi WebRTC khởi động, CameraX được tháo liên kết để tránh hai thành phần cùng mở camera. `video_call_fragment` tiếp quản camera ở cấu hình 640 × 480, 25 FPS và gửi một bản sao `VideoFrame` cục bộ về khối theo dõi. Khung hình được chuyển từ I420 sang RGB, giảm kích thước và xử lý trên một executor riêng; nếu lượt suy luận trước chưa kết thúc, khung hình mới bị bỏ qua thay vì tiếp tục xếp hàng. Khi cuộc gọi kết thúc, bộ thu hình của WebRTC được dừng và giải phóng trước khi CameraX được liên kết trở lại.

Bên cạnh việc quản lý nguồn hình ảnh, `tracking_fragment` duy trì một đối tượng `RobotControlManager` để điều khiển kết nối với ESP32. Chức năng gọi video sử dụng chung bộ quản lý này, giúp tập trung việc kiểm tra lệnh và xử lý dừng robot.

4.3.3 Nạp và suy luận mô hình YOLO26n-Pose

Các quy ước về bounding box, keypoint và hậu xử lý ở Mục 2.3.5 và Mục 2.3.4 được hiện thực trong `tracking_fragment.kt`. Chuỗi xử lý bao gồm lựa chọn mô hình, khởi tạo bộ suy luận, tiền xử lý khung hình, giải mã tensor và chuyển kết quả đến SORT, LSTM cùng lớp hiển thị.

Mô hình triển khai và cơ chế khởi tạo bộ tăng tốc. RobotApp đóng gói hai biến thể mô hình trong thư mục `assets`. Biến thể đầu ra thô có kích thước tensor tĩnh được ưu tiên để khai thác GPU; biến thể đầu cuối được sử dụng khi mô hình thứ nhất không thể khởi tạo. Bảng 4.2 trình bày quy ước đầu ra mà bộ giải mã hỗ trợ.

Sau khi chọn tệp, `setupAIModel()` lần lượt thử bộ tăng tốc GPU, NNAPI

Bảng 4.2: Hai mô hình YOLO26n-Pose được đóng gói trong RobotApp

Tệp mô hình	Kích thước	Đầu ra	Vai trò
yolo26n-pose_320_gpu.tflite	11.838.836 byte	[1, 56, 2100]	Đầu ra thô, tensor tĩnh; được ưu tiên để suy luận bằng GPU
yolo26n-pose_320.tflite	12.031.560 byte	[1, 300, 57]	Đầu ra E2E; mô hình dự phòng với số lượng kết quả giới hạn

và cuối cùng là CPU với bốn luồng. Nếu GPU không khởi tạo được, tài nguyên của bộ tăng tốc này được giải phóng trước khi chuyển sang NNAPI. Thay vì cố định kích thước theo tên tệp, chương trình đọc `getInputTensor(0).shape()` và `getOutputTensor(0).shape()`, sau đó cập nhật kích thước đầu vào và cấp phát bộ đệm đầu ra theo hình dạng tensor thực tế. Nhờ vậy, cùng một chuỗi xử lý có thể tiếp nhận các mô hình được xuất theo những quy ước khác nhau.

Tiền xử lý khung hình. CameraX sử dụng `STRATEGY_KEEP_ONLY_LATEST` để ưu tiên khung hình mới nhất khi tốc độ xử lý thấp hơn tốc độ thu nhận. Không có bộ định thời giới hạn riêng cho YOLO; mỗi khung hình được bộ phân tích cung cấp sẽ đi thẳng vào chuỗi suy luận. Nhờ đó, nhịp xử lý bám theo nguồn camera mà không phát sinh thêm độ trễ do chờ giữa hai lần suy luận. Trong cuộc gọi, bản sao `VideoFrame` của WebRTC được đưa đến cùng bộ thực thi tác vụ và hàm suy luận, cho phép hai nguồn hình ảnh sử dụng chung quy trình xử lý.

Khung hình nguồn có kích thước $W_c \times H_c$ được đưa về ảnh vuông $S \times S$, với $S = 320$, bằng phương pháp letterbox. Hệ số thu phóng được chọn thống nhất cho cả hai trục:

$$s_l = \min\left(\frac{S}{W_c}, \frac{S}{H_c}\right). \quad (4.1)$$

Kích thước nội dung sau thu phóng là $W_r = s_l W_c$, $H_r = s_l H_c$. Phần còn thiếu được đệm đều ở hai phía, với

$$p_x = \frac{S - W_r}{2}, \quad p_y = \frac{S - H_r}{2}. \quad (4.2)$$

Vùng đệm sử dụng giá trị RGB (114, 114, 114), tương đồng với quy trình tiền xử lý của YOLO. Một điểm (x_c, y_c) trên ảnh nguồn được ánh xạ sang đầu vào mô hình theo

$$x_i = s_l x_c + p_x, \quad y_i = s_l y_c + p_y. \quad (4.3)$$

Ảnh sau letterbox được nạp vào `TensorImage` kiểu `float32`; giá trị RGB tiếp tục

được chuẩn hóa từ $[0, 255]$ về $[0, 1]$ bằng `NormalizeOp(0f, 255f)`. Cách xử lý này tránh làm thay đổi tỷ lệ cơ thể khi khung hình nguồn không vuông, đặc biệt quan trọng đối với vị trí keypoint và tỷ lệ hình dạng dùng trong nhận biết tư thế.

Nhận dạng và giải mã cấu trúc tensor. Đầu ra thô chứa dự đoán tại ba mức bước lấy mẫu (stride) 8, 16 và 32. Với ảnh 320×320 , tổng số vị trí dự đoán là

$$N = \left(\frac{320}{8}\right)^2 + \left(\frac{320}{16}\right)^2 + \left(\frac{320}{32}\right)^2 = 2100. \quad (4.4)$$

Vì mô hình chỉ nhận diện lớp người, mỗi vị trí dự đoán gồm 4 giá trị hộp, 1 độ tin cậy và 17×3 giá trị keypoint, tổng cộng 56 kênh. Mô hình YOLO-Pose dùng cho suy luận GPU trả về tensor thô có kích thước $[1, 56, 2100]$, trong đó 56 là số thành phần của mỗi dự đoán và 2100 là số vị trí dự đoán ứng viên. Bốn kênh đầu biểu diễn (c_x, c_y, w_b, h_b) , kênh 4 là độ tin cậy, còn 51 giá trị từ kênh 5 tạo thành 17 bộ ba (x_j, y_j, p_j) .

Đầu ra đầu cuối (E2E) được nhận biết khi hai chiều còn lại sau chiều kích thước lô có giá trị 300 và 57. Mỗi hàng gồm bốn trường đầu biểu diễn $(x_{\min}, y_{\min}, x_{\max}, y_{\max})$, trường 4 là độ tin cậy, trường 5 là nhãn lớp và các trường từ 6 chứa dữ liệu của 17 điểm khớp. Hai dạng đầu ra cần được giải mã riêng: đọc đầu ra thô theo quy ước E2E sẽ làm tọa độ tâm và kích thước bounding box bị hiểu thành tọa độ hai góc, đồng thời làm lệch vị trí các kênh điểm khớp.

Hai mô hình có thể trả tọa độ chuẩn hóa hoặc tọa độ pixel trong không gian đầu vào. Hàm `detectCoordSystem()` khảo sát kênh tọa độ ở lần suy luận đầu tiên: nếu giá trị dương lớn nhất không vượt quá 2, tọa độ được xem là đã chuẩn hóa và được nhân với kích thước đầu vào; ngược lại, kết quả được xem là tọa độ pixel của ảnh mô hình. Tọa độ (x_i, y_i) sau đó được đưa về ảnh nguồn bằng phép biến đổi ngược

$$x_c = \frac{x_i - p_x}{s_l}, \quad y_c = \frac{y_i - p_y}{s_l}, \quad (4.5)$$

rồi giới hạn trong miền $[0, W_c] \times [0, H_c]$. Quy tắc nhận biết này đáp ứng hai mô hình được sử dụng trong đề tài; khi bổ sung quy trình xuất mô hình mới, hệ tọa độ nên được khai báo rõ bằng siêu dữ liệu để tránh suy đoán.

Đầu ra thô được đổi từ dạng `xcycywh` sang `xyxy`, sau đó loại các dự đoán có độ tin cậy dưới 0,20. Hàm `nms()` sắp xếp kết quả theo điểm tin cậy và loại hộp có IoU lớn hơn 0,45. Kết quả E2E cũng đi qua hàm này như một lớp lọc bổ sung, mặc dù mô hình đầu

cuối đã thực hiện bước lựa chọn dự đoán. Mỗi keypoint được giữ kèm độ tin cậy và chỉ được đánh dấu `visible` khi $p_j > 0,30$.

Chuyển đổi tọa độ và hiển thị kết quả. Quy trình xử lý sử dụng bốn hệ tọa độ như trình bày trong Bảng 4.3. Bounding box và các điểm khớp phải được chuyển đổi nhất quán giữa các hệ này. Các phép tính theo dõi và điều khiển sử dụng tọa độ ảnh camera; tọa độ trên giao diện chỉ phục vụ hiển thị.

Bảng 4.3: Các hệ tọa độ trong quy trình xử lý YOLO26n-Pose của RobotApp

Không gian	Miền tọa độ	Vai trò	Dữ liệu sử dụng
Ảnh camera $\mathcal{C}$	$[0, W_c] \times [0, H_c]$ pixel	Khung hình gốc nhận từ CameraX hoặc WebRTC	SORT, LSTM và điều khiển bám người
Ảnh mô hình $\mathcal{I}$	$[0, 320] \times [0, 320]$ pixel	Ảnh vuông sau khi thu phóng và thêm vùng đệm	Suy luận và đầu ra dạng pixel
Chuẩn hóa $\mathcal{N}$	$[0, 1] \times [0, 1]$	Biểu diễn không phụ thuộc độ phân giải	Đầu ra chuẩn hóa và các tỉ lệ hình học
Màn hình $\mathcal{V}$	$[0, W_v] \times [0, H_v]$ pixel	BoundingBoxOverlay chồng lên preview	Vẽ hộp, skeleton, nhãn và track ID

Camera trước được hiển thị như gương. Sau khi đổi về pixel camera, tọa độ ngang của keypoint được thay bằng $x'_c = W_c - x_c$. Với bounding box, hai cạnh phải đổi đồng thời:

$$x'_{\min} = W_c - x_{\max}, \quad x'_{\max} = W_c - x_{\min}. \quad (4.6)$$

Nếu chỉ phản chiếu từng cạnh mà không hoán đổi, kết quả sẽ có cạnh phải nhỏ hơn cạnh trái. Tọa độ y không thay đổi.

`BoundingBoxOverlay.onDraw()` tiếp tục ánh xạ pixel camera sang View theo center-crop giống `PreviewView`. Với

$$s_v = \max\left(\frac{W_v}{W_c}, \frac{H_v}{H_c}\right), \quad \Delta_x = \frac{W_v - s_v W_c}{2}, \quad \Delta_y = \frac{H_v - s_v H_c}{2}, \quad (4.7)$$

một điểm được vẽ tại $x_v = s_v x'_c + \Delta_x$, $y_v = s_v y_c + \Delta_y$. Phần bù có thể âm vì center-crop cắt bớt một chiều. Tọa độ View chỉ phục vụ hiển thị; các đại lượng như tâm mục tiêu, mức chiếm khung và khoảng cách keypoint vẫn được tính trong không gian camera rồi chuẩn hóa bởi W_c, H_c .

Chuyển kết quả đến các module tiếp theo. Sau khi giải mã, mỗi đối tượng gồm bounding box `bbox`, độ tin cậy `score`, nhãn lớp `classId` và 17 điểm khớp trước khi được chuyển đến `SortTracker`. Kết quả theo dõi được dùng để xác định mục tiêu ID0, tạo vector 69 đặc trưng cho LSTM, tính lệnh bám theo và hiển thị thông tin lên màn hình. Dữ liệu được thống nhất trong hệ tọa độ camera, nên các module phía sau không cần xử lý lại sự khác biệt về dạng đầu ra, thứ tự các chiều tensor hoặc quy ước tọa độ của từng mô hình.

4.3.4 SORT, Kalman, Hungarian và Re-ID

Các lớp `SortTracker`, `KalmanBoxTracker`, `HungarianAlgorithm` và `ReIdModule` hiện thực phương pháp theo dõi dựa trên kết quả phát hiện. Mỗi đối tượng đầu vào gồm bounding box, độ tin cậy và các keypoint. Quá trình xử lý một khung hình được thực hiện theo thứ tự: dự báo trạng thái các track đang có, lập ma trận chi phí giữa track và detection, giải bài toán gán ghép, cập nhật các cặp hợp lệ, sau đó xử lý những track và detection chưa được ghép. Cách tổ chức này giữ lại chu trình chính của SORT [27], đồng thời bổ sung tín hiệu tư thế và đặc trưng màu để phù hợp hơn với sự thay đổi hình dạng cơ thể.

Mỗi track sử dụng một bộ lọc Kalman độc lập. Các bước dự báo và hiệu chỉnh được xây dựng theo dạng chuẩn của bộ lọc Kalman rời rạc tuyến tính [28][29], với vector trạng thái

$$\mathbf{x} = \begin{bmatrix} c_x & c_y & s & r & v_x & v_y & v_s \end{bmatrix}^T, \quad (4.8)$$

trong đó (c_x, c_y) là tâm bounding box, s là diện tích, r là tỷ lệ chiều rộng trên chiều cao, còn v_x , v_y và v_s biểu diễn mức thay đổi của tâm và diện tích giữa hai khung hình liên tiếp. Các đại lượng vận tốc ở đây được tính theo mỗi khung hình, không phải theo giây. Khi một track vừa được tạo, bốn thành phần đầu được lấy từ bounding box ban đầu và ba thành phần vận tốc được đặt bằng 0.

Phần hiện thực sử dụng mô hình vận tốc không đổi với một bước thời gian tương ứng một khung hình. Ma trận chuyển trạng thái $\mathbf{F}$ có dạng

$$\mathbf{F} = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}.$$

Phép nhân $\mathbf{F}\mathbf{x}_{t-1}$ tương đương với việc tính

$$\begin{aligned} c_{x,t}^- &= c_{x,t-1} + v_{x,t-1}, & c_{y,t}^- &= c_{y,t-1} + v_{y,t-1}, \\ s_t^- &= s_{t-1} + v_{s,t-1}, & r_t^- &= r_{t-1}. \end{aligned}$$

Các thành phần vận tốc được giữ nguyên trong bước dự báo. Như vậy, bounding box ở khung hình mới được ước lượng bằng cách tiếp tục xu hướng dịch chuyển và thay đổi diện tích đã quan sát ở những khung hình trước. Đồng thời, độ bất định của dự báo được cập nhật theo

$$\mathbf{x}_t^- = \mathbf{F}\mathbf{x}_{t-1}, \quad \mathbf{P}_t^- = \mathbf{F}\mathbf{P}_{t-1}\mathbf{F}^T + \mathbf{Q}. \quad (4.9)$$

Trong đó, $\mathbf{P}_t^-$ biểu diễn độ bất định của trạng thái dự báo. Thành phần $\mathbf{F}\mathbf{P}_{t-1}\mathbf{F}^T$ truyền độ bất định từ khung hình trước sang khung hình hiện tại, còn $\mathbf{Q}$ bổ sung phần sai số do mô hình vận tốc không đổi không thể mô tả hoàn toàn chuyển động thực tế.

Với bounding box do YOLO phát hiện có chiều rộng w , chiều cao h và hai góc $(x_{\min}, y_{\min})$, $(x_{\max}, y_{\max})$, vector phép đo được tạo bởi

$$\mathbf{z}_t = \left[(x_{\min} + x_{\max})/2 \quad (y_{\min} + y_{\max})/2 \quad wh \quad w/h \right]^T.$$

Ma trận quan sát

$$\mathbf{H} = \begin{bmatrix} \mathbf{I}_4 & \mathbf{0}_{4 \times 3} \end{bmatrix}$$

chỉ lấy bốn thành phần (c_x, c_y, s, r) từ trạng thái dự báo để so sánh với phép đo.

Do đó, sai lệch $\mathbf{e}_t = \mathbf{z}_t - \mathbf{H}\mathbf{x}_t^-$ cho biết độ lệch về tâm, diện tích và tỷ lệ hình dạng giữa bounding box dự báo với bounding box vừa được YOLO phát hiện. Sai lệch này được dùng để hiệu chỉnh trạng thái:

$$\mathbf{K}_t = \mathbf{P}_t^- \mathbf{H}^T (\mathbf{H} \mathbf{P}_t^- \mathbf{H}^T + \mathbf{R})^{-1}, \quad (4.10)$$

$$\mathbf{x}_t = \mathbf{x}_t^- + \mathbf{K}_t (\mathbf{z}_t - \mathbf{H}\mathbf{x}_t^-), \quad (4.11)$$

$$\mathbf{P}_t = (\mathbf{I} - \mathbf{K}_t \mathbf{H}) \mathbf{P}_t^-. \quad (4.12)$$

Ma trận $\mathbf{K}_t$ là hệ số Kalman, quyết định mức độ điều chỉnh trạng thái dự báo theo phép đo mới. Khi độ bất định dự báo $\mathbf{P}_t^-$ lớn so với nhiễu phép đo $\mathbf{R}$, hệ số này tăng và kết quả bám sát detection của YOLO hơn. Ngược lại, khi phép đo được xem là nhiễu, mức hiệu chỉnh giảm để bounding box thay đổi ổn định hơn. Dòng cuối cập nhật lại $\mathbf{P}_t$ sau khi đã tiếp nhận phép đo.

Các ma trận hiệp phương sai được khởi tạo trong chương trình như sau:

$$\mathbf{P}_0 = \text{diag}(10, 10, 10, 10, 1000, 1000, 1000),$$

$$\mathbf{Q} = \text{diag}(100, 100, 1, 0, 0, 1, 0, 1, 0, 1),$$

$$\mathbf{R} = \text{diag}(10, 10, 100, 10).$$

Độ bất định ban đầu của ba thành phần vận tốc được đặt lớn vì chưa có thông tin chuyển động khi track mới hình thành. Nhiễu mô hình tại hai tọa độ tâm được chọn lớn hơn để bộ lọc có thể thích ứng nhanh với chuyển động của người và camera; ngược lại, nhiễu phép đo diện tích được đặt cao hơn tọa độ tâm vì kích thước bounding box thường dao động theo kết quả ước lượng tư thế. Các giá trị này là tham số cấu hình của bộ lọc trong nguyên mẫu. Bước dự báo giúp duy trì ước lượng vị trí khi detection tạm thời bị thiếu, còn bước hiệu chỉnh đưa trạng thái trở lại gần bounding box quan sát được. Khi tỷ lệ hình dạng thay đổi đột ngột giữa tư thế đứng và nằm, phần diện tích và tỷ lệ hộp được cập nhật nhanh theo phép đo mới để tránh tiếp tục duy trì hình dạng cũ.

Sau khi dự báo tất cả track, SortTracker tính chi phí C_{ij} giữa track thứ i và detection thứ j theo Công thức 3.1. Thuật toán Hungarian giải bài toán

$$\mathcal{A}^* = \arg \min_{\mathcal{A}} \sum_{(i,j) \in \mathcal{A}} C_{ij}, \quad (4.13)$$

với ràng buộc mỗi track và mỗi detection chỉ xuất hiện tối đa một lần trong tập ghép $\mathcal{A}$. Vì vậy, Hungarian không tự quyết định một cặp có đủ tin cậy hay không mà chỉ tìm phương án phân công có tổng chi phí thấp nhất. Mỗi cặp trả về còn phải vượt qua bước kiểm tra về IoU, khoảng cách tâm, khoảng cách tư thế, thay đổi diện tích và giới hạn chi phí. Cặp hợp lệ được dùng để cập nhật Kalman; các phần tử còn lại được chuyển sang nhánh xử lý mất dấu hoặc tạo track mới.

Ngưỡng IoU cơ sở của bộ theo dõi là 0,15; đối tượng mới cần hai lần ghép thành công để được xác nhận. Khi không tìm thấy detection phù hợp, định danh ID0 được duy trì tối đa 45 khung hình, còn các định danh khác được giữ tối đa 10 khung hình.

Trong phần hiện thực, ký hiệu B_{t-1} là hộp của detection gần nhất đã ghép với track, B_t^- là hộp do Kalman dự báo và D_t là detection đang xét. Hàm $\mathbf{c}(B)$, $A(B)$ và $\ell(B) = \sqrt{w_B^2 + h_B^2}$ lần lượt trả về tâm, diện tích và đường chéo của hộp. Đặt $\text{clip}(z) = \min(1, \max(0, z))$, hệ số kích thước tham chiếu và khoảng cách tâm được tính bởi

$$L_{\text{ref}} = \max(\ell(B_{t-1}), \ell(B_t^-), \ell(D_t), 1), \quad (4.14)$$

$$\delta_c = \frac{\|\mathbf{c}(B_t^-) - \mathbf{c}(D_t)\|_2}{L_{\text{ref}}}, \quad \hat{d}_c = \text{clip}\left(\frac{\delta_c}{0,90}\right). \quad (4.15)$$

Điểm neo tư thế $\mathbf{s}(\mathcal{K})$ là trung bình tọa độ của các keypoint vai trái, vai phải, hông trái và hông phải đang quan sát được; điểm neo chỉ hợp lệ khi có ít nhất hai trong bốn điểm. Thành phần khoảng cách tư thế là

$$\delta_p = \frac{\|\mathbf{s}(\mathcal{K}_{t-1}) - \mathbf{s}(\mathcal{K}_t)\|_2}{L_{\text{ref}}}, \quad \hat{d}_p = \text{clip}\left(\frac{\delta_p}{0,75}\right). \quad (4.16)$$

Nếu không đủ điểm vai–hông, mã nguồn dùng $\delta_p = \delta_c$. Mức thay đổi diện tích được tính đối xứng theo hai hướng phóng to và thu nhỏ:

$$r_A = \max\left(\frac{A(B_{t-1})}{A(D_t)}, \frac{A(D_t)}{A(B_{t-1})}\right), \quad p_a = \text{clip}\left(\frac{\ln r_A}{\ln 8}\right). \quad (4.17)$$

Diện tích hộp được chặn dưới ở 1 pixel vuông để tránh chia cho không. Sau khi lập ma trận chi phí, kết quả Hungarian chỉ được chấp nhận khi thỏa thêm điều kiện về IoU, khoảng cách tâm hoặc khoảng cách tư thế, tỷ lệ diện tích và chi phí cực đại. ID0 dùng các ngưỡng nới rộng hơn track thông thường nhưng vẫn có thể ở trạng thái không

ghép nếu không có detection hợp lệ.

Khối xử lý màu lấy phần trung tâm của bounding box bằng cách loại bỏ 18% chiều rộng ở mỗi biên trái–phải và 12% chiều cao ở mỗi biên trên–dưới. Việc thu hẹp vùng ảnh giúp giảm ảnh hưởng của nền và các phần cơ thể thường thay đổi mạnh khi người chuyển tư thế. Vùng ảnh còn lại được chuyển từ RGB sang HSV, sau đó xây dựng histogram hai chiều gồm 50 khoảng cho kênh sắc độ H và 60 khoảng cho kênh độ bão hòa S . Kênh độ sáng V không tham gia histogram nhằm giảm ảnh hưởng trực tiếp của thay đổi độ sáng. Histogram được chuẩn hóa về miền $[0, 1]$ và hai đặc trưng được so sánh bằng hệ số tương quan; giá trị càng gần 1 cho thấy phân bố màu càng tương đồng.

Đối với track đang được ghép bình thường, histogram đại diện được cập nhật sau mỗi 10 khung hình và chỉ khi quan sát được ít nhất 8 keypoint. Điều kiện này hạn chế việc thay thế đặc trưng đã lưu bằng một vùng cắt thiếu tin cậy. Khi mục tiêu chính có nhiều detection cùng thỏa điều kiện hình học, chi phí hình học chiếm trọng số 0,82 và sai khác màu chiếm trọng số 0,18 trong bước lựa chọn ứng viên. Vì vậy, màu sắc chỉ hỗ trợ phân giải trường hợp nhập nhằng, không thay thế thông tin chuyển động và hình học trong quá trình gán ghép.

Nếu ID0 không được ghép trong hơn 45 khung hình, histogram gần nhất của mục tiêu được đưa vào gallery và được giữ tối đa 150 khung hình tiếp theo. Với mỗi detection chưa được ghép, hệ thống trích xuất một histogram truy vấn và so sánh với các đặc trưng trong gallery. Khi hệ số tương quan đạt ít nhất 0,65, `restoreWithId()` khởi tạo một bộ lọc Kalman từ bounding box hiện tại và gán lại định danh cũ. Hàm này chỉ khôi phục ID, không phục hồi trạng thái chuyển động trước khi mục tiêu mất dấu. Nếu không có kết quả phù hợp, detection được dùng để tạo một track mới. Do không sử dụng mạng học sâu cho tái định danh, cơ chế này có chi phí tính toán thấp nhưng vẫn phụ thuộc vào màu sắc trang phục, điều kiện chiếu sáng và mức độ giống nhau giữa những người xuất hiện trong ảnh.

4.3.5 Duy trì mục tiêu ID0 và tạo lệnh bám theo

Người mục tiêu được gán định danh `PRIMARY_TRACK_ID=0`. Hàm lựa chọn ban đầu hiện thực Công thức 3.2. Với khung hình $W \times H$, hộp ứng viên $B = (x_{\min}, y_{\min}, x_{\max}, y_{\max})$, tâm hộp (c_x, c_y) , độ tin cậy s_{det} và N_{vis} keypoint quan sát được, các thành phần được tính như sau:

$$q_{area} = \text{clip}\left(\frac{A(B)}{0,45WH}\right), \quad q_{det} = s_{det}, \quad (4.18)$$

$$q_{vis} = \frac{N_{vis}}{17}, \quad q_{center} = \text{clip}\left(1 - \frac{\sqrt{(c_x/W - 0,5)^2 + (c_y/H - 0,5)^2}}{0,707}\right). \quad (4.19)$$

Giá trị q_{area} bão hòa khi hộp chiếm 45% diện tích ảnh; q_{center} bằng 1 tại tâm và giảm về gần 0 ở góc ảnh. Mức phạt biên trong mã nguồn là

$$p_{edge} = \begin{cases} 0,10, & x_{\min} \leq 0,01W \text{ hoặc } y_{\min} \leq 0,01H \\ & \text{hoặc } x_{\max} \geq 0,99W \text{ hoặc } y_{\max} \geq 0,99H, \\ 0, & \text{các trường hợp còn lại.} \end{cases} \quad (4.20)$$

Định danh ID0 được dùng chung cho phân tích té ngã và điều khiển bám theo. Khi mất dấu, hệ thống tìm lại mục tiêu trong khoảng 2 s; đối tượng được xét phải cách vị trí cuối không quá 35% đường chéo ảnh, nằm trong vùng trung tâm và được quan sát ổn định qua ba khung hình. Trong quá trình xử lý sự kiện nghi té ngã, hệ thống không tự chuyển sang một người khác.

Tạo vector điều khiển bám theo. Chức năng bám theo ước lượng độ gần từ bounding box và các mốc vai, hông, gối, mắt cá chân. Gọi w_b, h_b là kích thước bounding box, W, H là kích thước ảnh camera. Thành phần dựa trên vùng bao được tính bởi

$$d_b = \text{clip}_{[0; 1,5]}\left(\max\left(\frac{w_b}{W}, \frac{h_b}{H}\right)\right). \quad (4.21)$$

d_b là chỉ số độ gần ước lượng từ tỷ lệ vùng ảnh mà người mục tiêu chiếm giữ và không có đơn vị. Hai tỷ số w_b/W và h_b/H lần lượt cho biết bounding box chiếm bao nhiêu phần chiều rộng và chiều cao của ảnh. Giá trị lớn hơn được lựa chọn để chỉ số vẫn phản ánh được kích thước biểu kiến của người khi tư thế thay đổi: chiều cao thường chiếm ưu thế khi người đứng, trong khi chiều rộng có thể chiếm ưu thế khi người nằm. Khi người tiến gần camera, bounding box thường lớn hơn nên d_b tăng; khi người đi xa, d_b giảm. Phép $\text{clip}_{[0; 1,5]}$ giới hạn các giá trị bất thường do bounding box vượt ra ngoài

biên ảnh hoặc kết quả phát hiện chưa ổn định. Vì không sử dụng thông số hiệu chuẩn camera hay cảm biến chiều sâu, d_b chỉ biểu diễn độ gần tương đối, không phải khoảng cách từ robot đến người theo đơn vị mét.

Khi các keypoint cần thiết có độ tin cậy từ 0,45 trở lên, hệ thống tính thêm khoảng cách Euclid đã chuẩn hóa giữa hai mốc đại diện. Tùy phần cơ thể quan sát được, khoảng cách vai–mắt cá chân, vai–gối, hông–mắt cá chân hoặc vai–hông lần lượt được nhân với hệ số 1,33; 1,80; 2,20 hoặc 3,30 để đưa về gần cùng miền giá trị. Gọi kết quả này là d_p ; nếu chưa đủ keypoint tin cậy thì $d_p = 0$. Độ gần tức thời và độ gần sau làm mượt được xác định bởi

$$d_{\text{raw}} = \text{clip}_{[0; 1,5]}(\max(d_b, d_p)), \quad d_t = 0,55d_{t-1} + 0,45d_{\text{raw}}. \quad (4.22)$$

Việc chọn giá trị lớn hơn giữa hai nguồn đo giúp robot phản ứng thận trọng khi một nhóm keypoint bị che khuất hoặc nhận dạng chưa ổn định. Hệ thống chuyển từ vùng phù hợp sang quá xa khi $d_t < 0,56$ và chỉ trở lại khi $d_t \geq 0,64$. Tương tự, vùng quá gần được kích hoạt khi $d_t \geq 0,80$ và kết thúc khi $d_t \leq 0,72$. Khoảng chênh giữa ngưỡng đi vào và rời khỏi mỗi vùng tạo độ trễ, nhờ đó robot không liên tục đảo giữa tiến, dừng và lùi khi giá trị dao động gần biên.

Với quy ước giá trị âm là tiến và giá trị dương là lùi, thành phần tịnh tiến ban đầu được tạo bởi

$$x_{\text{raw}} = \begin{cases} -\lambda \left[0,60 + 0,30 \text{clip}_{[0; 1]} \left(\frac{0,64 - d_t}{0,34} \right) \right], & \text{quá xa,} \\ 0, & \text{phù hợp,} \\ 0,45 + 0,15 \text{clip}_{[0; 1]} \left(\frac{d_t - 0,72}{0,24} \right), & \text{quá gần.} \end{cases} \quad (4.23)$$

Hệ số λ bằng 0,70 khi chưa quan sát ổn định mắt cá chân, bằng 0,85 khi robot vừa tiến vừa chỉnh hướng và bằng 1 trong trường hợp còn lại. Nếu phần chân có khả năng nằm ngoài ảnh, lệnh lùi $x = 0,30$ được ưu tiên để mở rộng vùng quan sát. Khi bounding box chạm vùng biên hoặc chiếm quá 82% kích thước ảnh, hệ thống cũng ưu tiên lùi thay vì tiếp tục tiến.

Thành phần quay được xây dựng từ sai lệch ngang giữa tâm bounding box và tâm ảnh. Với c_x là tọa độ tâm mục tiêu, sai lệch chuẩn hóa $e_x = c_x/W - 0,5$ và lệnh quay được tính bởi

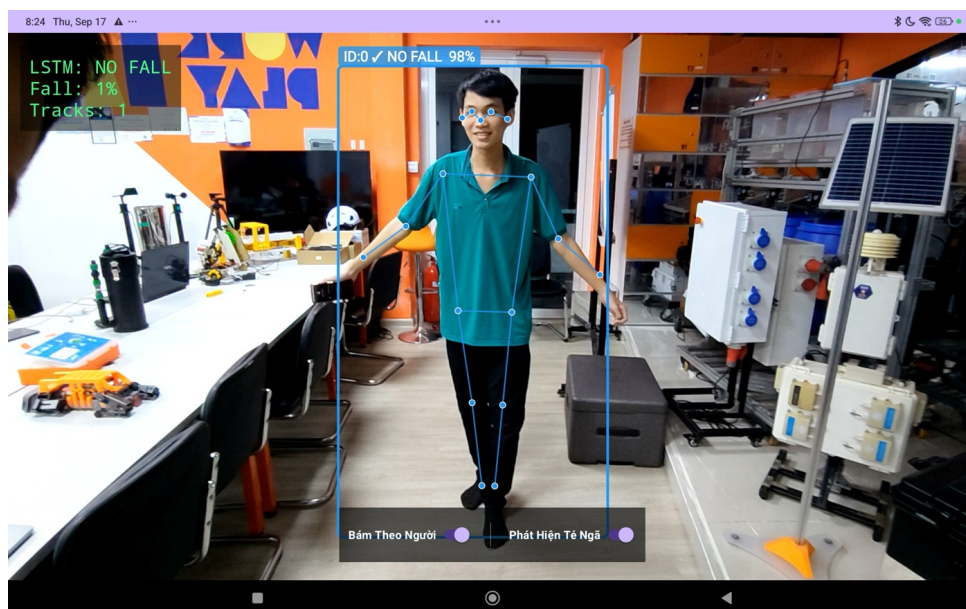
$$y_{\text{raw}} = \begin{cases} 0, & |e_x| \leq 0,07, \\ \text{clip}_{[-0,50; 0,50]}(2,2e_x), & |e_x| > 0,07. \end{cases} \quad (4.24)$$

Vùng chết quanh tâm ảnh loại bỏ các hiệu chỉnh nhỏ không cần thiết, còn giới hạn $\pm 0,50$ tránh robot quay quá mạnh. Khi chuyển động cùng một chiều, x và y tiếp tục được làm mượt theo $x_t = 0,40x_{t-1} + 0,60x_{\text{raw}}$ và $y_t = 0,65y_{t-1} + 0,35y_{\text{raw}}$. Lệnh dừng, lệnh đổi chiều và yêu cầu lùi khẩn cấp được áp dụng ngay để không tạo quãng trôi do bộ lọc. Nếu robot đang tịnh tiến mạnh nhưng thành phần quay sau lọc nhỏ hơn ngưỡng có hiệu lực, độ lớn của y được nâng lên 0,30 nhưng vẫn giữ nguyên dấu. Vector cuối cùng được chuyển đến bộ điều khiển với khoảng cách tối thiểu 120 ms giữa hai lần gửi.

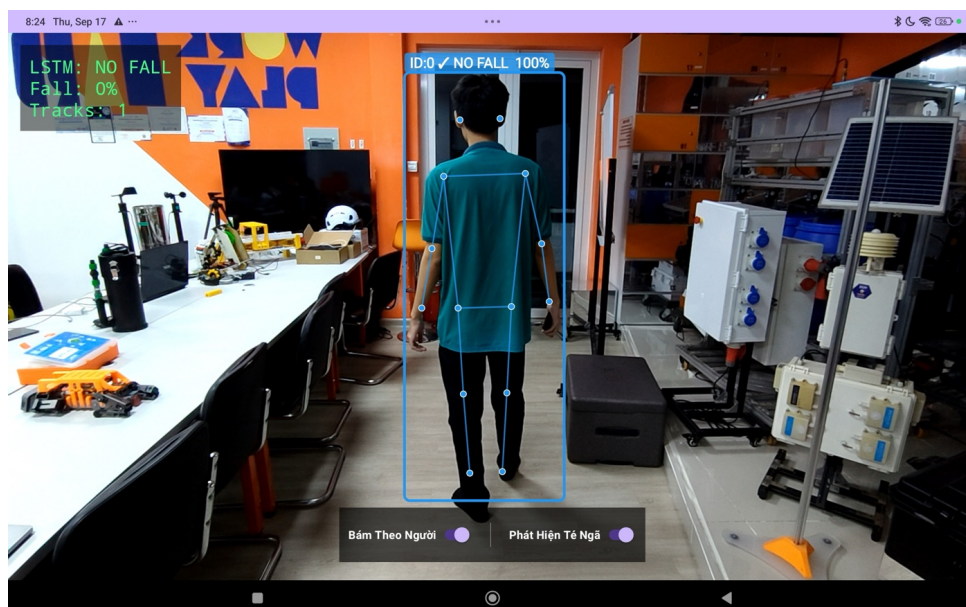
Một nhánh hình học riêng phân loại tư thế phục vụ điều khiển, không thay đổi kết quả FALL/NO FALL của LSTM. Nhánh này kết hợp tỷ lệ vùng bao, hướng phân bố keypoint, quan hệ vai–hông và bằng chứng từ thân dưới; *PostureGeometry* bổ sung trường hợp người nằm dọc theo hướng nhìn của camera nên hình chiếu có thể trông gần giống tư thế đứng. Khi tư thế nằm đã ổn định hoặc một sự cố té ngã đang được xử lý, robot không tiếp tục tiến. Các hành động lùi chậm, quay trái, quay phải và dừng lần lượt được biểu diễn bởi $(x, y) = (0,22; 0)$, $(0; -0,42)$, $(0; 0,42)$ và $(0; 0)$ để đưa người vào vùng quan sát phù hợp.

Sau khi làm mượt vector bám theo, `sendTrackingVector()` gọi `sendFollowCommand(x, y)` và chuyển lệnh trực tiếp đến `RobotControlManager` bằng `submitAutoFollowCommand()`. Đường truyền này nằm hoàn toàn trong `RobotApp`, không ghi vector bám theo lên Firebase. Bộ định tuyến lệnh chỉ chấp nhận nguồn cục bộ khi chế độ bám theo đang bật; ở chế độ điều khiển từ xa, nguồn cục bộ bị khóa và bộ lắng nghe `robot_control` mới được kích hoạt. Sau bước lựa chọn nguồn, `RobotControlManager` áp dụng chung phép giới hạn, ánh xạ vận tốc hai bánh và truyền lệnh đến ESP32 qua BLE.

Hình 4.4 minh họa lớp phủ của `tracking_fragment` khi chức năng bám theo đang hoạt động. Mục tiêu được gán định danh ID0 kèm bounding box, khung xương 17 điểm khớp và số lượng track đang tồn tại ở góc trái. Khi mục tiêu di chuyển ra xa, bounding box thu hẹp dần trong khi ID0 được giữ nguyên, cho thấy bước liên kết dữ liệu vẫn bám đúng đối tượng giữa các khung hình.



(a) Mục tiêu ID0 đứng đối diện robot



(b) Mục tiêu ID0 được giữ nguyên khi di chuyển ra xa

Hình 4.4: Lớp phủ theo dõi và duy trì mục tiêu ID0 trên RobotApp

4.3.6 LSTM phát hiện té ngã

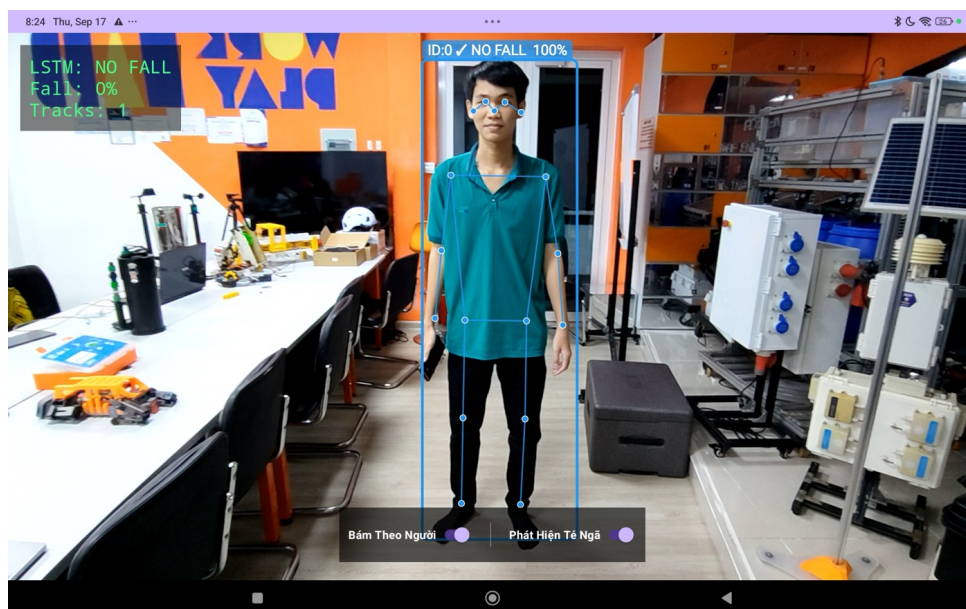
LstmFallDetector nạp mô hình `fall_detection_lstm_v3_ground_float.tflite` có dung lượng khoảng 59,6 KB và sử dụng hai luồng CPU. Mô hình chỉ dùng toán tử TFLite dựng sẵn nên ứng dụng không cần liên kết gói `tensorflow-lite-select-tf-ops`. Dữ liệu của mỗi đối tượng được lưu trong `FallDetectionQueue`; khi tích lũy đủ 15 khung hình, hàng đợi tạo tensor đầu vào $1 \times 15 \times 69$ kiểu số thực. Thành phần dịch chuyển được tính từ chênh lệch tọa độ chuẩn hóa giữa hai khung hình liên tiếp; riêng khung hình đầu tiên của một định danh mới có $\Delta x_i = \Delta y_i = 0$ vì chưa có quan sát trước đó. Với nhịp thu nhận mục tiêu 25 FPS, một cửa sổ đầu vào biểu diễn khoảng 0,6 s chuyển động. Tensor đầu ra 1×2 biểu diễn xác suất của hai lớp. Lớp dương được định nghĩa là trạng thái người đang nằm trên sàn, tính từ thời điểm bắt đầu ngã trở đi, chứ không riêng khoảnh khắc rơi; quy ước này và lý do lựa chọn được trình bày ở Mục 5.8.1. So với tệp Keras gốc khoảng 189,2 KB, tệp TFLite có dung lượng nhỏ hơn khi đóng gói vào ứng dụng.

Kết quả được gán nhãn FALL khi xác suất của lớp dương vượt ngưỡng quyết định, mặc định 0,985, sau đó chuyển đến `FallVerificationEngine` để kiểm tra bổ sung. Ngưỡng được tách thành tham số cấu hình vì giá trị tối ưu phụ thuộc vào môi trường lắp đặt, như phân tích ở Chương 5. Chỉ khi có năm kết quả FALL liên tiếp, hệ thống mới ghi nhận một chuỗi nghi té ngã. Khi bộ theo dõi khôi phục thành công ID0, hàng đợi đặc trưng vẫn được gán với người mục tiêu. Khả năng phân tích quan hệ thời gian của LSTM [25] được kết hợp với bước hậu kiểm trước khi tạo cảnh báo.

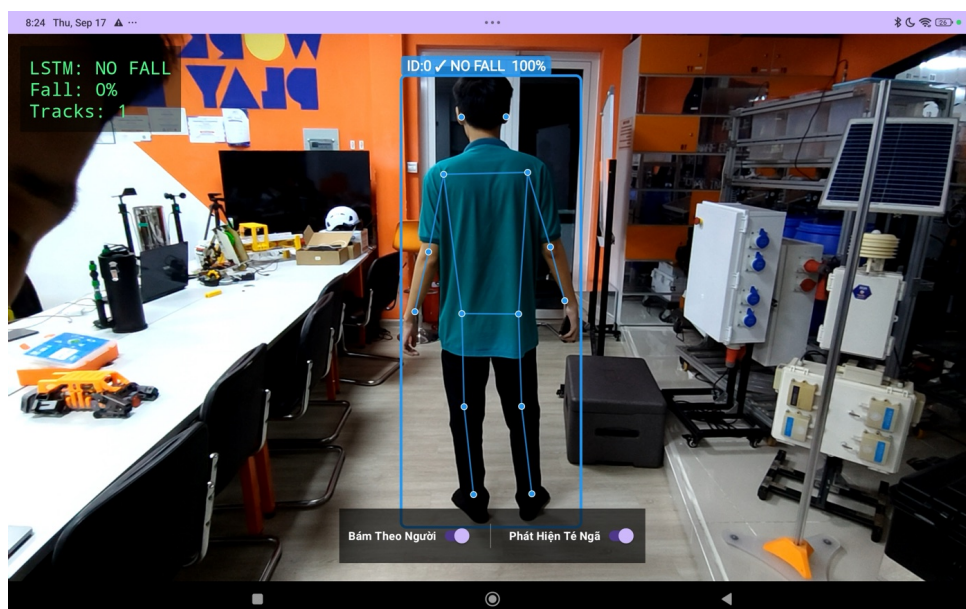
Kết quả của mô hình được hiển thị trực tiếp trên lớp phủ như Hình 4.5. Góc trái hiển thị nhãn dự đoán cùng xác suất lớp FALL, còn nhãn cạnh bounding box hiển thị trạng thái của riêng mục tiêu ID0. Trong hai tình huống đứng đối diện và quay lưng, mô hình đều cho nhãn NO FALL, cho thấy đặc trưng chuẩn hóa theo bounding box ít phụ thuộc vào hướng của người so với camera.

4.3.7 Hậu kiểm té ngã

`FallVerificationEngine` tiếp nhận `FrameEvidence`, gồm trạng thái quan sát mục tiêu, vị trí tương đối của thân trên, số điểm khớp, mức độ bounding box nằm trọn trong ảnh và tỷ lệ hình dạng. Sau khi chuỗi FALL kết thúc, bộ hậu kiểm xem xét vị trí mũi, vai ở các khung hình tiếp theo. Ba khung hình liên tiếp có các điểm thân trên trong vùng 70% phía trên ảnh sẽ loại bỏ tình huống nghi té ngã; ba khung hình liên



(a) Tư thế đứng đối diện: NO FALL



(b) Tư thế quay lưng: NO FALL

Hình 4.5: *Hiển thị kết quả suy luận LSTM trên lớp phủ theo dõi*

tiếp có các điểm này nằm dưới ngưỡng, hoặc 12 khung hình liên tiếp thiếu dữ liệu quan sát, sẽ dẫn đến bước hỏi xác nhận. Khi sự kiện đã được loại bỏ hoặc xác nhận, bộ hậu kiểm chờ 10 kết quả NO FALL liên tiếp trước khi trở lại trạng thái giám sát.

Bộ hậu kiểm trả về một trong ba kết quả: chưa đủ dữ liệu để kết luận, loại bỏ tình huống nghi ngờ qua `Rejected(reason)`, hoặc yêu cầu hỏi xác nhận qua `AskForConfirmation(reason, score, evidence)`. Việc tách phần xử lý khỏi giao diện cho phép kiểm thử riêng từng trường hợp bằng chuỗi dữ liệu đầu vào mà không cần camera. Trạng thái `isIncidentActive` cho biết sự kiện đang được xử lý, còn `canAcquireNewTarget` quy định khi nào được phép chọn mục tiêu mới. Các hàm `resolveAsSafe`, `resolveAsDanger` và `reset` cập nhật kết quả xác nhận hoặc đặt lại bộ xử lý.

4.3.8 Luồng xác nhận và cảnh báo

Trong mỗi sự cố, `TrackingService` chỉ chuyển một `FallEvent` đến chatbot. Robot dừng chuyển động, phát câu hỏi bằng TTS và lắng nghe câu trả lời trong tối đa 10 s. Những cụm từ thể hiện rõ trạng thái an toàn hoặc nguy hiểm được so khớp tại thiết bị để giảm độ trễ và hạn chế phụ thuộc vào LLM. Với câu trả lời không rõ nghĩa, hệ thống gọi mô hình phân loại theo chế độ không truyền luồng và yêu cầu kết quả có cấu trúc. Nếu người dùng xác nhận an toàn, sự cố được kết thúc tại robot; các trường hợp nguy hiểm, lỗi phân loại hoặc không có phản hồi sẽ được ghi vào `fall_alerts`, sau đó cuộc gọi đến người chăm sóc được thực hiện khi TTS phát xong.

Bản ghi cảnh báo lưu thời điểm, trạng thái xử lý, phản hồi của người dùng và các thông tin kiểm tra bổ sung như `verificationReason`, `visibleKeypoints`, `bboxFullyInside` và `upperBodyYRatio`. Các trường này lần lượt mô tả lý do yêu cầu xác nhận, số điểm khớp quan sát được, bounding box có nằm hoàn toàn trong ảnh hay không và vị trí tương đối của phần thân trên. Chúng hỗ trợ xem lại quá trình xử lý sự kiện, nhưng chưa giải thích đầy đủ cách mô hình LSTM đưa ra dự đoán.

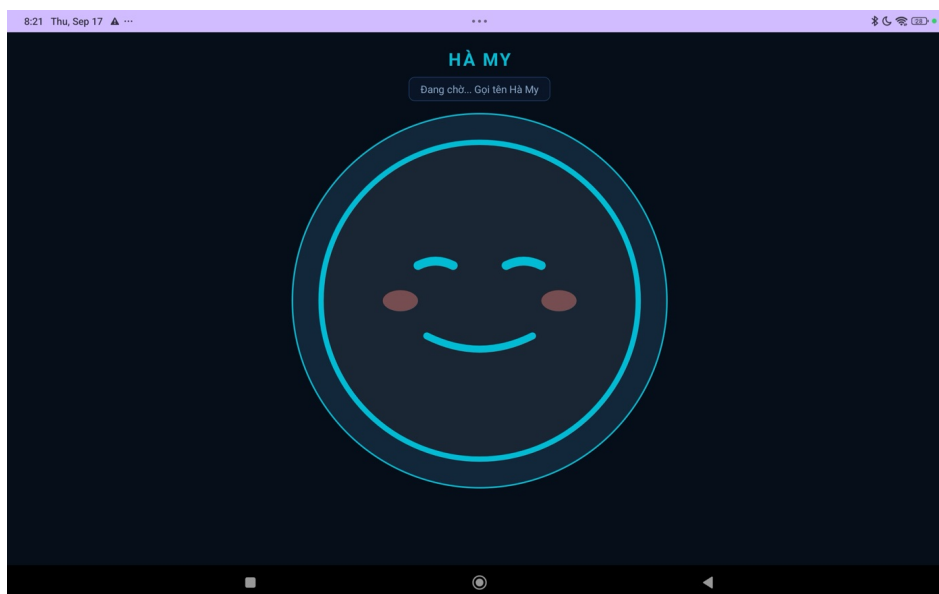
4.3.9 Chatbot giọng nói

`chatbot_fragment` phối hợp ba thành phần: `SpeechRecognizerManager` nhận dạng tiếng nói, `GroqChatService` xử lý hội thoại và `AndroidTtsService` tổng hợp tiếng nói. Giao diện hiển thị trạng thái hoạt động, biểu tượng lớn, tên robot và nội dung đang được nhận dạng hoặc phát ra. Hồ sơ tại `Users/{uid}` cung cấp cách xưng hô, tên người chăm sóc, tình trạng và sở thích để bổ sung ngữ cảnh cho câu trả lời.

GroqChatService gọi điểm cuối `/openai/v1/chat/completions` qua OkHttp. Hội thoại dạng truyền luồng sử dụng mô hình `qwen/qwen3.6-27b`, trong khi các tác vụ phân loại ngắn sử dụng `openai/gpt-oss-20b`. Lịch sử được giới hạn ở 20 thông điệp để kiểm soát độ dài ngữ cảnh. Với phản hồi theo cơ chế Server-Sent Events (SSE) có tiền tố `data:`, dịch vụ đọc nội dung tại `choices[0].delta.content`, phát từng phần qua Kotlin Flow và lưu câu trả lời hoàn chỉnh khi luồng kết thúc.

Các tác vụ liên quan đến an toàn không phụ thuộc hoàn toàn vào văn bản do LLM sinh ra. Hàm phân loại phản hồi té ngã kiểm tra trước tập từ khóa an toàn và nguy hiểm; API chỉ được gọi khi câu trả lời không rõ nghĩa, sau đó kết quả JSON được phân tích theo cấu trúc định trước. Hàm `detectCallIntent` áp dụng cách tương tự khi kết hợp từ khóa, tên hoặc quan hệ của người chăm sóc và mô hình phân loại dự phòng.

Hình 4.6 là trạng thái chờ của chatbot. Robot hiển thị tên được đặt trong hồ sơ và gợi ý cách đánh thức bằng cách gọi tên; khi nhận dạng được tiếng nói, vùng trạng thái phía trên chuyển sang hiển thị nội dung đang nghe hoặc đang phát.



Hình 4.6: Màn hình chatbot của RobotApp ở trạng thái chờ đánh thức

4.3.10 Nhắc nhở theo lịch

ReminderService lắng nghe lịch tại `Schedules/{uid}` và hồ sơ tại `Users/{uid}`. Khi đến thời điểm nhắc, dịch vụ tạo bản ghi dưới `reminder_logs/{uid}` và gửi sự kiện đến chatbot để phát lời nhắc. Người cao tuổi có thể xác nhận, từ chối hoặc không phản hồi. Bản ghi được cập nhật trạng thái,

câu trả lời, số lần nhắc lại và thời điểm xác nhận. Việc lưu riêng từng lần thực hiện cho phép người chăm sóc xem lịch sử mà không thay đổi nội dung lịch gốc.

4.3.11 WebRTC trong RobotApp

`video_call_fragment` khởi tạo `PeerConnectionFactory`, nguồn âm thanh và hình ảnh, thành phần hiển thị video cùng tham chiếu đến nhánh `webrtc_signal`. Khi chủ động gọi, RobotApp ghi bản đề nghị `offer` và các địa chỉ ứng viên ICE vào `robotCandidates`. Khi nhận cuộc gọi, ứng dụng đọc `offer`, tạo bản trả lời `answer` và theo dõi `caregiverCandidates`. Các bộ lắng nghe được đăng ký theo từng giai đoạn thiết lập phiên và hủy khi không còn sử dụng.

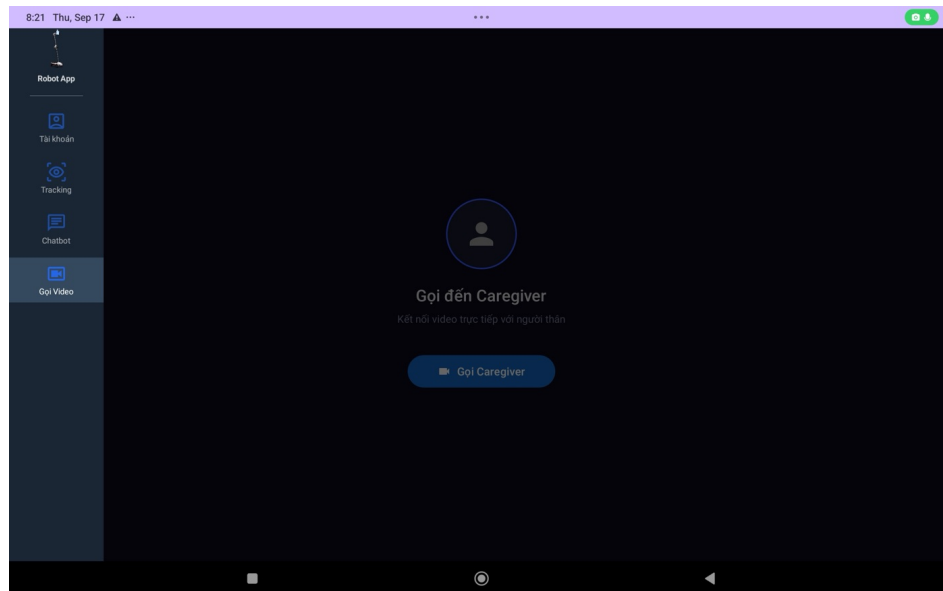
WebRTC đảm nhiệm mã hóa và truyền âm thanh, hình ảnh; Firebase trao đổi SDP, ICE và trạng thái cuộc gọi [74]. Trường `emergencyReason=fall` phân biệt cuộc gọi liên quan đến té ngã với cuộc gọi thông thường. Khi kết thúc, ứng dụng đặt trạng thái thành `ended`, giải phóng luồng dữ liệu, bộ thu hình, thành phần hiển thị và `PeerConnection`, sau đó trả quyền sử dụng camera cho chức năng theo dõi.

Hình 4.7 thể hiện hai trạng thái của `fragment` cuộc gọi trên RobotApp: màn hình khởi tạo cuộc gọi đến người chăm sóc và phiên gọi đang kết nối. Trong phiên gọi, khung hình lớn là luồng video nhận từ CaregiverApp, còn khung nhỏ là luồng hình ảnh cục bộ do camera robot cung cấp.

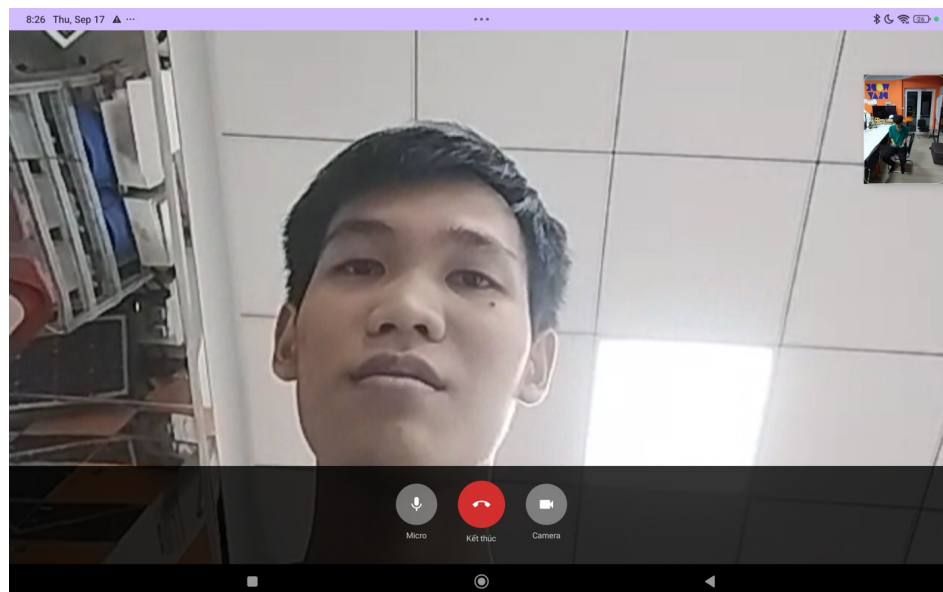
4.3.12 RobotControlManager và BLE

`RobotControlManager` quản lý hai chế độ điều khiển loại trừ nhau. Khi bấm theo tự động được bật, bộ quản lý nhận vector cục bộ từ chuỗi suy luận và không lắng nghe lệnh joystick trên RTDB. Khi bấm theo bị tắt, `ValueEventListener` tại nhánh `robot_control` được kích hoạt; trong `onDataChange()`, ba trường `x`, `y` và `ts` lần lượt được đọc thành thành phần tịnh tiến, thành phần quay và thời điểm tạo lệnh. Bản ghi đầu tiên nhận được khi đăng ký bộ lắng nghe chỉ phản ánh trạng thái đang lưu trên RTDB nên được bỏ qua, tránh thực thi lại một lệnh của phiên điều khiển trước.

Để không phụ thuộc vào độ lệch đồng hồ giữa hai điện thoại, RobotApp theo dõi `.info/serverTimeOffset` và ước lượng thời gian hiện tại của Firebase bằng tổng thời gian cục bộ với độ lệch nhận được. Giá trị này được dùng để tính tuổi của `ts` trên cùng mốc thời gian mà CaregiverApp sử dụng khi tạo lệnh. Robot chỉ thực thi lệnh khi đã nhận được thông tin đồng bộ thời gian, `x`, `y` là các giá trị hữu hạn và `ts` nằm trong giới



(a) Màn hình khởi tạo cuộc gọi đến người chăm sóc



(b) Phiên gọi đang kết nối với luồng cục bộ và từ xa

Hình 4.7: Chức năng gọi video trên RobotApp

hạn cho phép. Lệnh không hợp lệ hoặc được nhận khi chưa xác định được mốc thời gian chung không tạo chuyển động; bộ quản lý gửi lệnh dừng đến ESP32.

Ánh xạ vector điều khiển sang vận tốc hai bánh. Vector (x, y) chưa phải vận tốc động cơ. Thành phần x biểu diễn mức tịnh tiến tương đối, còn y biểu diễn mức quay tương đối; cả hai được giới hạn trong $[-1; 1]$. Trước khi trộn hai thành phần, bộ điều khiển loại bỏ lệnh quay có độ lớn nhỏ hơn 0,30 khi robot đang tiến hoặc lùi với $|x| > 0,30$. Bước này tạo vùng chết cho chuyển động quay, giúp robot đi thẳng ổn định hơn trước các dao động nhỏ của joystick hoặc sai lệch tâm ảnh. Thành phần quay còn lại được nhân với hệ số 0,15 để chuyển hướng diễn ra chậm hơn chuyển động tịnh tiến.

```

1 translation = clamp(x, -1, 1)
2 steering = clamp(y, -1, 1)
3
4 if abs(steering) < 0.30 and abs(translation) > 0.30:
5     steering = 0
6
7 steering = 0.15 * steering
8 left = clamp((steering + translation) * 20, -20, 20)
9 right = clamp((steering - translation) * 20, -20, 20)
10 payload = "L%.2fR%.2f\n".format(Locale.US, left, right)

```

Listing 2: Định dạng lệnh BLE ở mức logic

Hai biểu thức trộn có dấu khác nhau trước thành phần tịnh tiến vì hai động cơ được lắp đối xứng. Khi $x < 0$ và $y = 0$, vận tốc đặt của bánh trái mang dấu âm, còn bánh phải mang dấu dương; xét theo chiều lắp trục, cả hai bánh vẫn làm robot tiến. Khi $x = 0$, thành phần quay tạo hai giá trị đặt cùng dấu, khiến hai bánh quay theo hai hướng chuyển động vật lý ngược nhau và robot quay tại chỗ. Hằng số 20 chuyển lệnh chuẩn hóa sang vận tốc góc đặt cực đại 20 rad/s; phép `clamp` cuối cùng bảo đảm từng bánh không vượt quá giới hạn này.

Ví dụ, với lệnh $x = -0,60$ và $y = 0,50$, thành phần quay sau hiệu chỉnh là $s = 0,15 \times 0,50 = 0,075$. Vận tốc đặt nhận được là

$$v_L = (0,075 - 0,60) \times 20 = -10,50 \text{ rad/s}, \quad v_R = (0,075 + 0,60) \times 20 = 13,50 \text{ rad/s}. \quad (4.25)$$

RobotApp định dạng hai giá trị này thành chuỗi `L-10.50R13.50\n` trước khi ghi vào đặc tính BLE. Như vậy, ESP32 không nhận trực tiếp x , y hoặc ts , mà chỉ nhận vận tốc góc đặt của động cơ trái và phải. Firmware tách hai trường bắt đầu bằng ký tự L và R, sau đó cập nhật giá trị đặt cho hai bộ điều khiển động cơ tương ứng.

RobotControlManager tìm thiết bị theo tên OhmniRobot và UUID đã quy định. Thời gian quét tối đa là 10 s; khi cần khôi phục kết nối, ứng dụng thử kết nối lại sau 3 s. Sau khi tìm được dịch vụ và đặc tính nhận lệnh, ứng dụng sử dụng kiểu ghi `WRITE_TYPE_NO_RESPONSE`, với cách gọi API phù hợp từng phiên bản Android. Nếu nguồn đang được chọn không cung cấp lệnh mới trong 500 ms, bộ giám sát thời gian nhận lệnh gửi lệnh dừng xuống ESP32. Hàm `stop()` cũng gửi lệnh dừng trước khi hủy các bộ lắng nghe, hàm xử lý sự kiện, tác vụ quét và kết nối GATT.

4.4 Hiện thực CaregiverApp

4.4.1 Khởi động, xác thực và hồ sơ

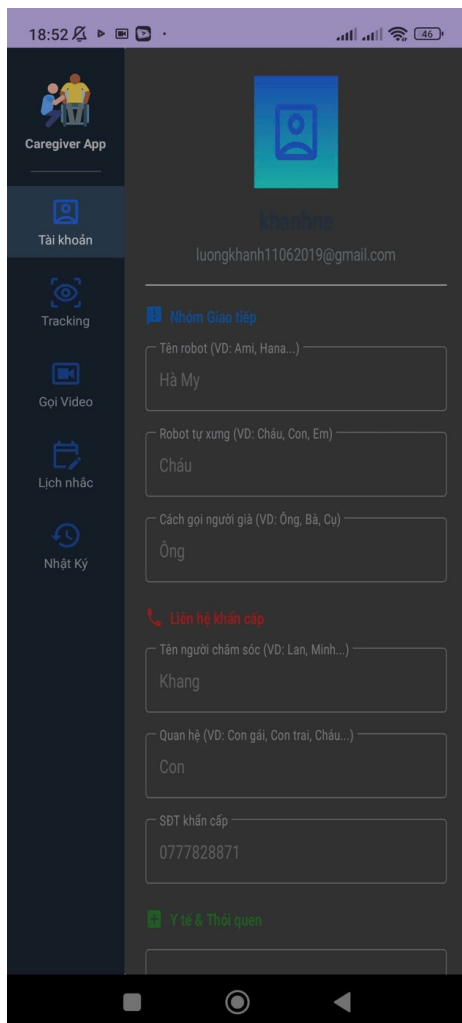
CaregiverApp sử dụng `LoginActivity` và `SignUpActivity` cho quá trình xác thực, `MainActivity` làm màn hình khởi động chính, còn `ProfileFragment` phục vụ quản lý hồ sơ. Firebase Authentication cung cấp UID; hồ sơ được lưu tại `Users/{uid}` để RobotApp cùng truy cập. `MainActivity` đồng thời khởi động `IncomingCallService` nhằm tiếp nhận cuộc gọi khi ứng dụng chạy nền.

Hình 4.8 thể hiện màn hình hồ sơ của CaregiverApp. Biểu mẫu được chia thành ba nhóm: nhóm giao tiếp cung cấp tên robot, cách tự xưng và cách gọi người cao tuổi; nhóm liên hệ khẩn cấp lưu tên, quan hệ và số điện thoại của người chăm sóc; nhóm y tế và thói quen lưu bệnh nền, lưu ý thị giác hoặc thính giác và sở thích. Các trường này được RobotApp đọc lại để bổ sung ngữ cảnh cho chatbot.

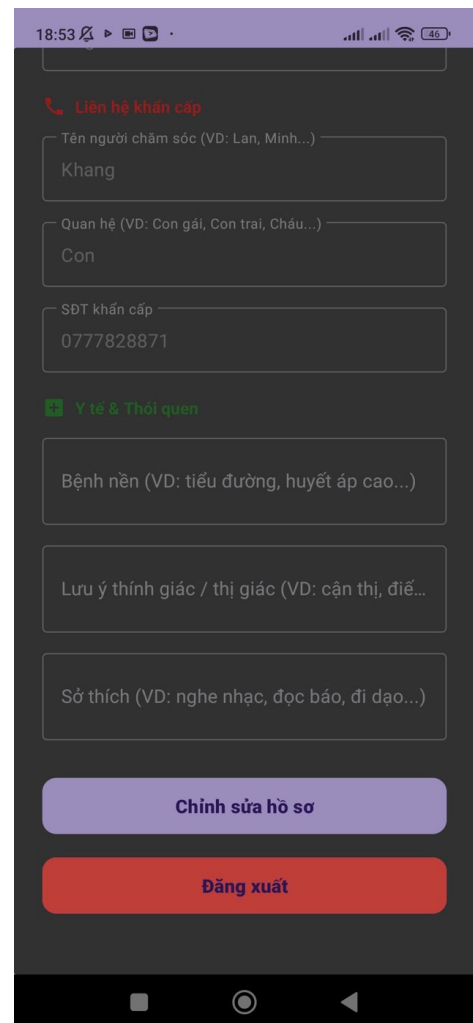
4.4.2 Bật/tắt chức năng theo dõi

`tracking_fragment` cập nhật hai giá trị trạng thái `robot_follow_enabled` và `ai_fall_detection_enabled` để bật, tắt các chức năng tương ứng. Fragment theo dõi `.info/connected` để hiển thị tình trạng kết nối RTDB. Khi tắt bấm theo tự động, CaregiverApp đồng thời gửi lệnh $x = 0, y = 0$; thời điểm của lệnh dừng được Firebase xác lập để không phụ thuộc vào đồng hồ cục bộ của điện thoại.

Giao diện tương ứng được trình bày ở Hình 4.9. Mỗi chức năng có một công tắc riêng kèm dòng trạng thái mô tả tình trạng hiện tại; chỉ báo kết nối RTDB được hiển thị



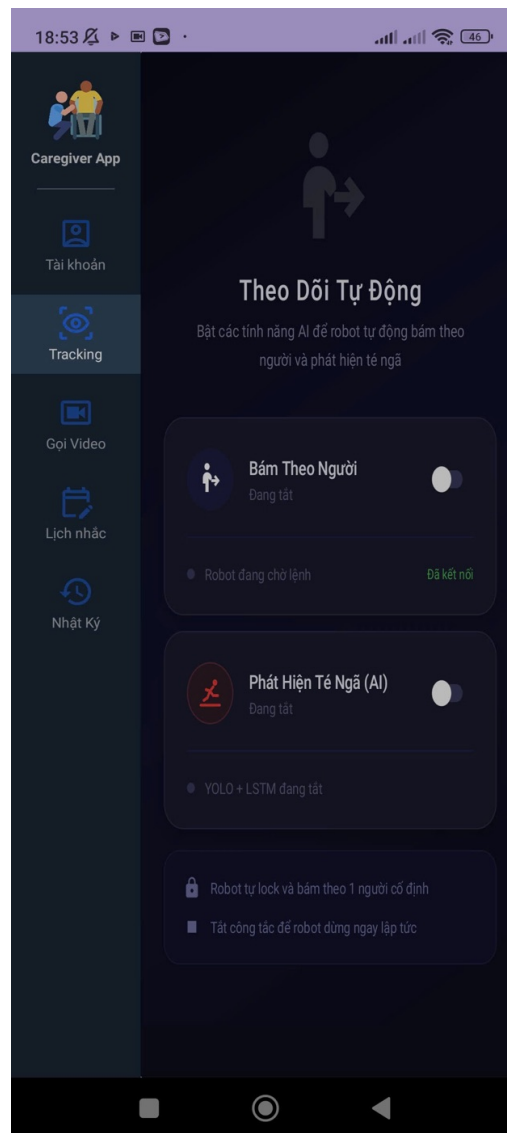
(a) Nhóm giao tiếp và liên hệ khẩn cấp



(b) Nhóm y tế, thói quen và thao tác tài khoản

Hình 4.8: Màn hình quản lý hồ sơ trên CaregiverApp

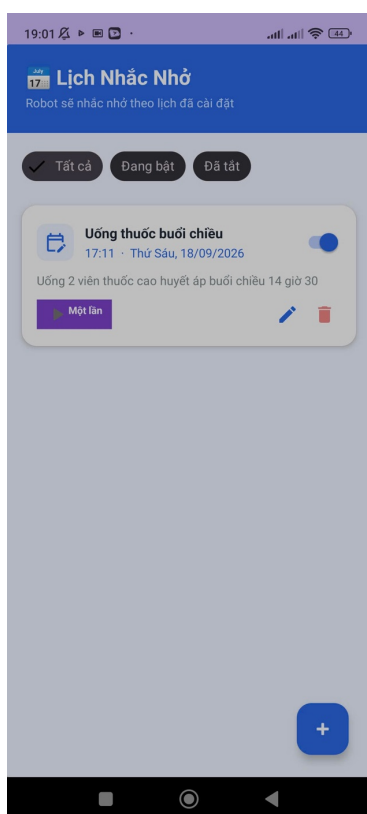
ngay dưới công tắc bấm theo để người chăm sóc biết lệnh có được đồng bộ hay không.



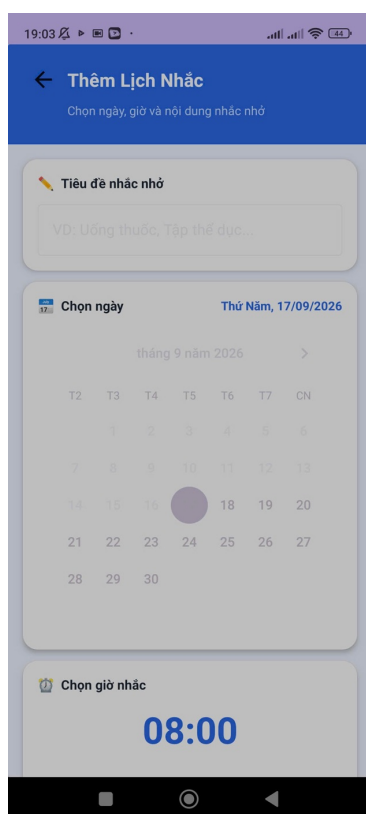
Hình 4.9: Màn hình bật, tắt bấm theo tự động và phát hiện té ngã

4.4.3 Lịch và nhật ký

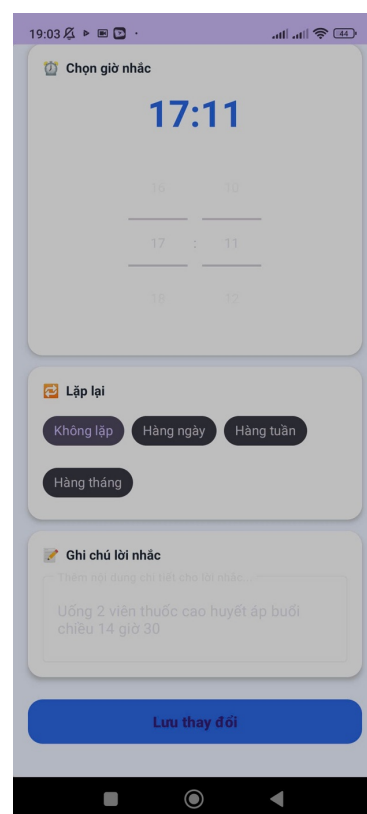
Danh sách lịch tại `Schedules/uid` được đọc bởi `ScheduleFragment`. Người chăm sóc tạo hoặc cập nhật lịch qua `AddScheduleFragment`; `ScheduleAdapter` hiển thị danh sách bằng `RecyclerView` và liên kết công tắc bật, tắt với trường `active`. Nhật ký lời nhắc, cảnh báo té ngã và cuộc gọi có các fragment cùng bộ hiển thị danh sách riêng. Cảnh báo và cuộc gọi được truy vấn theo thời điểm trong bảy ngày gần nhất.



(a) Danh sách lịch nhắc nhở đã tạo

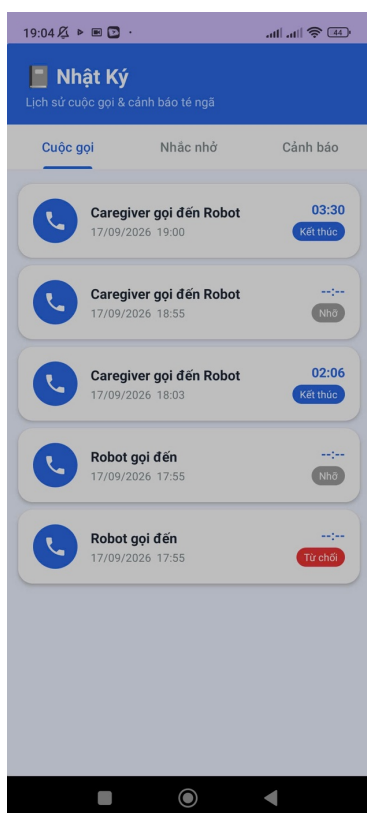


(b) Chọn ngày nhắc

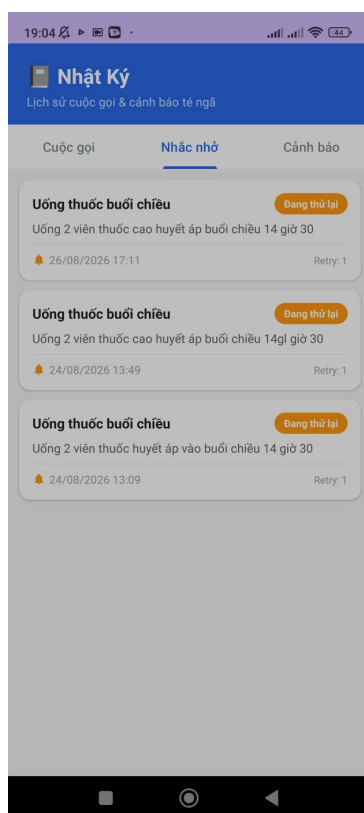


(c) Chọn giờ, chu kỳ lặp và nội dung nhắc

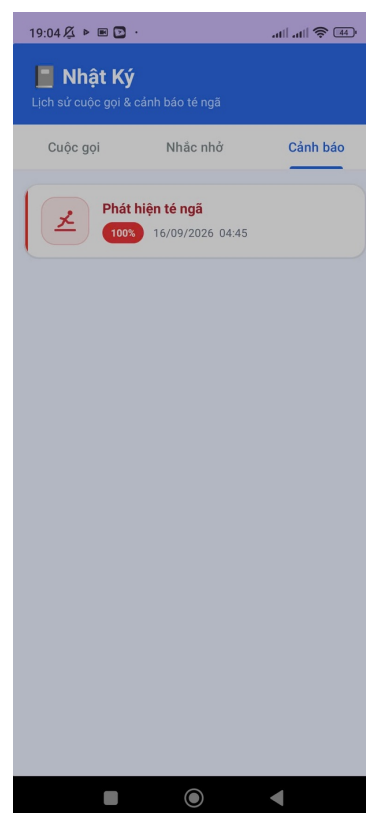
Hình 4.10: Quản lý lịch nhắc nhở trên CaregiverApp



(a) Lịch sử cuộc gọi



(b) Nhật ký lời nhắc



(c) Cảnh báo té ngã

Hình 4.11: Ba nhóm nhật ký hoạt động trên CaregiverApp

4.4.4 Tiếp nhận cuộc gọi khi ứng dụng chạy nền

IncomingCallService là dịch vụ chạy nền có thông báo thường trực (foreground service), lắng nghe nhánh `webrtc_signal`. Khi nhận `status=calling` và `callerRole=robot`, dịch vụ tạo thông báo và có thể mở `IncomingCallActivity` toàn màn hình. Nếu có `emergencyReason=fall`, nội dung thông báo cho biết cuộc gọi được tạo sau tình huống chưa xác nhận an toàn. Tập Manifest khai báo các quyền liên quan đến thông báo, dịch vụ nền và hiển thị toàn màn hình; khả năng hiển thị phụ thuộc vào quyền được cấp và chính sách của hệ điều hành.

Hình 4.12 là thông báo cuộc gọi đến được tạo khi CaregiverApp không ở tiền cảnh. Thông báo nêu rõ bên gọi là robot và cung cấp hai lựa chọn từ chối hoặc nhận cuộc gọi mà không cần mở ứng dụng trước.



Hình 4.12: Thông báo cuộc gọi đến từ robot khi ứng dụng chạy nền

4.4.5 Gọi video và điều khiển robot từ xa

`video_call_fragment` xử lý cả vai trò gọi đi và nhận cuộc gọi, hiển thị hình ảnh cục bộ, từ xa và trao đổi thông tin thiết lập phiên. Hàm `setupVirtualJoystick()` xác định tâm và bán kính di chuyển của cần điều khiển. Với mỗi sự kiện `ACTION_DOWN` hoặc `ACTION_MOVE`, độ lệch cảm ứng được giới hạn trong vùng tròn rồi ánh xạ theo $x = \Delta y/r$, $y = -\Delta x/r$. Hàm `sendControlIfThrottled()` giới hạn tần suất gửi khoảng 20 Hz. `CaregiverApp` theo dõi `.info/serverTimeOffset`, cộng độ lệch này với thời gian cục bộ tại thời điểm thao tác và ghi kết quả cùng x, y vào `robot_control`. Lệnh chuyển động chưa được phát khi thông tin đồng bộ thời gian chưa sẵn sàng.

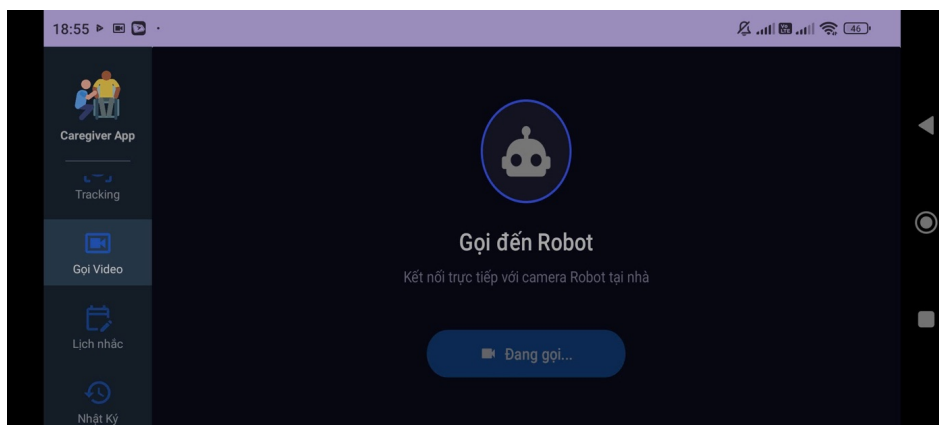
Khi người dùng bắt đầu điều khiển, ứng dụng tắt bấm theo tự động để tránh hai nguồn lệnh cùng hoạt động. Các sự kiện `ACTION_UP`, `ACTION_CANCEL`, kết thúc cuộc gọi và rời màn hình đều gọi luồng gửi lệnh dừng với $x = 0$, $y = 0$ và một ts mới. Nếu độ lệch thời gian chưa sẵn sàng, riêng lệnh dừng sử dụng `ServerValue.TIMESTAMP` để máy chủ xác lập thời điểm; phương án dự phòng này không được dùng cho lệnh gây chuyển động. Việc cập nhật lại ts giúp `RobotApp` phân biệt lệnh dừng vừa tạo với dữ liệu cũ còn lưu trên RTDB.

`CaregiverApp` gửi lệnh điều khiển chuẩn hóa qua Internet. `RobotApp` tiếp nhận, kiểm tra và chuyển lệnh thành vận tốc của hai động cơ theo quy ước lắp đặt, rồi truyền đến ESP32 qua BLE. Nhờ đó, người chăm sóc có thể điều khiển từ xa khi hai ứng dụng có kết nối Internet, còn việc giao tiếp với phần thân được thực hiện tại robot.

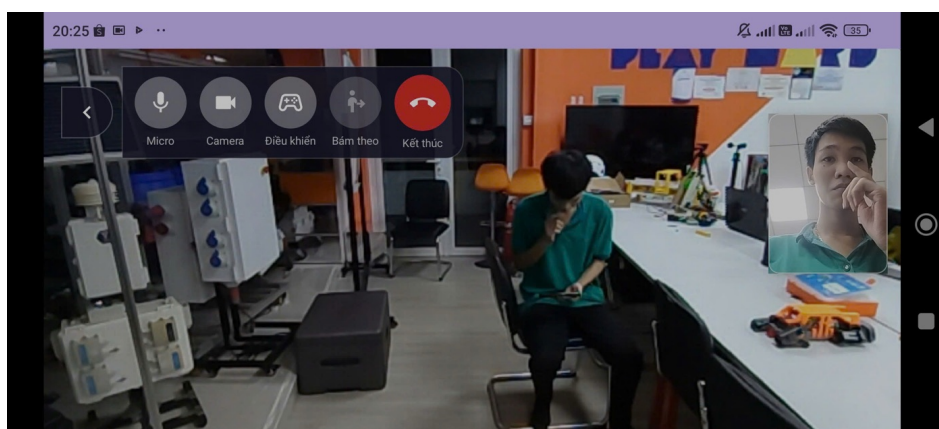
Hình 4.13 và Hình 4.14 minh họa các bước của luồng này trên `CaregiverApp`. Thanh chức năng trong cuộc gọi gồm micro, camera, điều khiển, bấm theo và kết thúc; khi chọn mục điều khiển, cần điều khiển ảo xuất hiện ở góc trái màn hình và các thao tác kéo được quy đổi thành cặp giá trị x, y như mô tả ở trên.

4.5 Hiện thực dữ liệu dùng chung

Các bộ lắng nghe Firebase cung cấp bản cập nhật gần thời gian thực, trong khi thao tác ghi có thể được phản ánh cục bộ trước khi đồng bộ với máy chủ [73]. Vì vậy, mỗi lệnh joystick truyền qua RTDB phải có timestamp; callback được gọi theo thời gian thực không có nghĩa dữ liệu nhận được luôn còn hiệu lực. Hai ứng dụng cùng quy đổi thời gian cục bộ theo `.info/serverTimeOffset` để việc kiểm tra tuổi lệnh không bị sai khi đồng hồ thiết bị lệch nhau. Các lớp `UserProfile`, `ScheduleItem`,

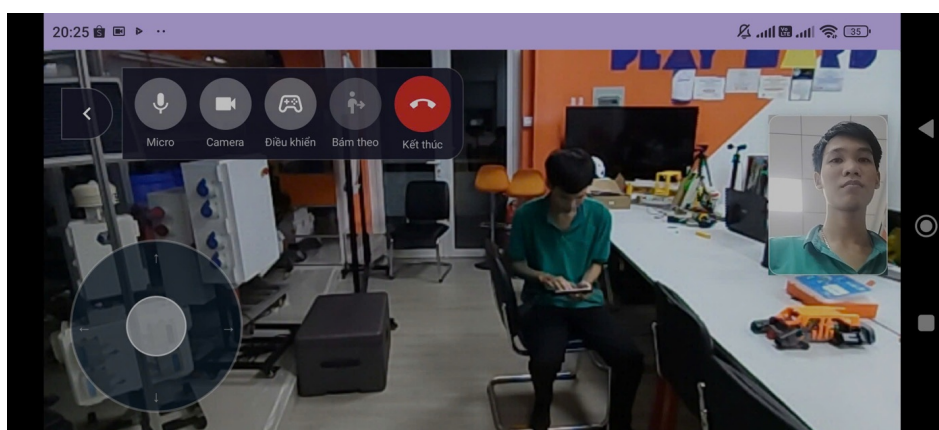


(a) Khởi tạo cuộc gọi đến robot



(b) Thanh chức năng trong cuộc gọi

Hình 4.13: Chức năng gọi video trên CaregiverApp



Hình 4.14: Cần điều khiển ảo dùng để điều khiển robot từ xa trong cuộc gọi

FallAlertEntry, ReminderLogEntry và CallHistoryEntry được dùng để ánh xạ dữ liệu JSON sang đối tượng trong ứng dụng.

Hồ sơ, lịch và nhật ký lời nhắc được lưu dưới các nhánh cấp gốc nhưng đều tiếp tục phân vùng theo UID. Các kênh vận hành gồm cờ chức năng, lệnh điều khiển, signaling, cảnh báo té ngã và lịch sử cuộc gọi được tập trung dưới `pair_sessions/{uid}`. Security Rules áp dụng nguyên tắc từ chối mặc định và chỉ cho phép tài khoản đã xác thực truy cập nhánh có UID tương ứng [81]. Cấu trúc này ngăn dữ liệu của các tài khoản khác nhau dùng chung kênh vận hành, nhưng hai thiết bị trong một cặp vẫn đăng nhập bằng cùng tài khoản nên chưa biểu diễn được vai trò robot và người chăm sóc như hai danh tính độc lập.

4.6 Huấn luyện và chuyển đổi mô hình AI

Mô hình phát hiện té ngã được chuẩn bị theo một quy trình liên tục từ xử lý dữ liệu video, trích xuất đặc trưng tư thế, huấn luyện LSTM đến chuyển đổi sang định dạng phù hợp với Android. Việc thống nhất quy ước đặc trưng giữa giai đoạn huấn luyện và suy luận có ý nghĩa quyết định, bởi chỉ một sai khác về thứ tự hoặc cách chuẩn hóa tọa độ cũng làm đầu vào trên thiết bị không còn tương thích với mô hình.

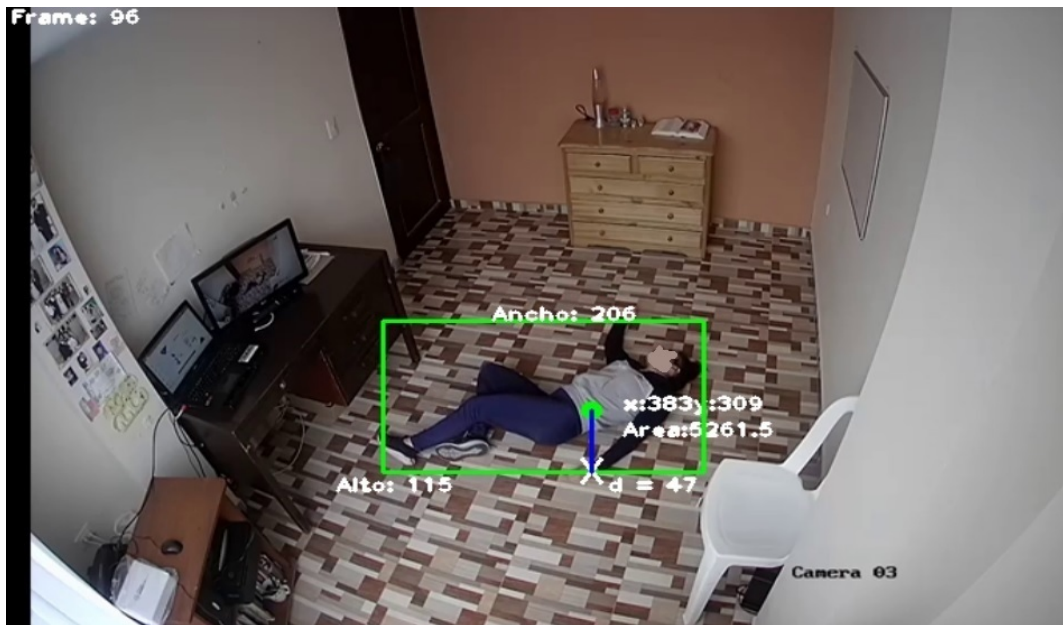
4.6.1 Xử lý dữ liệu

Dữ liệu LE2I được cung cấp dưới dạng các video liên tục gồm hoạt động bình thường và tình huống té ngã. Quá trình tiền xử lý gồm:

- **Trích xuất khung hình:** Toàn bộ khung hình của mỗi video được đọc tuần tự, không lấy mẫu lại. Video LE2I được ghi ở 25 FPS, nên nhịp khung hình của dữ liệu huấn luyện trùng với nhịp thu nhận mục tiêu của camera robot và một cửa sổ 15 khung hình biểu diễn cùng một khoảng thời gian ở cả hai giai đoạn.
- **Gán nhãn theo trạng thái:** Mỗi khung hình từ thời điểm bắt đầu ngã đến hết video được gán nhãn dương, biểu diễn trạng thái người nằm trên sàn. Quy ước này thay cho cách chỉ đánh dấu khoảng thời gian rơi của chú thích gốc; lý do và ảnh hưởng được phân tích ở Mục 5.2.
- **Tạo keypoints bằng YOLO-Pose:** Các khung hình sau khi được trích xuất được đưa qua mô hình YOLO-Pose để phát hiện tư thế và tạo ra tọa độ các điểm đặc trưng (keypoints) của đối tượng. Tập dữ liệu keypoints này được lưu lại để phục vụ huấn

luyện LSTM thay vì sử dụng ảnh thô. Ở phiên bản hiện tại, bước chuẩn bị dữ liệu sử dụng `yolov8n-pose.pt`, trong khi RobotApp suy luận bằng YOLO26n-Pose; chênh lệch này được thảo luận ở Chương 5.

- **Phân chia tập dữ liệu:** Tập huấn luyện, xác thực và kiểm tra được chia theo video và chủ thể, không chia ngẫu nhiên theo từng khung hình. Cách tổ chức này hạn chế rò rỉ dữ liệu khi những khung hình gần nhau của cùng một hành động xuất hiện đồng thời ở tập huấn luyện và tập kiểm tra.
- **Xử lý mất cân bằng lớp:** Số mẫu hoạt động bình thường thường lớn hơn số mẫu té ngã. Trọng số lớp và phương pháp lấy mẫu tăng cường được sử dụng để giảm xu hướng mô hình nghiêng về lớp NO FALL.



Hình 4.15: Ví dụ khung hình té ngã trong dữ liệu dùng để chuẩn bị mô hình

4.6.2 Trích xuất đặc trưng

Mô hình LSTM không nhận trực tiếp ảnh RGB mà học từ chuỗi đặc trưng tư thế do YOLO-Pose cung cấp. Mỗi khung hình của người mục tiêu được biểu diễn bởi 69 giá trị theo đúng quy ước sử dụng trong RobotApp:

1. **Tọa độ chuẩn hóa:** Tọa độ (x_i, y_i) của 17 keypoint được chuẩn hóa theo tâm hông và kích thước của vùng tư thế, giúp giảm ảnh hưởng của vị trí và khoảng cách giữa người với camera.

2. **Đặc trưng chuyển động:** Với mỗi keypoint, hai thành phần $(\Delta x_i, \Delta y_i)$ biểu diễn độ dịch chuyển so với khung hình trước. Ở khung hình đầu tiên của mỗi chuỗi, hai giá trị này được đặt bằng 0. Mỗi điểm vì vậy đóng góp bốn giá trị $(x_i, y_i, \Delta x_i, \Delta y_i)$.
3. **Tỷ lệ hình dạng và cửa sổ thời gian:** Tỷ lệ rộng/cao của bounding box được ghép với 68 giá trị keypoint để tạo vector 69 chiều. Mười lăm khung hình liên tiếp hình thành tensor đầu vào có kích thước $[1, 15, 69]$ cho LSTM.

Cấu trúc tế bào LSTM đã được giới thiệu ở Hình 2.3. Trong ứng dụng, chuỗi đặc trưng tư thế được đưa vào mô hình để phân tích sự thay đổi chuyển động theo thời gian.

4.6.3 Huấn luyện mô hình

Mô hình LSTM được xây dựng bằng TensorFlow/Keras. Quá trình huấn luyện gồm:

- Thiết lập seed cố định để tăng khả năng tái lập thí nghiệm.
- Sử dụng hàm mất mát Binary Cross-Entropy cho bài toán phân loại nhị phân.
- Áp dụng dừng sớm khi hàm mất mát trên tập xác thực không còn cải thiện, đồng thời lưu mô hình tốt nhất tại `best_model_lstm.h5`.

4.6.4 Chuyển đổi và tối ưu mô hình

Để triển khai trên điện thoại Android, mô hình sau huấn luyện được chuyển sang TensorFlow Lite nhằm giảm kích thước và tận dụng môi trường suy luận dành cho thiết bị biên.

1. Đối với mô hình LSTM:

Chương trình `convert_to_tflite.py` nạp `best_model_lstm.h5` và kiểm tra `input_shape` trước khi chuyển đổi bằng `TFLiteConverter.from_keras_model`. Tập toán tử gồm TFLite built-ins và Select TF Ops. Kết quả là `best_model_lstm_v2.tflite`, có thể đóng gói trực tiếp trong thư mục `assets` của RobotApp.

2. Đối với mô hình YOLO-Pose:

Mô hình PyTorch `yolo26n-pose.pt` trước hết được xuất sang ONNX với kích thước lô cố định bằng 1 và ảnh đầu vào 320×320 . Thư viện `onnx2tf` tiếp tục chuyển mô hình ONNX sang SavedModel và TFLite. Biến thể FP16 lưu trọng số bằng số dấu phẩy động

16 bit, qua đó giảm dung lượng và băng thông bộ nhớ so với FP32; mức ảnh hưởng đến chất lượng cần được xác nhận bằng phép kiểm tra tương đương sau chuyển đổi.

Bảng 4.4: Các mô hình chính sau khi huấn luyện và chuyển đổi

Tập mô hình	Định dạng	Dung lượng	Mục đích
best_model_lstm_v3_groundtruth	Keras H5	189,2 KB	Mô hình LSTM nguồn để huấn luyện và chuyển đổi
fall_detection_lstm_v3_groundtruth	TFLite float.tflite	59,6 KB	Mô hình LSTM được đóng gói trong RobotApp, chỉ dùng toán tử dựng sẵn
yolo26n-pose.pt	PyTorch	7,88 MB	Mô hình Pose gốc
yolo26n-pose_no_nms_float16.tflite	TFLite FP16	6,01 MB	Biến thể YOLO FP16 ở giai đoạn chuyển đổi, đầu vào 480×640 , dùng để đối chiếu kích thước và tốc độ
yolo26n-pose_no_nms_float32.tflite	TFLite FP32	11,87 MB	Biến thể float32 tương ứng, dùng làm mốc so sánh cho FP16

4.6.5 Quy trình kiểm tra tương đương sau chuyển đổi

Quy trình kiểm tra tương đương sử dụng cùng một tập dữ liệu đầu vào cho mô hình gốc và mô hình TFLite, sau đó đối chiếu bounding box, độ tin cậy, điểm khớp và vector xác suất của LSTM. Các phép so sánh nhằm phát hiện sai khác về toán tử, thứ tự đầu ra hoặc cách chuẩn hóa mà việc nạp mô hình thành công chưa phản ánh được.

Quy trình này đã được thực hiện đầy đủ cho mô hình LSTM được đóng gói. Toàn bộ 11.743 cửa sổ của tập kiểm thử được đưa qua interpreter TFLite và đối chiếu với đầu ra của mô hình Keras nguồn. Biến thể không lượng tử hóa có sai lệch xác suất tối đa $3,5 \times 10^{-6}$ và trùng 100% quyết định sau khi áp ngưỡng; biến thể lượng tử hóa có sai lệch tối đa 0,596 và trùng 99,85% quyết định. Kết quả cho thấy lượng tử hóa trọng số không làm thay đổi các chỉ số tổng hợp nhưng có thể làm lệch đáng kể xác suất của từng cửa sổ riêng lẻ.

Quá trình chuyển đổi cũng cho thấy việc nạp được mô hình không bảo đảm mô hình chạy đúng. Khi kích thước lô đầu vào để ở dạng động, trình chuyển đổi không hạ được các toán tử danh sách tensor của LSTM về tập toán tử dựng sẵn và yêu cầu bổ sung Select TF Ops. Một phương án chuyển đổi khác tạo ra tệp hợp lệ nhưng phát sinh lỗi READ_VARIABLE ngay khi suy luận vì biến trạng thái chưa được khởi tạo. Chỉ khi dựng lại mô hình với kích thước lô cố định bằng một rồi chuyển đổi trực tiếp từ mô hình Keras, tệp thu được mới vừa chỉ dùng toán tử dựng sẵn vừa chạy đúng. Nhờ đó, phiên

bản đóng gói không còn phụ thuộc gói `tensorflow-lite-select-tf-ops`, là một trong các thành phần làm tăng dung lượng gói cài đặt đã nêu ở Chương 5.

4.7 Hiện thực firmware phần thân robot

4.7.1 Khởi tạo phần cứng

Firmware sử dụng ESP32 để giao tiếp với hai cảm biến góc AS5600 qua hai bus I2C. Cảm biến của động cơ trái nối với $SDA=19/SCL=18$, còn cảm biến của động cơ phải nối với $SDA=23/SCL=5$. Mạch công suất bên trái dùng $IN1=32, IN2=33, IN3=25, EN=22$; bên phải dùng $IN1=26, IN2=27, IN3=14, EN=12$. Hai bus I2C riêng cho phép sử dụng đồng thời hai AS5600 có cùng địa chỉ mặc định.

Hai đối tượng `BLDCMotor` được cấu hình với 7 cặp cực và liên kết với cảm biến góc cùng mạch công suất điều khiển bằng ba tín hiệu PWM. Chương trình thiết lập phương pháp điều chế `SpaceVectorPWM`, chế độ điều khiển vận tốc `velocity`, các tham số PID và bộ lọc thông thấp trước khi gọi `init()` và `initFOC()`. Trong vòng lặp chính, `loopFOC()` được gọi liên tục, không chèn thời gian chờ bằng `delay()`.

4.7.2 Tiếp nhận lệnh qua BLE GATT

NimBLE khởi tạo thiết bị có tên `OhmniRobot`, thiết lập công suất phát và đưa UUID dịch vụ vào dữ liệu phản hồi quét để hỗ trợ tìm kiếm. Đặc tính nhận lệnh sử dụng thuộc tính ghi không phản hồi `WRITE_NR`. `ServerCallbacks::onConnect()` đánh dấu liên kết đã được thiết lập; `ServerCallbacks::onDisconnect()` đưa `target_left` và `target_right` về 0 để dừng robot khi mất kết nối.

Khi đặc tính được ghi, `CharCallbacks::onWrite()` lấy payload dưới dạng `std::string`, chuyển thành `String`, loại khoảng trắng rồi gọi `parseCommand()`. Hàm này tìm ký tự L và R, tách hai chuỗi số và chuyển chúng bằng `toFloat`. Kết quả được lưu vào `target_left, target_right`; đồng thời `lastCmdTime` được cập nhật bằng `millis()` để phục vụ `watchdog`. Hai giá trị target có đơn vị rad/s vì hai đối tượng động cơ được cấu hình ở chế độ `MotionControlType::velocity`. Phiên bản hiện thực hiện tại kiểm tra cấu trúc L-R nhưng chưa kiểm tra chặt toàn bộ chuỗi và tính hữu hạn sau chuyển đổi; đây là điểm cần bổ sung tại tầng firmware để không chỉ phụ thuộc vào bước kiểm tra của `RobotApp`.

4.7.3 Xử lý vùng không đáp ứng và dừng khi mất lệnh

Trong `loop()`, `motor.loopFOC()` và `motor1.loopFOC()` được gọi liên tục để đọc phản hồi góc từ hai AS5600 và cập nhật vòng điều khiển FOC. Sau bước kiểm tra watchdog, `applyDeadzone()` đưa $|v| < 0,5$ về 0, nâng độ lớn của lệnh khác 0 nhưng nhỏ hơn 3 lên 3, rồi giới hạn kết quả trong $[-40, 40]$. Giá trị sau xử lý được truyền lần lượt vào `motor.move(left_cmd)` và `motor1.move(right_cmd)`. Do chế độ điều khiển là vận tốc, `move()` đưa giá trị đặt vào bộ điều khiển vận tốc; PID kết hợp phản hồi encoder tạo điện áp điều khiển, còn hai đối tượng `BLDCDriver3PWM` phát tín hiệu PWM đến mạch công suất của động cơ trái và phải.

Cơ chế watchdog theo dõi thời gian kể từ lần nhận lệnh gần nhất khi BLE đang kết nối. Nếu thời gian này vượt 800 ms và vận tốc đặt khác 0, firmware ghi nhật ký và đưa vận tốc đặt của cả hai bánh về 0. Do `move()` được gọi sau bước kiểm tra, lệnh dừng được áp dụng ngay trong chu kỳ điều khiển hiện tại.

```
1 loopFOC(left); loopFOC(right)
2 if connected and now - lastCommand > 800 ms:
3     targetLeft = 0; targetRight = 0
4 leftCommand = applyDeadzone(targetLeft)
5 rightCommand = applyDeadzone(targetRight)
6 move(leftCommand); move(rightCommand)
```

Listing 3: Giả mã xử lý lệnh và điều khiển động cơ trên ESP32

4.8 Kết luận chương

Chương đã trình bày cách các module thiết kế được chuyển thành hai ứng dụng Android, các mô hình AI và firmware ESP32. RobotApp hiện thực các chức năng cảm nhận, hội thoại, điều phối và kết nối với thân robot; CaregiverApp hiện thực các chức năng quản lý, giám sát và can thiệp từ xa. Firebase, WebRTC và BLE lần lượt đảm nhiệm đồng bộ dữ liệu, truyền âm thanh–hình ảnh và chuyển lệnh đến phần thân robot. Quy trình chuẩn bị mô hình cùng chương trình firmware hoàn thiện chuỗi xử lý từ dữ liệu camera đến quyết định và chuyển động của robot. Chương 5 trình bày phương pháp kiểm thử, kết quả đánh giá và các giới hạn được ghi nhận.

Chương 5 – Kết quả và thảo luận

5.1 Mục tiêu đánh giá

Đánh giá nguyên mẫu nhằm trả lời bốn câu hỏi: các khối có được hiện thực đúng kiến trúc hay không; quy ước trao đổi dữ liệu giữa các ứng dụng có nhất quán hay không; các cơ chế an toàn có điểm kiểm chứng hay không; và bằng chứng hiện có cho phép kết luận tới mức nào. Do hệ thống gồm AI, cloud, mobile và phần cứng, một con số accuracy đơn lẻ không đại diện cho toàn hệ thống.

Chương này phân biệt ba loại bằng chứng. *Bằng chứng tĩnh* được rút từ mã nguồn, cấu hình và artifact. *Bằng chứng tự động* đến từ unit test/build có thể chạy lặp. *Bằng chứng tích hợp* đến từ thử nghiệm hai điện thoại, Internet, BLE và robot thật. Chỉ các kết quả có artifact hoặc được nhóm dự án xác nhận mới được ghi “đạt”; hạng mục thiếu log định lượng được ghi rõ để tránh tạo số liệu.

5.2 Cấu hình và tiêu chí đánh giá

Bảng 5.1: Cấu hình phần mềm được đánh giá

Hạng mục	Cấu hình
RobotApp	Android/Kotlin; minSdk 23; target/compile SDK 36; CameraX; TFLite; OpenCV; Firebase; WebRTC; BLE
CaregiverApp	Android/Kotlin; minSdk 23; target/compile SDK 36; Firebase; WebRTC
AI trên Android	Pose input 320×320 ; LSTM input 15×69 ; CPU/GPU TFLite artifacts
Firmware	ESP32; NimBLE; SimpleFOC; hai AS5600; hai BLDC; velocity limit 40 rad/s
Cloud	Firebase Authentication/RTDB; Groq HTTPS; STUN/TURN cho WebRTC
Giao thức điều khiển	RTDB x, y, ts ; BLE L<float>R<float>\n

Các chỉ số AI thích hợp gồm precision, recall, F1 và confusion matrix ở cấp sự cố. Với TP là phát hiện đúng sự cố, FP là cảnh báo giả, FN là bỏ sót:

$$Precision = \frac{TP}{TP + FP}, \quad (5.1)$$

$$Recall = \frac{TP}{TP + FN}, \quad (5.2)$$

$$F_1 = 2 \frac{Precision \cdot Recall}{Precision + Recall}. \quad (5.3)$$

Đơn vị đánh giá phải là một sự kiện té ngã, không phải từng frame; nhiều frame cảnh báo liên tiếp của cùng sự cố chỉ tính một sự kiện. Với tracking, chỉ số cần theo dõi là số lần ID0 đổi sai, tỷ lệ phục hồi đúng và thời gian mất mục tiêu. Với điều khiển, chỉ số là độ trễ end-to-end, tần số lệnh, tỷ lệ lệnh bị loại và thời gian dừng sau mất kênh. Với WebRTC, chỉ số gồm thời gian thiết lập, tỷ lệ cuộc gọi thành công, RTT, jitter và packet loss từ getStats.

5.3 Kết quả hiện thực và artifact

5.3.1 Mức độ hoàn thành thành phần

Bảng 5.2: Kết quả hiện thực theo thành phần

Thành phần	Trạng thái	Bằng chứng
Thành phần	Trạng thái	Bằng chứng
RobotApp Android	Đã hiện thực	28 tệp Kotlin; manifest; tài nguyên giao diện và ba mô hình trong assets
CaregiverApp Android	Đã hiện thực	25 tệp Kotlin; giao diện quản lý, nhật ký, lịch nhắc nhở và cuộc gọi
Pose trên thiết bị	Đã tích hợp	Hai model TFLite CPU/GPU và pipeline decode/NMS
Tracking và Re-ID	Đã tích hợp	SORT, Kalman, Hungarian, HSV histogram, ID0
LSTM và hậu kiểm	Đã tích hợp	Model TFLite, queue 15×69, state machine, verification engine
Hội thoại giọng nói	Đã tích hợp RobotApp	SpeechRecognizer, Groq streaming/classify, Android TTS

Lịch nhắc nhở và nhật ký	Đã tích hợp	Schedules, reminder_logs, adapters và service
WebRTC	Đã tích hợp	Caller/callee, SDP/ICE RTDB, incoming foreground service
Điều khiển robot	Đã tích hợp	Joystick, RTDB, RobotControlManager, BLE GATT
Firmware BLDC	Đã hiện thực	Sketch ESP32, NimBLE, SimpleFOC, dual I2C, watchdog
Backend riêng/MQTT	Không thuộc hiện thực	Hệ thống dùng Firebase RTDB; không có broker/server signaling riêng
Chatbot CaregiverApp	Placeholder	Logic hội thoại thật chỉ nằm ở RobotApp

5.3.2 Kết quả đóng gói Android

Nhóm dự án xác nhận cả hai ứng dụng build thành công. Trong cây build có APK debug RobotApp khoảng 504,24 MiB và CaregiverApp khoảng 51,03 MiB. Chênh lệch chủ yếu do RobotApp đóng gói nhiều runtime native và thư viện AI (TFLite, GPU/Select TF Ops, OpenCV, WebRTC) cùng model; CaregiverApp không chứa model pose/LSTM và stack AI.

Kích thước 504 MiB là chấp nhận được cho prototype debug nhưng không phù hợp phát hành. Bản release cần bật shrinking, loại ABI không dùng, kiểm tra trùng native library, cân nhắc Play App Bundle/split APK và bỏ dependency AI dư. Model thực chỉ chiếm khoảng vài chục MB; phần lớn dung lượng tăng đến từ runtime/native binaries và ký hiệu debug.

Bảng 5.3: Artifact đóng gói hiện có

Artifact	Kích thước	Ý nghĩa
RobotApp debug APK	504,24 MiB	Bản debug tích hợp AI/WebRTC/BLE
CaregiverApp debug APK	51,03 MiB	Bản debug quản lý/WebRTC

5.3.3 Kết quả artifact mô hình

Chuỗi chuyển đổi đã tạo đủ artifact H5, PT, ONNX, SavedModel và TFLite. LSTM giảm từ khoảng 189,2 KB ở dạng Keras xuống 59,6 KB ở dạng TFLite không lượng tử hóa. Biến thể pose FP16 trong cây chuyển đổi có dung lượng 6,01 MB, khoảng một nửa biến thể float32 tương ứng là 11,87 MB; hai biến thể được đóng gói trong ứng dụng có dung lượng lần lượt khoảng 12,03 MB và 11,84 MB. Đây là kết quả về kích thước tệp, không tự động chứng minh tốc độ hoặc độ chính xác.

Bảng 5.4: Đánh giá định tính các biến thể mô hình

Biến thể	Lợi ích kỳ vọng	Điều phải đo
Float32 CPU	Tương thích rộng, dễ debug	Latency, nhiệt, FPS, bộ nhớ
Graph GPU	Tăng throughput nếu delegate hỗ trợ toàn graph	Tỷ lệ op chạy GPU, overhead copy, fallback
FP16	Giảm dung lượng/bảng thông bộ nhớ	Sai số keypoint, mAP/PCK, latency thật
NMS ngoài graph	Tăng tính tương thích delegate	Độ đúng decode/NMS Kotlin so với nguồn

5.4 Kiểm thử tự động ở mức logic

Các lớp xử lý độc lập với giao diện được kiểm thử bằng JUnit để có thể tái hiện các điều kiện biên mà không cần camera hoặc robot thật. Phiên bản hiện tại có 24 phương thức kiểm thử trực tiếp cho logic của đề tài, phân bố ở các nhóm đường dẫn Firebase, ánh xạ lệnh động cơ, chính sách bám người, hình học tư thế và hậu kiểm té ngã. Hai kiểm thử mẫu do Android Studio tạo sẵn không được tính vào nhóm này.

5.4.1 Ánh xạ lệnh và chính sách bám người

RobotControlManagerTest gồm sáu trường hợp bảo vệ quy ước dấu và hệ số chuyển đổi từ vector (x, y) sang vận tốc hai bán kính. Bốn trường hợp đầu kiểm tra tiến, lùi và quay tại chỗ; hai trường hợp còn lại xác nhận nhiều quay nhỏ bị loại khi robot đang tịnh tiến mạnh, trong khi lệnh quay đủ lớn vẫn được giữ lại. Các phép so sánh sử dụng sai số 0,001.

TrackingMotionPolicyTest bổ sung bốn trường hợp cho chính sách vận tốc. Các kiểm thử xác nhận tốc độ tiến tăng theo độ xa nhưng không vượt giới hạn, tốc độ

Bảng 5.5: Các trường hợp kiểm thử ánh xạ điều khiển

Nhóm kiểm thử	Đầu vào tiêu biểu	Kết quả mong đợi
Tiến và lùi theo hiệu chỉnh	$(-1, 0); (1, 0)$	$(-20, 20); (20, -20)$
Quay tại chỗ	$(0, 1); (0, -1)$	$(3, 3); (-3, -3)$
Loại nhiều quay nhỏ	$(0,4; 0,2)$	$(8, -8)$, thành phần quay nhỏ bị loại
Duy trì lệnh quay có chủ đích	$(0,4; 0,5)$	$(9,5; -6,5)$

được giảm khi robot đang quay hoặc chưa quan sát ổn định mắt cá chân, tốc độ lùi tăng khi người ở quá gần, và thành phần hiệu chỉnh hướng nhỏ vẫn vượt được vùng không đáp ứng của động cơ khi robot đang tịnh tiến. Nhóm kiểm thử này phản ánh thay đổi từ các lệnh rời rạc sang vector chuyển động biến thiên theo khoảng cách.

5.4.2 Hậu kiểm té ngã và hình học tư thế

`FallVerificationEngineTest` gồm sáu trường hợp kiểm tra trực tiếp chuỗi trạng thái của bộ hậu kiểm. Một kết quả FALL đơn lẻ không được mở sự cố; chỉ năm kết quả FALL liên tiếp mới tạo ứng viên, và một kết quả NO FALL xen giữa phải xóa bộ đếm ứng viên. Sau khi chuỗi FALL kết thúc, ba khung hình có điểm thân trên trong 70% phía trên ảnh làm ứng viên bị loại; ba khung hình có điểm thân trên thấp hơn ngưỡng hoặc 12 khung hình liên tiếp không còn bằng chứng quan sát sẽ dẫn đến yêu cầu xác nhận.

Bảng 5.6: Các trường hợp kiểm thử hậu kiểm té ngã

Trường hợp	Chuỗi đầu vào	Kết quả mong đợi
Nhiều FALL đơn lẻ	Một kết quả FALL	Chưa mở sự cố
Chuỗi FALL ổn định	Năm FALL liên tiếp, sau đó ba khung hình thân trên ở cao	Hình thành ứng viên rồi loại sau hậu kiểm
Chuỗi ứng viên bị gián đoạn	Bốn FALL, một NO FALL, sau đó bốn FALL	Bộ đếm được đặt lại, chưa mở sự cố
Thân trên còn ở vùng cao	Ba khung hình có $y/H \leq 0,70$ sau chuỗi FALL	Loại ứng viên
Thân trên ở vùng thấp	Ba khung hình có $y/H > 0,70$ sau chuỗi FALL	Yêu cầu xác nhận
Mất mục tiêu sau chuỗi FALL	12 khung hình không có bằng chứng thân trên	Yêu cầu xác nhận

`PostureGeometryTest` có bốn trường hợp được xây dựng từ các mẫu tư thế đứng, nằm, đang đứng dậy và thiếu keypoint thân dưới. Mục đích của nhóm này là kiểm tra riêng cách nhận biết tư thế nằm bị co ngắn theo phối cảnh camera. Kết quả chỉ được

dùng để lựa chọn hành vi chuyển động an toàn, không thay thế nhãn FALL/NO FALL của LSTM.

5.4.3 Phân vùng đường dẫn Firebase

Hai ứng dụng đều có `FirebasePairPathsTest` với cùng hai nguyên tắc kiểm tra. Các UID khác nhau phải tạo ra những đường dẫn điều khiển, cuộc gọi và cảnh báo khác nhau; toàn bộ kênh vận hành của một cặp thiết bị phải nằm dưới `pair_sessions/{uid}`. Việc duy trì cùng bộ kiểm thử ở RobotApp và CaregiverApp giúp phát hiện sớm sai lệch đường dẫn khi một trong hai ứng dụng thay đổi cấu trúc dữ liệu.

5.5 Đánh giá tính nhất quán khi tích hợp bằng kiểm tra tĩnh

5.5.1 Nhất quán Firebase

Đối chiếu hai ứng dụng cho thấy các đường dẫn vận hành được tạo thống nhất bởi `FirebasePairPaths`: `robot_control`, `robot_follow_enabled`, `ai_fall_detection_enabled`, `webrtc_signal`, `fall_alerts` và `call_history` đều là nhánh con của `pair_sessions/{uid}`. Hồ sơ, lịch và nhật ký lời nhắc lần lượt nằm dưới `Users/{uid}`, `Schedules/{uid}` và `reminder_logs/{uid}`. Offer/answer dùng cùng các trường `sdp`, `type`; ICE dùng `candidate`, `sdpMid`, `sdpMLineIndex` và được tách nhánh theo vai trò.

Security Rules từ chối truy cập ở cấp gốc và chỉ mở từng nhánh khi UID xác thực trùng với UID trên đường dẫn. Cách tổ chức hiện tại vì vậy đã cách ly dữ liệu giữa các tài khoản. Giới hạn còn lại là RobotApp và CaregiverApp trong một cặp dùng chung tài khoản, nên hệ thống chưa phân biệt quyền của hai thiết bị hoặc biểu diễn quan hệ nhiều người chăm sóc–nhiều robot.

5.5.2 Nhất quán BLE

Tên thiết bị, UUID của service và characteristic trong `RobotControlManager` trùng với firmware. RobotApp tạo chuỗi gồm giá trị bánh trái đứng trước bánh phải, sử dụng dấu chấm thập phân và kết thúc bằng ký tự xuống dòng; firmware loại khoảng trắng rồi tách hai giá trị tại ký tự L và R. RobotApp giới hạn vận tốc trong $[-20, 20]$ rad/s, hẹp hơn giới hạn $[-40, 40]$ rad/s của firmware. Characteristic ở hai phía đều sử dụng chế độ ghi không phản hồi.

5.5.3 Chuỗi timeout

Ba mốc thời gian tạo thành các lớp bảo vệ kế tiếp: RobotApp gửi lệnh dừng sau 500 ms không nhận cập nhật từ RTDB, firmware đưa vận tốc đặt về 0 sau 800 ms không nhận payload BLE, còn lệnh RTDB có tuổi lớn hơn 2.000 ms trên mốc thời gian đã đồng bộ bị loại khi ứng dụng nhận dữ liệu. Với joystick cập nhật khoảng 20 Hz, chu kỳ lệnh danh định là 50 ms, nhỏ hơn đáng kể so với hai mốc dừng. RobotApp vì vậy chủ động dừng trước, trong khi firmware đóng vai trò bảo vệ cuối cùng nếu ứng dụng không thể gửi lệnh dừng hoặc liên kết BLE bị gián đoạn.

5.5.4 Quyền sở hữu tài nguyên

Mã nguồn chỉ định tracking fragment là owner của RobotControlManager. Luồng cuộc gọi chia sẻ local WebRTC frame về AI và unbind CameraX. Đây là kết quả kiến trúc quan trọng vì xung đột camera/GATT thường không xuất hiện trong unit test. Kiểm thử vòng đời trên thiết bị vẫn cần bao phủ gọi đến khi app nền, từ chối, mất mạng, xoay màn hình và kết thúc bất thường.

5.6 Kịch bản kiểm thử chức năng đầu cuối

Bảng 5.7: Danh mục kiểm thử đầu cuối

Mã	Kịch bản	Thao tác/kích thích	Kết quả mong đợi
E01	Đăng nhập dùng chung	Đăng nhập cùng tài khoản hai app	Cùng UID/hồ sơ, không lộ user khác
E02	Tạo lịch nhắc nhở	Tạo lịch từ caregiver, chờ thời điểm	Robot phát TTS, log xuất hiện
E03	Hội thoại bình thường	Nói câu tiếng Việt khi robot lắng nghe	STT–LLM–TTS hoàn thành
E04	Bật auto-follow	Chỉ một người đứng trước robot	ID0 ổn định, robot bám trong giới hạn
E05	Người thứ hai đi qua	Người nhiều cắt ngang mục tiêu	ID0 không đổi sai
E06	Mất mục tiêu ngắn	ID0 ra khỏi ảnh rồi trở lại	Khôi phục trong điều kiện cho phép

Mã	Kịch bản	Thao tác/kích thích	Kết quả mong đợi
E07	Nằm chủ động	Người từ từ nằm nhưng còn rõ/an toàn	Không cảnh báo sai hoặc xác nhận an toàn
E08	Té ngã dựng lại	Thực hiện với đệm và người giám sát	Robot dừng, hỏi, tạo một alert
E09	Phản hồi an toàn	Trả lời “tôi ổn” rõ ràng	Cập nhật safe, không gọi khẩn cấp
E10	Không phản hồi	Im lặng quá cửa sổ nghe	Cập nhật danger/no-response, gọi caregiver
E11	Video call	Gọi giữa hai mạng khác nhau	Offer/answer/ICE, audio/video kết nối
E12	AI trong cuộc gọi	Bật tracking khi call	Một camera, frame chia sẻ, không crash
E13	Joystick	Kéo bốn hướng rồi thả	Đúng dấu; thả gửi STOP
E14	Mất Internet	Ngắt mạng khi joystick đang kéo	Không nhận lệnh mới; watchdog dừng
E15	Mất BLE	Tắt Bluetooth/ngắt nguồn tablet	Callback/watchdog firmware dừng
E16	Lệnh cũ/sai mốc thời gian	Ghi timestamp đã quy đổi cũ hơn 2 s hoặc tạm thời chưa có thông tin đồng bộ thời gian	RobotApp bỏ lệnh và dừng
E17	Input lỗi	NaN/vượt miền/thiếu timestamp	Không chuyển động

Mã	Kịch bản	Thao tác/kích thích	Kết quả mong đợi
E18	Cuộc gọi khẩn nên	Caregiver app ở nền, robot gọi fall	Notification/full-screen nêu đúng lý do

Ảnh chụp màn hình của các kịch bản đã thực hiện được trình bày ở Chương 4. Kịch bản E01 và E02 tương ứng với Hình 4.3, Hình 4.8 và Hình 4.10; E04–E06 tương ứng với Hình 4.4; E07–E08 tương ứng với Hình 4.5 và mục cảnh báo ở Hình 4.11; E11–E13 tương ứng với Hình 4.7, Hình 4.13 và Hình 4.14; E18 tương ứng với Hình 4.12. Các ảnh này là bằng chứng tích hợp ở mức giao diện, cho thấy kịch bản đã chạy được trên thiết bị thật, nhưng chưa thay thế được log định lượng về độ trễ và độ chính xác.

Bảng là bộ kiểm thử hồi quy đề xuất. Các kịch bản phần cứng phải bắt đầu với bánh xe nhấc khỏi mặt đất, tốc độ thấp, vùng trống và nút ngắt nguồn sẵn sàng. Không cho người cao tuổi thật thực hiện té ngã; chỉ dùng diễn viên khỏe mạnh, đệm bảo vệ và giám sát.

5.7 Thiết kế phép đo hiệu năng

5.7.1 Độ trễ pipeline AI

Mỗi frame cần gắn timestamp t_0 khi nhận, t_1 sau preprocess, t_2 sau interpreter, t_3 sau decode/NMS, t_4 sau tracking/LSTM và t_5 khi UI/lệnh được cập nhật. Báo cáo median, P90, P95 thay vì chỉ trung bình. FPS hiệu dụng phải tính frame thật được xử lý; drop-frame count phản ánh quá tải.

$$T_{AI} = (t_1 - t_0) + (t_2 - t_1) + (t_3 - t_2) + (t_4 - t_3). \quad (5.4)$$

Nhiệt và pin ảnh hưởng mạnh sau vài phút. Do đó benchmark cần warm-up 30–60 s, chạy ít nhất 10 phút cho mỗi CPU/GPU/FP16, ghi model thiết bị, Android version, delegate, thread count và độ phân giải camera.

5.7.2 Độ trễ điều khiển

CaregiverApp gắn thời điểm tạo t_c đã quy đổi theo mốc thời gian Firebase vào lệnh; RobotApp ghi nhận t_r khi bộ lắng nghe nhận dữ liệu và t_b khi gửi qua BLE; ESP32 ghi thời điểm callback tiếp nhận lệnh. Hai ứng dụng phải lưu cả độ lệch thời gian do

Firestore cung cấp tại thời điểm đo để có thể đối chiếu. Độ trễ đầu cuối cần được báo cáo cùng tỷ lệ mất lệnh, loại kết nối Internet và khoảng cách BLE; phép đo bằng video tốc độ cao có thể được dùng để kiểm chứng độc lập.

5.7.3 WebRTC

Thu thập `RTCStatsReport` định kỳ: `candidate pair`, `currentRoundTripTime`, `packetsLost`, `jitter`, `framesEncoded/Decoded`, `framesDropped` và `availableOutgoingBitrate`. Thử cùng Wi-Fi, Wi-Fi-4G và hai mạng NAT khó; ghi rõ có dùng TURN. Thời gian setup tính từ `calling` đến `connected`.

5.7.4 Độ chính xác té ngã

Tập kiểm thử phải độc lập theo chủ thể/video, bao gồm đi, ngồi, nằm, cúi nhặt đồ, khuất người, ra khỏi ảnh, té nhiều hướng và nhiều điều kiện sáng. So sánh ít nhất ba cấu hình: `pose/luật một frame`; `pose+LSTM`; `pose+LSTM+hậu kiểm`. Ablation này đo được đóng góp của mỗi tầng thay vì chỉ nêu pipeline cuối.

5.8 Kết quả Benchmark Mô hình AI

Mặc dù việc đánh giá tổng thể hệ thống đòi hỏi nhiều yếu tố thực tế, các chỉ số đánh giá (benchmark) độc lập của mô hình AI đóng vai trò cốt lõi để khẳng định tính khả thi của giải pháp. Phân hệ AI được đánh giá trên hai khía cạnh: Độ chính xác (Accuracy/Precision/Recall) và Tốc độ suy luận (Inference Speed) trên thiết bị di động biên.

5.8.1 Dữ liệu, quy ước gán nhãn và giao thức đánh giá

Bộ dữ liệu LE2I gồm 190 video chia theo sáu bối cảnh. Tuy nhiên chỉ 130 video trong số đó có tệp chú thích gốc đi kèm; toàn bộ 27 video của bối cảnh *Lecture room* và 33 video của bối cảnh *Office* thiếu thư mục `Annotation_files`, dù tài liệu của bộ dữ liệu nêu rằng mọi bối cảnh đều phải có.

Sáu mươi video thiếu chú thích này *không* được xem là “không ngã”. Kiểm tra bằng tay cho thấy nhiều video trong đó chứa pha ngã rõ rệt: người nằm trên sàn, ghé bị hất đổ, và tẩm nệm đặt sẵn trong khung hình đóng vai trò đệm bảo hộ cho diễn viên. Một phép đo định lượng xác nhận nhận định này. Với các video *đã* có chú thích, chỉ 3% số video mang nhãn “không ngã” xuất hiện đoạn liên tục từ 1,0 s trở lên có tỷ lệ hình

dạng vượt 1,5, tức thân người nằm ngang, trong khi con số này ở nhóm có ngã là 54%. Ở nhóm 60 video thiếu chú thích, tỷ lệ là 27%, tức cao gấp chín lần nhóm không ngã đã xác minh. Ước tính có khoảng 16 video chứa pha ngã trong số này.

Vì không thể suy ra `fall_start` cho những video đó, toàn bộ 60 video được loại khỏi tập dữ liệu. Nguyên tắc áp dụng ở đây là tệp chú thích tồn tại nhưng thiếu hai dòng đầu có nghĩa là không ngã, còn tệp không tồn tại có nghĩa là chưa được chú thích và không suy ra được nhân.

Tập dữ liệu cuối cùng gồm 130 video với nhãn đã xác minh, tương ứng 42.066 khung hình. Sau khi tạo cửa sổ trượt 15 khung hình với bước trượt 1, tập dữ liệu có 40.246 mẫu. Trong 130 video, 96 video chứa pha ngã và 34 video không chứa.

Quy ước gán nhãn. Chú thích gốc của LE2I chỉ đánh dấu khoảng thời gian rơi, có độ dài trung vị 24 khung hình tương đương khoảng 1,0 s. Sau thời điểm `fall_end`, mỗi video còn trung vị 76 khung hình, tương đương 3,1 s, trong đó người vẫn nằm trên sàn nhưng mang nhãn bình thường. Quy ước này không phù hợp với bài toán giám sát người cao tuổi, vì trạng thái cần phát hiện là “có người đang nằm trên sàn” chứ không riêng khoảnh khắc rơi. Một mô hình huấn luyện theo chú thích gốc bị phạt đúng vào lúc nó nhận ra người đã nằm xuống.

Phân tích các cửa sổ bị phân loại sai của một mô hình huấn luyện theo chú thích gốc xác nhận nhận định trên. Trong 615 cửa sổ báo động giả thu được từ ba lần chạy, 289 cửa sổ chiếm 47,0% nằm sau `fall_end` và 120 cửa sổ chiếm 19,5% nằm ngay trước `fall_start`. Tổng cộng 66,5% số trường hợp không phải lỗi suy luận mà là hệ quả của ranh giới chú thích: ở những khung hình đó người vẫn đang nằm trên sàn hoặc đang trong pha mất thăng bằng, nhưng chú thích gốc đã gán nhãn bình thường. Chỉ 206 cửa sổ chiếm 33,5% thuộc video không chứa pha ngã, tức là lỗi thực sự.

Vì vậy, hệ thống sử dụng quy ước gán nhãn theo trạng thái: mọi khung hình từ `fall_start` đến hết video mang nhãn dương. Lớp dương biểu diễn trạng thái người nằm trên sàn, chiếm 24,37% số cửa sổ thay vì 5,43% như chú thích gốc. Mức mất cân bằng giảm đủ để không cần dùng trọng số lớp khi huấn luyện; kết luận này được kiểm chứng ở Bảng 5.12.

Giao thức đánh giá. Vì bước trượt bằng 1, hai cửa sổ liên tiếp dùng chung 14 trên 15 khung hình. Chia ngẫu nhiên ở mức cửa sổ sẽ đưa các cửa sổ gần như trùng nhau vào cả

tập huấn luyện lẫn tập đánh giá và không tạo ra tập kiểm thử độc lập. Do đó 130 video được chia theo video thành 70% huấn luyện, 15% kiểm định và 15% kiểm thử, một video chỉ thuộc đúng một tập. Ngưỡng quyết định được chọn trên tập kiểm định; tập kiểm thử chỉ dùng để báo cáo.

Kết quả được trình bày ở hai mức. *Mức cửa sổ* đo trực tiếp đầu ra của mô hình. *Mức sự kiện* áp dụng quy tắc năm kết quả dương liên tiếp như mô tả ở Chương 4, quy mỗi video về một quyết định nhị phân và đối chiếu với việc video có chứa pha ngã hay không; đây mới là đơn vị đánh giá phản ánh hành vi mà người dùng cảm nhận.

5.8.2 Quy trình lựa chọn cấu hình mô hình

Cấu hình của khối LSTM được xác định bằng một quy trình lựa chọn nối tiếp gồm sáu giai đoạn. Mỗi giai đoạn chỉ thay đổi một nhóm tham số và kế thừa toàn bộ lựa chọn đã chốt ở giai đoạn trước, nhờ đó mỗi bảng đo đúng một yếu tố và các bảng liên kết được với nhau thành một chuỗi. Mọi thí nghiệm dùng chung cách chia tập theo video đã mô tả ở trên, chạy với ba hạt giống ngẫu nhiên 42, 7 và 2024, và chọn ngưỡng quyết định trên tập kiểm định. Các giá trị báo cáo là trung bình kèm độ lệch chuẩn của ba lần chạy.

Toàn bộ các bảng trong mục này báo cáo chỉ số của riêng lớp Fall ở mức cửa sổ, vì đây là đơn vị đầu ra trực tiếp của mô hình và cũng là tiêu chí dùng để lựa chọn ở cả sáu giai đoạn. Chỉ số của lớp No-fall không được đưa vào: lớp này chiếm 75,63% số cửa sổ nên các giá trị của nó luôn nằm quanh 0,95 bất kể chất lượng mô hình, và việc gộp trung bình hai lớp sẽ che mất đúng phần khác biệt cần quan sát. Chỉ số đầy đủ của cả hai lớp được trình bày ở Bảng 5.14 sau khi cấu hình đã chốt, còn kết quả ở mức hệ thống được trình bày ở Bảng 5.15.

Cần lưu ý ngay rằng độ lệch chuẩn giữa ba lần chạy nằm trong khoảng 0,008–0,055, xuất phát từ việc mỗi tập kiểm thử chỉ có khoảng 20 video. Vì vậy chỉ những chênh lệch vượt rõ khoảng này mới được xem là có ý nghĩa. Ở những chỗ hai cấu hình gần nhau, bài báo cáo dùng thêm phép so sánh bắt cặp theo hạt giống: vì cả sáu giai đoạn dùng chung ba cách chia tập, chênh lệch được tính trên từng cặp cùng hạt giống rồi lấy trung bình, cách này loại bỏ được phần dao động do chia tập và nhạy hơn so với việc chỉ đối chiếu độ lệch chuẩn.

Kiến trúc mạng. Giai đoạn đầu so sánh sáu kiến trúc hồi quy trên cùng một cấu hình dữ liệu, với cửa sổ 15 khung hình và đặc trưng chuẩn hóa theo bounding box.

Bảng 5.8: So sánh kiến trúc mạng hồi quy, cửa sổ 15 khung hình (chỉ số lớp Fall ở mức cửa sổ)

Mô hình	Tham số	Precision	Recall	F1
LSTM32	13.250	0,776	0,932	0,845 ± 0,032
GRU64	26.306	0,793	0,897	0,842 ± 0,034
LSTM64	34.690	0,787	0,937	0,854 ± 0,029
LSTM64×2	67.714	0,791	0,851	0,820 ± 0,021
BiLSTM64	69.378	0,799	0,943	0,864 ± 0,027
LSTM128	102.146	0,795	0,928	0,855 ± 0,022

Sáu kiến trúc trong Bảng 5.8 nằm trong dải hẹp từ 0,820 đến 0,864, trong khi độ lệch chuẩn của từng dòng là 0,021–0,034. So sánh bắt cặp với LSTM32 cho thấy không kiến trúc nào vượt trội có ý nghĩa: BiLSTM64 hơn 0,019 với thống kê t bằng 3,19 và LSTM64 hơn 0,009 với t bằng 2,07, cả hai đều dưới ngưỡng 4,30 ứng với mức tin cậy 95% khi chỉ có hai bậc tự do. Tăng dung lượng mô hình gấp 7,7 lần, từ 13.250 lên 102.146 tham số, không tạo ra cải thiện xác thực được; bổ sung lớp thứ hai thậm chí làm kết quả kém đi.

LSTM32 được chọn vì đạt hiệu năng không phân biệt được với các kiến trúc lớn hơn trong khi chỉ dùng 13% số tham số của LSTM128 và 19% của BiLSTM64. Với một mô hình phải chạy liên tục trên tablet song song với YOLO-Pose, đây là đánh đổi rõ ràng có lợi.

Độ dài cửa sổ trượt. Giai đoạn thứ hai giữ nguyên LSTM32 và thay đổi số khung hình trong mỗi cửa sổ.

Bảng 5.9: Ảnh hưởng của độ dài cửa sổ trượt, mô hình LSTM32 (chỉ số lớp Fall ở mức cửa sổ)

Cửa sổ	Độ trễ tối thiểu	Precision	Recall	F1
10 khung	0,40 s	0,758	0,871	0,806 ± 0,038
15 khung	0,60 s	0,776	0,932	0,845 ± 0,032
20 khung	0,80 s	0,793	0,921	0,850 ± 0,034
30 khung	1,20 s	0,804	0,914	0,852 ± 0,008
45 khung	1,80 s	0,805	0,885	0,840 ± 0,044

Cửa sổ 10 khung hình là mức duy nhất kém rõ rệt, thấp hơn 15 khung hình 0,039: 0,4 giây không đủ để bao trọn một động tác ngã. Từ 15 khung hình trở lên, bốn mức còn lại nằm trong dải 0,840–0,852 và so sánh bắt cặp với cửa sổ 15 khung hình cho t lớn nhất chỉ bằng 1,17, tức không mức nào khác biệt có ý nghĩa.

Kéo dài cửa sổ vì vậy không mang lại gì, trong khi chi phí thì có thật: cửa sổ 45 khung hình buộc hệ thống tích lũy 1,8 s dữ liệu trước khi đưa ra dự đoán đầu tiên cho

một định danh mới, và làm tăng gấp ba bộ đệm keypoint mà `FallDetectionQueue` phải giữ cho mỗi đối tượng đang theo dõi. Cửa sổ 15 khung hình tương ứng 0,6 s được giữ lại vì đó là mức ngắn nhất đã đạt hiệu năng bão hòa.

Biểu diễn đặc trưng. Giai đoạn thứ ba so sánh bảy cách biểu diễn đầu vào trên cùng LSTM32 và cửa sổ 15 khung hình.

Bảng 5.10: So sánh cách biểu diễn đặc trưng đầu vào, LSTM32 và cửa sổ 15 khung hình (chỉ số lớp *Fall* ở mức cửa sổ)

Biểu diễn	Chiều	Tham số	Precision	Recall	F1
Chuẩn hóa theo hộp bao	69	13.250	0,776	0,932	$0,845 \pm 0,032$
Chuẩn hóa theo khung ảnh	69	13.250	0,794	0,977	$0,875 \pm 0,034$
Tọa độ gốc	69	13.250	0,793	0,977	$0,874 \pm 0,038$
Bỏ thành phần vận tốc	34	8.770	0,787	0,925	$0,850 \pm 0,022$
Tọa độ và tỷ lệ hình dạng	35	8.898	0,776	0,974	$0,863 \pm 0,020$
Đầu ra YOLO nguyên dạng	51	10.946	0,742	0,911	$0,818 \pm 0,049$
Bổ sung độ tin cậy khớp	86	15.426	0,789	0,960	$0,865 \pm 0,033$

Điểm chung của hai cách biểu diễn đứng đầu là cùng giữ lại thông tin về chiều cao biểu kiến của người trong khung hình. Chuẩn hóa theo hộp bao lấy tọa độ trừ tâm hông rồi chia cho chiều rộng và chiều cao của hộp bao, nên theo định nghĩa, biên độ tọa độ dọc của bộ xương sau biến đổi luôn bằng 1 bất kể người đang đứng hay đang nằm. Phép chia này xóa mất đúng tín hiệu phân biệt mạnh nhất của bài toán. Đo trên một video của *Coffee_room_01*, ở khung hình người còn đứng biên độ tọa độ dọc của các khớp quan sát được là 1,54 đơn vị, còn ở khung hình người đã nằm trên sàn chỉ còn 0,43 đơn vị, tức bằng 28%; sau khi chuẩn hóa theo hộp bao, cả hai đều bằng 1,00. Hệ quả thể hiện rõ ở cột recall: 0,932 với hộp bao so với 0,977 của hai cách còn lại.

Tuy vậy mức chênh lệch không đủ để khẳng định. Chuẩn hóa theo khung ảnh và tọa độ gốc lần lượt hơn hộp bao 0,030 và 0,029, với thống kê t bắt cặp là 1,49 và 1,71, đều dưới ngưỡng 4,30. Xu hướng nhất quán ở cả ba hạt giống nên nhiều khả năng là thật, nhưng ba lần chạy chưa đủ để chứng minh.

Giữa hai phương án dẫn đầu, chênh lệch chỉ là 0,001 với t bằng $-0,18$, tức không phân biệt được. Trong trường hợp đó, chuẩn hóa theo khung ảnh là lựa chọn nên ưu tiên: nó chia tọa độ cho kích thước khung ảnh 320×240 nên độc lập với vị trí và độ phân giải của camera, trong khi tọa độ gốc giữ nguyên tọa độ pixel và vì thế gắn mô hình vào đúng hình học camera của tập LE2I. Đây là một hạn chế được nêu lại ở phần đánh giá tính hợp lệ, và có thể tránh được mà không mất gì về hiệu năng.

Hai thí nghiệm còn lại làm rõ đóng góp của từng nhóm đặc trưng. Bỏ hoàn toàn 34 chiều vận tốc làm F1 giảm 0,024, nhưng thêm lại duy nhất một chiều tỷ lệ hình dạng đã thu về 0,013 trong số đó; nói cách khác, một đại lượng vô hướng gánh hơn nửa đóng góp của toàn bộ thông tin chuyển động. Cấu hình 35 chiều này chỉ kém phương án đầy đủ 0,011 nhưng dùng ít hơn 33% tham số và có độ lệch chuẩn nhỏ hơn, nên là ứng viên đáng cân nhắc nếu cần thu gọn thêm mô hình. Ngược lại, dùng trực tiếp đầu ra YOLO gồm tọa độ và độ tin cậy từng khớp cho kết quả thấp nhất bằng, 0,818: độ tin cậy phản ánh chất lượng phát hiện chứ không phản ánh tư thế, nên nó thêm nhiễu mà không thêm tín hiệu.

Khối lượng dữ liệu huấn luyện. Để xác định xem hiệu năng hiện tại đã bão hòa theo lượng dữ liệu hay chưa, mô hình được huấn luyện lại với các tỷ lệ video khác nhau, giữ nguyên tập kiểm định và kiểm thử.

Bảng 5.11: Ảnh hưởng của khối lượng dữ liệu huấn luyện (chỉ số lớp Fall ở mức cửa sổ)

Tỷ lệ video huấn luyện	Precision	Recall	F1
25% (khoảng 23 video)	0,699	0,934	$0,800 \pm 0,055$
50% (khoảng 46 video)	0,737	0,969	$0,835 \pm 0,039$
75% (khoảng 68 video)	0,763	0,938	$0,841 \pm 0,055$
100% (91 video)	0,793	0,977	$0,874 \pm 0,038$

F1 tăng đơn điệu theo khối lượng dữ liệu, từ 0,800 với một phần tư số video lên 0,874 khi dùng toàn bộ. Mức tăng tổng cộng 0,074 vượt rõ độ lệch chuẩn của từng dòng, và phần lớn đến từ precision, từ 0,699 lên 0,793, trong khi recall đã cao ngay từ đầu.

Đường cong chưa có dấu hiệu bão hòa: bước nhảy từ 75% lên 100% vẫn còn 0,033, lớn hơn bước từ 50% lên 75%. Kết hợp với Bảng 5.8, hai kết quả này chỉ ra hai hướng khác nhau. Dung lượng mô hình đã bão hòa, nên tăng số tham số không giúp gì; nhưng lượng dữ liệu thì chưa, nên bổ sung video huấn luyện vẫn là hướng cải thiện có hiệu lực và nên được ưu tiên trước khi thử các kiến trúc phức tạp hơn. Điều này đặc biệt đáng lưu ý vì tập dữ liệu hiện chỉ còn 130 video sau khi loại các video thiếu chú thích, tức nhỏ hơn đáng kể so với quy mô ban đầu của bộ LE2I.

Trọng số lớp trong hàm mất mát. Vì lớp dương chiếm 24,37% số cửa sổ, tức vẫn ít hơn lớp âm khoảng ba lần, giai đoạn thứ năm thử tăng trọng số của lớp này trong hàm mất mát.

F1 giảm đơn điệu theo trọng số: 0,874 rồi 0,867, 0,864 và 0,861. Mỗi lần nhân

Bảng 5.12: Ảnh hưởng của trọng số lớp dương trong hàm mất mát (chỉ số lớp Fall ở mức cửa sổ)

Trọng số lớp dương	Ngưỡng	Precision	Recall	F1
$\times 1$	0,24	0,793	0,977	0,874 $\pm$ 0,038
$\times 2$	0,16	0,775	0,985	0,867 $\pm$ 0,019
$\times 4$	0,30	0,774	0,980	0,864 $\pm$ 0,038
$\times 8$	0,42	0,766	0,985	0,861 $\pm$ 0,032

đôi trọng số lại mất thêm một phần nhỏ, và cấu hình không dùng trọng số cho kết quả tốt nhất. Kết quả này xác nhận nhận định ở phần quy ước gán nhãn rằng mức mất cân bằng khoảng ba lần không đủ lớn để cần can thiệp.

Cơ chế của sự suy giảm thể hiện qua hai cột còn lại. Tăng trọng số đẩy recall lên sát trần, từ 0,977 tới 0,985, nhưng precision giảm tương ứng từ 0,793 xuống 0,766; phần thu được ở recall nhỏ hơn phần mất ở precision vì recall vốn đã gần bão hòa. Đồng thời ngưỡng quyết định dò được trên tập kiểm định tăng dần theo trọng số, cho thấy trọng số chỉ dịch chuyển thang xác suất đầu ra chứ không cải thiện khả năng phân biệt giữa hai lớp. Vì phép dò ngưỡng đã tự điều chỉnh được điểm cân bằng giữa precision và recall, việc can thiệp thêm bằng trọng số là thừa.

Số vòng huấn luyện. Giai đoạn cuối kiểm tra số vòng huấn luyện trên cấu hình đã chốt.

Bảng 5.13: Ảnh hưởng của số vòng huấn luyện trên cấu hình đã chọn (F1 lớp Fall ở mức cửa sổ)

Cấu hình	F1
5 vòng, cố định	0,857 $\pm$ 0,028
10 vòng, cố định	0,857 $\pm$ 0,033
30 vòng, cố định	0,874 $\pm$ 0,038
Tối đa 60 vòng, dừng sớm	0,873 $\pm$ 0,041

Ba dòng đầu của Bảng 5.13 huấn luyện đủ số vòng đã đặt rồi lấy trọng số cuối cùng. Dòng thứ tư dùng cơ chế dừng sớm với patience bằng 10 và `restore_best_weights`: quá trình huấn luyện dừng lại khi mất mát kiểm định không cải thiện suốt mười vòng liên tiếp, sau đó khôi phục lại trọng số của vòng tốt nhất. Giá trị 60 ở đây chỉ là trần chứ không phải số vòng thực tế; ba lần chạy lần lượt dừng ở vòng 30, 22 và 16.

Kết quả cho thấy huấn luyện dưới 10 vòng là chưa đủ: hai cấu hình đầu dừng lại ở 0,857 trong khi mất mát kiểm định vẫn còn đang giảm, với vòng tốt nhất rơi đúng vào

vòng cuối hoặc sát vòng cuối. Từ 30 vòng trở lên, F1 đạt 0,874 và cơ chế dừng sớm cho kết quả gần như trùng khít, 0,873, với chênh lệch 0,001 nằm sâu trong sai số giữa ba lần chạy. Nói cách khác, mô hình hội tụ trong khoảng từ vòng 18 đến vòng 24 và huấn luyện thêm không làm hỏng kết quả.

Vì hai cấu hình cho kết quả tương đương, hệ thống giữ 30 vòng cố định để quy trình huấn luyện đơn giản và tái lập được, đồng thời tránh phụ thuộc vào một tham số *patience* phải hiệu chỉnh riêng. Cần lưu ý rằng kết luận này khác với kết quả đo trên tập dữ liệu chưa được làm sạch nhãn, nơi dừng sớm làm F1 giảm rõ rệt; khi nhãn còn nhiều, mất mát kiểm định là tín hiệu không đáng tin để quyết định thời điểm dừng.

Ngưỡng quyết định vẫn dao động mạnh giữa các lần chia tập, lần lượt là 0,385, 0,050 và 0,285 ở cấu hình đã chọn. Vì vậy ngưỡng phải được hiểu là một giá trị dò được trên dữ liệu kiểm định của chính lần chia tập đó, không phải mức độ tin cậy của dự đoán và cũng không chuyển giao được sang môi trường lắp đặt khác.

Cấu hình được chọn. Kết quả của sáu giai đoạn trên là cấu hình gồm một lớp LSTM 32 đơn vị với 13.250 tham số, cửa sổ trượt 15 khung hình tương ứng 0,6 s, đặc trưng 69 chiều xây dựng từ tọa độ của 17 điểm khớp cùng thành phần vận tốc và tỷ lệ hình dạng, huấn luyện 30 vòng không dùng trọng số lớp và không dùng sớm. Cấu hình này đạt F1 lớp Fall bằng $0,874 \pm 0,038$ ở mức cửa sổ.

Điều đáng ghi nhận là *không giai đoạn nào* trong sáu giai đoạn tạo ra cải thiện vượt ngưỡng ý nghĩa thống kê. Kiến trúc, độ dài cửa sổ, cách biểu diễn đặc trưng, trọng số lớp và số vòng huấn luyện đều chỉ dịch chuyển F1 trong dải hẹp quanh 0,85. Để so sánh, riêng việc sửa hai lỗi chú thích mô tả ở Mục 5.8.1 đã nâng F1 từ 0,650 lên 0,845 trên cùng cấu hình xuất phát, tức gấp nhiều lần tổng mức cải thiện thu được từ toàn bộ quá trình tinh chỉnh mô hình. Đây là kết luận thực tiễn quan trọng nhất của mục này: với một bộ dữ liệu nhỏ và được chú thích thủ công, công sức bỏ vào việc kiểm chứng nhãn có hiệu quả cao hơn hẳn công sức bỏ vào việc dò tìm siêu tham số.

5.8.3 Độ chính xác của mô hình

Bảng 5.14 trình bày chỉ số của từng lớp ở mức cửa sổ, trung bình ba lần chạy với ba cách chia tập khác nhau.

Hai dòng của Bảng 5.14 phải được đọc khác nhau. Lớp bình thường chiếm 75,63% số cửa sổ, nên các chỉ số quanh 0,95 của lớp này phần lớn phản ánh tỷ lệ phân bố chứ

Bảng 5.14: Độ chính xác của cấu hình đã chọn ở mức cửa sổ, chia tập theo video, trung bình ba lần chạy

Lớp (Class)	Precision	Recall	F1-Score	Accuracy
Bình thường (No-fall)	$0,989 \pm 0,009$	$0,914 \pm 0,010$	$0,950 \pm 0,007$	$0,930 \pm 0,006$
Nằm trên sàn (Fall)	$0,793 \pm 0,071$	$0,977 \pm 0,015$	$0,874 \pm 0,038$	

không phản ánh chất lượng mô hình: một bộ phân loại luôn trả về “bình thường” đã đạt accuracy 75,63%. Vì lý do đó, accuracy tổng thể 0,930 cũng không phải chỉ số có ý nghĩa cho bài toán này. Thông tin thực sự nằm ở dòng thứ hai: recall 0,977 nghĩa là mô hình bỏ sót chưa tới ba trên một trăm cửa sổ thuộc giai đoạn người nằm trên sàn, còn precision 0,793 nghĩa là trong mười cửa sổ được gán nhãn nằm trên sàn có khoảng hai cửa sổ sai.

Sự chênh lệch giữa hai chỉ số này là có chủ ý. Với bài toán giám sát người cao tuổi, bỏ sót một cú ngã có hậu quả nghiêm trọng hơn nhiều so với một cảnh báo thừa, nên ngưỡng quyết định được chọn nghiêng về phía recall. Phần precision còn thiếu được bù lại ở tầng hậu kiểm trình bày ở cuối mục này.

Ảnh hưởng của quy tắc hậu kiểm. Kết quả ở mức cửa sổ mới chỉ mô tả đầu ra thô của mô hình. Ở mức hệ thống, một cảnh báo chỉ được phát khi có K cửa sổ dương liên tiếp, và giá trị K là tham số cấu hình chứ không phải hằng số. Bảng 5.15 đo ảnh hưởng của tham số này trên cùng ba lần chạy của cấu hình đã chọn.

Ba cột cuối cần được đọc riêng. *Báo giả mỗi giờ* đếm từng lần phát cảnh báo trên mỗi giờ video không chứa pha ngã, nên một video có thể đóng góp nhiều lần. *Độ trễ* là khoảng thời gian từ `fall_start` đến lúc cảnh báo nổ ra; giá trị âm nghĩa là cảnh báo phát ra *trước* khi cú ngã bắt đầu, dấu hiệu cho thấy hệ thống đang báo gần như liên tục chứ không phải phát hiện đúng thời điểm. *Bắt được* là số video có ngã mà hệ thống vẫn phát hiện, trên tổng 43 video của ba lần chạy.

Cần nêu rõ một giới hạn trước khi đọc bảng. Sau khi loại 60 video không có chú thích, tập dữ liệu chỉ còn 34 video không chứa pha ngã, nên mỗi tập kiểm thử chỉ có từ 3 đến 9 video âm. Precision và F1 ở mức sự kiện vì vậy được tính trên rất ít mẫu và chỉ nên dùng để so sánh tương đối giữa các giá trị K , không nên hiểu là ước lượng cho hiệu năng khi triển khai. Cột báo giả mỗi giờ không mang giới hạn này vì nó chuẩn hóa theo thời lượng video chứ không theo số video.

Ngưỡng 25 cửa sổ, tương ứng một giây liên tục, là điểm vận hành tốt nhất trong dải khảo sát: số báo động giả giảm còn một nửa, từ 121,4 xuống 62,2 lần mỗi giờ, trong khi recall vẫn giữ nguyên ở mức tuyệt đối. Cái giá phải trả là độ trễ cảnh báo tăng lên

Bảng 5.15: Ảnh hưởng của ngưỡng hậu kiểm theo số cửa sổ dương liên tiếp

Số cửa sổ	Thời lượng	P	R	F1	Báo giả/giờ	Độ trễ	Bắt được
1 (không hậu kiểm)	—	0,793	1,000	0,883	213,2	−0,84 s	43/43
5 (cấu hình hiện tại)	0,2 s	0,793	1,000	0,883	206,0	−0,66 s	43/43
25	1,0 s	0,823	1,000	0,902	181,6	+0,68 s	43/43
50	2,0 s	0,887	0,939	0,911	115,5	+1,77 s	40/43
75	3,0 s	0,866	0,775	0,817	115,5	+3,04 s	33/43

khoảng 1,0 s, mức hoàn toàn chấp nhận được với một tình huống mà người bệnh nằm trên sàn trong nhiều phút.

Vượt quá ngưỡng này, hệ thống bắt đầu đánh đổi bằng chính những cú ngã thật. Ở ngưỡng 50 cửa sổ, số báo động giả chỉ giảm thêm được 12,6 lần mỗi giờ nhưng recall rơi từ 1,000 xuống 0,806, tức sáu trên 34 video bị bỏ sót. Nguyên nhân là nhiều cú ngã trong LE2I kết thúc bằng tư thế mà mô hình chỉ nhận đúng trong một đoạn ngắn, không đủ dài để thỏa điều kiện liên tiếp; kéo dài điều kiện càng lâu thì càng nhiều trường hợp như vậy bị loại oan. Kết quả này cho thấy hậu kiểm theo thời lượng là công cụ hữu hiệu để lọc báo động giả nhưng có giới hạn rõ ràng, và cấu hình năm cửa sổ hiện tại nằm ở phía quá lỏng của dải khảo sát.

Cần nêu rõ ba giới hạn kèm các con số trên. Thứ nhất, mỗi lần chạy chỉ có khoảng 29 video kiểm thử, trong đó khoảng 14 video chứa pha ngã, nên khoảng tin cậy rất rộng; độ lệch chuẩn 0,088 của F1 ở mức sự kiện phản ánh điều này. Thứ hai, các chỉ số đo riêng khối LSTM trên keypoint trích sẵn, chưa bao gồm sai số của khối theo dõi và của bước hậu kiểm khi chạy trên thiết bị, nên là cận dưới cho phần AI chứ chưa phải hiệu năng đầu cuối. Thứ ba, các tình huống té ngã trong LE2I được dàn dựng bởi diễn viên khỏe mạnh và có thể khác đáng kể so với người cao tuổi trong sinh hoạt thực tế.

5.8.4 Kết quả đóng gói mô hình

Tập mô hình hiện được đóng gói trong RobotApp được tạo ra *trước* toàn bộ quy trình ở Mục 5.8.2 và trước khi hai lỗi chủ thích được phát hiện. Nó được huấn luyện trên 190 video với nhãn chưa làm sạch và dùng cách chuẩn hóa theo hộp bao. Các số liệu trong mục này mô tả đúng artifact đang tồn tại, không phải cấu hình khuyến nghị, và không so sánh trực tiếp được với các bảng ở trên vì khác cả tập dữ liệu lẫn nhãn. Việc huấn luyện và đóng gói lại được nêu ở phần hạn chế.

Tập này được huấn luyện trên 133 video của tập dữ liệu cũ, chọn ngưỡng trên 28 video kiểm định và đánh giá trên 29 video kiểm thử chưa từng xuất hiện. Ngưỡng chọn được là 0,985. Bảng 5.16 trình bày kết quả đo trực tiếp trên từng tập TFLite, chạy đủ

11.743 cửa sổ của tập kiểm thử qua interpreter thay vì đo trên mô hình Keras.

Bảng 5.16: Kết quả đo trên tập TFLite được đóng gói

Tệp	Dung lượng	P / R / F1	Sai lệch so với Keras
..._v3_ground_float.tflite	59,6 KB	0,838 / 0,615 / 0,709	tối đa $3,5 \times 10^{-6}$; trùng quyết định 100%
..._v3_ground.tflite (lượng tử hóa)	24,8 KB	0,836 / 0,621 / 0,712	tối đa 0,596; trùng quyết định 99,85%

Cả hai biến thể đạt kết quả giống nhau ở mức sự kiện: phát hiện 13/14 video có ngã và sinh 6 video báo động giả, tương ứng precision 0,684, recall 0,929 và F1 0,788. Giá trị này chỉ ứng với một lần chia tập cụ thể, một cách biểu diễn đặc trưng khác và tập dữ liệu trước khi làm sạch nhãn, nên không so sánh trực tiếp được với Bảng 5.15.

Biến thể float được chọn để đóng gói. Bản lượng tử hóa nhỏ hơn 35 KB nhưng có sai lệch xác suất tối đa 0,596 trên từng cửa sổ riêng lẻ, đủ để đảo quyết định ở khoảng 18 trên 11.743 cửa sổ khi so với ngưỡng 0,985; mức tiết kiệm dung lượng này không tương xứng với rủi ro đó trong một ứng dụng có kích thước như RobotApp.

Ngưỡng quyết định không chuyển giao được giữa các lần chia tập. Với tập đang đóng gói, ba lần chạy cho ba giá trị tối ưu lần lượt là 0,985, 0,360 và 0,120; với cấu hình đã chọn ở Mục 5.8.2, ba giá trị là 0,390, 0,425 và 0,135. Trong cả hai trường hợp, biên độ dao động đủ lớn để khẳng định ngưỡng chỉ là điểm khởi đầu và phải được hiệu chỉnh lại bằng dữ liệu thu tại chính môi trường lắp đặt, không phải hằng số cố định.

5.8.5 Benchmark Tốc độ suy luận (Inference Speed / FPS)

Để đáp ứng yêu cầu cảnh báo kịp thời, chuỗi xử lý từ khung hình, YOLO-Pose đến LSTM cần duy trì tốc độ tối thiểu khoảng 10–15 FPS trên thiết bị Android. Nguồn CameraX và WebRTC của RobotApp được cấu hình ở mức mục tiêu 25 FPS; chuỗi suy luận không bổ sung bộ giới hạn thời gian riêng mà xử lý theo các khung hình được nguồn camera cung cấp. Vì vậy, 25 FPS là tốc độ thu nhận mục tiêu, còn tốc độ AI thực tế vẫn phụ thuộc vào thời gian tiền xử lý, suy luận và hậu xử lý trên thiết bị. Bảng 5.17 ghi lại kết quả đo của biến thể FP16 trong giai đoạn chuyển đổi mô hình. Phiên bản RobotApp hiện tại đóng gói yolo26n-pose_320_gpu.tflite làm mô hình ưu tiên và yolo26n-pose_320.tflite làm phương án dự phòng; vì vậy số liệu này được dùng làm mốc tham khảo và cần được đo lại với đúng mô hình, bộ tăng tốc và thiết bị sử dụng khi nghiệm thu.

Bảng 5.17: Thời gian xử lý trung bình của cấu hình FP16 thử nghiệm trên Android

Thành phần AI	Tệp mô hình hoặc công cụ	Thời gian xử lý mỗi khung hình (ms)
Trích xuất khung hình	Android CameraX	~ 10 ms
YOLO-Pose (FP16)	yolo26n-pose_no_nms_float16.tflite (đầu vào 480 × 640)	~ 45 ms
LSTM (Sequence)	best_model_lstm_v2.tflite	~ 5 ms
Tổng thời gian xử lý	Toàn bộ chuỗi xử lý	~ 60 ms (tương đương ~ 16 FPS)

Kết quả trong Bảng 5.17 cho thấy cấu hình FP16 thử nghiệm đạt thời gian xử lý trung bình khoảng 60 ms cho mỗi khung hình, tương đương gần 16 FPS. Giá trị này cho thấy hướng triển khai trên thiết bị có khả năng đáp ứng mức tốc độ đặt ra. Tuy nhiên, đây chưa phải kết quả đại diện cho mọi cấu hình của RobotApp; độ trễ còn phụ thuộc thiết bị, cơ chế tăng tốc, nguồn khung hình và biến thể mô hình được lựa chọn khi ứng dụng khởi tạo.

5.9 Thảo luận kết quả

5.9.1 Điểm mạnh

Kết quả nổi bật nhất là mức tích hợp xuyên suốt. Lệnh caregiver không dừng ở giao diện mà đi đến hai setpoint motor. Cảnh báo AI không dừng ở nhấn mà nổi với hỏi xác nhận, bản ghi và cuộc gọi. Camera được dùng chung đúng vòng đời. Các chi tiết timestamp, watchdog, khóa sự cố và ID0 cho thấy thiết kế đã xử lý nhiều lỗi đặc thù hệ thống thời gian thực.

Việc chạy pose và LSTM tại RobotApp giảm nhu cầu gửi video liên tục lên cloud. Hệ thống vẫn dùng Internet cho điều phối/hội thoại/call, nhưng quyết định thị giác cơ bản ở gần nguồn. Tách CaregiverApp khỏi BLE cũng giảm phụ thuộc khoảng cách vật lý và giữ hiệu chỉnh motor ở một nơi.

5.9.2 Hạn chế

Thứ nhất, các chỉ số của LSTM ở Bảng 5.14 và Bảng 5.15 được đo với cách chia tập theo video, nhưng cấu hình đo tốc độ vẫn chưa lưu kèm log thiết bị và kết quả đo thô. Vì vậy, các số liệu hiện có chưa đủ để đánh giá độc lập toàn bộ chuỗi YOLO-Pose-tracking-LSTM-hậu kiểm trên phiên bản ứng dụng hiện tại. Thứ hai, số báo động giả vẫn ở mức 181,6 lần mỗi giờ ngay cả ở ngưỡng hậu kiểm tốt nhất, nghĩa là khối LSTM

vẫn phụ thuộc hoàn toàn vào hậu kiểm hình học và bước hỏi xác nhận để không phát cảnh báo nhầm ra ngoài. Thứ ba, tệp mô hình đang đóng gói trong RobotApp được tạo trước quy trình lựa chọn ở Mục 5.8.2 nên vẫn dùng cách chuẩn hóa theo hộp bao; huấn luyện và đóng gói lại theo cấu hình đã chọn là việc cần làm trước nghiệm thu, và vì phép chuẩn hóa được thực hiện ở phía ứng dụng, thay đổi này kéo theo phải sửa cả bước tạo đặc trưng trong `FullDetectionQueue`. Thứ tư, camera RGB có điểm mù và đặt ra yêu cầu bảo vệ quyền riêng tư. Thứ năm, hội thoại, đồng bộ từ xa và thiết lập cuộc gọi phụ thuộc vào Groq, Firebase và hạ tầng STUN/TURN. Thứ sáu, dữ liệu đã được phân vùng theo UID nhưng hai thiết bị trong một cặp vẫn dùng chung tài khoản, nên chưa có cơ chế phân quyền riêng cho robot và người chăm sóc. Thứ bảy, dung lượng APK debug của RobotApp còn lớn.

Một điểm cần lưu ý về dữ liệu huấn luyện: keypoint dùng để huấn luyện LSTM được trích bằng YOLOv8n-Pose ở giai đoạn chuẩn bị dữ liệu, trong khi RobotApp suy luận bằng YOLO26n-Pose. Hai mô hình có sai số định vị khớp khác nhau, nên phân bố đặc trưng khi chạy thực tế lệch so với phân bố lúc huấn luyện. Giả thuyết này đã được kiểm chứng bằng cách trích xuất lại toàn bộ 190 video bằng YOLO26n-Pose và huấn luyện lại trên cùng giao thức: F1 mức cửa sổ đạt 0,713 so với 0,721 của dữ liệu gốc, tức không cải thiện mà còn thấp hơn chút ít, và khác biệt vẫn nằm trong sai số giữa ba lần chạy. Như vậy sai lệch giữa hai bộ trích xuất keypoint không phải nguyên nhân chính của giới hạn hiện tại. Cần lưu ý rằng phép đo này được thực hiện trên tập dữ liệu trước khi làm sạch nhãn, nên nó chỉ cho biết việc đổi bộ trích xuất không mang lại cải thiện, chứ chưa loại trừ được khả năng có cải thiện trên dữ liệu sạch; việc đo lại là công việc còn tồn đọng.

Cuối cùng, cả hai cách biểu diễn dẫn đầu ở Bảng 5.10 đều phụ thuộc vào hình học camera, chỉ khác mức độ. Tọa độ gốc giữ nguyên giá trị pixel nên gắn chặt vào đúng cấu hình camera của LE2I, gồm độ phân giải 320×240 , góc đặt và khoảng cách tới người; một camera đặt ở độ cao khác sẽ cho dải giá trị tọa độ dọc khác và làm mô hình hoạt động sai. Chuẩn hóa theo khung ảnh loại bỏ được sự phụ thuộc vào độ phân giải nhưng vẫn giữ phụ thuộc vào góc đặt và khoảng cách. Vì hai cách cho hiệu năng không phân biệt được, chuẩn hóa theo khung ảnh là lựa chọn an toàn hơn và nên được dùng khi đóng gói lại mô hình.

Để triển khai trên nhiều phòng và nhiều góc camera, cả hai vẫn chưa đủ; cần một phép chuẩn hóa vừa giữ được tỷ lệ chiều cao vừa độc lập với camera, chẳng hạn chia theo chiều cao người khi đứng, đo một lần lúc hiệu chuẩn lắp đặt. Việc này nằm ngoài

phạm vi đồ án và được nêu như hướng phát triển.

Re-ID histogram nhẹ không đủ mạnh khi nhiều người mặc màu giống nhau. Logic người có bbox lớn nhất chỉ là heuristic ban đầu. LSTM 15 frame tạo trễ phụ thuộc FPS và có thể nhận cửa sổ không đều khi drop-frame. Bộ hậu kiểm dùng ngưỡng thủ công, cần calibration theo góc camera/nhà. Parser firmware chấp nhận `String.toFloat`, nên cần validate nghiêm hơn.

Chất lượng chú thích của bộ dữ liệu. Trong quá trình đánh giá, hai lỗi chú thích đã được phát hiện và cần được nêu lại vì chúng ảnh hưởng tới mọi số liệu trước đó.

Lỗi thứ nhất nằm ở quy trình chuẩn bị dữ liệu. Bối cảnh *Coffee room 02* đặt tên thư mục chú thích là `Annotations_files` trong khi năm bối cảnh còn lại dùng `Annotation_files`. Mã chuẩn bị dữ liệu chỉ khớp dạng thứ hai nên bỏ sót toàn bộ bối cảnh này, khiến 12 trên 22 video có pha ngã bị gán nhãn “không ngã”. Hệ quả kép: mô hình được huấn luyện với 12 cú ngã bị dạy là bình thường, đồng thời bị tính là báo động giả khi nó phát hiện đúng những cú ngã đó lúc kiểm thử.

Lỗi thứ hai nằm ở chính bộ dữ liệu: 60 video của hai bối cảnh *Lecture room* và *Office* không có tệp chú thích nào. Cách xử lý và bằng chứng đã trình bày ở Mục 5.8.1.

Hai lỗi này chỉ lộ ra khi các video bị phân loại sai được xem lại trực tiếp bằng mắt, chứ không thể phát hiện qua bất kỳ chỉ số tổng hợp nào. Đây là lý do việc kiểm tra thủ công một mẫu các trường hợp sai là bước bắt buộc chứ không phải tùy chọn, và cũng là giới hạn của toàn bộ phần đánh giá: các chỉ số chỉ đáng tin bằng đúng độ tin cậy của nhãn mà chúng dựa vào.

5.9.3 Môi đe dọa tới tính hợp lệ

Nội tại: kết quả có thể phụ thuộc actor, trình tự kịch bản, vị trí camera và việc điều chỉnh ngưỡng trên cùng dữ liệu. *Ngoại tại:* một hoặc vài nhà/thiết bị không đại diện mọi căn hộ, mạng và người cao tuổi. *Đo lường:* accuracy theo frame có thể thổi phồng hiệu năng; clock khác nhau gây sai độ trễ. *Kết luận:* số mẫu nhỏ làm confidence interval rộng. Giảm rủi ro bằng protocol cố định, split theo chủ thể, log tự động, blind annotation và công bố cả trường hợp thất bại.

5.10 Mức đáp ứng mục tiêu

Bảng 5.18: Đối chiếu mục tiêu và kết quả

Mục tiêu	Mức đạt	Nhận xét
Hai ứng dụng theo vai trò	Đạt nguyên mẫu	Có APK và module chức năng chính
Firmware thân robot	Đạt nguyên mẫu	BLE, dual BLDC, encoder, FOC, watchdog
Theo dõi người và auto-follow	Đạt hiện thực	Có ID0, SORT/Re-ID và cầu nối motor; cần metric dài hạn
Phát hiện/xác nhận té ngã	Đạt hiện thực	Đã tích hợp pose, LSTM, hậu kiểm, xác nhận giọng nói và cảnh báo; chưa đánh giá độc lập ở cấp sự cố trên toàn bộ chuỗi
Hội thoại và nhắc nhở	Đạt nguyên mẫu	STT–Groq–TTS và Schedules/logs
Video call/điều khiển xa	Đạt nguyên mẫu	WebRTC signaling RTDB và joystick
Đánh giá định lượng toàn diện	Chưa hoàn tất	Cần log dataset, benchmark thiết bị và thử nghiệm nhiều bối cảnh
Sẵn sàng sản phẩm/y tế	Ngoài phạm vi	Cần security, safety, UX, pháp lý và thử nghiệm thực địa

5.11 Khuyến nghị trước nghiệm thu và trình diễn

Trước buổi trình diễn, nên đóng băng commit của bốn khối, lưu checksum model, build release/debug từ môi trường ghi phiên bản và chạy danh mục E01–E18. Mỗi thất bại cần lưu thời gian, thiết bị, logcat/serial và video. Chuẩn bị chế độ demo offline tối thiểu cho AI/BLE, kiểm tra credential TURN/Groq/Firebase, sạc đầy thiết bị và thử nút dừng vật lý.

Các bảng kết quả định lượng nên được điền từ log thật theo mẫu ở Phụ lục. Việc để trống có chú thích tốt hơn một con số không truy nguồn. Nếu thời gian hạn chế, ưu tiên đo recall/false alarm theo sự cố, latency P95 AI, thời gian dừng khi mất lệnh, tỷ lệ cuộc gọi thành công và số lần ID switch.

5.12 Kết luận chương

Nguyên mẫu đã hiện thực các khối kiến trúc chính, gồm hai ứng dụng Android, mô hình triển khai và firmware của phần thân robot. Việc kiểm tra tĩnh cho thấy cấu trúc dữ liệu Firebase và giao thức BLE được sử dụng nhất quán giữa các thành phần. Các kiểm thử tự động hiện bao phủ việc phân vùng đường dẫn, ánh xạ lệnh động cơ, chính sách vận tốc bám người, nhận biết hình học tư thế và hậu kiểm té ngã. Kết quả định lượng hiện có mới cung cấp đánh giá ban đầu cho mô hình LSTM và một cấu hình suy luận thử

nghiệm; việc đo toàn bộ hệ thống trên đúng phiên bản ứng dụng và thiết bị đích vẫn cần được hoàn thiện. Chương cuối tổng kết những kết quả đạt được, các hạn chế và hướng phát triển tiếp theo.

Chương 6 – Kết luận

6.1 Tổng kết

Đồ án đã nghiên cứu và xây dựng nguyên mẫu robot trợ lý ảo hỗ trợ chăm sóc, giám sát người cao tuổi tại nhà. Hệ thống kết hợp hội thoại bằng giọng nói, nhắc nhở theo lịch, theo dõi người, phát hiện tình huống nghi té ngã và liên lạc với người chăm sóc. Điện thoại Android gắn trên robot đảm nhiệm thu nhận hình ảnh, âm thanh và xử lý các chức năng hỗ trợ; phần thân robot thực hiện chuyển động theo lệnh từ điện thoại. Cách tổ chức này kế thừa ý tưởng sử dụng điện thoại thông minh làm bộ xử lý của nền tảng OpenBot và được phát triển theo nhu cầu của đề tài.

Hai ứng dụng RobotApp và CaregiverApp tạo thành các phương tiện tương tác dành cho người cao tuổi và người chăm sóc. RobotApp thực hiện hội thoại, phát lời nhắc và xử lý hình ảnh tại robot. CaregiverApp cho phép quản lý thông tin, cấu hình chức năng giám sát, xem nhật ký, tham gia cuộc gọi và điều khiển robot từ xa. Dữ liệu chăm sóc, thông tin cuộc gọi và lệnh chuyển động được trao đổi qua các kênh phù hợp với từng chức năng.

Đối với giám sát té ngã, hệ thống kết hợp ước lượng tư thế, duy trì định danh người mục tiêu và phân tích chuỗi chuyển động bằng LSTM. Kết quả phân loại được kiểm tra bổ sung trước khi robot hỏi xác nhận tình trạng. Khi người dùng cần hỗ trợ hoặc chưa thể xác nhận an toàn, ứng dụng lưu cảnh báo và thực hiện cuộc gọi đến người chăm sóc. Các cơ chế kiểm tra lệnh và dừng khi mất cập nhật được tổ chức ở cả ứng dụng lẫn phần thân robot.

6.2 Kết quả đạt được

Những kết quả chính của đồ án gồm:

- **Xây dựng hệ thống hỗ trợ chăm sóc có sự phối hợp giữa robot và người chăm sóc.** Hai ứng dụng cung cấp các chức năng quản lý hồ sơ, nhắc nhở, hội thoại, theo dõi sự kiện và liên lạc từ xa trong cùng một hệ thống.
- **Tích hợp thị giác máy tính với chức năng bám theo và phát hiện té ngã.** Kết quả ước lượng tư thế được sử dụng đồng thời để duy trì người mục tiêu, phân tích chuyển động và điều chỉnh vị trí robot. Bước hậu kiểm và xác nhận bằng giọng nói bổ sung thông tin trước khi phát cảnh báo.

- **Hiện thực khả năng di chuyển và điều khiển từ xa.** Firmware điều khiển độc lập hai động cơ, tiếp nhận lệnh từ RobotApp và đưa vận tốc đặt về 0 khi kết nối hoặc luồng lệnh bị gián đoạn. Việc kiểm tra lệnh được kết hợp giữa ứng dụng và vi điều khiển.
- **Triển khai mô hình AI trên thiết bị Android.** Quy trình chuẩn bị và chuyển đổi đã tạo các mô hình TFLite để tích hợp vào RobotApp. Dữ liệu đầu vào, đầu ra và các phép chuyển đổi tọa độ được tổ chức thống nhất giữa những bước xử lý.
- **Xây dựng các nhóm kiểm thử và bước đầu đánh giá hiệu năng.** Các kiểm thử ở mức logic bao phủ đường dẫn dữ liệu, ánh xạ lệnh động cơ, chính sách bám theo, hình học tư thế và hậu kiểm té ngã. Chương 5 trình bày các chỉ số ban đầu của mô hình LSTM, một cấu hình đo tốc độ suy luận và các kịch bản kiểm thử chức năng đầu cuối.

Kết quả hiện tại thể hiện khả năng tích hợp các thành phần của nguyên mẫu. Các số liệu đánh giá ban đầu hỗ trợ xem xét tính khả thi của hướng triển khai, nhưng chưa đại diện cho chất lượng phát hiện té ngã và hiệu năng của toàn bộ hệ thống trong mọi điều kiện sử dụng.

6.3 Hạn chế

Phạm vi đánh giá còn giới hạn về dữ liệu, thiết bị và bối cảnh thử nghiệm. Các số liệu đã trình bày chưa đi kèm đầy đủ lịch sử huấn luyện, thông tin chia tập dữ liệu, cấu hình thiết bị và kết quả đo thô để tái lập toàn bộ thí nghiệm. Cấu hình đo tốc độ của biến thể FP16 cũng chưa đại diện cho mọi mô hình được đóng gói trong RobotApp. Vì vậy, cần tiếp tục đánh giá trên đúng phiên bản ứng dụng và thiết bị sử dụng, đặc biệt khi nhiều chức năng hoạt động đồng thời.

Chất lượng giám sát phụ thuộc vào góc nhìn camera, điều kiện chiếu sáng và mức độ che khuất. Dữ liệu té ngã dàn dựng có thể khác với tình huống xảy ra trong sinh hoạt của người cao tuổi. Đặc trưng màu dùng để nhận lại mục tiêu có thể nhầm lẫn khi nhiều người mặc trang phục tương tự; các ngưỡng theo dõi và hậu kiểm cũng cần được hiệu chỉnh theo vị trí camera và tốc độ xử lý. Nguyên mẫu hiện phục vụ hỗ trợ giám sát, chưa được đánh giá lâm sàng và không thay thế thiết bị chẩn đoán y khoa.

Hội thoại qua LLM, đồng bộ dữ liệu từ xa và thiết lập cuộc gọi phụ thuộc vào kết nối Internet. Khi mạng gián đoạn, xử lý hình ảnh và kết nối điều khiển cục bộ không phụ

thuộc trực tiếp vào các dịch vụ này, nhưng việc chuyển cảnh báo và liên lạc với người chăm sóc có thể bị gián đoạn. Hai ứng dụng trong một cặp hiện sử dụng cùng tài khoản, nên chưa có cơ chế phân quyền riêng cho robot và người chăm sóc hoặc quản lý nhiều robot trong một gia đình. Cơ chế quản lý khóa truy cập, thông tin xác thực và kiểm tra dữ liệu trên máy chủ cũng cần được hoàn thiện.

Phần thân robot mới đáp ứng các chuyển động cơ bản và cơ chế dừng khi mất lệnh. Việc bổ sung cảm biến tránh va chạm, nút dừng vật lý, giám sát nguồn điện và kiểm tra chặt chẽ chuỗi lệnh nhận vào vẫn cần được xem xét. Dung lượng ứng dụng, mức sử dụng bộ nhớ, nhiệt độ và năng lượng khi vận hành kéo dài cũng chưa được tối ưu đầy đủ.

6.4 Hướng phát triển

6.4.1 Hoàn thiện dữ liệu và phương pháp đánh giá

Hướng ưu tiên là chuẩn hóa quy trình thí nghiệm để có thể lặp lại và so sánh giữa các phiên bản. Dữ liệu cần được phân chia theo video và người tham gia, đồng thời lưu cấu hình huấn luyện, phiên bản mô hình và kết quả đo. Tập kiểm tra nên mở rộng các tình huống sinh hoạt dễ nhầm với té ngã, nhiều góc camera và điều kiện che khuất. Việc đánh giá cần bao gồm tỷ lệ phát hiện đúng, cảnh báo nhầm và thời gian phản ứng ở cấp sự kiện. Các thí nghiệm lần lượt loại bỏ từng thành phần, chẳng hạn bộ hậu kiểm hoặc thông tin tư thế, sẽ giúp xác định đóng góp của chúng đối với kết quả chung.

6.4.2 Nâng cao độ ổn định của theo dõi và phát hiện té ngã

Chức năng theo dõi có thể được cải thiện bằng đặc trưng ngoại hình phù hợp với tài nguyên của điện thoại, kết hợp vị trí và diễn biến chuyển động để nhận lại người mục tiêu. Việc sử dụng độ tin cậy của các điểm khớp và khoảng thời gian thực giữa các khung hình cũng cần được nghiên cứu, nhất là khi tốc độ xử lý thay đổi. Ngoài camera, có thể khảo sát thêm cảm biến khoảng cách hoặc thiết bị đeo để bổ sung thông tin trong những tình huống quan sát bị hạn chế. Mỗi phương án cần được đánh giá về hiệu quả, chi phí tính toán và mức độ thuận tiện đối với người cao tuổi.

6.4.3 Hoàn thiện khả năng di chuyển và an toàn vận hành

Phần điều khiển cần được mở rộng theo hướng kiểm tra đầy đủ dữ liệu nhận vào, giới hạn tốc độ thay đổi vận tốc và quy định rõ mức ưu tiên giữa dừng khẩn cấp, điều

hiển từ xa và bám theo tự động. Cảm biến khoảng cách, cơ cấu phát hiện va chạm, giám sát pin và nút dừng vật lý là những thành phần có thể bổ sung để hỗ trợ vận hành trong nhà. Các thay đổi cần được đánh giá qua thử nghiệm cơ khí, thời gian dừng và phản ứng khi kết nối bị gián đoạn.

6.4.4 Mở rộng quản lý người dùng và khả năng duy trì liên lạc

Cấu trúc quản lý có thể mở rộng để mỗi người chăm sóc sử dụng tài khoản riêng và được cấp quyền truy cập vào robot phù hợp. Thông tin cuộc gọi, lịch sử và lệnh điều khiển cần gắn với từng phiên hoặc thiết bị để tránh nhầm lẫn khi số lượng người dùng tăng. Bên cạnh đó, cơ chế xác thực dịch vụ và bảo vệ dữ liệu cần được hoàn thiện.

Để giảm ảnh hưởng của mất mạng, có thể lưu tạm cảnh báo tại thiết bị và đồng bộ lại khi kết nối được khôi phục, kèm cơ chế tránh tạo bản ghi trùng. Các câu hỏi xác nhận mẫu và phản hồi bằng giọng nói tại thiết bị cũng cần được chuẩn bị cho trường hợp dịch vụ trực tuyến không khả dụng. Khả năng sử dụng cuộc gọi hoặc tin nhắn qua mạng di động làm kênh dự phòng có thể được khảo sát trên thiết bị hỗ trợ.

6.4.5 Cải thiện trải nghiệm và tối ưu tài nguyên

Giao diện và tương tác giọng nói cần được đánh giá với người dùng mục tiêu, tập trung vào kích thước chữ, độ tương phản, tốc độ đọc, cách xưng hô, phương ngữ và thời gian chờ phản hồi. Robot cần thông báo rõ trạng thái sử dụng camera, microphone và cho phép người dùng chủ động bật hoặc tắt các chức năng.

Về hiệu năng, cần so sánh các phương án suy luận trên CPU và bộ tăng tốc ở đúng thiết bị sử dụng, đồng thời theo dõi bộ nhớ, nhiệt độ và mức tiêu thụ năng lượng khi chạy lâu. Việc giảm kích thước mô hình, tái sử dụng bộ đệm ảnh và loại bỏ thư viện không cần thiết có thể giúp giảm tải cho điện thoại. Mỗi thay đổi tối ưu cần được kiểm tra lại về chất lượng nhận dạng và độ ổn định của ứng dụng.

6.5 Kết luận cuối cùng

Đề án tạo cơ sở để tiếp tục nghiên cứu việc kết hợp tương tác giọng nói, giám sát bằng thị giác và robot di động trong hỗ trợ chăm sóc tại nhà. Giá trị của nguyên mẫu nằm ở khả năng phối hợp các chức năng này để người cao tuổi nhận được hỗ trợ hằng ngày và người chăm sóc có thêm phương tiện theo dõi, liên lạc từ xa. Những kết quả và giới hạn đã xác định giúp định hướng các bước thử nghiệm, cải tiến tiếp theo, hướng đến giải



pháp phù hợp hơn với nhu cầu sử dụng của người cao tuổi.

TÀI LIỆU THAM KHẢO

1. World Health Organization, “Ageing and health,” 2025. [Online]. Available: [WHO – Ageing and health](#). [Accessed: 31-Aug-2026].
2. General Statistics Office of Viet Nam and UNFPA Viet Nam, “Population ageing and projections of older population: The Population and Housing Census 2019,” 2021. [Online]. Available: https://vietnam.unfpa.org/sites/default/files/resource-pdf/unfpa_6_projection_en_7.11.pdf.
3. UNFPA Viet Nam, “Applying foresight to curate a care economy for older persons in Viet Nam – Executive summary,” 2025. [Online]. Available: https://vietnam.unfpa.org/sites/default/files/pub-pdf/2025-12/Applying%20foresight%20to%20curate%20a%20care%20economy%20for%20OP%20in%20VN_EN_FINAL.pdf.
4. World Health Organization, “Falls,” Apr. 2021. [Online]. Available: <https://www.who.int/news-room/fact-sheets/detail/falls>.
5. A. Smaje, M. Weston-Clark, R. Raj, M. Orlu, D. Davis, and M. Rawle, “Factors associated with medication adherence in older patients: A systematic review,” *Aging Medicine*, vol. 1, no. 3, pp. 254–266, 2018, doi: 10.1002/agm2.12045.
6. World Health Organization, “Mental health of older adults,” 2025. [Online]. Available: <https://www.who.int/news-room/fact-sheets/detail/mental-health-of-older-adults>.
7. M. Müller and V. Koltun, “OpenBot: Turning smartphones into robots,” in *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, 2021, pp. 9305–9311, doi: 10.1109/ICRA48506.2021.9561788.
8. N. T. Newaz and E. Hanada, “The methods of fall detection: A literature review,” *Sensors*, vol. 23, no. 11, Art. no. 5212, 2023, doi: 10.3390/s23115212.
9. A. Alam et al., “Vision-based human fall detection systems using deep learning: A review,” *Computers in Biology and Medicine*, vol. 146, Art. no. 105626, 2022, doi: 10.1016/j.combiomed.2022.105626.

10. H. Khan, I. Ullah, M. Shabaz, M. F. Omer, M. T. Usman, M. S. Guellil, and J. Koo, “Visionary vigilance: Optimized YOLOv8 for fallen person detection with large-scale benchmark dataset,” *Image and Vision Computing*, vol. 149, Art. no. 105195, 2024, doi: 10.1016/j.imavis.2024.105195.
11. L. Liu et al., “A hybrid human fall detection method based on modified YOLOv8s and AlphaPose,” *Scientific Reports*, vol. 15, Art. no. 2636, 2025, doi: 10.1038/s41598-025-86429-6.
12. D. Kim, X. Yu, and S. Xiong, “Robust skeleton-based AI for automatic multi-person fall detection on construction sites with occlusions,” *Automation in Construction*, vol. 175, Art. no. 106216, 2025, doi: 10.1016/j.autcon.2025.106216.
13. N. C. Bach, N. V. Chi, D. D. Cuong, N. A. Tuan, T. Q. T. Kieu, N. T. Phuong, N. D. Thien, and L. Q. Thao, “A lightweight and efficient deep learning model for real-time fall detection on edge device,” *Biomedical Signal Processing and Control*, vol. 113, part B, Art. no. 108942, 2026, doi: 10.1016/j.bspc.2025.108942.
14. A. K. Bourke, J. V. O’Brien, and G. M. Lyons, “Evaluation of a threshold-based tri-axial accelerometer fall detection algorithm,” *Gait & Posture*, vol. 26, no. 2, pp. 194–199, 2007, doi: 10.1016/j.gaitpost.2006.09.012.
15. S. Abbate, M. Avvenuti, F. Bonatesta, G. Cola, P. Corsini, and A. Vecchio, “A smartphone-based fall detection system,” *Pervasive and Mobile Computing*, vol. 8, no. 6, pp. 883–899, 2012, doi: 10.1016/j.pmcj.2012.08.003.
16. A. Sucerquia, J. D. López, and J. F. Vargas-Bonilla, “SisFall: A fall and movement dataset,” *Sensors*, vol. 17, no. 1, Art. no. 198, 2017, doi: 10.3390/s17010198.
17. F. S. Butt et al., “Fall detection from electrocardiogram (ECG) signals and classification by deep transfer learning,” *Information*, vol. 12, no. 2, Art. no. 63, 2021, doi: 10.3390/info12020063.
18. Y. Li, K. C. Ho, and M. Popescu, “A microphone array system for automatic fall detection,” *IEEE Transactions on Biomedical Engineering*, vol. 59, no. 5, pp. 1291–1301, 2012, doi: 10.1109/TBME.2012.2186449.
19. E. E. Stone and M. Skubic, “Fall detection in homes of older adults using the Microsoft Kinect,” *IEEE Journal of Biomedical and Health Informatics*, vol. 19, no. 1, pp. 290–301, 2015, doi: 10.1109/JBHI.2014.2312180.

20. M. D. Solbach and J. K. Tsotsos, “Vision-based fallen person detection for the elderly,” in *Proc. IEEE International Conference on Computer Vision Workshops (ICCVW)*, 2017, pp. 1433–1442.
21. F. Jin, A. Sengupta, and S. Cao, “mmFall: Fall detection using 4-D mmWave radar and a hybrid variational RNN autoencoder,” *IEEE Transactions on Automation Science and Engineering*, vol. 19, no. 2, pp. 1245–1257, 2022, doi: 10.1109/TASE.2020.3042158.
22. B. Kwolek and M. Kepski, “Human fall detection on embedded platform using depth maps and wireless accelerometer,” *Computer Methods and Programs in Biomedicine*, vol. 117, no. 3, pp. 489–501, 2014, doi: 10.1016/j.cmpb.2014.09.005.
23. A. Núñez-Marcos, G. Azkune, and I. Arganda-Carreras, “Vision-based fall detection with convolutional neural networks,” *Wireless Communications and Mobile Computing*, vol. 2017, Art. no. 9474806, 2017, doi: 10.1155/2017/9474806.
24. S. Juraev, A. Ghimire, J. Alikhanov, V. Kakani, and H. Kim, “Exploring human pose estimation and the usage of synthetic data for elderly fall detection in real-world surveillance,” *IEEE Access*, vol. 10, pp. 94249–94261, 2022, doi: 10.1109/ACCESS.2022.3203174.
25. S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997, doi: 10.1162/neco.1997.9.8.1735.
26. L. E. Sucerquia, J. D. López, and J. F. Vargas-Bonilla, “Dataset for human fall recognition in an uncontrolled environment,” *Data in Brief*, vol. 45, Art. no. 108610, 2022, doi: 10.1016/j.dib.2022.108610.
27. A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, “Simple online and realtime tracking,” in *Proc. IEEE International Conference on Image Processing (ICIP)*, 2016, pp. 3464–3468, doi: 10.1109/ICIP.2016.7533003.
28. R. E. Kalman, “A new approach to linear filtering and prediction problems,” *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960, doi: 10.1115/1.3662552.
29. G. Welch and G. Bishop, “An introduction to the Kalman filter,” in *Course 8, Proc. ACM SIGGRAPH*, Los Angeles, CA, USA, 2001. [Online]. Available:

- <https://cis.temple.edu/~latecki/Courses/CIS750-03/Papers/KalmanFilterSIGGRAPH2001.pdf>. [Accessed: 17-Sep-2026].
30. H. W. Kuhn, “The Hungarian method for the assignment problem,” *Naval Research Logistics Quarterly*, vol. 2, no. 1–2, pp. 83–97, 1955, doi: 10.1002/nav.3800020109.
 31. N. Wojke, A. Bewley, and D. Paulus, “Simple online and realtime tracking with a deep association metric,” in *Proc. IEEE ICIP*, 2017, pp. 3645–3649, doi: 10.1109/ICIP.2017.8296962.
 32. Y. Zhang et al., “ByteTrack: Multi-object tracking by associating every detection box,” in *Proc. ECCV*, 2022, pp. 1–21, doi: 10.1007/978-3-031-20047-2_1.
 33. 78, “xiaozi-esp32: An MCP-based chatbot,” GitHub, [Online]. Available: <https://github.com/78/xiaozi-esp32>. [Accessed: 05-Sep-2026].
 34. Google, “SpeechRecognizer,” Android Developers, [Online]. Available: <https://developer.android.com/reference/android/speech/SpeechRecognizer>. [Accessed: 05-Sep-2026].
 35. Google, “TextToSpeech,” Android Developers, [Online]. Available: <https://developer.android.com/reference/android/speech/tts/TextToSpeech>. [Accessed: 05-Sep-2026].
 36. Alpha Cephei, “Vosk offline speech recognition API for Android,” [Online]. Available: <https://alphacephei.com/vosk/android>. [Accessed: 05-Sep-2026].
 37. k2-fsa, “sherpa-onnx Android pre-built applications,” [Online]. Available: <https://k2-fsa.github.io/sherpa/onnx/android/prebuilt-apk.html>. [Accessed: 05-Sep-2026].
 38. Microsoft, “About the Speech SDK,” Microsoft Learn, [Online]. Available: <https://learn.microsoft.com/en-us/azure/ai-services/speech-service/speech-sdk>. [Accessed: 11-Sep-2026].
 39. Microsoft, “Language and voice support for Azure Speech,” Microsoft Learn, [Online]. Available: <https://learn.microsoft.com/en-us/azure/ai-services/speech-service/language-support>. [Accessed: 11-Sep-2026].

40. Groq, “API reference: Chat completions,” [Online]. Available: <https://console.groq.com/docs/api-reference>. [Accessed: 31-Aug-2026].
41. Groq, “Rate limits,” [Online]. Available: <https://console.groq.com/docs/rate-limits>. [Accessed: 05-Sep-2026].
42. Google, “Gemini Developer API pricing,” [Online]. Available: <https://ai.google.dev/gemini-api/docs/pricing>. [Accessed: 05-Sep-2026].
43. Google AI Edge, “Google AI Edge Gallery,” GitHub, [Online]. Available: <https://github.com/google-ai-edge/gallery>. [Accessed: 05-Sep-2026].
44. MLC team, “MLC LLM Android SDK,” [Online]. Available: <https://llm.mlc.ai/docs/deploy/android.html>. [Accessed: 05-Sep-2026].
45. L. Laranjo et al., “Conversational agents in healthcare: A systematic review,” *Journal of the American Medical Informatics Association*, vol. 25, no. 9, pp. 1248–1258, 2018, doi: 10.1093/jamia/ocy072.
46. D. Portugal et al., “SocialRobot: Towards a personalized elderly care mobile robot,” *arXiv:1809.05518*, 2018, doi: 10.48550/arXiv.1809.05518.
47. OpenBot Foundation, “OpenBot source-code repository,” GitHub. [Online]. Available: github.com/ob-f/OpenBot. [Accessed: 05-Sep-2026].
48. OpenBot Foundation, “OpenBot—Turning smartphones into robots,” [Online]. Available: [OpenBot official website](#). [Accessed: 05-Sep-2026].
49. OpenBot Foundation, “OpenBot firmware,” GitHub documentation. [Online]. Available: [OpenBot firmware documentation](#). [Accessed: 05-Sep-2026].
50. OpenBot Foundation, “OpenBot controllers,” GitHub documentation. [Online]. Available: [OpenBot controller documentation](#). [Accessed: 05-Sep-2026].
51. OpenBot Foundation, “OpenBot Robot App,” GitHub documentation. [Online]. Available: [OpenBot Robot App documentation](#). [Accessed: 05-Sep-2026].
52. Y. LeCun and Y. Bengio, “Convolutional networks for images, speech, and time series,” in *The Handbook of Brain Theory and Neural Networks*, M. A. Arbib, Ed. Cambridge, MA, USA: MIT Press, 1998, pp. 255–258.

53. Z. Zou, K. Chen, Z. Shi, Y. Guo, and J. Ye, “Object detection in 20 years: A survey,” *Proceedings of the IEEE*, vol. 111, no. 3, pp. 257–276, 2023, doi: 10.1109/JPROC.2023.3238520.
54. R. Girshick, J. Donahue, T. Darrell, and J. Malik, “Rich feature hierarchies for accurate object detection and semantic segmentation,” in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2014, pp. 580–587, doi: 10.1109/CVPR.2014.81.
55. S. Ren, K. He, R. Girshick, and J. Sun, “Faster R-CNN: Towards real-time object detection with region proposal networks,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 39, no. 6, pp. 1137–1149, 2017, doi: 10.1109/TPAMI.2016.2577031.
56. W. Liu et al., “SSD: Single Shot MultiBox Detector,” in *Proc. European Conference on Computer Vision (ECCV)*, 2016, pp. 21–37, doi: 10.1007/978-3-319-46448-0_2.
57. J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You Only Look Once: Unified, real-time object detection,” in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788, doi: 10.1109/CVPR.2016.91.
58. M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn, and A. Zisserman, “The PASCAL Visual Object Classes (VOC) challenge,” *International Journal of Computer Vision*, vol. 88, pp. 303–338, 2010, doi: 10.1007/s11263-009-0275-4.
59. T.-Y. Lin et al., “Microsoft COCO: Common objects in context,” in *Proc. European Conference on Computer Vision (ECCV)*, 2014, pp. 740–755, doi: 10.1007/978-3-319-10602-1_48.
60. A. G. Howard et al., “MobileNets: Efficient convolutional neural networks for mobile vision applications,” *arXiv:1704.04861*, 2017, doi: 10.48550/arXiv.1704.04861.
61. M. Tan, R. Pang, and Q. V. Le, “EfficientDet: Scalable and efficient object detection,” in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 10781–10790, doi: 10.1109/CVPR42600.2020.01079.
62. J. Redmon and A. Farhadi, “YOLOv3: An incremental improvement,” *arXiv:1804.02767*, 2018, doi: 10.48550/arXiv.1804.02767.

63. G. Jocher, “Ultralytics YOLOv5,” Zenodo, 2020, doi: 10.5281/zenodo.3908559.
64. Ultralytics, “YOLO architecture explained,” [Online]. Available: <https://docs.ultralytics.com/guides/yolo-architecture/>. [Accessed: 07-Sep-2026].
65. A. Wang, H. Chen, L. Liu, K. Chen, Z. Lin, J. Han, and G. Ding, “YOLOv10: Real-time end-to-end object detection,” *arXiv:2405.14458*, 2024, doi: 10.48550/arXiv.2405.14458.
66. R. Khanam and M. Hussain, “YOLOv11: An overview of the key architectural enhancements,” *arXiv:2410.17725*, 2024, doi: 10.48550/arXiv.2410.17725.
67. G. Jocher, J. Qiu, M. Liu, S. Lyu, F. C. Akyon, and M. E. Kalfaoglu, “Ultralytics YOLO26: Unified real-time end-to-end vision models,” *arXiv:2606.03748*, 2026, doi: 10.48550/arXiv.2606.03748.
68. Ultralytics, “YOLO26-pose model configuration,” GitHub. [Online]. Available: <https://github.com/ultralytics/ultralytics/blob/main/ultralytics/cfg/models/26/yolo26-pose.yaml>. [Accessed: 07-Sep-2026].
69. J. L. Elman, “Finding structure in time,” *Cognitive Science*, vol. 14, no. 2, pp. 179–211, 1990, doi: 10.1207/S15516709COG1402_1.
70. I. Charfi, J. Miteran, J. Dubois, M. Atri, and R. Tourki, “Optimized spatio-temporal descriptors for real-time fall detection: Comparison of support vector machine and Adaboost-based classification,” *Journal of Electronic Imaging*, vol. 22, no. 4, Art. no. 041106, 2013, doi: 10.1117/1.JEI.22.4.041106.
71. Google, “TensorFlow Lite: Deploy machine learning models on mobile and edge devices,” [Online]. Available: <https://www.tensorflow.org/lite>. [Accessed: 31-Aug-2026].
72. Google Firebase, “Firebase Authentication,” [Online]. Available: <https://firebase.google.com/docs/auth>. [Accessed: 11-Sep-2026].
73. Google Firebase, “Firebase Realtime Database,” [Online]. Available: <https://firebase.google.com/docs/database>. [Accessed: 31-Aug-2026].

74. W3C, “WebRTC: Real-Time Communication in Browsers,” W3C Recommendation, Mar. 2025. [Online]. Available: <https://www.w3.org/TR/webrtc/>.
75. A. Keränen, C. Holmberg, and J. Rosenberg, “Interactive Connectivity Establishment (ICE): A protocol for Network Address Translator traversal,” RFC 8445, IETF, Jul. 2018, doi: 10.17487/RFC8445.
76. E. Rescorla, “WebRTC security architecture,” RFC 8827, IETF, Jan. 2021, doi: 10.17487/RFC8827.
77. Bluetooth SIG, “The Bluetooth Low Energy primer,” 2024. [Online]. Available: <https://www.bluetooth.com/bluetooth-le-primer/>.
78. Bluetooth SIG, “Bluetooth Core Specification,” [Online]. Available: <https://www.bluetooth.com/specifications/specs/core-specification/>. [Accessed: 31-Aug-2026].
79. ISO/IEC, *ISO/IEC 25010:2023 Systems and software engineering—SQuaRE—Product quality model*, 2023.
80. Ultralytics, “Pose estimation with Ultralytics YOLO,” [Online]. Available: <https://docs.ultralytics.com/tasks/pose/>. [Accessed: 07-Sep-2026].
81. Google Firebase, “Understand Firebase Realtime Database Security Rules,” [Online]. Available: <https://firebase.google.com/docs/database/security>. [Accessed: 31-Aug-2026].
82. J. R. Hendershot and T. J. E. Miller, *Design of Brushless Permanent-Magnet Machines*. Venice, FL, USA: Motor Design Books, 2010.
83. SimpleFOC Community, “Arduino SimpleFOC library documentation,” [Online]. Available: <https://docs.simplefoc.com/>. [Accessed: 31-Aug-2026].
84. Google, “Bluetooth permissions,” Android Developers. [Online]. Available: <https://developer.android.com/develop/connectivity/bluetooth/bt-permissions>. [Accessed: 31-Aug-2026].
85. Google, “CameraX architecture,” Android Developers. [Online]. Available: [Android Developers – CameraX architecture](#). [Accessed: 31-Aug-2026].

PHỤ LỤC

Phụ lục A – Đặc tả use case

UC01 – Đăng ký và đăng nhập

Tác nhân	Người chăm sóc/người quản trị thiết bị robot
Mục tiêu	Tạo hoặc sử dụng tài khoản Firebase để nhận UID và truy cập dữ liệu chăm sóc tương ứng.
Tiền điều kiện	Thiết bị có Internet, ứng dụng trở đúng Firebase project.
Luồng chính	(1) Nhập email/mật khẩu; (2) ứng dụng gửi Firebase Auth; (3) nhận phiên và UID; (4) tải Users/uid; (5) chuyển màn hình chính.
Ngoại lệ	Email sai định dạng, mật khẩu sai, tài khoản chưa xác minh, mất mạng hoặc profile chưa tồn tại.
Hậu điều kiện	Phiên hợp lệ được duy trì; listener dữ liệu chỉ dùng UID được xác thực.
Yêu cầu liên quan	FR01, FR02, NFR05.

UC02 – Quản lý hồ sơ chăm sóc

Tác nhân	Người chăm sóc
Mục tiêu	Cập nhật thông tin giúp robot xưng hô và hỗ trợ phù hợp.
Tiền điều kiện	Đã đăng nhập và có quyền trên profile.
Luồng chính	Mở Profile; sửa họ tên, tên robot, đại từ, người liên hệ, quan hệ, tình trạng, ghi chú, sở thích; xác nhận; ghi Users/uid; RobotApp nhận thay đổi.
Ngoại lệ	Trường bắt buộc rỗng, số điện thoại sai, mất mạng, write bị Rules từ chối.
Hậu điều kiện	Profile mới được dùng khi tạo prompt/lời nhắc; thay đổi được ghi log nếu triển khai audit.
Yêu cầu liên quan	FR02, NFR05, NFR06.

UC03 – Quản lý lịch nhắc nhở

Tác nhân	Người chăm sóc
Mục tiêu	Tạo, sửa, bật/tắt hoặc xóa lịch thuốc/sinh hoạt.
Tiền điều kiện	Đăng nhập; thời gian và ngày lập hợp lệ.
Luồng chính	Chọn thêm lịch; nhập tiêu đề, ghi chú, thời gian, ngày; lưu dưới Schedules/uid/id; danh sách cập nhật; RobotApp nhận lịch active.
Ngoại lệ	Lịch trùng, múi giờ thay đổi, thiết bị khởi động lại, mất mạng, thời gian quá khứ.
Hậu điều kiện	Lịch định nghĩa được lưu riêng với từng lần thực hiện.
Yêu cầu liên quan	FR03, FR04, NFR10.

UC04 – Nhận và xác nhận lời nhắc

Tác nhân	Người cao tuổi; RobotApp
Mục tiêu	Phát lời nhắc dễ hiểu và lưu phản hồi.
Tiền điều kiện	Lịch active tới hạn; RobotApp hoạt động; TTS sẵn sàng.
Luồng chính	ReminderService tạo log delivered; phát sự kiện; chatbot phát TTS; nghe phản hồi; cập nhật confirmed/completed và confirmedAt.
Ngoại lệ	Người dùng đang gọi, TTS/STT bận, không phản hồi, lịch bị tắt đồng thời, dịch vụ LLM lỗi.
Hậu điều kiện	Một log phản ánh kết quả; lịch gốc không bị thay đổi ngoài quy tắc lặp.
Yêu cầu liên quan	FR04, FR05, FR19.

UC05 – Hội thoại trợ lý

Tác nhân	Người cao tuổi
Mục tiêu	Trao đổi bằng giọng nói và có câu trả lời theo hồ sơ.
Tiền điều kiện	Micro, STT, Internet, Groq và TTS khả dụng.
Luồng chính	Lắng nghe; STT tạo text; thêm history/prompt; gọi stream; ghép theo câu; TTS phát; trở về listening.

Ngoại lệ	Không nghe rõ, API timeout/rate limit, nội dung trống, TTS lỗi, người dùng ngắt.
Hậu điều kiện	History giữ tối đa số message cấu hình; không lưu secret trong log.
Yêu cầu liên quan	FR05, NFR04, NFR06.

UC06 – Theo dõi và tự bám người mục tiêu

Tác nhân	Người chăm sóc; RobotApp; ESP32
Mục tiêu	Robot duy trì khoảng cách/hướng tương đối với ID0.
Tiền điều kiện	Camera, model, BLE và cờ follow khả dụng; vùng di chuyển an toàn.
Luồng chính	Bật follow; pose detection; tracker chọn/duy trì ID0; sinh x, y, ts ; RobotControlManager ánh xạ; ESP32 điều khiển bánh.
Ngoại lệ	Nhiều người, mất ID0, vật cản, thiếu sáng, BLE mất, caregiver giành manual control.
Hậu điều kiện	Khi không đủ điều kiện, lệnh dừng; không tự đổi mục tiêu trong cảnh mờ hồ.
Yêu cầu liên quan	FR06–FR09, FR17–FR20.

UC07 – Phát hiện và xác nhận té ngã

Tác nhân	RobotApp; người cao tuổi; người chăm sóc
Mục tiêu	Phát hiện sự cố, hỏi xác nhận và chuyển cảnh báo có thể can thiệp.
Tiền điều kiện	Fall detection bật; ID0 và model sẵn sàng.
Luồng chính	Pose-tracking–LSTM phát raw fall; hậu kiểm; dừng robot; tạo một alert pending; hỏi TTS; phân loại phản hồi; cập nhật safe/danger.
Ngoại lệ	Mất bbox, bbox bị cắt, API lỗi, STT lỗi, không phản hồi, call đang tồn tại.

Hậu điều kiện	Safe: đóng sự cố; danger/no-response: gọi caregiver với emergencyReason; không phát lập đến khi phục hồi.
Yêu cầu liên quan	FR10–FR14, NFR01, NFR12.

UC08 – Gọi video

Tác nhân	Người cao tuổi/RobotApp; người chăm sóc/CaregiverApp
Mục tiêu	Thiết lập âm thanh, hình ảnh hai chiều.
Tiền điều kiện	Internet, camera/mic, signaling RTDB và STUN/TURN sẵn sàng.
Luồng chính	Caller ghi calling/role, offer; callee đọc và ghi answer; hai bên trao đổi ICE; connected; truyền media; ended và cleanup.
Ngoại lệ	Từ chối, timeout, mất mạng, permission thiếu, camera bận, TURN lỗi.
Hậu điều kiện	Listener, PeerConnection, capturer và renderer được giải phóng; camera trả về tracking; call history được ghi.
Yêu cầu liên quan	FR14, FR15, NFR03, NFR06.

UC09 – Điều khiển joystick

Tác nhân	Người chăm sóc; CaregiverApp; RobotApp; ESP32
Mục tiêu	Lái robot từ xa trong cuộc gọi.
Tiền điều kiện	Call/control screen hoạt động, Firebase và BLE kết nối, khu vực an toàn.
Luồng chính	Chạm joystick; tắt auto-follow; chuẩn hóa x, y ; gửi khoảng 20 Hz với timestamp; RobotApp lọc/ánh xạ; firmware chấp hành; thả để STOP.
Ngoại lệ	Mất touch, fragment pause, timestamp cũ, lệnh không hữu hạn, RTDB/BLE mất.
Hậu điều kiện	Khi thao tác kết thúc hoặc lỗi, target motor về 0.
Yêu cầu liên quan	FR16–FR20, NFR01–NFR03.

UC10 – Xem nhật ký

Tác nhân	Người chăm sóc
Mục tiêu	Xem các hoạt động nhắc, cảnh báo và cuộc gọi theo thời gian.
Tiền điều kiện	Đăng nhập và có quyền đọc dữ liệu.
Luồng chính	Chọn loại diary; query log; sắp xếp theo timestamp; hiển thị trạng thái, nội dung, phản hồi và lý do.
Ngoại lệ	Không có dữ liệu, schema cũ, offline, quyền bị từ chối.
Hậu điều kiện	Không sửa bản ghi khi chỉ xem; query được giới hạn để tránh tải toàn bộ lịch sử.
Yêu cầu liên quan	FR19, NFR05, NFR10.

Phụ lục B – Cấu trúc dữ liệu và giao thức truyền thông

B.1. Lệnh điều khiển RTDB

```

1 {
2   "robot_control": {
3     "x": -0.65,
4     "y": 0.10,
5     "ts": 1788381000000
6   }
7 }
```

Listing 4: Ví dụ lệnh điều khiển hợp lệ

Ràng buộc: x, y phải là số hữu hạn trong $[-1, 1]$ sau clipping; ts là epoch millisecond; tại RobotApp $now - ts \leq 2000$ ms và $ts - now \leq 5000$ ms. Lệnh STOP là $x = 0, y = 0$ với timestamp mới. Không xóa node thay cho STOP vì listener có thể diễn giải trường thiếu theo giá trị mặc định không mong muốn.

B.2. Signaling WebRTC

```

1 webrtc_signal: {
2   status: "calling" | "connected" | "ended",
3   callerRole: "robot" | "caregiver",
4   emergencyReason: "fall" | null,
```

```
5 offer: { type: "offer", sdp: "..."},
6 answer: { type: "answer", sdp: "..."},
7 robotCandidates: {
8   id: { candidate: "...", sdpMid: "0", sdpMLineIndex: 0 }
9 },
10 caregiverCandidates: {
11   id: { candidate: "...", sdpMid: "0", sdpMLineIndex: 0 }
12 }
13 }
```

Listing 5: Cấu trúc signaling khái quát

Mỗi cuộc gọi tương lai nên có `callId`, `robotId`, `caller UID`, `callee UID`, `createdAt`, `expiresAt` và `state transition` được `rules` kiểm soát. `Candidate/SDP` phải được xóa khi kết thúc để tránh client mới đọc dữ liệu cũ.

B.3. Bản ghi cảnh báo

```
1 fall_alerts/{alertId}: {
2   timestamp: Long,
3   status: "pending" | "safe" | "danger" | "no_response",
4   elderlyResponse: String,
5   verificationReason: String,
6   evidenceScore: Int,
7   visibleKeypoints: Int,
8   bboxFullyInside: Boolean,
9   upperBodyYRatio: Number
10 }
```

Listing 6: Các trường cảnh báo té ngã

B.4. BLE GATT

Phụ lục C – Danh mục module mã nguồn

Bảng 6.11: Thông số BLE triển khai

Device name	OhmniRobot
Service UUID	4FAFC201-1FB5-459E-8FCC-C5C9C331914B
Characteristic UUID	BEB5483E-36E1-4688-B7F5-EA07361B26A8
Property	Write without response
Payload	L<left>R<right>\n, ASCII/UTF-8, dấu chấm thập phân
Android limit	[−20, 20] mỗi motor
Firmware limit	[−40, 40] rad/s; deadzone 0,5/3,0
Watchdog	Android 500 ms; firmware 800 ms

Bảng 6.12: Module chính của RobotApp

Tệp/lớp	Trách nhiệm
MainActivity	Điều hướng, vòng đời fragment/call và listener lời mời
tracking_fragment	CameraX/WebRTC frame, AI, ID0, fall và auto-follow
BoundingBoxOverlay	Vẽ vùng bao, keypoint, ID và trạng thái
SortTracker	Ghép detection-track, bảo vệ ID0, tích hợp Re-ID
KalmanBoxTracker	Dự báo/cập nhật trạng thái vùng bao
HungarianAlgorithm	Tối ưu phép gán từ ma trận chi phí
ReIdModule	Histogram vùng thân và khôi phục track
FallDetectionQueue	Bộ đệm cửa sổ 15 frame, 69 feature
LstmFallDetector	Interpreter và máy trạng thái fall/lying/recovery
FallVerificationEngine	Hậu kiểm bằng chứng, khóa sự cố
TrackingService/ ReminderEvent	Chuyển sự kiện giữa tracking/reminder và chatbot
chatbot_fragment	UI giọng nói, xử lý reminder/fall/call intent
SpeechRecognizer Manager	Đóng gói Android SpeechRecognizer
AndroidTtsService	Đóng gói TextToSpeech và callback
GroqChatService	Chat streaming và các tác vụ phân loại
ReminderService	Theo dõi lịch và phát sự kiện nhắc nhở

Tệp/lớp	Trách nhiệm
video_call_fragment	PeerConnection, signaling, media và camera sharing
RobotControlManager	RTDB command, validation, motor mapping và BLE

Bảng 6.13: Module chính của CaregiverApp

Tệp/lớp	Trách nhiệm
Login/SignUpActivity	Firebase Authentication và tạo profile
MainActivity	Điều hướng chính và khởi động incoming-call service
ProfileFragment	Đọc/cập nhật Users/uid
tracking_fragment	Cờ follow/fall và STOP khi tắt
Schedule/ AddScheduleFragment	Thêm, đọc, cập nhật và xóa lịch nhắc nhở
ScheduleAdapter/ ScheduleItem	Danh sách và trạng thái active
Diary*Fragment/ Adapter	Reminder log, fall alert và call history
IncomingCallService	Listener nền và notification cuộc gọi
IncomingCallActivity	UI nhận/từ chối call toàn màn hình
video_call_fragment	WebRTC caller/callee, joystick và call log
chatbot_fragment	Placeholder trong snapshot hiện tại

Phụ lục D – Manifest artifact AI

Mỗi model phát hành nên đi kèm manifest theo mẫu:

```

1 model_id: pose-320-fp16-v1
2 source_weight: yolo26n-pose.pt
3 source_sha256: <hash>
4 converter:
5   ultralytics: <version>
6   onnx: <version>
7   onnx2tf: <version>

```

```
8   tensorflow: <version>
9   input:
10    shape: [1, 320, 320, 3]
11    dtype: float32
12    color: RGB
13    normalization: /255
14   output:
15    shape: <recorded shape>
16    decoding: android-decoder-v1
17   quantization: fp16-weights
18   validation:
19    dataset_split_sha256: <hash>
20    max_abs_tensor_error: <value>
21    final_metric_delta: <value>
```

Listing 7: *Mẫu manifest model có thể tải lập*

Đối với LSTM, manifest phải ghi thứ tự 69 feature, cách normalize keypoint, class order của output, sequence length, stride, threshold và hysteresis. Nếu thiếu một trong các thông tin này, model có thể nạp thành công nhưng cho kết quả sai ngữ nghĩa.

Phụ lục E – Quy trình build và triển khai

E.1. Hai ứng dụng Android

Yêu cầu JDK 11 trở lên, Android SDK phù hợp compile SDK 36 và tệp cấu hình Firebase của từng applicationId. Các lệnh chuẩn:

```
1 cd DATN_RobotApp_Smartphone_v2
2 gradlew.bat testDebugUnitTest
3 gradlew.bat assembleDebug
4
5 cd DATN_CaregiverApp_Smartphone_v2
6 gradlew.bat testDebugUnitTest
7 gradlew.bat assembleDebug
```

Listing 8: *Build và unit test Android trên Windows*

Không đưa local.properties, API key hoặc credential TURN vào Git. Build release cần signing key bảo mật, minify/resource shrinking và kiểm thử lại

reflection/native library.

E.2. Firmware

Trong Arduino IDE, chọn ESP32 Dev Module; cài SimpleFOC phiên bản tương thích và NimBLE-Arduino; mở `robot_ble_control.ino`; kiểm tra pin; compile/upload; mở Serial 115200. Lần đầu initFOC phải nâng bánh khỏi sàn. Kiểm tra encoder quay đúng chiều và chỉ thử setpoint nhỏ trước.

E.3. Dịch vụ

Thiết lập Firebase Auth email/password, RTDB và Rules. Cấu hình STUN/TURN trên cả hai ứng dụng. Cấp Groq key qua cơ chế bí mật. Tạo một tài khoản thử, profile, lịch và cặp thiết bị; xóa SDP/ICE cũ trước test. Đảm bảo notification/full-screen intent được cấp trên caregiver.

Phụ lục F – Phiếu ghi kết quả kiểm thử

Test ID	
Commit/model hash	
Ngày, người thực hiện	
Thiết bị/Android	
Mạng/BLE/khoảng cách	
Tiền điều kiện	
Các bước	
Kết quả mong đợi	
Kết quả quan sát	
Metric/log/video	
Pass/Fail/Blocked	
Mã lỗi và ghi chú	

Bảng 6.15: Mẫu tổng hợp benchmark AI

Cấu hình	Median (ms)	P95 (ms)	FPS hiệu dụng	Drop frame (%)
CPU float32				
GPU graph				
FP16				

Bảng 6.16: Mẫu tổng hợp phát hiện té ngã theo sự cố

Cấu hình	Precision	Recall	F1	Cảnh báo giả/giờ
Pose/luật				
Pose + LSTM				
Pose + LSTM + hậu kiểm				

Bảng 6.17: Mẫu tổng hợp độ trễ và an toàn điều khiển

Kịch bản	Median	P95	Max/ghi chú
Caregiver tới RobotApp			
RobotApp tới BLE callback			
Thời gian dừng khi mất RTDB			
Thời gian dừng khi mất BLE			

Phụ lục G – Checklist an toàn trình diễn

- Bánh xe được nâng khỏi sàn khi kiểm tra dầu/FOC lần đầu.
- Khu vực chạy phẳng, không có bậc thang, trẻ nhỏ, vật nuôi hoặc vật cản dễ vỡ.
- Pin, dây, driver và động cơ không quá nhiệt; có cầu chì/bảo vệ phù hợp.
- Có người giữ công tắc ngắt nguồn; STOP đã được thử trước auto-follow.
- Không dùng người cao tuổi thật để dựng lại té ngã; có đệm và giám sát.
- Không trình chiếu API key, token, email/số điện thoại thật hoặc dữ liệu sức khỏe.
- Kiểm tra RTDB Rules và dùng tài khoản/dữ liệu demo.
- Kiểm tra Internet, TURN, notification và quyền camera/micro/BLE.
- Lưu commit/model checksum và phương án khôi phục bản APK/firmware ổn định.
- Ghi log, nhưng tắt/xóa log nhạy cảm sau trình diễn.